

Hi2131V100 芯片

用户指南

文档版本 03

发布日期 2025-06-20

前言

概述

本文档主要介绍 Hi2131 系列芯片的各项基本功能。适用于 Hi2131、Hi2131C 芯片。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
Hi2131	V100
Hi2131C	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 客户开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
----	----

符号	说明
 危险	表示如不可避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
03	2025-06-20	更新 “A 订购须知” 小节。
02	2025-06-06	更新 “A 订购须知” 小节。
01	2025-04-07	第一次正式版本发布。 - 在 “1 产品概述” 中更新 “1.1 概述” 、 “1.2.1 功能特性” 、 “1.2.2 遵从的标准与协议” 、 “1.4 应用场景” 小节。 - 在 “2 系统” 中更新 “2.1.2 复位控制” 、 “2.2 时钟” 、 “2.3 电源管理与低功耗模式控制” 、 “ ” 小节。 - 在 “4 PMU” 中更新 “4.1 概述” 、 “4.2 管脚说明” 、 “4.3.1 初次上电” 、 “4.4 应用场景” 。 - 在 “5 外围设备” 中更新 “5.2.2 功能描述” 、 “5.4.2 功能描述” 、 “5.6.2 功能描述” 、 “5.7.2

文档版本	发布日期	修改说明
		功能描述”、“5.10.2 功能描述”、“5.13.2 功能描述” - 更新 “5.12.2 功能描述” 中的 Tsensor 的精度。 - 更新 “5.12.5 寄存器描述” 中 “TSENSOR_CTRL” 寄存器 bit[2:1]的取值含义。 - 更新 “6 安全” 小节。 - 更新 “A 订购须知” 小节。
00B05	2025-03-07	- 更新 “1.4 应用场景” 小节。 - 在 “2 系统” 中更新 “2.3.2 系统工作模式”、“2.10.1 概述” 小节。 - 在 “4 PMU” 中更新 “4.1 概述”、“4.2 管脚说明” 小节。 - 在 “5 外围设备” 中更新 “5.1.1 概述”、“5.1.2 软件复用管脚描述”、“5.2.2 功能描述”、“5.4.5 寄存器描述”、“5.6.3.1 接口信号”、“5.6.3.3 应用说明” 小节。
00B04	2025-02-24	更新 “4.3.1 初次上电” 小节。
00B03	2025-01-14	在 “5 外围设备” 中更新 “5.11.3 工作方式”、“5.12.5 寄存器描述” 小节。
00B02	2025-01-03	- 更新 “1.2.1 功能特性” 小节。 - 更新 “2.4 处理器子系统” 小节。 - 在 “5 外围设备” 中更新 “5.5.4.1 初始化配置”、“5.6.2 功能描述”、“5.11.2 功能描述”、“5.12.5 寄存器描述” 小节。
00B01	2024-11-29	第一次临时版本发布。

目 录

前言.....i

1 产品概述.....1

1.1 概述..... 1

1.2 功能描述..... 1

1.2.1 功能特性..... 1

1.2.2 遵从的标准与协议 3

1.3 芯片架构..... 3

1.4 应用场景..... 6

2 系统.....8

2.1 复位..... 8

2.1.1 概述..... 8

2.1.2 复位控制..... 8

2.1.3 复位约束..... 9

2.2 时钟..... 10

2.3 电源管理与低功耗模式控制..... 10

2.3.1 概述..... 10

2.3.2 系统工作模式11

2.4 处理器子系统11

2.5 中断系统..... 12

2.5.1 中断分配..... 12

2.5.2 中断结构..... 14

2.5.3 寄存器概览..... 15

2.5.4 寄存器描述 16

2.6 DMA..... 18

2.6.1 概述.....	18
2.6.2 功能描述.....	19
2.6.3 工作方式.....	19
2.6.4 寄存器概览.....	20
2.6.5 寄存器描述.....	22
2.7 安全加解密.....	49
2.7.1 概述.....	49
2.7.2 功能描述.....	49
2.8 RTC.....	49
2.8.1 概述.....	49
2.8.2 功能描述.....	50
2.8.3 工作方式.....	50
2.8.4 寄存器概览.....	50
2.8.5 寄存器描述.....	51
2.9 TIMER.....	56
2.9.1 概述.....	56
2.9.2 功能描述.....	57
2.9.3 工作方式.....	57
2.9.3.1 初始化配置.....	57
2.9.3.2 中断处理.....	57
2.9.3.3 Timer 复位.....	58
2.9.4 寄存器概览.....	58
2.9.5 寄存器描述.....	59
2.10 WDT.....	95
2.10.1 概述.....	95
2.10.2 功能描述.....	96
2.10.3 工作方式.....	96
2.10.4 寄存器概览.....	97
2.10.5 寄存器描述.....	98
3 RF 系统.....	106
4 PMU.....	107
4.1 概述.....	107

4.2 管脚说明.....	107
4.3 上下电流程	110
4.3.1 初次上电.....	110
4.3.2 下电流程.....	112
4.3.3 下电后再上电流程	112
4.4 应用场景.....	112
4.4.1 系统的供电.....	112
4.4.2 系统的时钟	113
5 外围设备.....	114
5.1 IO 复用.....	114
5.1.1 概述.....	114
5.1.2 软件复用管脚描述	115
5.1.3 控制寄存器	126
5.2 GPIO	127
5.2.1 概述.....	127
5.2.2 功能描述.....	128
5.2.3 工作方式.....	128
5.2.3.1 初始化配置.....	129
5.2.3.2 边沿中断配置	130
5.2.3.3 电平中断配置	130
5.2.4 寄存器概览.....	131
5.2.5 寄存器描述.....	132
5.3 UART	139
5.3.1 概述.....	139
5.3.2 功能描述.....	139
5.3.3 接口时序.....	140
5.3.4 工作方式.....	141
5.3.4.1 中断或查询方式下的数据传输	141
5.3.4.2 DMA 方式下的数据传输	143
5.3.5 寄存器概览.....	144
5.3.6 寄存器描述.....	146
5.4 LP_UART	168

5.4.1 概述.....	168
5.4.2 功能描述.....	168
5.4.3 工作方式.....	169
5.4.3.1 初始化配置.....	169
5.4.3.2 数据传输配置.....	170
5.4.4 寄存器概览.....	171
5.4.5 寄存器描述.....	172
5.5 I2C	182
5.5.1 概述.....	182
5.5.2 功能描述.....	182
5.5.3 接口时序.....	182
5.5.4 工作方式.....	183
5.5.4.1 初始化配置.....	183
5.5.4.1.1 I2C 控制器的时钟和复位.....	183
5.5.4.1.2 I2C 初始化配置.....	184
5.5.4.2 写操作（中断方式）.....	184
5.5.4.3 读操作（中断方式）.....	185
5.5.4.4 写操作（查询方式）.....	186
5.5.4.5 读操作（查询方式）.....	187
5.5.5 寄存器概览.....	187
5.5.6 寄存器描述.....	188
5.6 I2S/PCM	198
5.6.1 概述.....	198
5.6.2 功能描述.....	199
5.6.3 工作方式.....	199
5.6.3.1 接口信号.....	199
5.6.3.2 操作模式.....	200
5.6.3.3 应用说明.....	200
5.6.4 寄存器概览.....	205
5.6.5 寄存器描述.....	206
5.6.6 注意事项.....	226
5.7 SPI	227

5.7.1 概述.....	227
5.7.2 功能描述.....	227
5.7.3 接口时序.....	228
5.7.4 工作方式.....	232
5.7.4.1 SPI/SSP 协议+MASTER 模式数据收发配置.....	235
5.7.4.2 MICROWIRE 协议+MASTER 模式数据收发配置.....	236
5.7.4.3 SPI/SSP 协议+SLAVE 模式数据收发配置.....	237
5.7.4.4 MICROWIRE 协议+SLAVE 模式数据收发配置.....	238
5.7.4.5 DMA 配置流程.....	240
5.7.5 寄存器概览.....	241
5.7.6 寄存器描述.....	242
5.8 PWM.....	266
5.8.1 概述.....	266
5.8.2 功能描述.....	266
5.8.3 工作方式.....	266
5.8.3.1 单脉冲输出.....	266
5.8.3.2 repeat 模式.....	267
5.8.3.3 continuous 模式.....	267
5.8.4 寄存器概览.....	267
5.8.5 寄存器描述.....	268
5.9 UICC.....	274
5.9.1 概述.....	274
5.9.2 功能描述.....	274
5.9.3 工作方式.....	275
5.9.4 寄存器概览.....	277
5.9.5 寄存器描述.....	278
5.10 USB2.0.....	285
5.10.1 概述.....	285
5.10.2 功能描述.....	285
5.11 Vsensor.....	286
5.11.1 概述.....	286
5.11.2 功能描述.....	286

5.11.3 工作方式	287
5.11.4 寄存器概览	288
5.11.5 寄存器描述	289
5.12 Tsensor	301
5.12.1 概述	301
5.12.2 功能描述	302
5.12.3 工作方式	302
5.12.3.1 正常检测温度模式	302
5.12.3.2 高低温门限中断模式	303
5.12.3.3 过温保护中断模式	304
5.12.4 寄存器概览	304
5.12.5 寄存器描述	306
5.13 Audio PWM	313
5.13.1 概述	313
5.13.2 功能描述	313
6 安全	315
6.1 概述	315
6.2 功能描述	315
7 JTAG	316
7.1 概述	316
7.2 功能描述	316
A 订购须知	318
B 缩略语	320

插图目录

图 1-1 Hi2131V100 逻辑框图	4
图 1-2 应用场景通信连接示意图	7
图 3-1 RF 收发交互示意图	106
图 4-1 系统初次上电时序图	111
图 5-1 GPIO 模块原理图	128
图 5-2 GPIO 中断配置流程	129
图 5-3 UART 数据帧格式	141
图 5-4 UART 流控功能时序	141
图 5-5 I2C 起始和终止信号示意图	183
图 5-6 I2S 操作模式	200
图 5-7 SPI 功能框图	227
图 5-8 SPI 单帧帧格式 (SPO=0、SPH=0)	228
图 5-9 SPI 连续帧帧格式 (SPO=0、SPH=0)	229
图 5-10 SPI 单帧帧格式 (SPO=0、SPH=1)	229
图 5-11 SPI 连续帧帧格式 (SPO=0、SPH=1)	230
图 5-12 SPI 单帧帧格式 (SPO=1、SPH=0)	230
图 5-13 SPI 连续帧帧格式 (SPO=1、SPH=0)	231
图 5-14 SPI 单帧帧格式 (SPO=1、SPH=1)	231
图 5-15 SPI 连续帧帧格式 (SPO=1、SPH=1)	232
图 5-16 UICC Physical interface	274

图 A-1 芯片 Mark 命名规则 1.....	318
图 A-2 芯片 Mark 命名规则 2.....	319

Draft

表格目录

表 1-1 逻辑框图中各模块列表.....	4
表 2-1 复位控制说明	9
表 2-2 PWR_KEY_N 管脚	10
表 2-3 非标准中断编号列表 (ACPU)	12
表 2-4 ACPU 中断寄存器概览 (基址是 0x3000_6000)	15
表 2-5 DMA 寄存器概览 (基址是 0x3804_1000)	20
表 2-6 RTC 寄存器概览	50
表 2-7 TIMER 寄存器概览 (应用核 TIMER0 基址是 0x3000_1000, TIMER1 基址是 3000_1100, 协议核 TIMER0 的基址是 0x4040_0000, TIMER1 基址为 0x4040_0100)	58
表 2-8 WDT 寄存器概览 (应用核基址是 0x3000_4000, 协议核基址是 0x4040_1000)	97
表 4-1 电源引脚	107
表 4-2 信号引脚	108
表 4-3 时钟引脚	109
表 4-4 地引脚	109
表 5-1 VDDIO_A 的软件复用管脚.....	115
表 5-2 VDDIO_A 的软件复用管脚描述	115
表 5-3 VDDIO_B 的软件复用管脚.....	116
表 5-4 VDDIO_B 的软件复用管脚描述	117
表 5-5 VDDIO_C 的软件复用管脚	117
表 5-6 VDDIO_C 的软件复用管脚描述	120
表 5-7 VDDIO_F 的软件复用管脚	125

表 5-8 VDDIO_F 的软件复用管脚描述.....	125
表 5-9 VDDIO_A、VDDIO_F 管脚驱动强度 (1.8V)	126
表 5-10 VDDIO_B、VDDIO_C 管脚驱动强度 (3.3V)	126
表 5-11 VDDIO_B、VDDIO_C 管脚驱动强度 (1.8V)	127
表 5-12 GPIO 寄存器概览 (GPIO0~9 在 CSYS_SUB, 基址是 0x4040_5000, GPIO10~48 在 AON, 基址是 0x3006_9000, N 表示 GPIO 组序号)	131
表 5-13 UART 接口信号描述	140
表 5-14 自动流控配置参考	142
表 5-15 UART 寄存器概览 (UART0 基址是 0x3000_2000, UART1 基址是 0x3000_3000)	144
表 5-16 LP_UART 寄存器概览 (基址是 0x3006_0600)	171
表 5-17 I2C 总线描述.....	182
表 5-18 I2C 寄存器概览 (I2C0 的基址是 0x3000_0000, I2C1 的基址是 30000700)	187
表 5-19 I2S 接口信号	199
表 5-20 I2S/PCM 寄存器概览 (基址是 0x3000_0300)	205
表 5-21 SPI 接口信号描述	228
表 5-22 SPI 寄存器概览 (SPI0 基址是 0x3000_0100, SPI1 基址是 0x0x3000_0200)	241
表 5-23 PWM0 寄存器概览 (基址是 0x3000_0500)	267
表 5-24 UICC 寄存器概览 (基址是 0x3000_0400)	277
表 5-25 VSENSOR_RB 寄存器概览 (基址是 0x3002_8000)	289
表 5-26 Tsensor 寄存器概览 (基址是 0x3002_A000)	304
表 5-27 Tsensor 寄存器偏移地址变量表	306
表 7-1 JTAG 接口信号	316
表 A-1 芯片应用的配置 1	318
表 A-2 芯片应用的配置 2	319

1 产品概述

1.1 概述

1.2 功能描述

1.3 芯片架构

1.4 应用场景

1.1 概述

Hi2131 系列芯片是基于 3GPP R14 的 LTE Cat1 bis 芯片。产品高度集成，内置电源管理、时钟管理、应用处理器、协议处理器、射频、内存等功能模块；该芯片支持典型的 3GPP 协议规范频段。该产品具有优异的通信性能和待机功耗。

1.2 功能描述

1.2.1 功能特性

关键特性

- 支持频段范围：
 - LB：703MHz ~ 960MHz
 - MB：1710MHz ~ 2200MHz
 - HB：2300MHz ~ 2690MHz
- 集成电源管理单元、集成射频收发机、集成 2 个核心处理器。

- 支持 DCXO 或 TCXO 灵活配置，支持内外置 RTC 灵活选择。
- 集成丰富的外设接口：UART/LPUART、USB 2.0 Device、I2C、SPI、PWM/Audio PWM、I2S、MIPI-RFFE、JTAG、SWD 等。
- 支持多种低功耗模式，DRX 模式下功耗优。

其他特性

- **处理器**
 - 共 2 个自研 CPU (ACPU/CCPU)，均集成内存保护单元 (MPU)。
 - CPU 频率可配：ACPU 和 CCPU 频率可配置。
- **存储**
 - 支持 2MB/4MB Flash。
- **系统**
 - 时钟源：支持 XO 晶振时钟 (DCXO/TCXO 26MHz 或 TXCO 32MHz 晶振)，内置 RC_32kHz 时钟，可支持外置 32kHz 晶振。
 - 集成 48bit 位宽，时钟频率为 32kHz 的定时器。
- **外设接口**
 - 支持 49 个可编程 IO。
 - 支持 2 个 UART，支持波特率 4800/9600/115200/921600/2M/4M bit/s 可配置。
 - 支持低功耗 UART。
 - 支持 2 路 PWM 输出，可配置为 continue 模式或 repeat 模式。
 - 支持 2 个主从模式的 SPI。
 - 支持 JTAG/SWD 总线。
 - 支持 2 个 I2C 通信接口，最大支持 400kb/s，波特率可配。
 - 支持 1 个 MIPI RFFE 接口。最大支持 33MHz 端口速率，支持软件可配置。
 - 支持 4 通道的 ADC。
 - 支持 1 个 4 通道的 DMA。
 - 内置 1 个温度传感器。
- **低功耗**

芯片支持非业务时，支持进入睡眠模式低功耗状态。
- **通信**

- 支持 3GPP R14 Cat1 bis。
- 支持 FDD/TDD。
- 支持带宽 1.4/3/5/10/15/20 MHz。
- 支持 WIFI SCAN。

- **射频**

支持频段： Band 1/2/3/4/5/7/8/18/19/20/26/28/34/38/39/40/41/66。

- **安全**

- 安全启动、安全升级、安全存储、安全调试。
- 真随机数生成。
- 硬件密码：高级加密标准 (AES)，安全哈希算法 (SHA)。
- 硬件公钥加密 (PKE)：椭圆曲线密码学 (ECC)。
- 支持 eFuse。

- **封装**

5.8mm×5.8mm ball grid array (TFBGA) 封装，Ball Pitch 0.4mm，高度为 0.76mm (典型值)。

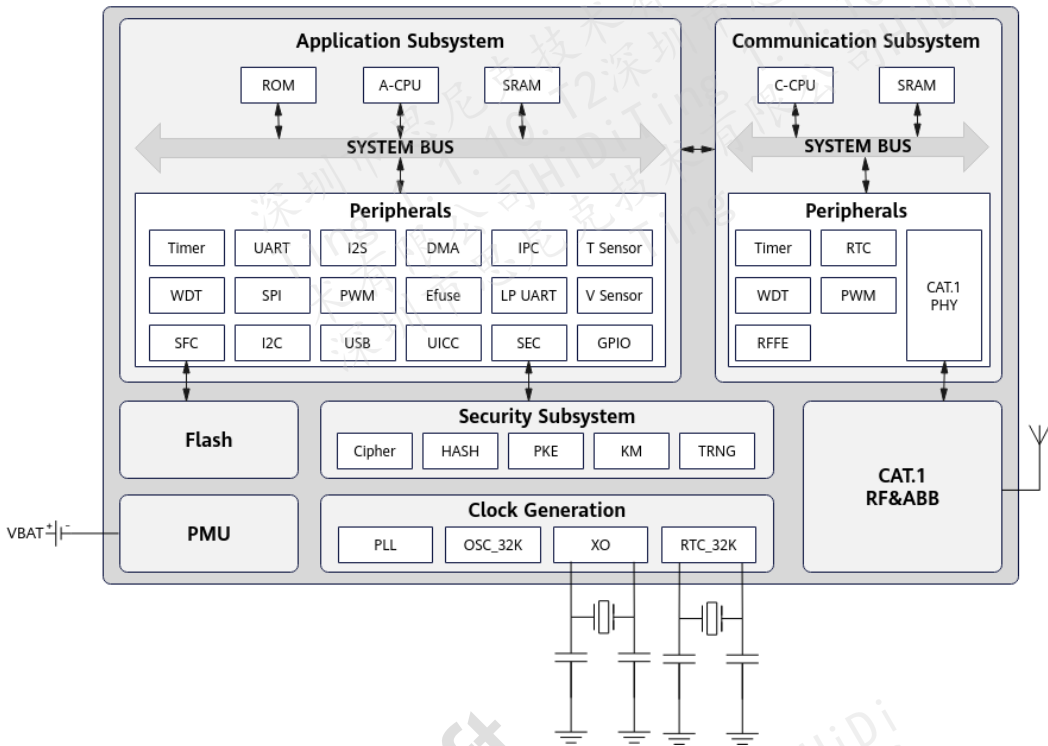
1.2.2 遵从的标准与协议

支持 3GPP R14 Cat1 bis 空口协议。

1.3 芯片架构

Hi2131V100 逻辑框图如图 1-1 所示。

图1-1 Hi2131V100 逻辑框图



逻辑框图中各模块功能介绍如表 1-1 所示。

表1-1 逻辑框图中各模块列表

模块名	功能
A CPU（Application Processor）	应用处理器，最高工作频率为 264MHz。
C CPU（Cat.1 Protocol Processor）	协议处理器，最高工作频率为 264MHz。
SRAM（Static Random Access Memory）	静态随机存取存储器，CPU 的内存。
ROM（Read-Only Memory）	只读存储器，用于 CPU 启动数据读取。
UART（Universal Asynchronous Receiver & Transmitter）	通用异步收发器。
DMA（Direct Memory Access）	直接内存访问模块。
USB（Universal Serial Bus）	通用串行总线。

模块名	功能
I2C (Inter-Integrated Circuit)	集成电路总线。
LP_UART (Low Power Universal Asynchronous Receiver & Transmitter)	低功耗通用异步收发器。
TIMER	定时器。
SPI (Serial Peripheral Interface)	串行外设接口。
GPIO (General Purpose Input/Output)	通用输入输出。
RTC (Real-Time Clock)	实时时钟。
PWM (Pulse-Width Modulation)	脉冲宽度调制。
RF FE (RF Front End)	射频前端接口。
UICC (Universal Integrated Circuit Card)	通用集成电路卡接口。
Cipher	协议加密模块。
EFUSE (Electronic Fuse)	一次性可编程器件。
HASH	哈希加密算法。
PKE (Public Key Encryption)	公开密钥加密算法。
KM (Key Management)	密钥管理。
OSC_32K	内置 32kHz 时钟源。
XO (Crystal Oscillator)	晶体振荡器。
RTC_32K	外置 32.768kHz 时钟源。
XO (Crystal Oscillators)	晶振。
PLL (Phase-Locked Loops)	锁相环。
Flash	闪存。
SFC (SPI Flash Controller)	串行接口闪存控制器。

模块名	功能
PMU (Power Manager Unit)	电源管理模块。
IPC (Inter Process Communication)	核间通信模块。
WDT (Watch Dog Timer)	看门狗。
CAT1.PHY	CAT1 的 PHY 层处理单元。
CAT1.RF&ABB	CAT1 的射频基带单元。
I2S (Inter—IC Sound)	音频数据传输接口控制单元。
TSensor	温度传感器。
VSensor	电压传感器。
SEC	安全管理模块。
TRNG (True Random Number Generator)	真随机数生成器。

1.4 应用场景

Hi2131 系列芯片主要应用场景如下：

- Hi2131CV100：主要用于数传场景，支持 OpenCPU 能力，可支持集成少量应用，用于 IP 摄像头、金融支付、跟踪定位等。
- Hi2131V100：主要用于 OpenCPU 开发场景，可支撑客户集成丰富的应用，如共享经济、金融支付、跟踪定位等。

不同场景中的终端应用芯片时，通过天线与蜂窝网络基站进行通信，如图 1-2 所示。

图1-2 应用场景通信连接示意图



2 系统

- 2.1 复位
- 2.2 时钟
- 2.3 电源管理与低功耗模式控制
- 2.4 处理器子系统
- 2.5 中断系统
- 2.6 DMA
- 2.7 安全加解密
- 2.8 RTC
- 2.9 TIMER
- 2.10 WDT

2.1 复位

2.1.1 概述

复位模块根据输入控制信号产生各模块的复位信号，支持整个芯片全局复位和各个模块的单独复位。

2.1.2 复位控制

复位控制类型包括：

- 整芯片复位控制

- 子系统复位控制
- 单独模块复位控制

芯片可以使用的复位控制方式如表 2-1 所示。

表2-1 复位控制说明

复位方式	来源	复位范围	复位后芯片模式	说明
硬件复位	芯片管脚 RSTN	全芯片	ACTIVE	芯片的管脚复位，会滤除管脚输入信号 100μs 长度的毛刺，避免误触发复位；可配置屏蔽；低电平有效。
POR (Power On Reset) 复位	模拟 PMU 模块输出	全芯片	ACTIVE	模拟 PMU 输出的复位信号，用于芯片的上电复位。
应用子系统及协议子系统 Watch Dog 复位	应用子系统及协议子系统中 Watch Dog 模块输出	全芯片	ACTIVE	系统的看门狗复位。
全局软复位控制	系统控制寄存器，软件配置	全芯片	ACTIVE	软件使用的复位，用于复位整系统。
低功耗唤醒硬复位	低功耗控制器模块	掉电区域	ACTIVE	低功耗复位，用于上电后恢复掉电寄存器的默认状态。

2.1.3 复位约束

为了防止管脚复位 RSTN 误触发，在芯片内部实现滤毛刺机制，可防止误触，并且可以滤除 100μs 以内的毛刺。

2.2 时钟

系统时钟在高性能模式下最高可运行在 264MHz，低功耗模式下最低运行在 XO (26MHz/32MHz) 时钟下，支持中间多档位 (198MHz/132MHz/99MHz 等) 可配。

2.3 电源管理与低功耗模式控制

2.3.1 概述

芯片提供多种低功耗的控制来动态降低芯片的功耗：

- 系统工作模式控制

整 SOC (System-On-a-Chip) 系统有多种工作模式，可以根据实际的功耗要求和性能要求选择不同的工作模式。

- 时钟门控和时钟频率调整

提供时钟关断功能，可以关闭没有必要的时钟，减少芯片的功耗。系统工作的时钟频率可以进行调整，在满足整系统降频条件后，系统可以自动切换时钟频率以降低芯片功耗。

- 模块级低功耗控制

提供模块级的低功耗控制，可以在协议核不工作的情况下，关断该模块电源以减少芯片的功耗。

提供模块级的低功耗控制，可以在应用核不工作的情况下，关断该模块时钟或者配置进入复位状态以减少芯片的功耗。

- 支持按键开关机

表2-2 PWR_KEY_N 管脚

信号名称	含义	I/O
PWR_KEY_N	芯片上下电管脚： • 有效按键时间大于 10ms，芯片开机。 • 芯片处于上电状态时拉低 PWR_KEY_N 管脚，单次长按时间大于 1s 芯片关机。	I

2.3.2 系统工作模式

系统提供以下工作模式：

- normal 工作模式：应用核以及协议核均处于工作模式。
在该模式下，系统中双核 CPU 和射频处于高性能高带宽模式下。
- OPEN MCU 模式：仅应用核处于工作模式，协议核不会被启动。
在该模式下，芯片作为 MCU 开放，不进行 RF 通信，因此协议核处于掉电状态以降低芯片功耗。
- DRX 模式：系统仅协议核处于工作模式，RF 处于 RX 接收模式。
在该模式下，应用核处于复位状态，协议核和射频处于低速模式状态。
- 深睡工作模式：系统进入深睡模式。
在该模式下，系统仅保留常电区（少部分数字逻辑和常电 IO 以及保证系统唤醒后正常工作的 retention ram）工作，关闭各子系统的电源，等待唤醒中断后重新上电。

2.4 处理器子系统

应用子系统、协议子系统均采用自研的 CPU，完成系统的任务和控制工作。

该芯片用户指南中，应用子系统处理器简称为 ACPU，协议子系统处理器简称为 CCPU。

该芯片 CPU 具有以下特性：

- 最高支持 264MHz 工作频率，双核工作在统一频率下。
- 应用子系统 ACPU 支持 64 个中断，协议子系统 CCPU 支持 32 个中断，支持电平触发或边沿触发。
- 支持 Flash Patch 功能，ACPU 包含 64 个指令比较器和 2 个地址比较器，CCPU 包含 32 个指令比较器和 2 个地址比较器。
- 支持边沿和电平两种终端触发方式。
- 支持 PMP（Physical Memory Protection）功能。
- 支持 JTAG（Joint Test Action Group）和 SWD（Serial Wire Debug）接口调试。

2.5 中断系统

2.5.1 中断分配

芯片支持向量模式和直接模式的中断方式。ACPU 支持边沿和电平两种中断触发方式，支持优先级可编程，优先级配置寄存器（共 3bit）可配置 8 级的优先级。配置相同的情况下，中断编号小的优先级高。全部中断默认配置为 3'b000。

中断系统包括：

- ACPU 外部的非标准中断：ACPU 所支持的非标准中断编号分别如表 2-3 所示。

表2-3 非标准中断编号列表（ACPU）

中断编号	中断名称	中断描述
NMI	a_wdg_nmi_intr	ACPU 的 NMI 中断，包含看门狗状态中断和软件 NMI 中断。
16	i2c1_intr	I2C1 控制器收发状态中断。
17	uart0_intr	UART0 控制器收发状态中断。
18	uart1_intr	UART1 控制器收发状态中断。
19	pwm0_intr	PWM0 控制器输出状态中断。
20	a_dma_intr	DMA 控制器上报 ACPU 状态中断。
21	clkcali_intr	32K 时钟计数校准控制器完成中断。
22	share_mem_monitor_intr	SHARE 内存维测中断。
23	spi0_intr	SPI0 控制器收发状态中断。
24	spi1_intr	SPI1 控制器收发状态中断。
25	i2s_sio_intr	I2S1 控制器状态中断。
26	i2s_sio_tx_intr	I2S1 控制器发送状态中断。
27	i2s_sio_rx_intr	I2S1 控制器接收状态中断。
28	i2c0_intr	I2C0 控制器收发状态中断。
29	vsensor_intr	Vsensor 控制器测量完成状态中断。

中断编号	中断名称	中断描述
30	lp_uart_tx_intr	LP_UART 控制器发送状态中断。
31	lp_uart_rx_intr	LP_UART 控制器接收状态中断。
32	audio_pwm_intr	Audio PWM 控制器输出状态中断。
33	cipher_intr	Cipher 控制器完成状态中断。
34	acpu_ipc_intr	ACPU 核间通信状态中断。
35	aon_gpio_intr	常电 GPIO 状态中断。
36	acpu_soft0_intr	ACPU 软中断 0。
37	acpu_soft1_intr	ACPU 软中断 1。
38	ccpu_soft0_intr	CCPU 软中断 0。
39	ccpu_soft1_intr	CCPU 软中断 1。
40	a_timer0_intr	ACPU Timer0 计数状态上报中断。
41	a_timer1_intr	ACPU Timer1 计数状态上报中断。
42	a_rtc0_intr	ACPU RTC0 计数状态上报中断。
43	a_rtc1_intr	ACPU RTC1 计数状态上报中断。
44	usb_intr	USB 控制器状态上报中断。
45	uicc_intr	UICC 控制器状态上报中断。
46	uicc_etu_intr	UICC 控制器超时状态上报中断。
47	pwm1_intr	PWM1 控制器输出状态中断。
48	tsensor_intr	Tsensor 控制器温度检测状态上报中断。
49	sec0_intr	安全相关状态上报 0 中断
50	sec1_intr	安全相关状态上报 1 中断
51	sec2_intr	安全相关状态上报 2 中断
52	sfc_intr	FLASH 控制器状态上报中断。
53	usb_err_intr	USB 错误状态上报中断。

中断编号	中断名称	中断描述
54	cmu_err_intr	CMU 错误状态上报中断。
55	lp_ctl_err_intr	低功耗状态错误上报中断。
56	usb_sys_err_intr	USB 控制器内部状态错误上报中断。
57	asys_wakeup_intr	应用系统唤醒原因上报中断。
58	usb_vbus_intr	USB VBUS 插拔状态上报中断。
59	sim_hpd_intr	SIM 卡插拔状态上报中断。
60	vol_temp_intr	PMU 电压温度检测状态上报中断。

2.5.2 中断结构

说明

- 芯片使用 CPU 内部集成的中断控制器。所有的外设或寄存器触发的非标准中断均直接连至 CPU 内部进行处理。
- 本节主要介绍非 IP 中断，各个 IP 的中断说明请参见各 IP 对应章节。

NMI 中断

NMI (Non-Maskable Interrupt) 中断用于控制软件，处理系统的特殊场景，在触发 NMI 中断后，CPU 会进入异常处理程序。

ACPU 可以通过 WDT 中断或写寄存器 NMI_SOFT_INTR[soft_tee_nmi_int]触发 NMI 中断。

说明

NMI_SOFT_INTR[soft_tee_nmi_int]不会自动清零，在触发中断后需要将该值写回 0。

软中断

软中断通过配置寄存器触发中断。ACPU 包含 2 个可以通过写寄存器触发的中断。

软中断的触发方式相同，以 ACPU 的软中断 0 为例，配置步骤如下：

- 步骤 1 写 **ASYS_SOFT_IRQ_EN**[asys_soft_irq_en] bit0 为 1，打开软中断 0 使能。
- 步骤 2 写 **ASYS_SOFT_IRQ_MASK**[asys_soft_irq_mask] bit0 为 0，关闭软中断上报屏蔽。

- 0：软中断置位后信号上报 CPU。
- 1：屏蔽软中断置位后信号，不上报 CPU，仅上报 `ASYS_SOFT_IRQ_STAT`[`asys_soft_irq_stat`]供查询。

步骤 3 写 `ASYS_SOFT_IRQ_IN`[`asys_soft_irq_in`] bit0 为 1，将软中断 0 置位。
此寄存器为自清零寄存器，写 1 后寄存器生效使能自动回 0，写 0 无效。

步骤 4 软件进入软中断 0 处理程序，此时可读取 `ASYS_SOFT_IRQ_STAT`[`asys_soft_irq_stat`] bit0，查询中断状态：

- 0：无中断。
- 1：有中断。

步骤 5 写 `ASYS_SOFT_IRQ_CLEAR`[`asys_soft_irq_clr`] bit0 为 1，清除软中断 0。
此寄存器为自清零寄存器，写 1 后自动回零，写 0 无效。

----结束

 说明

ACPU 的软中断 1 和 CCPU 的 2 个软中断配置步骤相同，按照上述配置对应功能寄存器即可，不再赘述。

2.5.3 寄存器概览

ACPU 中断寄存器概览如表 2-4 所示。

表2-4 ACPU 中断寄存器概览（基址是 0x3000_6000）

偏移地址	名称	描述
0x0204	<code>ACPU_NMI_SOFT_INTR</code>	A 核的 NMI 中断触发配置寄存器。
0x0260	<code>ASYS_SOFT_IRQ_IN</code>	A 核产生的软中断寄存器。
0x0264	<code>ASYS_SOFT_IRQ_CLEAR</code>	A 核产生的软中断清除寄存器。
0x0268	<code>ASYS_SOFT_IRQ_EN</code>	A 核软中断使能寄存器。
0x026C	<code>ASYS_SOFT_IRQ_MASK</code>	A 核软中断屏蔽寄存器。
0x0270	<code>ASYS_SOFT_IRQ_STAT</code>	A 核屏蔽前软中断查询寄存器。

2.5.4 寄存器描述

ACPU_NMI_SOFT_INTR

A 核产生的 NMI 中断寄存器。

Offset Address: 0x204 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000 0
0	RW	acpu_soft_nmi_intr	A 核 NMI 软中断触发寄存器; 1: 拉高 NMI 中断; 0: 拉低 NMI 中断。	0x0

ASYS_SOFT_IRQ_IN

A 核产生的软中断寄存器。

Offset Address: 0x260 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:2	RO	reserved	保留。	0x00000000
1:0	W1_PULSE	asys_soft_irq_in	A 核软中断触发寄存器，在软中断对使能寄存器使能的情况下，对应 bit 写 1 表示触发中断。 00: 写 00 无效; 01: 写 01 触发软中断 0; 10: 写 10 触发软中断 1; 11: 写 11 触发软中断 0 和 1。	0x0

ASYS_SOFT_IRQ_CLEAR

A 核产生的软中断清除寄存器。

Offset Address: 0x264 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:2	RO	reserved	保留。	0x00000000
1:0	W1_PULSE	asys_soft_irq_clr	A 核软中断清除寄存器，对应 bit 写 1 表示清除。 00: 写 00 无效; 01: 写 01 清除软中断 0; 10: 写 10 清除软中断 1; 11: 写 11 清除软中断 0 和 1。	0x0

ASYS_SOFT_IRQ_EN

A 核软中断使能寄存器。

Offset Address: 0x268 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:2	RO	reserved	保留。	0x00000000
1:0	RW	asys_soft_irq_en	A 核软中断使能寄存器，对应 bit 写 1 表示使能，0 表示不使能，2bit 分别对应两个软中断。 00: 不使能软中断; 01: 使能软中断 0; 10: 使能软中断 1; 11: 使能软中断 0 和 1。	0x0

ASYS_SOFT_IRQ_MASK

A 核软中断屏蔽寄存器。

Offset Address: 0x26C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
31:2	RO	reserved	保留。	0x00000000
1:0	RW	asys_soft_irq_mask	A 核软中断屏蔽寄存器，对应 bit 写 1 表示屏蔽，0 表示不屏蔽，2bit 分别对应两个软中断。 00：不屏蔽软中断； 01：屏蔽软中断 0； 10：屏蔽软中断 1； 11：屏蔽软中断 0 和 1。	0x0

ASYS_SOFT_IRQ_STAT

A 核屏蔽前软中断查询寄存器。

Offset Address: 0x270 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:2	RO	reserved	保留。	0x00000000
1:0	RO	asys_soft_irq_stat	A 核屏蔽前软中断查询寄存器，1 表示触发中断，0 表示未触发，2bit 分别对应两个软中断。 00：未触发软中断； 01：触发了软中断 0； 10：触发了软中断 1； 11：触发了软中断 0 和 1。	0x0

2.6 DMA

2.6.1 概述

DMA 直接在存储器和外设之间、存储器和存储器之间、外设和外设之间搬移数据，可以避免处理器干预，减少处理器的中断处理开销。

DMA 控制器收到通道的 Request 请求后，根据 CPU 已经配置的源地址、目的地址、通道的控制信息完成数据的搬移同时，对发送的数据表项进行计数，并通过中断的形式上报数据传输状态（即指示数据是否传输完成或传输错误）到 CPU。DMA 方式适用于数据块的高速传输。

2.6.2 功能描述

DMA 具有以下功能特点：

- 支持 4 个通道。
- 支持 8bit/16bit/32bit 数据传输。
- DMA 通道优先级固定，优先级从高到低对应的通道号依次为 0~3，通道 0 优先级最高，通道 3 优先级最低。
- DMA 每个通道各包含 1 个 16×32bit 的 FIFO (First In First Out)。
- 支持外设的单次传输 (Single) 和连续传输 (Burst) 两种 DMA 请求。
- 支持 16 组 DMA 请求，可通过配置设为传输的源端请求或目的端请求。
- DMA 请求支持以下外设：UART0/1、SPI0/1、UICC、I2S。
- 支持 DMA 传输长度可编程。
- 支持源地址、目的地址配置为 DMA 传输时地址递增或不变。
- 支持 4 个数据传输方向：
 - Memory 到外设
 - Memory 到 Memory
 - 外设到 Memory
 - 外设到外设
- 支持 DMA 时钟门控关闭功能。
- 支持链表传输。
- 支持 2 个电平中断，1 号中断上报 A 核，2 号中断上报 C 核。2 个中断支持单独屏蔽通道 0~3 的传输完成中断和错误中断。

2.6.3 工作方式

时钟门控

使用 DMA 时，需要配置 `CRG_SHARE_CLKEN_RB_0[cfg_share_dma_clken]` 为 1，打开 DMA 时钟门控。

DMA 初始化

以配置通道 0 为例，初始化 DMA 步骤如下：

- 步骤 1 清除历史中断，配置寄存器 `DMAC_INT_CLR[int_trans_clr]` 为 1，
`DMAC_INT_CLR[int_err_clr]` 为 1。
- 步骤 2 使能 DMAC，配置寄存器 `DMAC_CONFIG[dmac_en]` 为 1。

----结束

DMA 通道 0 搬移配置

- 步骤 1 配置通道 0 控制寄存器 `DMAC_CHN_CONTROL_0`，配置源端/目的端的地址递增方式、传输位宽、Burst 长度以及传输的长度。
- 步骤 2 配置通道 0 配置寄存器 `DMAC_CHN_CONFIG_0`，配置中断屏蔽、流控与传输类型、源设备/目的设备编号。
- 步骤 3 使能通道 0，配置 `DMAC_CHN_CONFIG_0[dmac_chn_en_0]` 为 1。

----结束

2.6.4 寄存器概览

DMA 寄存器概览如表 2-5 所示。

表2-5 DMA 寄存器概览（基址是 0x3804_1000）

偏移地址	名称	描述
0x0000	<code>DMAC_ID</code>	DMA 版本寄存器。
0x0004	<code>DMAC_INT1_ST</code>	中断 1 状态寄存器。
0x0040	<code>DMAC_INT2_ST</code>	中断 2 状态寄存器。
0x0044	<code>DMAC_TC_INT2_ST</code>	中断 2 传输状态寄存器。
0x0048	<code>DMAC_ERR_INT2_ST</code>	中断 2 错误状态寄存器。
0x0008	<code>DMAC_INT_CLR</code>	传输中断寄存器。
0x000C	<code>DMAC_ORI_INT_ST</code>	原始中断状态寄存器。

偏移地址	名称	描述
0x0010	DMAC_EN_CHNS	通道使能查询寄存器。
0x0014	DMAC_BURST_REQ	BURST 软件配置寄存器。
0x0018	DMAC_SINGLE_REQ	SINGLE 软件配置寄存器。
0x001C	DMAC_CONFIG	配置寄存器。
0x0020	DMAC_SYNC	同步寄存器。
0x0024	DMAC_AUTOCG_EXTEND	自动门控配置寄存器。
0x0100	DMAC_LLI_0	通道 0 链表寄存器。
0x0120	DMAC_LLI_1	通道 1 链表寄存器。
0x0140	DMAC_LLI_2	通道 2 链表寄存器。
0x0160	DMAC_LLI_3	通道 3 链表寄存器。
0x0110	DMAC_S_ADDR_0	通道 0 源地址寄存器。
0x0130	DMAC_S_ADDR_1	通道 1 源地址寄存器。
0x0150	DMAC_S_ADDR_2	通道 2 源地址寄存器。
0x0170	DMAC_S_ADDR_3	通道 3 源地址寄存器。
0x0104	DMAC_D_ADDR_0	通道 0 目的地址寄存器。
0x0124	DMAC_D_ADDR_1	通道 1 目的地址寄存器。
0x0144	DMAC_D_ADDR_2	通道 2 目的地址寄存器。
0x0164	DMAC_D_ADDR_3	通道 3 目的地址寄存器。
0x0114	DMAC_CHN_CONTROL_0	通道 0 控制寄存器。
0x0134	DMAC_CHN_CONTROL_1	通道 1 控制寄存器。
0x0154	DMAC_CHN_CONTROL_2	通道 2 控制寄存器。
0x0174	DMAC_CHN_CONTROL_3	通道 3 控制寄存器。
0x0108	DMAC_CHN_CONFIG_0	通道 0 配置寄存器。
0x0128	DMAC_CHN_CONFIG_1	通道 1 配置寄存器。

偏移地址	名称	描述
0x0148	DMAC_CHN_CONFIG_2	通道 2 配置寄存器。
0x0168	DMAC_CHN_CONFIG_3	通道 3 配置寄存器。

2.6.5 寄存器描述

DMAC_ID

DMA 版本寄存器。

Offset Address: 0x0000 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:8	-	reserved	保留。	0x000000
7:0	RO	dmac_id	版本: v00。	0x00

DMAC_INT1_ST

中断 1 状态寄存器。

Offset Address: 0x0004 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:20	-	reserved	保留。	0x000
19:16	RO	int_err_st	错误中断 1 状态查询。 i=0 ~ 3 int_err_st[i]=0: 通道 i 未产生错误中断 (经过中断屏蔽) ; int_err_st[i]=1: 通道 i 产生错误中断 (经过中断屏蔽) 。	0x0
15:12	-	reserved	保留。	0x0
11:8	RO	int_trans_st	传输中断 1 状态查询。 i=0 ~ 3	0x0

Bits	Access	Name	Description	Reset
			int_trans_st[i]=0: 通道 i 未产生传输中断 (经过中断屏蔽); int_trans_st[i]=1: 通道 i 产生传输中断 (经过中断屏蔽)。	
7:4	-	reserved	保留。	0x0
3:0	RO	int_st	中断 1 状态查询。 i=0 ~ 3 int_st[i]=0: 通道 i 未产生中断 (经过中断屏蔽); int_st[i]=1: 通道 i 产生中断 (经过中断屏蔽)。	0x0

DMAC_INT2_ST

中断 2 状态寄存器。

Offset Address: 0x0040 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:4	-	reserved	保留。	0x00000000
3:0	RO	int2_st	中断 2 状态查询。 i=0 ~ 3 int_st[i]=0: 通道 i 未产生中断 (经过中断屏蔽); int_st[i]=1: 通道 i 产生中断 (经过中断屏蔽)。	0x0

DMAC_TC_INT2_ST

中断 2 传输状态寄存器。

Offset Address: 0x0044 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:4	-	reserved	保留。	0x00000000
3:0	RO	int2_trans_st	传输中断 2 状态查询。 i=0 ~ 3 int_trans_st[i]=0: 通道 i 未产生传输中断 (经过中断屏蔽) ; int_trans_st[i]=1: 通道 i 产生传输中断 (经过中断屏蔽) 。	0x0

DMAC_ERR_INT2_ST

中断 2 错误状态寄存器。

Offset Address: 0x0048 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:4	-	reserved	保留。	0x00000000
3:0	RO	int2_err_st	错误中断 2 状态查询。 i=0 ~ 3 int_err_st[i]=0: 通道 i 未产生错误中断 (经过中断屏蔽) ; int_err_st[i]=1: 通道 i 产生错误中断 (经过中断屏蔽) 。	0x0

DMAC_INT_CLR

传输中断寄存器。

Offset Address: 0x0008 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:12	-	reserved	保留。	0x000000
11:8	WO	int_err_clr	i=0 ~ 3	0x0

Bits	Access	Name	Description	Reset
			int_err_clr[i]=0: 不清除通道 i 的错误中断; int_err_clr[i]=1: 清除通道 i 的错误中断。	
7:4	-	reserved	保留。	0x0
3:0	WO	int_trans_clr	i=0 ~ 3 int_trans_clr[i]=0: 不清除通道 i 的传输中断; int_trans_clr[i]=1: 清除通道 i 的传输中断。	0x0

DMAC_ORI_INT_ST

原始中断状态寄存器。

Offset Address: 0x000C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:12	-	reserved	保留。	0x00000
11:8	RO	ori_int_err_st	i=0 ~ 3 ori_int_err_st[i]=0: 通道 i 未产生错误中断 (未经中断屏蔽); ori_int_err_st[i]=1: 通道 i 产生错误中断 (未经中断屏蔽)。	0x0
7:4	-	reserved	保留。	0x0
3:0	RO	ori_int_trans_st	i=0 ~ 3 ori_int_trans_st[i]=0: 通道 i 未产生传输中断 (未经中断屏蔽); ori_int_trans_st[i]=1: 通道 i 产生传输中断 (未经中断屏蔽)。	0x0

DMAC_EN_CHNS

通道使能查询寄存器。

Offset Address: 0x0010 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:4	-	reserved	保留。	0x00000000
3:0	RO	en_chns	i=0 ~ 3 en_chns[i]=0: 通道 i 未使能; en_chns[i]=1: 通道 i 使能。	0x0

DMAC_BURST_REQ

BURST 软件配置寄存器。

Offset Address: 0x0014 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:16	RW	burst_req	i=0 ~ 15 burst_req[i]=0: 无影响; burst_req[i]=1: 产生 DMA burst 传输请求, 当传输结束时该寄存器中的相应位被清零。	0x0000
15:0	RW	last_burst_req	i=0 ~ 15 last_burst_req[i]=0: 无影响; last_burst_req[i]=1: 产生 DMA last burst 传输请求, 当传输结束时该寄存器中的相应位被清零。	0x0000

DMAC_SINGLE_REQ

SINGLE 软件配置寄存器。

Offset Address: 0x0018 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
31:16	RW	single_req	i=0 ~ 15 single_req[i]=0: 无影响; single_req[i]=1: 产生 DMA single 传输请求, 当传输结束时该寄存器中的相应位被清零。	0x0000
15:0	RW	last_single_req	i=0 ~ 15 last_single_req[i]=0: 无影响; last_single_req[i]=1: 产生 DMA last single 传输请求, 当传输结束时该寄存器中的相应位被清零。	0x0000

DMAC_CONFIG

DMAC_CONFIG 为配置寄存器。

Offset Address: 0x001C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:2	-	reserved	保留。	0x00000000
1	RW	dmac_m1	Master 1 endianness 配置位。 0: little endian 模式; 1: big endian 模式。	0x0
0	RW	dmac_en	DMAC 使能。 0: 禁止 DMAC; 1: 使能 DMAC。	0x0

DMAC_SYNC

同步寄存器。

Offset Address: 0x0020 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
31:16	-	reserved	保留。	0x0000
15:0	RW	damc_sync	控制是否需要请求线进行同步。 0：使能对应外设的 DMA 请求信号同步逻辑； 1：禁止对应外设的 DMA 请求信号同步逻辑。	0x0000

DMAC_AUTO CG_EXTEND

自动门控配置寄存器。

Offset Address: 0x0024 Total Reset Value: 0x0000_0002

Bits	Access	Name	Description	Reset
31:5	-	reserved	保留。	0x00000000
4	RW	damc_master_auto_cg_en	控制 DMA MASTER AUTO CG。 0：正常； 1：bypass。	0x0
3	RW	damc_slave_auto_cg_en	控制 DMA SLAVE AUTO CG。 0：正常； 1：bypass。	0x0
2:0	RW	damc_auto_cg_extend	控制 DMA AUTO CG 延长拍数，范围 0~7。	0x2

DMAC_LLI_0

通道 0 链表寄存器。

Offset Address: 0x0100 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:2	RW	dmac_lli_0	链式地址[31:2]。	0x00000000

Bits	Access	Name	Description	Reset
1	-	reserved	保留。	0x0
0	RW	dmac_lm_0	用于载入下一个链表结点的 Master。 0: Master1; 1: Master2。 注意：本芯片只支持 Master1。	0x0

DMAC_LLI_1

通道 1 链表寄存器。

Offset Address: 0x0120 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:2	RW	dmac_lli_1	链式地址[31:2]。	0x00000000
1	-	reserved	保留。	0x0
0	RW	dmac_lm_1	用于载入下一个链表结点的 Master。 0: Master1; 1: Master2。 注意：本芯片只支持 Master1。	0x0

DMAC_LLI_2

通道 2 链表寄存器。

Offset Address: 0x0140 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:2	RW	dmac_lli_2	链式地址[31:2]。	0x00000000
1	-	reserved	保留。	0x0
0	RW	dmac_lm_2	用于载入下一个链表结点的	0x0

Bits	Access	Name	Description	Reset
			Master。 0: Master1; 1: Master2。 注意：本芯片只支持 Master1。	

DMAC_LLI_3

通道 3 链表寄存器。

Offset Address: 0x0160 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:2	RW	dmac_lli_3	链式地址[31:2]。	0x00000000
1	-	reserved	保留。	0x0
0	RW	dmac_lm_3	用于载入下一个链表结点的 Master。 0: Master1; 1: Master2。 注意：本芯片只支持 Master1。	0x0

DMAC_S_ADDR_0

通道 0 源地址寄存器。

Offset Address: 0x0110 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	dmac_s_addr_0	DMAC 通道 0 源地址寄存器。	0x00000000

DMAC_S_ADDR_1

通道 1 源地址寄存器。

Offset Address: 0x0130 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	dmac_s_addr_1	DMAC 通道 1 源地址寄存器。	0x00000000

DMAC_S_ADDR_2

通道 2 源地址寄存器。

Offset Address: 0x0150 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	dmac_s_addr_2	DMAC 通道 2 源地址寄存器。	0x00000000

DMAC_S_ADDR_3

通道 3 源地址寄存器。

Offset Address: 0x0170 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	dmac_s_addr_3	DMAC 通道 3 源地址寄存器。	0x00000000

DMAC_D_ADDR_0

通道 0 目的地址寄存器。

Offset Address: 0x0104 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	dmac_d_addr_0	DMAC 通道 0 目的地址寄存器。	0x00000000

DMAC_D_ADDR_1

通道 1 目的地址寄存器。

Offset Address: 0x0124 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	dmac_d_addr_1	DMAC 通道 1 目的地址寄存器。	0x00000000

DMAC_D_ADDR_2

通道 2 目的地址寄存器。

Offset Address: 0x0144 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	dmac_d_addr_2	DMAC 通道 2 目的地址寄存器。	0x00000000

DMAC_D_ADDR_3

通道 3 目的地址寄存器。

Offset Address: 0x0164 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	dmac_d_addr_3	DMAC 通道 3 目的地址寄存器。	0x00000000

DMAC_CHN_CONTROL_0

通道 0 控制寄存器。

Offset Address: 0x0114 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31	RW	dmac_trans_int_0	传输完成中断使能位，该位用于决定当前链表结点是否触发传输完成中断。 0：当前链表结点不触发传输完成中断； 1：当前链表结点触发传输完成中	0x0

Bits	Access	Name	Description	Reset
			断。	
30:28	RW	dmac_prot_0	Master 发出的访问保护 HPROT[2:0]信号。	0x0
27	RW	dmac_d_inc_0	目的地址递增。 0: 目的地址不递增; 1: 目的地址每传一个数就递增一次。 目的设备为外设时目的地址不递增; 目的设备为存储器时目的地址递增。	0x0
26	RW	dmac_s_inc_0	源地址递增。 0: 源地址不递增; 1: 源地址每传一个数就递增一次。 源设备为外设时源地址不递增; 源设备为存储器时源地址递增。	0x0
25	RW	dmac_d_master_0	设置访问目的设备的 Master。 0: 使用 Master1 作为目的设备传输; 1: 使用 Master2 作为目的设备传输。 注意: 本芯片只支持 Master1。	0x0
24	RW	dmac_s_master_0	设置访问源设备的 Master。 0: 使用 Master1 作为源设备传输; 1: 使用 Master2 作为源设备传输。 注意: 本芯片只支持 Master1。	0x0
23:21	RW	dmac_d_width_0	目的设备传输位宽。	0x0

Bits	Access	Name	Description	Reset
			000: Byte (8bit) ; 001: Halfword (16bit) ; 010: Word (32bit) ; 其他: reserved。	
20:18	RW	dmac_s_width_0	源设备传输位宽。 000: Byte (8bit) ; 001: Halfword (16bit) ; 010: Word (32bit) ; 其他: reserved。	0x0
17:15	RW	dmac_d_bsize_0	目的设备 burst 长度。 000: 1; 001: 4; 010: 8; 011: 16; 100: 32; 101: 64; 110: 128; 111: 256。	0x0
14:12	RW	dmac_s_bsize_0	源设备 burst 长度。 000: 1; 001: 4; 010: 8; 011: 16; 100: 32; 101: 64; 110: 128; 111: 256。	0x0
11:0	RW	dmac_trans_si	当 DMAC 是流控制器时, 通过写	0x000

Bits	Access	Name	Description	Reset
		ze_0	该寄存器可设定 DMA 传输的长度。	

DMAC_CHN_CONTROL_1

通道 1 控制寄存器。

Offset Address: 0x0134 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31	RW	dmac_trans_int_1	传输完成中断使能位。该位用于决定当前链表结点是否触发传输完成中断。 0: 当前链表结点不触发传输完成中断; 1: 当前链表结点触发传输完成中断。	0x0
30:28	RW	dmac_prot_1	Master 发出的访问保护 HPROT[2:0]信号。	0x0
27	RW	dmac_d_inc_1	目的地址递增。 0: 目的地址不递增; 1: 目的地址每传一个数就递增一次。 目的设备为外设时目的地址不递增, 目的设备为存储器时目的地址递增。	0x0
26	RW	dmac_s_inc_1	源地址递增。 0: 源地址不递增; 1: 源地址每传一个数就递增一次。 源设备为外设时源地址不递增, 源	0x0

Bits	Access	Name	Description	Reset
			设备为存储器时源地址递增。	
25	RW	dmac_d_master_1	设置访问目的设备的 Master。 0：使用 Master1 作为目的设备传输； 1：使用 Master2 作为目的设备传输。 注意：本芯片只支持 Master1。	0x0
24	RW	dmac_s_master_1	设置访问源设备的 Master。 0：使用 Master1 作为源设备传输； 1：使用 Master2 作为源设备传输。 注意：本芯片只支持 Master1。	0x0
23:21	RW	dmac_d_width_1	目的设备传输位宽。 000: Byte (8bit) ; 001: Halfword (16bit) ; 010: Word (32bit) ; 其他: reserved。	0x0
20:18	RW	dmac_s_width_1	源设备传输位宽。 000: Byte (8bit) ; 001: Halfword (16bit) ; 010: Word (32bit) ; 其他: reserved。	0x0
17:15	RW	dmac_d_bsize_1	目的设备 burst 长度。 000: 1; 001: 4; 010: 8; 011: 16;	0x0

Bits	Access	Name	Description	Reset
			100: 32; 101: 64; 110: 128; 111: 256。	
14:12	RW	dmac_s_bsize_1	源设备 burst 长度。 000: 1; 001: 4; 010: 8; 011: 16; 100: 32; 101: 64; 110: 128; 111: 256。	0x0
11:0	RW	dmac_trans_size_1	当 DMAC 是流控制器时, 通过写该寄存器可设定 DMA 传输的长度。	0x000

DMAC_CHN_CONTROL_2

通道 2 控制寄存器。

Offset Address: 0x0154 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31	RW	dmac_trans_int_2	传输完成中断使能位。该位用于决定当前链表结点是否触发传输完成中断。 0: 当前链表结点不触发传输完成中断; 1: 当前链表结点触发传输完成中断。	0x0

Bits	Access	Name	Description	Reset
30:28	RW	dmac_prot_2	Master 发出的访问保护 HPROT[2:0]信号。	0x0
27	RW	dmac_d_inc_2	目的地址递增。 0: 目的地址不递增; 1: 目的地址每传一个数就递增一次。 目的设备为外设时目的地址不递增, 目的设备为存储器时目的地址递增。	0x0
26	RW	dmac_s_inc_2	源地址递增。 0: 源地址不递增; 1: 源地址每传一个数就递增一次。 源设备为外设时源地址不递增; 源设备为存储器时源地址递增。	0x0
25	RW	dmac_d_master_2	设置访问目的设备的 master。 0: 使用 Master1 作为目的设备传输; 1: 使用 Master2 作为目的设备传输。 注意: 本芯片只支持 Master1。	0x0
24	RW	dmac_s_master_2	设置访问源设备的 master。 0: 使用 Master1 作为源设备传输; 1: 使用 Master2 作为源设备传输。 注意: 本芯片只支持 Master1。	0x0
23:21	RW	dmac_d_width_2	目的设备传输位宽。 000: Byte (8bit) ;	0x0

Bits	Access	Name	Description	Reset
			001: Halfword (16bit) ; 010: Word (32bit) ; 其他: reserved。	
20:18	RW	dmac_s_width_2	源设备传输位宽。 000: Byte (8bit) ; 001: Halfword (16bit) ; 010: Word (32bit) ; 其他: reserved。	0x0
17:15	RW	dmac_d_bsize_2	目的设备 burst 长度。 000: 1; 001: 4; 010: 8; 011: 16; 100: 32; 101: 64; 110: 128; 111: 256。	0x0
14:12	RW	dmac_s_bsize_2	源设备 burst 长度。 000: 1; 001: 4; 010: 8; 011: 16; 100: 32; 101: 64; 110: 128; 111: 256。	0x0
11:0	RW	dmac_trans_size_2	当 DMAC 是流控制器时, 通过写该寄存器可设定 DMA 传输的长	0x000

Bits	Access	Name	Description	Reset
			度。	

DMAC_CHN_CONTROL_3

通道 3 控制寄存器。

Offset Address: 0x0174 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31	RW	dmac_trans_int_3	传输完成中断使能位。该位用于决定当前链表结点是否触发传输完成中断。 0: 当前链表结点不触发传输完成中断; 1: 当前链表结点触发传输完成中断。	0x0
30:28	RW	dmac_prot_3	Master 发出的访问保护 HPROT[2:0]信号。	0x0
27	RW	dmac_d_inc_3	目的地址递增。 0: 目的地址不递增; 1: 目的地址每传一个数就递增一次。 目的设备为外设时目的地址不递增, 目的设备为存储器时目的地址递增。	0x0
26	RW	dmac_s_inc_3	源地址递增。 0: 源地址不递增; 1: 源地址每传一个数就递增一次。 源设备为外设时源地址不递增, 源设备为存储器时源地址递增。	0x0

Bits	Access	Name	Description	Reset
25	RW	dmac_d_master_3	设置访问目的设备的 Master。 0：使用 Master1 作为目的设备传输； 1：使用 Master2 作为目的设备传输。 注意：本芯片只支持 Master1。	0x0
24	RW	dmac_s_master_3	设置访问源设备的 Master。 0：使用 Master1 作为源设备传输； 1：使用 Master2 作为源设备传输。 注意：本芯片只支持 Master1。	0x0
23:21	RW	dmac_d_width_3	目的设备传输位宽。 000: Byte (8bit) ; 001: Halfword (16bit) ; 010: Word (32bit) ; 其他: reserved。	0x0
20:18	RW	dmac_s_width_3	源设备传输位宽。 000: Byte (8bit) ; 001: Halfword (16bit) ; 010: Word (32bit) ; 其他: reserved。	0x0
17:15	RW	dmac_d_bsize_3	目的设备 burst 长度。 000: 1; 001: 4; 010: 8; 011: 16; 100: 32;	0x0

Bits	Access	Name	Description	Reset
			101: 64; 110: 128; 111: 256。	
14:12	RW	dmac_s_bsize_3	源设备 burst 长度。 000: 1; 001: 4; 010: 8; 011: 16; 100: 32; 101: 64; 110: 128; 111: 256。	0x0
11:0	RW	dmac_trans_size_3	当 DMAC 是流控制器时，通过写该寄存器可设定 DMA 传输的长度。	0x000

DMAC_CHN_CONFIG_0

通道 0 配置寄存器。

Offset Address: 0x0108 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:17	-	reserved	保留。	0x0000
16	RW	dmac_halt_0	Halt 位。 0: 允许 DMA 请求; 1: 忽略后来的 DMA 请求, 通道 FIFO 中的内容均被传完。	0x0
15	RO	dmac_active_0	Active 位。 0: 通道 FIFO 中没有数据;	0x0

Bits	Access	Name	Description	Reset
			1: 通道 FIFO 中有数据。	
14	RW	dmac_lock_0	中断 2 屏蔽位, 包含传输完成中断和错误中断。 0: 屏蔽; 1: 不屏蔽。	0x0
13	RW	dmac_int_tc_0	中断 1 传输完成中断屏蔽位。 0: 屏蔽; 1: 不屏蔽。	0x0
12	RW	dmac_int_en_0	中断 1 错误中断屏蔽位。 0: 屏蔽; 1: 不屏蔽。	0x0
11:9	RW	dmac_flow_ctl_0	流控及传输类型字段。 000: 存储器至存储器 DMAC; 001: 存储器至外设 DMAC; 010: 外设至存储器 DMAC; 011: 源设备至目的设备 DMAC; 100: 源设备至目的设备; 101: 存储器至外设目的设备; 110: 外设至存储器源设备; 111: 源设备至目的设备。	0x0
8:5	RW	dmac_d_peripheral_0	目的设备。该字段用于选择一个外设请求信号作为本通道的 DMA 目的设备的请求信号。 如果 DMA 传输的目的设备是存储器则该字段被忽略。	0x0
4:1	RW	dmac_s_peripheral_0	源设备。该字段用于选择一个外设请求信号作为本通道的 DMA 源设备的请求信号。	0x0

Bits	Access	Name	Description	Reset
			如果 DMA 传输的源设备是存储器则该字段被忽略。	
0	RW	dmac_chn_en_0	通道使能位。 0: 关闭通道; 1: 启动通道。	0x0

DMAC_CHN_CONFIG_1

通道 1 配置寄存器。

Offset Address: 0x0128 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:17	-	reserved	保留。	0x0000
16	RW	dmac_halt_1	Halt 位。 0: 允许 DMA 请求; 1: 忽略后来的 DMA 请求, 通道 FIFO 中的内容均被传完。	0x0
15	RO	dmac_active_1	Active 位。 0: 通道 FIFO 中没有数据; 1: 通道 FIFO 中有数据。	0x0
14	RW	dmac_lock_1	中断 2 屏蔽位, 包含传输完成中断和错误中断。 0: 屏蔽; 1: 不屏蔽。	0x0
13	RW	dmac_int_tc_1	中断 1 传输完成中断屏蔽位。 0: 屏蔽; 1: 不屏蔽。	0x0
12	RW	dmac_int_en_1	中断 1 错误中断屏蔽位。	0x0

Bits	Access	Name	Description	Reset
			0: 屏蔽; 1: 不屏蔽。	
11:9	RW	dmac_flow_ctl_1	流控及传输类型字段。 000: 存储器至存储器 DMAC; 001: 存储器至外设 DMAC; 010: 外设至存储器 DMAC; 011: 源设备至目的设备 DMAC; 100: 源设备至目的设备; 101: 存储器至外设目的设备; 110: 外设至存储器源设备; 111: 源设备至目的设备。	0x0
8:5	RW	dmac_d_peripheral_1	目的设备。该字段用于选择一个外设请求信号作为本通道的 DMA 目的设备的请求信号。 如果 DMA 传输的目的设备是存储器则该字段被忽略。	0x0
4:1	RW	dmac_s_peripheral_1	源设备。该字段用于选择一个外设请求信号作为本通道的 DMA 源设备的请求信号。 如果 DMA 传输的源设备是存储器则该字段被忽略。	0x0
0	RW	dmac_chn_en_1	通道使能位。 0: 关闭通道; 1: 启动通道。	0x0

DMAC_CHN_CONFIG_2

通道 2 配置寄存器。

Offset Address: 0x0148 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:17	-	reserved	保留。	0x0000
16	RW	dmac_halt_2	Halt 位。 0: 允许 DMA 请求; 1: 忽略后来的 DMA 请求, 通道 FIFO 中的内容均被传完。	0x0
15	RO	dmac_active_2	Active 位。 0: 通道 FIFO 中没有数据; 1: 通道 FIFO 中有数据。	0x0
14	RW	dmac_lock_2	中断 2 屏蔽位, 包含传输完成中断和错误中断。 0: 屏蔽; 1: 不屏蔽。	0x0
13	RW	dmac_int_tc_2	中断 1 传输完成中断屏蔽位。 0: 屏蔽; 1: 不屏蔽。	0x0
12	RW	dmac_int_en_2	中断 1 错误中断屏蔽位。 0: 屏蔽; 1: 不屏蔽。	0x0
11:9	RW	dmac_flow_ctl_2	流控及传输类型字段。 000: 存储器至存储器 DMAC; 001: 存储器至外设 DMAC; 010: 外设至存储器 DMAC; 011: 源设备至目的设备 DMAC; 100: 源设备至目的设备; 101: 存储器至外设目的设备; 110: 外设至存储器源设备; 111: 源设备至目的设备。	0x0

Bits	Access	Name	Description	Reset
8:5	RW	dmac_d_peripheral_2	目的设备。该字段用于选择一个外设请求信号作为本通道的 DMA 目的设备的请求信号。 如果 DMA 传输的目的设备是存储器则该字段被忽略。	0x0
4:1	RW	dmac_s_peripheral_2	源设备。该字段用于选择一个外设请求信号作为本通道的 DMA 源设备的请求信号。 如果 DMA 传输的源设备是存储器则该字段被忽略。	0x0
0	RW	dmac_chn_en_2	通道使能位。 0: 关闭通道; 1: 启动通道。	0x0

DMAC_CHN_CONFIG_3

通道 3 配置寄存器。

Offset Address: 0x0168 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:17	-	reserved	保留。	0x0000
16	RW	dmac_halt_3	Halt 位。 0: 允许 DMA 请求; 1: 忽略后来的 DMA 请求, 通道 FIFO 中的内容均被传完。	0x0
15	RO	dmac_active_3	Active 位。 0: 通道 FIFO 中没有数据; 1: 通道 FIFO 中有数据。	0x0
14	RW	dmac_lock_3	中断 2 屏蔽位, 包含传输完成中断	0x0

Bits	Access	Name	Description	Reset
			和错误中断。 0: 屏蔽; 1: 不屏蔽。	
13	RW	dmac_int_tc_3	中断 1 传输完成中断屏蔽位。 0: 屏蔽; 1: 不屏蔽。	0x0
12	RW	dmac_int_en_3	中断 1 错误中断屏蔽位。 0: 屏蔽; 1: 不屏蔽。	0x0
11:9	RW	dmac_flow_ctl_3	流控及传输类型字段。 000: 存储器至存储器 DMAC; 001: 存储器至外设 DMAC; 010: 外设至存储器 DMAC; 011: 源设备至目的设备 DMAC; 100: 源设备至目的设备; 101: 存储器至外设目的设备; 110: 外设至存储器源设备; 111: 源设备至目的设备。	0x0
8:5	RW	dmac_d_peripheral_3	目的设备。该字段用于选择一个外设请求信号作为本通道的 DMA 目的设备的请求信号。 如果 DMA 传输的目的设备是存储器则该字段被忽略。	0x0
4:1	RW	dmac_s_peripheral_3	源设备。该字段用于选择一个外设请求信号作为本通道的 DMA 源设备的请求信号。 如果 DMA 传输的源设备是存储器则该字段被忽略。	0x0

Bits	Access	Name	Description	Reset
0	RW	dmac_chn_en_3	通道使能位。 0：关闭通道； 1：启动通道。	0x0

2.7 安全加解密

2.7.1 概述

应用子系统与协议子系统共用一个安全加解密模块 Cipher，Cipher 具备基本的数据搬移功能（仅支持内存之间的数据搬移）和安全加解密与完整性保护校验功能。

2.7.2 功能描述

Cipher 具有以下功能特点：

- 支持 AHB LITE MASTER 总线访问接口，支持对 MEM 进行读写访问。
- 支持 APB 3.0 SLAVE 配置接口。
- 支持 3GPP LTE AES/ZUC/SNOW3G 的加解密算法和完整性保护校验算法。
- 算法 Key Size 仅支持 128bit。
- 算法码流长度支持 2Byte ~ 8192Byte。

2.8 RTC

2.8.1 概述

RTC 模块作为低功耗计时模块，用于系统计时；同时在系统进入深睡后，可作为深睡唤醒源唤醒整个系统。RTC 模块内部计数器为 48bit，以 RTC 时钟递增计数。应用子系统、协议子系统均可使用 2 个 RTC 中断，各中断独立互不影响，每个子系统独立配置各自对应的中断寄存器。

2.8.2 功能描述

RTC 具有以下功能特点：

- 支持一个 48bit 计数器，在 32K 时钟下每个周期加一。
- 支持一个带校准功能的 48bit 计数器，补偿系数可配。
- 应用子系统、协议子系统可分别配置 RTC 产生中断的触发条件。
- ACPU、CCPU 各支持 2 个阈值中断。
- 每个中断单独配置使能、屏蔽，支持中断清除。

2.8.3 工作方式

- 步骤 1 读 RTC_TIMER_COUNTER_LSBS_ADDR、
RTC_TIMER_COUNTER_MSBS_ADDR[rtc_cnt_h]，获取当前 RTC 计数时间。
- 步骤 2 写 RTC_TIME_MSB，根据 RTC 计数到达时间配置 RTC 计数值的高 16bit。
- 步骤 3 写 RTC_TIME_LSB，根据 RTC 计数到达时间配置 RTC 计数值的低 32bit。
- 步骤 4 等待 RTC 上报中断后，可配置 RTC_IRQ_CLR[RTC_IRQ_CLR]为 1，清除该中断。

----结束

2.8.4 寄存器概览

应用核 RTC0 基地址是 0x30060000，仅此 RTC 支持校准配置，其余 RTC 不支持校准配置，所有 RTC 支持读取校准后的 RTC 计数值。

应用核 RTC1 基地址是 0x30060100 (以应用核 RTC1 为例进行说明)；

协议核 RTC0 基地址是 0x30060200；

协议核 RTC1 基地址是 0x30060300。

RTC 寄存器概览如表 2-6 所示。

表2-6 RTC 寄存器概览

偏移地址	名称	描述
0x04	RTC_TIME_LSB	RTC 中断阈值低位寄存器。
0x08	RTC_TIME_MSB	RTC 中断阈值高位寄存器。

偏移地址	名称	描述
0x0C	RTC_IRQ_CLR	RTC 中断清除寄存器。
0x10	RTC_IRQ_MASK	RTC 中断屏蔽寄存器。
0x14	RTC_IRQ_INT	RTC 中断寄存器。
0x18	RTC_IRQ_INT_MASK	带屏蔽的 RTC 中断寄存器。
0x1C	RTC_TIMER_COUNTER_LSB	RTC 实时计数值寄存器，实时计数值的低 32bit。
0x20	RTC_TIMER_COUNTER_MSB	RTC 实时计数值寄存器，实时计数值的高 16bit。
0x24	RTC_TIMER_COUNTER_CALI_LSB	RTC 校准计数值寄存器，校准计数值的低 32bit。
0x2C	RTC_TIMER_COUNTER_CALI_MSB	RTC 校准计数值寄存器，校准计数值的高 16bit。
0x30	RTC_TIMER_CALI	RTC 校准值寄存器。
0x34	RTC_ID	RTC 版本寄存器。

2.8.5 寄存器描述

RTC_TIME_LSB

RTC 中断阈值低位寄存器。

Offset Address: 0x04 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW_RE G_AND_ VLD	RTC_IRQ_TIME_LSB	RTC 中断阈值低位。	0x00000000

RTC_TIME_MSB

RTC 中断阈值高位寄存器。

Offset Address: 0x08 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:16	RO	reserved	保留。	0x0000
15:0	RW_RE G_AND _VLD	RTC_IRQ _TIME_M _SBS	RTC 中断阈值高位。	0x0000

RTC_IRQ_CLR

RTC 中断清除寄存器。

Offset Address: 0x0C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	W1_PU LSE	RTC_IRQ _CLR	RTC 中断清除寄存器。 0: 不清除; 1: 清除。	0x0

RTC_IRQ_MASK

RTC 中断屏蔽寄存器。

Offset Address: 0x10 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RW	RTC_IRQ _MASK	RTC 中断屏蔽寄存器。 0: 不屏蔽。 1: 屏蔽;	0x0

RTC_IRQ_INT

RTC 中断寄存器。

Offset Address: 0x14 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	RTC_IRQ_INT	RTC 中断状态寄存器。 0: RTC 不产生中断。 1: RTC 产生中断;	0x0

RTC_IRQ_INT_MASK

带屏蔽的 RTC 中断寄存器。

Offset Address: 0x18 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	RTC_IRQ_INT_MASK	中断屏蔽的 RTC 中断状态寄存器。 0: RTC_IRQ_INT 为 1, RTC_IRQ_INT_MASK 为 0, 表示中断已产生且中断被屏蔽; RTC_IRQ_INT 为 0, RTC_IRQ_INT_MASK 为 0, 表示中断未产生且中断被屏蔽。 1: 异常。	0x0

RTC_TIMER_COUNTER_LSB

RTC 实时计数值寄存器, 实时计数值的低 32bit。

Offset Address: 0x1C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RO	RTC_TIMER_COUNTER_LSB	RTC 实时计数值[31:0]。 内部 RTC cnt 不受全局复位影响, 复位不清 0。	0x00000000

RTC_TIMER_COUNTER_MSB

RTC 实时计数值寄存器，实时计数值的高 16bit。

Offset Address: 0x20 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:16	RO	reserved	保留。	0x0000
15:0	RO	RTC_TIMER_COUNTER_MSB	RTC 实时计数值[47:32]。 内部 RTC cnt 不受全局复位影响，复位不清 0。	0x0000

RTC_TIMER_COUNTER_CALI_LSB

RTC 校准计数值寄存器，校准计数值的低 32bit。

Offset Address: 0x24 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RO	RTC_TIMER_COUNTER_CALI_LSB	带校准功能时，为校准后的 RTC 计数值[31:0]；不支持校准功能时，该寄存器无效。 内部 RTC cnt 不受全局复位影响，复位不清 0。	0x00000000

RTC_TIMER_COUNTER_CALI_MSB

RTC 校准计数值寄存器，校准计数值的高 16bit。

Offset Address: 0x2C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:16	RO	reserved	保留。 不支持校准功能时，配置该寄存器无效。	0x0000
15:0	RO	RTC_TIMER_C	带校准功能时，为校准后的	0x0000

Bits	Access	Name	Description	Reset
		OUNTE R_CALI _MSB	RTC 计数器的[47:32]; 不支持校准功能时, 该寄存器无效。 内部 RTC cnt 不受全局复位影响, 复位不清 0。	

RTC_TIMER_CALI

RTC 校准值寄存器。

Offset Address: 0x30 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW_RE G_AND _VLD	RTC_TI MER_C ALI	RTC 校准配置。 bit[14:0]: rtc_calib_frac, RTC 时钟与 32768 相除的余数值; bit[15]: rtc_calib_int, RTC 时钟与 32768 相除的整数值; bit[31:16]: rtc_freq, RTC 频率配置寄存器。 不支持校准功能时, 配置该寄存器无效。	0x00000000

RTC_ID

RTC 版本寄存器。

Offset Address: 0x34 Total Reset Value: 0x20240111

Bits	Access	Name	Description	Reset
31:0	RO	RTC_ID	RTC 版本寄存器。	0x20240111

2.9 TIMER

2.9.1 概述

Timer 模块主要实现定时、计数功能，可以供操作系统用作系统时钟，也可以供应用程序用于定时和计数。

说明

- 应用核的 Timer 模块包含 2 个功能完全相同的 Timer 模块。
- 协议核的 Timer 模块包含 2 个功能完全相同的 Timer 模块。

Timer 为基于一个 32bit 减法的计数器。计数器的值在每个计数时钟的上升沿减 1。当计数值递减到 0 且中断使能，Timer 将产生一个中断。

Timer 有以下 3 种计数模式：

- 自由运行模式

定时器持续计数，当计数值减到 0 时又自动回转到其最大值，并继续计数。在自由模式下，也可以载入计数值，并立即从载入值递减计数，但计数到 0 时回转到其最大值。

- 周期模式

定时器持续计数，当计数值减到 0 时从 `TIMERN_BGLOAD` 中再次载入初值并继续计数。

- 单次计数模式

向定时器中载入计数初值。当定时器的计数值减到 0 时就停止计数，直到重新被载入新值且定时器处于使能状态，才再次开始计数。

每个 Timer 具有一个预分频计数器 (prescaler)，可将其工作时钟在 Timer 内部再次进行 1 分频、16 分频或 256 分频，进一步提高计数时钟频率的选择灵活性。

对定时器载入计数初值的方法如下：

- 通过写 `TIMERN_LOAD` 可对定时器载入计数初值。当定时器处于工作状态时，如果向该寄存器写入值，会导致定时器立刻从新值开始重新计数。适用于所有计数模式。
- 通过写 `TIMERN_BGLOAD` 可以设定周期计数模式的计数周期。写该寄存器不会立刻影响定时器的当前计数，定时器会继续计数直到计数值减到 0，然后载入该寄存器中的新值开始计数。适用于所有计数模式。

2.9.2 功能描述

Timer 模块具有以下功能特点：

- 支持可编程 8bit 预分频器的 32bit 减法定时器/计数器。
- 计数时钟与配置时钟可以异步。
- 支持 3 种计数模式：自由运行模式、周期模式、单次计数模式。
- 有 2 种载入计数初值的方法，分别通过 `TIMERN_LOAD` 和 `TIMERN_BGLOAD` 寄存器实现。
- 当前的计数值可随时读取。
- 中断使能时，当前计数值递减到 0 会产生中断。

2.9.3 工作方式

2.9.3.1 初始化配置

系统初始化时应对 Timer 进行初始化。初始化 `TIMERx` 中的“x”取值为（应用核：0~1）时应按以下步骤进行配置：

步骤 1 配置 `CRG` 相关寄存器，选择 `TIMERx` 的计数时钟频率，打开 `TIMERx` 时钟门控。

步骤 2 配置 `TIMER_x_LOAD_COUNT0` 寄存器，设置 `TIMERx` 的计数初始值。

步骤 3 配置 `TIMER_x_CONTROL[timer_x_mode]`，选择 `TIMERx` 的计数模式。

步骤 4 配置 `TIMER_x_CONTROL[timer_x_imsk]`，根据需要决定是否屏蔽中断。

步骤 5 配置 `TIMER_x_CONTROL[timer_x_en]=1`，启动 `TIMERx` 计数。

步骤 6 查询 `TIMER_x_CURRENT_VALUE0`，获取 `TIMERx` 的计时值。如未屏蔽中断，则计时值为 0 时上报中断。

----结束

2.9.3.2 中断处理

中断查询及中断清除操作如下：

- 查询 `TIMER_x_RAW_INTR[timer_x_raw_intr]`，获取 `TIMERx` 的原始中断信息。
- 查询 `TIMER_x_INTR[timer_x_intr]`，获取 `TIMERx` 的屏蔽后中断信息。
- 配置 `TEIMER_x_EOI_REN[timer_0_eoi]=1`，清除中断。

2.9.3.3 Timer 复位

当需要关闭 TIMERx (TIMERx 正在运行, 即 `TIMER_x_CONTROL[timer_x_en] = 1`), 重新开启下一次的计数器时:

步骤 1 配置 `TIMER_x_CONTROL[timer_x_rstfsm]`, 复位 TIMERx 状态机, 该动作只在 (`TIMER_x_CONTROL[timer_x_en] = 1`) 时有效。

步骤 2 轮询 `TIMER_x_CONTROL[timer_x_en]`, 直到回读值为 0, 证明 TIMERx 已被复位。

步骤 3 重新进行 TIMERx 的初始化配置流程。

----结束

2.9.4 寄存器概览

TIMER 寄存器概览如表 2-7 所示。

表2-7 TIMER 寄存器概览 (应用核 TIMER0 基地址是 0x3000_1000, TIMER1 基地址是 3000_1100, 协议核 TIMER0 的基地址是 0x4040_0000, TIMER1 基地址为 0x4040_0100)

偏移地址	名称	描述
0x20	<code>TIMER_VERSION</code>	Timer 版本号寄存器。
0x100+0x100×0	<code>TIMER_0_LOAD_COUNT0</code>	Timer0 载入值的低 32bit 寄存器。
0x104+0x100×0	<code>TIMER_0_LOAD_COUNT1</code>	Timer0 载入值的高 32bit 寄存器。
0x108+0x100×0	<code>TIMER_0_CURRENT_VALUE0</code>	Timer0 当前值的低 32bit 寄存器。
0x10C+0x100×0	<code>TIMER_0_CURRENT_VALUE1</code>	Timer0 当前值的高 32bit 寄存器。
0x110 + 0x100×0	<code>TIMER_0_CONTROL</code>	Timer0 控制寄存器。
0x114+0x100×0	<code>TEIMER_0_EOI_REGISTER</code>	Timer0 中断清除寄存器。
0x118+0x100×0	<code>TIMER_0_RAW_INTERRUPT</code>	Timer0 原始中断寄存器。
0x11C+0x100×0	<code>TIMER_0_INTR</code>	Timer0 屏蔽后的中断寄存器。

2.9.5 寄存器描述

TIMER_VERSION

Timer 版本号寄存器。

Offset Address: 0x20 Total Reset Value: 0x0000_0002

Bits	Access	Name	Description	Reset
31:0	RW	timer_version	Timer 版本号。	0x00000002

TIMER_0_LOAD_COUNT0

Timer0 载入值的低 32bit 寄存器。

Offset Address: 0x100+0x100×0 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	timer_0_load_count0	Timer0 载入值的低 32bit，如果 Timer0 只有 32bit 时，需要软件保证该值大于 8。该载入值可以直接在如下情况下加载到 Timer0：在 one-shot 模式和 periodic 模式下，该载入值为计数器进行递减操作的起始值。 one-shot 模式：只有在使能 Timer0 时，才能载入该值到 Timer0； periodic 模式：该值随时可以载入 Timer0；如果 Timer0 为 64bit，则该载入值作为载入值的低 32bit，和高 32bit 绑定一起作为载入值，加载情况和 32bit 载入值一样。	0x00000000

TIMER_1_LOAD_COUNT0

Timer1 载入值的低 32bit 寄存器。

Offset Address: 0x100+0x100×1 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	timer_1_load_count0	Timer1 载入值的低 32bit, 如果 Timer1 只有 32bit 时, 需要软件保证该值大于 8。该载入值可以直接在如下情况下加载到 Timer1: 在 one-shot 模式和 periodic 模式下, 该载入值为计数器进行递减操作的起始值。 one-shot 模式: 只有在使能 Timer1 时, 才能载入该值到 Timer1; periodic 模式: 该值随时可以载入 Timer1; 如果 Timer1 为 64bit, 则该载入值作为载入值的低 32bit, 和高 32bit 绑定一起作为载入值, 加载情况和 32bit 载入值一样。	0x00000000

TIMER_2_LOAD_COUNT0

Timer2 载入值的低 32bit 寄存器。

Offset Address: 0x100+0x100×2 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	timer_2_load_count0	Timer2 载入值的低 32bit, 如果 Timer2 只有 32bit 时, 需要软件保证该值大于 8。该载入值可以直接在如下情况下加载到 Timer2: 在 one-shot 模式和 periodic 模式下, 该载入	0x00000000

Bits	Access	Name	Description	Reset
			值为计数器进行递减操作的起始值。 one-shot 模式：只有在使能 Timer2 时，才能载入该值到 Timer2； periodic 模式：该值随时可以载入 Timer2；如果 Timer2 为 64bit，则该载入值作为载入值的低 32bit，和高 32bit 绑定一起作为载入值，加载情况和 32bit 载入值一样。	

TIMER_3_LOAD_COUNT0

Timer3 载入值的低 32bit 寄存器。

Offset Address: 0x100+0x100×3 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	timer_3_load_count0	Timer3 载入值的低 32bit，如果 Timer3 只有 32bit 时，需要软件保证该值大于 8。该载入值可以直接在如下情况下加载到 Timer3：在 one-shot 模式和 periodic 模式下，该载入值为计数器进行递减操作的起始值。 one-shot 模式：只有在使能 Timer3 时，才能载入该值到 Timer3； periodic 模式：该值随时可以载入 Timer3；如果 Timer3 为 64bit，则该载入值作为载入值的低 32bit，和高 32bit 绑定一起作为载入值，加载情况和	0x00000000

Bits	Access	Name	Description	Reset
			32bit 载入值一样。	

TIMER_4_LOAD_COUNT0

Timer4 载入值的低 32bit 寄存器。

Offset Address: 0x100+0x100×4 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	timer_4_load_count0	Timer4 载入值的低 32bit，如果 Timer4 只有 32bit 时，需要软件保证该值大于 8。该载入值可以直接在如下情况下加载到 Timer4：在 one-shot 模式和 periodic 模式下，该载入值为计数器进行递减操作的起始值。 one-shot 模式：只有在使能 Timer4 时，才能载入该值到 Timer4； periodic 模式：该值随时可以载入 Timer4；如果 Timer4 为 64bit，则该载入值作为载入值的低 32bit，和高 32bit 绑定一起作为载入值，加载情况和 32bit 载入值一样。	0x00000000

TIMER_5_LOAD_COUNT0

Timer5 载入值的低 32bit 寄存器。

Offset Address: 0x100+0x100×5 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	timer_5_load_c	Timer5 载入值的低 32bit，如果 Timer5 只有 32bit 时，需	0x00000000

Bits	Access	Name	Description	Reset
		ount0	要软件保证该值大于 8。该载入值可以直接在如下情况下加载到 Timer5：在 one-shot 模式和 periodic 模式下，该载入值为计数器进行递减操作的起始值。 one-shot 模式：只有在使能 Timer5 时，才能载入该值到 Timer5； periodic 模式：该值随时可以载入 Timer5；如果 Timer5 为 64bit，则该载入值作为载入值的低 32bit，和高 32bit 绑定一起作为载入值，加载情况和 32bit 载入值一样。	

TIMER_6_LOAD_COUNT0

Timer6 载入值的低 32bit 寄存器。

Offset Address: 0x100+0x100×6 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	timer_6_load_count0	Timer6 载入值的低 32bit，如果 Timer6 只有 32bit 时，需要软件保证该值大于 8。该载入值可以直接在如下情况下加载到 Timer6：在 one-shot 模式和 periodic 模式下，该载入值为计数器进行递减操作的起始值。 one-shot 模式：只有在使能 Timer6 时，才能载入该值到 Timer6； periodic 模式：该值随时可以	0x00000000

Bits	Access	Name	Description	Reset
			载入 Timer6；如果 Timer6 为 64bit，则该载入值作为载入值的低 32bit，和高 32bit 绑定一起作为载入值，加载情况和 32bit 载入值一样。	

TIMER_7_LOAD_COUNT0

Timer7 载入值的低 32bit 寄存器。

Offset Address: 0x100+0x100×7 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	timer_7_load_count0	Timer7 载入值的低 32bit，如果 Timer7 只有 32bit 时，需要软件保证该值大于 8。该载入值可以直接在如下情况下加载到 Timer7：在 one-shot 模式和 periodic 模式下，该载入值为计数器进行递减操作的起始值。 one-shot 模式：只有在使能 Timer7 时，才能载入该值到 Timer7； periodic 模式：该值随时可以载入 Timer7；如果 Timer7 为 64bit，则该载入值作为载入值的低 32bit，和高 32bit 绑定一起作为载入值，加载情况和 32bit 载入值一样。	0x00000000

TIMER_0_LOAD_COUNT1

Timer0 载入值的高 32bit 寄存器。

Offset Address: 0x104+0x100×0 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	timer_0_load_count1	Timer0 载入值的高 32bit, 如果 Timer0 只有 32bit 时, 该载入值不起作用, 如果 Timer0 为 64bit 时, 需要保证 32bit 和低 32bit 绑定的值大于 8, 并且一起在如下情况下加载到 Timer0: 在 one-shot 模式和 periodic 模式下, 该载入值为计数器进行递减操作的起始值。 one-shot 模式: 只有在使能 Timer0 时, 才能载入该值到 Timer0; periodic 模式: 该值随时可以载入 Timer0。 注意: 该寄存器受写锁 Lock 保护。	0x00000000

TIMER_1_LOAD_COUNT1

Timer1 载入值的高 32bit 寄存器。

Offset Address: 0x104+0x100×1 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	timer_1_load_count1	Timer1 载入值的高 32bit, 如果 Timer1 只有 32bit 时, 该载入值不起作用, 如果 Timer1 为 64bit 时, 需要保证 32bit 和低 32bit 绑定的值大于 8, 并且一起在如下情况下加载到 Timer1: 在 one-shot 模式和 periodic 模式下, 该载入	0x00000000

Bits	Access	Name	Description	Reset
			值为计数器进行递减操作的起始值。 one-shot 模式：只有在使能 Timer1 时，才能载入该值到 Timer1； periodic 模式：该值随时可以载入 Timer1。 注意：该寄存器受写锁 Lock 保护。	

TIMER_2_LOAD_COUNT1

Timer2 载入值的高 32bit 寄存器。

Offset Address: 0x104+0x100×2 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	timer_2_load_count1	Timer2 载入值的高 32bit，如果 Timer2 只有 32bit 时，该载入值不起作用，如果 Timer2 为 64bit 时，需要保证 32bit 和低 32bit 绑定的值大于 8，并且一起在如下情况下加载到 Timer2：在 one-shot 模式和 periodic 模式下，该载入值为计数器进行递减操作的起始值。 one-shot 模式：只有在使能 Timer2 时，才能载入该值到 Timer2； periodic 模式：该值随时可以载入 Timer2。 注意：该寄存器受写锁 Lock 保护。	0x00000000

TIMER_3_LOAD_COUNT1

Timer3 载入值的高 32bit 寄存器。

Offset Address: 0x104+0x100×3 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	timer_3_load_count1	Timer3 载入值的高 32bit, 如果 Timer3 只有 32bit 时, 该载入值不起作用, 如果 Timer3 为 64bit 时, 需要保证 32bit 和低 32bit 绑定的值大于 8, 并且一起在如下情况下加载到 Timer3: 在 one-shot 模式和 periodic 模式下, 该载入值为计数器进行递减操作的起始值。 one-shot 模式: 只有在使能 Timer3 时, 才能载入该值到 Timer3; periodic 模式: 该值随时可以载入 Timer3。 注意: 该寄存器受写锁 Lock 保护。	0x00000000

TIMER_4_LOAD_COUNT1

Timer4 载入值的高 32bit 寄存器。

Offset Address: 0x104+0x100×4 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	timer_4_load_count1	Timer4 载入值的高 32bit, 如果 Timer4 只有 32bit 时, 该载入值不起作用, 如果 Timer4 为 64bit 时, 需要保证	0x00000000

Bits	Access	Name	Description	Reset
			32bit 和低 32bit 绑定的值大于 8，并且一起在如下情况下加载到 Timer4：在 one-shot 模式和 periodic 模式下，该载入值为计数器进行递减操作的起始值。 one-shot 模式：只有在使能 Timer4 时，才能载入该值到 Timer4； periodic 模式：该值随时可以载入 Timer4。 注意：该寄存器受写锁 Lock 保护。	

TIMER_5_LOAD_COUNT1

Timer5 载入值的高 32bit 寄存器。

Offset Address: 0x104+0x100×5 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	timer_5_load_count1	Timer5 载入值的高 32bit，如果 Timer5 只有 32bit 时，该载入值不起作用，如果 Timer5 为 64bit 时，需要保证 32bit 和低 32bit 绑定的值大于 8，并且一起在如下情况下加载到 Timer5：在 one-shot 模式和 periodic 模式下，该载入值为计数器进行递减操作的起始值。 one-shot 模式：只有在使能 Timer5 时，才能载入该值到 Timer5； periodic 模式：该值随时可以	0x00000000

Bits	Access	Name	Description	Reset
			载入 Timer5。 注意：该寄存器受写锁 Lock 保护。	

TIMER_6_LOAD_COUNT1

Timer6 载入值的高 32bit 寄存器。

Offset Address: 0x104+0x100×6 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	timer_6_load_count1	Timer6 载入值的高 32bit，如果 Timer6 只有 32bit 时，该载入值不起作用，如果 Timer6 为 64bit 时，需要保证 32bit 和低 32bit 绑定的值大于 8，并且一起在如下情况下加载到 Timer6：在 one-shot 模式和 periodic 模式下，该载入值为计数器进行递减操作的起始值。 one-shot 模式：只有在使能 Timer6 时，才能载入该值到 Timer6； periodic 模式：该值随时可以载入 Timer6。 注意：该寄存器受写锁 Lock 保护。	0x00000000

TIMER_7_LOAD_COUNT1

Timer7 载入值的高 32bit 寄存器。

Offset Address: 0x104+0x100×7 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	timer_7_load_count1	Timer7 载入值的高 32bit, 如果 Timer7 只有 32bit 时, 该载入值不起作用, 如果 Timer7 为 64bit 时, 需要保证 32bit 和低 32bit 绑定的值大于 8, 并且一起在如下情况下加载到 Timer7: 在 one-shot 模式和 periodic 模式下, 该载入值为计数器进行递减操作的起始值。 one-shot 模式: 只有在使能 Timer7 时, 才能载入该值到 Timer7; periodic 模式: 该值随时可以载入 Timer7。 注意: 该寄存器受写锁 Lock 保护。	0x00000000

TIMER_0_CURRENT_VALUE0

Timer0 当前值的低 32bit 寄存器。

Offset Address: 0x108+0x100×0 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
31:0	RO	timer_0_current_value0	Timer0 当前值的低 32bit。	0xFFFFFFFF

TIMER_1_CURRENT_VALUE0

Timer1 当前值的低 32bit 寄存器。

Offset Address: 0x108+0x100×1 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
31:0	RO	timer_1_current_value0	Timer1 当前值的低 32bit。	0xFFFFFFFF

TIMER_2_CURRENT_VALUE0

Timer2 当前值的低 32bit 寄存器。

Offset Address: 0x108+0x100×2 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
31:0	RO	timer_2_current_value0	Timer2 当前值的低 32bit。	0xFFFFFFFF

TIMER_3_CURRENT_VALUE0

Timer3 当前值的低 32bit 寄存器。

Offset Address: 0x108+0x100×3 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
31:0	RO	timer_3_current_value0	Timer3 当前值的低 32bit。	0xFFFFFFFF

TIMER_4_CURRENT_VALUE0

Timer4 当前值的低 32bit 寄存器。

Offset Address: 0x108+0x100×4 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
31:0	RO	timer_4_current_value0	Timer4 当前值的低 32bit。	0xFFFFFFFF

TIMER_5_CURRENT_VALUE0

Timer5 当前值的低 32bit 寄存器。

Offset Address: 0x108+0x100×5 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
31:0	RO	timer_5_current_value0	Timer5 当前值的低 32bit。	0xFFFFFFFF

TIMER_6_CURRENT_VALUE0

Timer6 当前值的低 32bit 寄存器。

Offset Address: 0x108+0x100×6 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
31:0	RO	timer_6_current_value0	Timer6 当前值的低 32bit。	0xFFFFFFFF

TIMER_7_CURRENT_VALUE0

Timer7 当前值的低 32bit 寄存器。

Offset Address: 0x108+0x100×7 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
31:0	RO	timer_7_current_value0	Timer7 当前值的低 32bit。	0xFFFFFFFF

TIMER_0_CURRENT_VALUE1

Timer0 当前值的高 32bit 寄存器。

Offset Address: 0x10C+0x100×0 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
31:0	RO	timer_0_current_value1	Timer0 当前值的高 32bit。	0xFFFFFFFF

TIMER_1_CURRENT_VALUE1

Timer1 当前值的高 32bit 寄存器。

Offset Address: 0x10C+0x100×1 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
31:0	RO	timer_1_current_value1	Timer1 当前值的高 32bit。	0xFFFFFFFF

TIMER_2_CURRENT_VALUE1

Timer2 当前值的高 32bit 寄存器。

Offset Address: 0x10C+0x100×2 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
31:0	RO	timer_2_current_value1	Timer2 当前值的高 32bit。	0xFFFFFFFF

TIMER_3_CURRENT_VALUE1

Timer3 当前值的高 32bit 寄存器。

Offset Address: 0x10C+0x100×3 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
31:0	RO	timer_3_current_value1	Timer3 当前值的高 32bit。	0xFFFFFFFF

TIMER_4_CURRENT_VALUE1

Timer4 当前值的高 32bit 寄存器。

Offset Address: 0x10C+0x100×4 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
31:0	RO	timer_4_current_value1	Timer4 当前值的高 32bit。	0xFFFFFFFF

TIMER_5_CURRENT_VALUE1

Timer5 当前值的高 32bit 寄存器。

Offset Address: 0x10C+0x100×5 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
31:0	RO	timer_5_current_value1	Timer5 当前值的高 32bit。	0xFFFFFFFF

TIMER_6_CURRENT_VALUE1

Timer6 当前值的高 32bit 寄存器。

Offset Address: 0x10C+0x100×6 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
31:0	RO	timer_6_current_value1	Timer6 当前值的高 32bit。	0xFFFFFFFF

TIMER_7_CURRENT_VALUE1

Timer7 当前值的高 32bit 寄存器。

Offset Address: 0x10C+0x100×7 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
31:0	RO	timer_7_current_value1	Timer7 当前值的高 32bit。	0xFFFFFFFF

TIMER_0_CONTROL

TIMER_0_CONTROL 为 Timer0 控制寄存器。

Offset Address: 0x110 + 0x100×0 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:7	-	reserved	保留。	0x00000000
6	RO	timer_0_cnt_lock	Timer0 count 读锁存信号。 0: 保持当前的 count 值不变; 1: 锁存当前最新的 count 值。	0x0
5	W1_PULSE	timer_0_cnt_req	Timer0 count 读锁存信号。 0: 保持当前的 count 值不变; 1: 请求锁存当前最新的 count 值。 配置为 1 后自动降为 b0。	0x0
4	W1_PULSE	timer_0_rstfsm	Timer0 状态机的复位信号。 0: 不复位 Timer2 的状态机; 1: 复位 Timer2 的状态机。 配置为 1 后自动降为 0。	0x0
3	RW	timer_0_imsk	Timer0 的中断屏蔽信号。 0: 不屏蔽中断; 1: 屏蔽中断。	0x0
2:1	RW	timer_0_mode	Timer0 的计数模式。 00: one-shot 模式; 01: periodic 模式; 10: one-shot 模式; 11: free_running 模式。	0x0
0	RW	timer_0_en	Timer0 的使能信号, 在使能信号从无效变为有效时, 通过宏定义配置决定是否加载计数器初始值, 使	0x0

Bits	Access	Name	Description	Reset
			能信号有效期间可以生成中断，使能信号无效时 Timer0 保持原值不变，中断被清除。 0：使能无效； 1：使能有效。 Timer 使能从有效变成无效，再从无效变成有效时，需要保证期间无效的时间大于等于 Timer 工作的一个时钟周期。	

TIMER_1_CONTROL

TIMER_1_CONTROL 为 Timer1 控制寄存器。

Offset Address: 0x110 + 0x100×1 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:7	-	reserved	保留。	0x00000000
6	RO	timer_1_cnt_lock	Timer1 count 读锁存信号。 0：保持当前的 count 值不变； 1：锁存当前最新的 count 值。	0x0
5	W1_PULSE	timer_1_cnt_req	Timer1 count 读锁存信号。 0：保持当前的 count 值不变； 1：请求锁存当前最新的 count 值。 配置为 1 后自动降为 0。	0x0
4	W1_PULSE	timer_1_rstfsm	Timer1 状态机的复位信号。 0：不复位 Timer1 的状态机； 1：复位 Timer1 的状态机。 配置为 1 后自动降为 0。	0x0

Bits	Access	Name	Description	Reset
3	RW	timer_1_imsk	Timer1 的中断屏蔽信号。 0: 不屏蔽中断; 1: 屏蔽中断。	0x0
2:1	RW	timer_1_mode	Timer1 的计数模式。 00: one-shot 模式; 01: periodic 模式; 10: one-shot 模式; 11: free_running 模式。	0x0
0	RW	timer_1_en	Timer1 的使能信号, 在使能信号从无效变为有效时, 通过宏定义配置决定是否加载计数器初始值, 使能信号有效期间可以生成中断, 使能信号无效时 Timer1 保持原值不变, 中断被清除。 0: 使能无效; 1: 使能有效。 Timer 使能从有效变成无效, 再从无效变成有效时, 需要保证期间无效的时间大于等于 Timer 工作的一个时钟周期。	0x0

TIMER_2_CONTROL

TIMER_2_CONTROL 为 Timer2 控制寄存器。

Offset Address: 0x110 + 0x100×2 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:7	-	reserved	保留。	0x00000000
6	RO	timer_2_cnt_lock	Timer2 count 读锁存信号。	0x0

Bits	Access	Name	Description	Reset
			0: 保持当前的 count 值不变; 1: 锁存当前最新的 count 值。	
5	W1_PULSE	timer_2_cnt_req	Timer2 count 读锁存信号。 0: 保持当前的 count 值不变; 1: 请求锁存当前最新的 count 值。 配置为 1 后自动降为 0。	0x0
4	W1_PULSE	timer_2_rstfsm	Timer2 状态机的复位信号。 0: 不复位 Timer2 的状态机; 1: 复位 Timer2 的状态机。 配置为 1 后自动降为 0。	0x0
3	RW	timer_2_imsk	Timer2 的中断屏蔽信号。 0: 不屏蔽中断; 1: 屏蔽中断。	0x0
2:1	RW	timer_2_mode	Timer2 的计数模式。 00: one-shot 模式; 01: periodic 模式; 10: one-shot 模式; 11: free_running 模式。	0x0
0	RW	timer_2_en	Timer2 的使能信号, 在使能信号从无效变为有效时, 通过宏定义配置决定是否加载计数器初始值, 使能信号有效期间可以生成中断, 使能信号无效时 Timer2 保持原值不变, 中断被清除。 0: 使能无效; 1: 使能有效。 Timer 使能从有效变成无效, 再从无效变成有效时, 需要保证期间无	0x0

Bits	Access	Name	Description	Reset
			效的时间大于等于 Timer 工作的一个时钟周期。	

TIMER_3_CONTROL

TIMER_3_CONTROL 为 Timer3 控制寄存器。

Offset Address: 0x110 + 0x100×3 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:7	-	reserved	保留。	0x00000000
6	RO	timer_3_cnt_lock	Timer3 count 读锁存信号。 0: 保持当前的 count 值不变; 1: 锁存当前最新的 count 值。	0x0
5	W1_PULSE	timer_3_cnt_req	Timer3 count 读锁存信号。 0: 保持当前的 count 值不变; 1: 请求锁存当前最新的 count 值。 配置为 1 后自动降为 0。	0x0
4	W1_PULSE	timer_3_rstfsm	Timer3 状态机的复位信号。 0: 不复位 Timer3 的状态机; 1: 复位 Timer3 的状态机。 配置为 1 后自动降为 0。	0x0
3	RW	timer_3_imsk	Timer3 的中断屏蔽信号。 0: 不屏蔽中断; 1: 屏蔽中断。	0x0
2:1	RW	timer_3_mode	Timer3 的计数模式。 00: one-shot 模式; 01: periodic 模式;	0x0

Bits	Access	Name	Description	Reset
			10: one-shot 模式; 11: free_running 模式。	
0	RW	timer_3_en	Timer3 的使能信号, 在使能信号从无效变为有效时, 通过宏定义配置决定是否加载计数器初始值, 使能信号有效期间可以生成中断, 使能信号无效时 Timer3 保持原值不变, 中断被清除。 0: 使能无效; 1: 使能有效。 Timer 使能从有效变成无效, 再从无效变成有效时, 需要保证期间无效的时间大于等于 Timer 工作的一个时钟周期	0x0

TIMER_4_CONTROL

TIMER_4_CONTROL 为 Timer4 控制寄存器。

Offset Address: 0x110 + 0x100×4 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:7	-	reserved	保留。	0x00000000
6	RO	timer_4_cnt_lock	Timer4 count 读锁存信号。 0: 保持当前的 count 值不变; 1: 锁存当前最新的 count 值。	0x0
5	W1_PULSE	timer_4_cnt_req	Timer4 count 读锁存信号。 0: 保持当前的 count 值不变; 1: 请求锁存当前最新的 count 值。	0x0

Bits	Access	Name	Description	Reset
			配置为 1 后自动降为 0。	
4	W1_PULSE	timer_4_rstfsm	Timer4 状态机的复位信号。 0: 不复位 Timer4 的状态机; 1: 复位 Timer4 的状态机。 配置为 1 后自动降为 0。	0x0
3	RW	timer_4_imsk	Timer4 的中断屏蔽信号。 0: 不屏蔽中断; 1: 屏蔽中断。	0x0
2:1	RW	timer_4_mode	Timer4 的计数模式。 00: one-shot 模式; 01: periodic 模式; 10: one-shot 模式; 11: free_running 模式。	0x0
0	RW	timer_4_en	Timer4 的使能信号, 在使能信号从无效变为有效时, 通过宏定义配置决定是否加载计数器初始值, 使能信号有效期间可以生成中断, 使能信号无效时 Timer4 保持原值不变, 中断被清除。 0: 使能无效; 1: 使能有效。 Timer 使能从有效变成无效, 再从无效变成有效时, 需要保证期间无效的时间大于等于 Timer 工作的一个时钟周期。	0x0

TIMER_5_CONTROL

TIMER_5_CONTROL 为 Timer5 控制寄存器。

Offset Address: 0x110 + 0x100×5 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:7	-	reserved	保留。	0x00000000
6	RO	timer_5_cnt_lock	Timer5 count 读锁存信号。 0: 保持当前的 count 值不变; 1: 锁存当前最新的 count 值。	0x0
5	W1_PULSE	timer_5_cnt_req	Timer5 count 读锁存信号。 0: 保持当前的 count 值不变; 1: 请求锁存当前最新的 count 值。 配置为 1 后自动降为 0。	0x0
4	W1_PULSE	timer_5_rstfsm	Timer5 状态机的复位信号。 0: 不复位 Timer5 的状态机; 1: 复位 Timer5 的状态机。 配置为 1 后自动降为 0。	0x0
3	RW	timer_5_imsk	Timer5 的中断屏蔽信号。 0: 不屏蔽中断; 1: 屏蔽中断。	0x0
2:1	RW	timer_5_mode	Timer5 的计数模式。 00: one-shot 模式; 01: periodic 模式; 10: one-shot 模式; 11: free_running 模式。	0x0
0	RW	timer_5_en	Timer5 的使能信号, 在使能信号从无效变为有效时, 通过宏定义配置决定是否加载计数器初始值, 使能信号有效期间可以生成中断, 使能信号无效时 Timer5 保持原值不变, 中断被清除。	0x0

Bits	Access	Name	Description	Reset
			0: 使能无效; 1: 使能有效。 Timer 使能从有效变成无效, 再从无效变成有效时, 需要保证期间无效的时间大于等于 Timer 工作的一个时钟周期。	

TIMER_6_CONTROL

TIMER_6_CONTROL 为 Timer6 控制寄存器。

Offset Address: 0x110 + 0x100×6 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:7	-	reserved	保留。	0x00000000
6	RO	timer_6_cnt_lock	Timer6 count 读锁存信号。 0: 保持当前的 count 值不变; 1: 锁存当前最新的 count 值。	0x0
5	W1_PULSE	timer_6_cnt_req	Timer6 count 读锁存信号。 0: 保持当前的 count 值不变; 1: 请求锁存当前最新的 count 值。 配置为 1 后自动降为 0。	0x0
4	RW	timer_6_rstfsm	Timer6 状态机的复位信号。 0: 不复位 Timer6 的状态机; 1: 复位 Timer6 的状态机。 配置为 1 后自动降为 0。	0x0
3	RW	timer_6_imsk	Timer6 的中断屏蔽信号。 0: 不屏蔽中断;	0x0

Bits	Access	Name	Description	Reset
			1: 屏蔽中断。	
2:1	RW	timer_6_mode	Timer6 的计数模式。 00: one-shot 模式; 01: periodic 模式; 10: one-shot 模式; 11: free_running 模式。	0x0
0	RW	timer_6_en	Timer6 的使能信号, 在使能信号从无效变为有效时, 通过宏定义配置决定是否加载计数器初始值, 使能信号有效期间可以生成中断, 使能信号无效时 Timer6 保持原值不变, 中断被清除。 0: 使能无效; 1: 使能有效。 Timer 使能从有效变成无效, 再从无效变成有效时, 需要保证期间无效的时间大于等于 Timer 工作的一个时钟周期。	0x0

TIMER_7_CONTROL

TIMER_7_CONTROL 为 Timer7 控制寄存器。

Offset Address: 0x110 + 0x100×7 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:7	-	reserved	保留。	0x00000000
6	RO	timer_7_cnt_lock	Timer7 count 读锁存信号。 0: 保持当前的 count 值不变; 1: 锁存当前最新的 count 值。	0x0

Bits	Access	Name	Description	Reset
5	W1_PULSE	timer_7_cnt_req	Timer7 count 读锁存信号。 0: 保持当前的 count 值不变; 1: 请求锁存当前最新的 count 值。 配置为 1 后自动降为 0。	0x0
4	W1_PULSE	timer_7_rstfsm	Timer7 状态机的复位信号。 0: 不复位 Timer7 的状态机; 1: 复位 Timer7 的状态机。 配置为 1 后自动降为 0。	0x0
3	RW	timer_7_imsk	Timer7 的中断屏蔽信号。 0: 不屏蔽中断; 1: 屏蔽中断。	0x0
2:1	RW	timer_7_mode	Timer7 的计数模式。 00: one-shot 模式; 01: periodic 模式; 10: one-shot 模式; 11: free_running 模式。	0x0
0	RW	timer_7_en	Timer7 的使能信号, 在使能信号从无效变为有效时, 通过宏定义配置决定是否加载计数器初始值, 使能信号有效期间可以生成中断, 使能信号无效时 Timer7 保持原值不变, 中断被清除。 0: 使能无效; 1: 使能有效。 Timer 使能从有效变成无效, 再从无效变成有效时, 需要保证期间无效的时间大于等于 Timer 工作的一个时钟周期。	0x0

TEIMER_0_EOI_REN

Timer0 中断清除寄存器。

Offset Address: 0x114+0x100×0 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	WO	timer_0_eoi	Timer0 的中断清除信号。 0: 不清除 Timer0 中断; 1: 清除 Timer0 中断。	0x0

TEIMER_1_EOI_REN

Timer1 中断清除寄存器。

Offset Address: 0x114+0x100×1 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	WO	timer_1_eoi	Timer1 的中断清除信号。 0: 不清除 Timer1 中断; 1: 清除 Timer1 中断。	0x0

TEIMER_2_EOI_REN

Timer2 中断清除寄存器。

Offset Address: 0x114+0x100×2 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	WO	timer_2_eoi	Timer2 的中断清除信号。	0x0

Bits	Access	Name	Description	Reset
			0: 不清除 Timer2 中断; 1: 清除 Timer2 中断。	

TEIMER_3_EOI_REN

Timer3 中断清除寄存器。

Offset Address: 0x114+0x100×3 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	WO	timer_3_eoi	Timer3 的中断清除信号。 0: 不清除 Timer3 中断; 1: 清除 Timer3 中断。	0x0

TEIMER_4_EOI_REN

Timer4 中断清除寄存器。

Offset Address: 0x114+0x100×4 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	WO	timer_4_eoi	Timer4 的中断清除信号。 0: 不清除 Timer4 中断; 1: 清除 Timer4 中断。	0x0

TEIMER_5_EOI_REN

Timer5 中断清除寄存器。

Offset Address: 0x114+0x100×5 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	WO	timer_5_eoi	Timer5 的中断清除信号。 0: 不清除 Timer5 中断; 1: 清除 Timer5 中断。	0x0

TEIMER_6_EOI_REN

Timer6 中断清除寄存器。

Offset Address: 0x114+0x100×6 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	WO	timer_6_eoi	Timer6 的中断清除信号。 0: 不清除 Timer6 中断; 1: 清除 Timer6 中断。	0x0

TEIMER_7_EOI_REN

Timer7 中断清除寄存器。

Offset Address: 0x114+0x100×7 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	WO	timer_7_eoi	Timer7 的中断清除信号。 0: 不清除 Timer7 中断; 1: 清除 Timer7 中断。	0x0

TIMER_0_RAW_INTR

Timer0 原始中断寄存器。

Offset Address: 0x118+0x100×0 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	timer_0_raw_intr	Timer0 的原始中断信号。 0: 未产生原始中断; 1: 产生原始中断。 如果是脉冲中断, 该寄存器没有意义。	0x0

TIMER_1_RAW_INTR

Timer1 原始中断寄存器。

Offset Address: 0x118+0x100×1 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	timer_1_raw_intr	Timer1 的原始中断信号。 0: 未产生原始中断; 1: 产生原始中断。 如果是脉冲中断, 该寄存器没有意义。	0x0

TIMER_2_RAW_INTR

Timer2 原始中断寄存器。

Offset Address: 0x118+0x100×2 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	timer_2_raw_intr	Timer2 的原始中断信号。	0x0

Bits	Access	Name	Description	Reset
		tr	0: 未产生原始中断; 1: 产生原始中断。 如果是脉冲中断, 该寄存器没有意义。	

TIMER_3_RAW_INTR

Timer3 原始中断寄存器。

Offset Address: 0x118+0x100×3 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	timer_3_raw_intr	Timer3 的原始中断信号。 0: 未产生原始中断; 1: 产生原始中断。 如果是脉冲中断, 该寄存器没有意义。	0x0

TIMER_4_RAW_INTR

Timer4 原始中断寄存器。

Offset Address: 0x118+0x100×4 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	timer_4_raw_intr	Timer4 的原始中断信号。 0: 未产生原始中断; 1: 产生原始中断。 如果是脉冲中断, 该寄存器没有意义。	0x0

TIMER_5_RAW_INTR

Timer5 原始中断寄存器。

Offset Address: 0x118+0x100×5 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	timer_5_raw_intr	Timer5 的原始中断信号。 0: 未产生原始中断; 1: 产生原始中断。 如果是脉冲中断, 该寄存器没有意义。	0x0

TIMER_6_RAW_INTR

Timer6 原始中断寄存器。

Offset Address: 0x118+0x100×6 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	timer_6_raw_intr	Timer6 的原始中断信号。 0: 未产生原始中断; 1: 产生原始中断。 如果是脉冲中断, 该寄存器没有意义。	0x0

TIMER_7_RAW_INTR

Timer7 原始中断寄存器。

Offset Address: 0x118+0x100×7 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	timer_7_raw_intr	Timer7 的原始中断信号。 0：未产生原始中断； 1：产生原始中断。 如果是脉冲中断，该寄存器没有意义。	0x0

TIMER_0_INTR

Timer0 屏蔽后的中断寄存器。

Offset Address: 0x11C+0x100×0 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	timer_0_intr	Timer0 屏蔽后的中断信号： 0：屏蔽后未产生中断； 1：屏蔽后产生中断。 如果是脉冲中断，该寄存器没有意义。	0x0

TIMER_1_INTR

Timer1 屏蔽后的中断寄存器。

Offset Address: 0x11C+0x100×1 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	timer_1_intr	Timer1 屏蔽后的中断信号： 0：屏蔽后未产生中断；	0x0

Bits	Access	Name	Description	Reset
			1: 屏蔽后产生中断。 如果是脉冲中断, 该寄存器没有意义。	

TIMER_2_INTR

Timer2 屏蔽后的中断寄存器。

Offset Address: 0x11C+0x100×2 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	timer_2_intr	Timer2 屏蔽后的中断信号: 0: 屏蔽后未产生中断; 1: 屏蔽后产生中断。 如果是脉冲中断, 该寄存器没有意义。	0x0

TIMER_3_INTR

Timer3 屏蔽后的中断寄存器。

Offset Address: 0x11C+0x100×3 Total Reset Value: 0x0000_0000

Offset Address: 0x11C+0x100×3			Total Reset Value: 0x00000000	
Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	timer_3_intr	Timer3 屏蔽后的中断信号: 0: 屏蔽后未产生中断; 1: 屏蔽后产生中断。 如果是脉冲中断, 该寄存器没有意义。	0x0

TIMER_4_INTR

Timer4 屏蔽后的中断寄存器。

Offset Address: 0x11C+0x100×4 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	timer_4_intr	Timer4 屏蔽后的中断信号： 0：屏蔽后未产生中断； 1：屏蔽后产生中断。 如果是脉冲中断，该寄存器没有意义。	0x0

TIMER_5_INTR

Timer5 屏蔽后的中断寄存器。

Offset Address: 0x11C+0x100×5 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	timer_5_intr	Timer5 屏蔽后的中断信号： 0：屏蔽后未产生中断； 1：屏蔽后产生中断。 如果是脉冲中断，该寄存器没有意义。	0x0

TIMER_6_INTR

Timer6 屏蔽后的中断寄存器。

Offset Address: 0x11C+0x100×6 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	timer_6_intr	Timer6 屏蔽后的中断信号： 0：屏蔽后未产生中断； 1：屏蔽后产生中断。 如果是脉冲中断，该寄存器没有意义。	0x0

TIMER_7_INTR

Timer7 屏蔽后的中断寄存器。

Offset Address: 0x11C+0x100×7 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	timer_7_intr	Timer7 屏蔽后的中断信号： 0：屏蔽后未产生中断； 1：屏蔽后产生中断。 如果是脉冲中断，该寄存器没有意义。	0x0

2.10 WDT

2.10.1 概述

WatchDog 用于系统异常恢复，如果未得到更新则隔一定时间（可编程）产生一个系统复位信号，当 WatchDog 在此之前关闭工作时钟或更新计数器或进入 cpu debug_mode，复位信号不会产生。

Hi2131 系列芯片中集成了 2 个 WatchDog：应用子系统、协议子系统各使用 1 个 WatchDog。

2.10.2 功能描述

WatchDog 具有以下功能特点：

- 内置 1 个 32bit 减法计数器。
- 支持超时时间间隔（即计数初值）可配置。
- 支持寄存器锁定，防止寄存器被误改。
- 支持超时中断产生。
- 支持复位信号产生。工作方式：
 - 在第一次超时时，WatchDog 产生中断。
 - 在第二次超时时，如果中断未被清除，WatchDog 产生系统复位。
- 支持调试模式。

2.10.3 工作方式

系统初始化配置

系统上电复位后 WatchDog 计数器处于停止计数状态，在系统初始化过程中需要将 WatchDog 初始化并启动其运行。WatchDog 的初始化过程如下：

- 步骤 1 配置 `WDT_LOCK`，解锁 WatchDog。
- 步骤 2 配置 `WDT_CR[wdt_mode]`，设定 WatchDog 工作模式。
- 步骤 3 配置 `WDT_CR[wdt_imsk]`，根据需要决定是否屏蔽中断。
- 步骤 4 配置 `WDT_CR[rst_en]`，根据需要决定是否产生复位信号。
- 步骤 5 配置 `WDT_CR[wdt_en]=1`，使能 WatchDog，WatchDog 开始工作。
- 步骤 6 配置寄存器 `WDT_LOCK`，给 WatchDog 上锁，防止软件错误修改 WatchDog 的配置。

----结束

中断处理过程

收到 WatchDog 发出的中断后，应及时清除其中断状态，并使其载入计数初值重新开始计数。WatchDog 中断处理的过程如下：

- 步骤 1 向 `WDT_LOCK` 写 `0x5A5A_5A5A`，为 WatchDog 开锁。

步骤 2 写寄存器 **WDT_EOI**[wdt_eoi]，清除 WatchDog 的中断状态，同时也使 WatchDog 自动载入计数初值重新开始计数。

步骤 3 向寄存器 **WDT_LOCK** 写入 0x5A5A_5A5A 以外的任何值，给 WatchDog 上锁。

----结束

关闭 WatchDog

向寄存器 **WDT_CR**[wdt_en]控制位写入 0 或 1 控制 WatchDog 的状态：

步骤 1 向 **WDT_LOCK** 写 0x5A5A_5A5A，为 WatchDog 开锁。

步骤 2 配置 **WDT_CR**[wdt_en]=0，关闭 WatchDog。

----结束

2.10.4 寄存器概览

WDT 寄存器概览如表 2-8 所示。

表2-8 WDT 寄存器概览（应用核基址是 0x3000_4000，协议核基址是 0x4040_1000）

偏移地址	名称	描述
0x000	WDT_LOCK	WDT 锁定寄存器。
0x004	WDT_LOAD	WDT 加载值寄存器。
0x008	WDT_RESTART	WDT 重新启动寄存器。
0x00C	WDT_EOI	WDT 中断清除寄存器。
0x010	WDT_CR	WDT 控制寄存器。
0x014	WDT_CNT	WDT 递减计数器。
0x018	WDT_RAW_INTR	WDT 原始中断寄存器。
0x01C	WDT_INTR	WDT 屏蔽后中断寄存器。
0x0020	WDT_LPIF_STATE	WDT 低功耗状态机状态寄存器。
0x024	WDT_STATUS	WDT 工作状态寄存器。
0x0028	WDT_CCVR_EN	WDT 计数锁定控制寄存器。

偏移地址	名称	描述
0x030	WDT_ID	WDT ID 版本寄存器。

2.10.5 寄存器描述

WDT_LOCK

WDT 锁定寄存器。

Offset Address: 0x0000 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	wdt_lock	写 LOCK 寄存器。 向 LOCK 寄存器写 32bit 密钥 (密钥为 0x5A5A_5A5A) , 解锁寄存器, 表示所有寄存器可写; 向 LOCK 寄存器写其他任意值, 锁定寄存器, LOCK 保护范围内的寄存器不能写, Lock 保护范围外的寄存器可以写。 读任何寄存器不受 LOCK 寄存器的影响, 任何时刻都可以读。 读 LOCK 寄存器 (LOCK 寄存器读出值表示寄存器的锁定状态) : 0x0000_0000: 寄存器处于解锁状态; 0x0000_0001: 寄存器处于锁定状态。 LOCK 寄存器锁定范围请见每个寄存器的描述	0x00000000

WDT_LOAD

WDT 加载值寄存器。

Offset Address: 0x0004 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
31:8	RW	wdt_load	WDT 中递减计数器初始值寄存器的高 24bit, 低 8bit 补全 1。 注意: 该寄存器受写锁 LOCK 寄存器保护。	0xFFFFFFFF
7:0	RO	reserved	WDT 中递减计数器的初始值寄存器低 8bit。 注意: 该寄存器受写锁 LOCK 寄存器保护。	0xFF

WDT_RESTART

WDT 重新启动寄存器。

Offset Address: 0x0008 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	WO	wdt_restart	WDT 重启寄存器。 写入值不是 32'h5A5A_5A5A: 不重启 WDT 计数器; 写入值是 32'h5A5A_5A5A: 重启 WDT 计数器。 注意: 本芯片中该寄存器无效。	0x00000000

WDT_EOI

WDT 中断清除寄存器。

Offset Address: 0x000C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	WO	wdt_eoi	中断清除寄存器。 0: 不清除原始中断; 1: 清除原始中断。 注意: 该寄存器受写锁 LOCK 寄存器保护。 本芯片中表示为高时清除中断, 并将计数值更改为 WDT_LOAD。	0x0

WDT_CR

WDT 控制寄存器。

Offset Address: 0x0010 Total Reset Value: 0x0000_00FC

Bits	Access	Name	Description	Reset
31:8	RO	reserved	保留。	0x00000000
7	RW	wdt_mode	WDT 工作模式配置寄存器。 0: 一次中断产生复位信号; 1: 两次中断产生复位信号。 在 wdt_en 有效时, wdt_mode 保持不变。 注意: 该寄存器受写锁 LOCK 寄存器保护。 本芯片中该寄存器无效。	0x1
6	RW	wdt_imsk	中断屏蔽寄存器。 0: 不屏蔽中断; 1: 屏蔽中断。 注意: 该寄存器受写锁 LOCK 寄存器保护。	0x1

Bits	Access	Name	Description	Reset
			本芯片中该寄存器无效。	
5:3	RW	rst_pl	系统复位信号有效长度配置寄存器。 000: 系统复位信号有效长度为 8'h2; 001: 系统复位信号有效长度为 8'h4; 010: 系统复位信号有效长度为 8'h8; 011: 系统复位信号有效长度为 8'h10; 100: 系统复位信号有效长度为 8'h20; 101: 系统复位信号有效长度为 8'h40; 110: 系统复位信号有效长度为 8'h80; 111: 系统复位信号有效长度为 8'h100。 注意: 该寄存器受写锁 LOCK 寄存器保护。 本芯片中该寄存器无效。	0x7
2	RW	rst_en	系统复位信号产生使能寄存器。 0: 不能产生复位信号; 1: 可以产生复位信号。 注意: 该寄存器受写锁 LOCK 寄存器保护。默认值为 0。	0x1
1	RO	reserved	保留。	0x0
0	RW	wdt_en	计数器工作的使能寄存器。	0x0

Bits	Access	Name	Description	Reset
			0: 计数器不使能; 1: 计数器使能。 注意: 该寄存器受写锁 LOCK 寄存器保护。默认值为 0。	

WDT_CNT

WDT 递减计数器。

Offset Address: 0x0014 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
31:0	RO	wdt_cnt	递减计数器的当前值, 32bit, 解复位后立即跳转加载寄存器的值。 当配置使能开始有效时刻、配置重新启动时刻、计数器递减到 0 且未被暂停时刻加载计数器的初始值, 在 wdt_en 有效且未被暂停时刻, 计数器进行减 1 操作, 否则计数器保持不变。当 wdt 使能无效时, 加载计数器的初始值。	0xFFFFFFFF

WDT_RAW_INTR

WDT 原始中断寄存器。

Offset Address: 0x0018 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	wdt_raw_intr	原始中断寄存器。 0: 未产生原始中断;	0x0

Bits	Access	Name	Description	Reset
			1: 产生原始中断。	

WDT_INTR

WDT 屏蔽后中断寄存器。

Offset Address: 0x001C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	wdt_intr	屏蔽后的中断寄存器。 0: 未产生屏蔽后中断; 1: 产生屏蔽后中断。	0x0

WDT_LPIF_STATE

WDT 低功耗状态机状态寄存器。

Offset Address: 0x0020 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:3	RO	reserved	保留。	0x00000000
2:0	RO	wdt_lpfif_state	低功耗状态指示信号。 000: IDLE 状态; 001: 正常工作状态; 010: 接收低功耗请求状态; 011: 进入低功耗确认状态; 100: 拒绝进入低功耗状态; 101: 等待退出低功耗请求状态; 110: 等待确认退出低功耗状态; 111: 保留。	0x0

Bits	Access	Name	Description	Reset
			低功耗复位时为 000 状态，解复位后 3 个 cycle，跳转到 101 状态，接下来根据低功耗请求的状态值产生状态机的跳转，最快 5 个 cycle 跳转到 001 状态。 注意：本芯片中该寄存器无效。	

WDT_STATUS

WDT 工作状态寄存器。

Offset Address: 0x0024 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	wdt_status	WDT 工作状态寄存器。 0: WDT 空闲； 1: WDT 忙。 注意：本芯片中该寄存器无效。	0x0

WDT_CCVR_EN

WDT 计数锁定控制寄存器。

Offset Address: 0x0028 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:2	RO	rsvd_wdt_eoi	保留。	0x00000000
1	RO	wdt_ccvr_lock	wdt counter 的锁存指示信号。 发起 wdt counter 读请求之	0x0

Bits	Access	Name	Description	Reset
			后，检测到该 bit 有效时，即可通过 WDT_CCVR 读取到有效的 wdt counter 数据 注意：本芯片中该寄存器无效。	
0	RW	wdt_ccvr_req	wdt counter 的读取使能（上升沿有效）。 使用规则如下：读取 wdt counter 时，先将该 bit 置为 1（脉冲信号，只有一拍），代码内部检测 wdt_ccvr_req 的上升沿。 注意：本芯片中该寄存器无效。	0x0

WDT_ID

WDT ID 版本寄存器。

Offset Address: 0x0030 Total Reset Value: 0x20230606

Bits	Access	Name	Description	Reset
31:0	RO	wdt_id	wdt_id 内部赋值，定义版本。	0x20230606

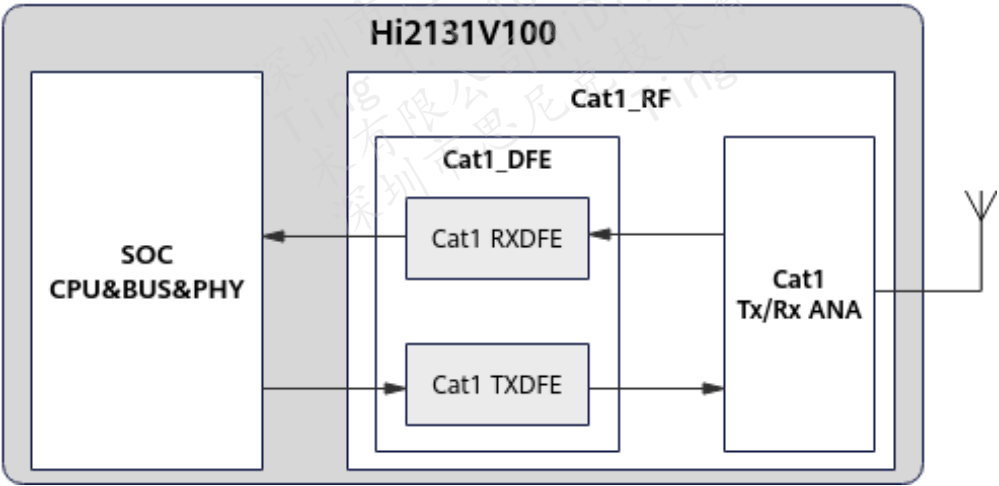
3 RF 系统

RF 系统包括了射频的数字基带处理 DFE (Digital Front-End) 以及主要由 PLL、ADC、DAC、TRX 收发前端通路构成的 RF 模拟部分。

业务的收发交互过程如下：

- 上行发送过程：CAT1.PHY 输出基带数字的调制信号，经过 DFE 和 RF 模拟前端后调制到载波上，通过天线发送出去，基站收到后进行解调。
- 下行接收过程：UE 通过天线接收到空口的高频信号，经过 RF 模拟前端和 DFE 后将调制信号从高频搬移到零中频，CAT1.PHY 再进行解调制（与上行发送是相反的过程）。

图3-1 RF 收发交互示意图



注：ANA (Analog)。

4 PMU

- 4.1 概述
- 4.2 管脚说明
- 4.3 上下电流程
- 4.4 应用场景

4.1 概述

PMU 部分为 SOC 提供供电、时钟信号以及相关辅助功能的完整电源配套解决方案。PMU 内主要集成了 2 路高效率 BUCK、13 路 LDO、2 路功率开关等电源模块；集成了低功耗基准、POR、XO32K 和 RC32K 等系统控制、时钟等模块。PMU 为 SOC 芯片提供了完整的片上电源供电解决方案。

4.2 管脚说明

表4-1 电源引脚

编号	名称	I/O	pin number	数量	功能描述
1	PVDD_ ANA	P	A12	1	ANA BUCK 的功率电源，2.5V ~ 4.8V。
2	PVDD_ DIG	P	A14	1	DIG BUCK 的功率电源，2.5V ~ 4.8V。

编号	名称	I/O	pin number	数量	功能描述
3	VDD_BAT	P	A13	1	电池电压, 2.5V ~ 4.8V。
4	VDDIO_1P8	P	A9	1	IO/模拟 1.8V 电源供电。
5	VDD_DIG_1P05	P	C14	1	数字/RFLDO 电源供电, 电压 1.05V。
6	VDD_ANA_1P9	P	A10	1	RFLDO1P7/TCXO/PLLLDO1P8/LDOA 的供电电源, 电压 1.9V。
7	VDD_RF_SW_1P05	P	E14	1	供电电源, 供给内部 RF 模块, 电压 1.01V。
8	VDD_CLAMP	P	F14	1	PMU 内部 1.86V 供电电源。
9	VDD_1P7_LDO	P	G14	1	RFLDO1P7 的输出电源, 电压 1.7V。
10	VDDIO_E	P	F13	1	USB 供电电源: 3.3V typ
11	VDDIO_D	P	G13	1	PA/SWITCH 供电: 2.8/3/3.3 (default) V
12	VDDIO_A	P	B9	1	IO/MIPI 供电: 1.8V
13	VDDIO_C	P	A8	1	GPIO 供电: 1.8(default)/3.3V
14	VDDIO_F	P	A7	1	EFUSE/TSENSOR/FLASH 供电, 电压 1.8V
15	VDDIO_XO	P	A4	1	XO 供电: 电压 1.8V
16	VDDIO_B	P	A3	1	SIM 供电: 电压 1.8(default)/3V

表4-2 信号引脚

编号	名称	I/O	pin number	数量	功能描述
1	LX_DIG	I/O	B14	1	DIG BUCK 开关信号管脚。
2	LX_ANA	I/O	A11	1	ANA BUCK 开关信号管脚。

编号	名称	I/O	pin number	数量	功能描述
3	AIO_0	I/O	B4	1	电压检测输入管脚 0。 高低电源电压：0.5V ~ 4.8V
4	AIO_1	I/O	B5	1	电压检测输入管脚 1。 0.1V ~ 1.5V
5	AIO_2	I/O	B6	1	电压检测输入管脚 2。 0.1V ~ 1.5V
6	AIO_3	I/O	B7	1	电压检测输入管脚 3。 0.1V ~ 1.5V
7	PWR_KEY_N	I/O	D9	1	开关机按键引脚，按下接低电位，低电平有效。

表4-3 时钟引脚

编号	名称	I/O	pin number	数量	功能描述
1	XTAL32_IN	I/O	A5	1	32K 晶体时钟驱动 XIN。
2	XTAL32_OUT	I/O	A6	1	32K 晶体时钟驱动 XOUT。

表4-4 地引脚

编号	名称	I/O	pin number	数量	功能描述
1	PGND_ANA	G	B11	1	ANA BUCK 功率地。
2	PGND_DIG	G	B13	1	DIG BUCK 功率地。
3	VSS_BAT	G	C8	2	PMU 模拟地。
4	VSS_BAT_1	G	C9	2	PMU 模拟地。

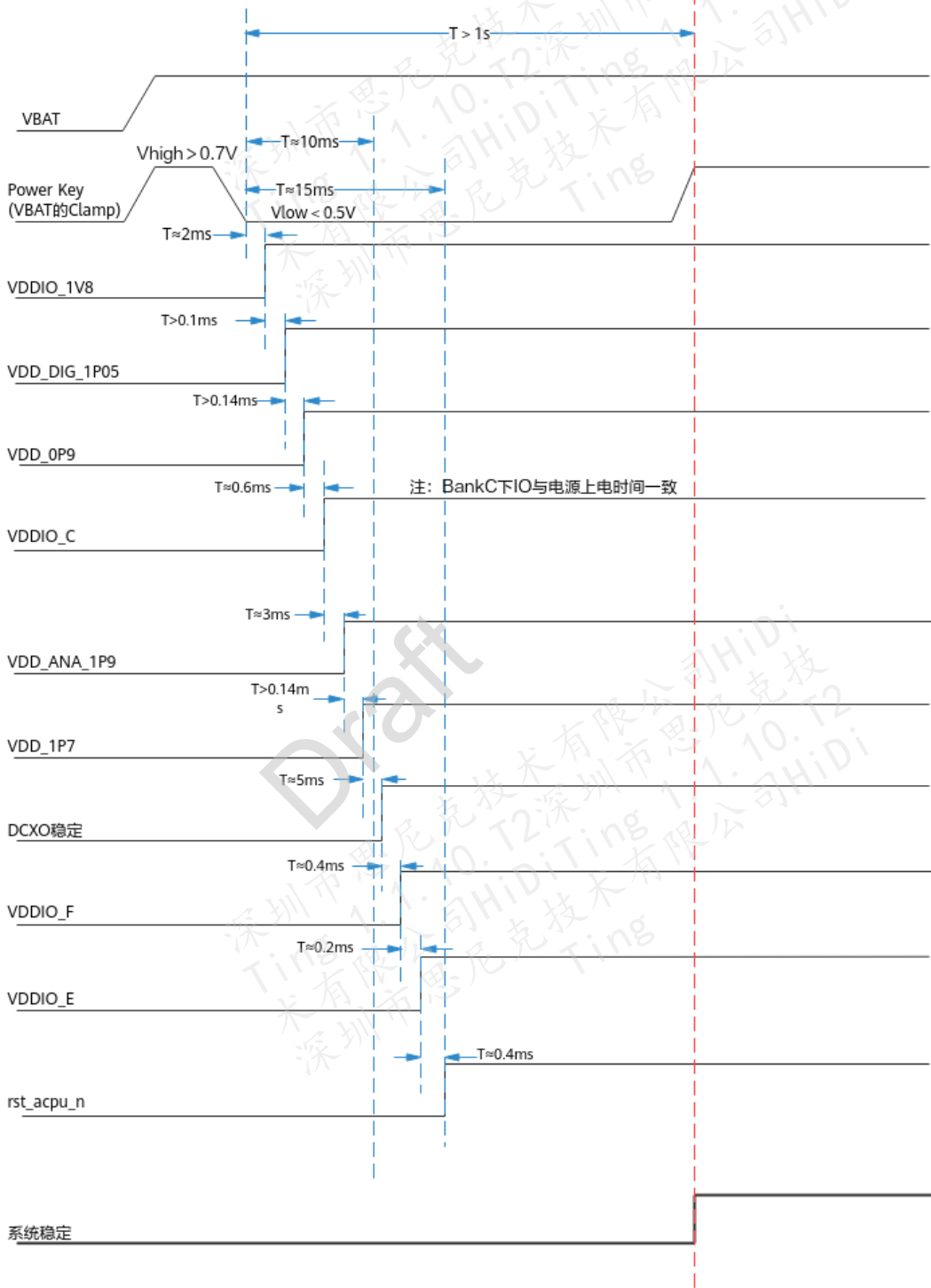
4.3 上下电流程

4.3.1 初次上电

初始上电，系统接通电源后的第一次上电。

Draft

图4-1 系统初次上电时序图



初次的上电时序如图 4-1 所示，整个系统保留最小系统，待 PWRON_KEY_N 信号拉低 $\geq 10ms$ ，系统开始开机；上电后 PWRON_KEY_N 拉低 $\geq 1s$ ，系统关机。

4.3.2 下电流程

PMU 跟随 VBAT 下电，要求 VBAT 下电过程必须下到 2V 以下才能保证 SOC 能有效复位。

4.3.3 下电后再上电流程

下电后重新上电的 PMU 约束为：VBAT 下电到低于 2V 后再重新上电。

下电后重新上电分两种情况：

- VBAT 下电到 0 后重新上电：PMU 上电时序与初始上电流程基本相同。
- VBAT 下电到中间电压后马上上电或等待一段时间重新上电：第二次开始上电时，需要 VBAT 下电能到 2V 以下。

4.4 应用场景

PMU 整体包含如下几个重要的应用场景：

- 系统的供电
- 系统的时钟
- 系统的 PSM 进入和退出

4.4.1 系统的供电

PMU 供电包含几大电源平面：

- VBAT 供电平面，电压 2.5 ~ 4.8V。
- ANA_BUCK 1.9V 供电平面，作为 LDO_BANKA/F/XO、RFLDO1P7/PLLLDO1P8 输入级供电。
- DIG_BUCK 1.05V 供电平面，作为 PMU 的数字级供电，给 RF 的数字使用/SOC 的数字使用（包括 GPIO 的 CORE 电源）。
- VDDIO_1P8/A/C/F 供电平面，向 GPIO 等 IO 电源域供电，其中还包括 Flash 和经过 Power switch 向 EFUSE/Tsensor 供电；VDDIO_A 也会给 MIPI 供电，VDDIO_1P8 会给 PMU 内部 BUCK CLAMP 供电。
- 其余 IP 如 RFLDO1P7/1P01、LDO_BANKB/C/D/E，分别向 RF/SIM/PA/USB 等外设供电。

4.4.2 系统的时钟

PMU 模块输出 32.768kHz 时钟，占空比在 40% ~ 60%之间，供芯片内部使用。

Draft

5 外围设备

- 5.1 IO 复用
- 5.2 GPIO
- 5.3 UART
- 5.4 LP_UART
- 5.5 I2C
- 5.6 I2S/PCM
- 5.7 SPI
- 5.8 PWM
- 5.9 UICC
- 5.10 USB2.0
- 5.11 Vsensor
- 5.12 Tsensor
- 5.13 Audio PWM

5.1 IO 复用

5.1.1 概述

芯片数字管脚数量有限，通过 IO 复用的方式丰富管脚功能。

- BANKC 的 IO 在常电区，在 PSM 模式下依旧维持有电状态，带载能力不变，可以正常工作（GPIO_16 除外，该 IO 不支持 PSM 下作为输入使用）。
- 其他 BANK 的 IO 可以根据场景灵活开关供电实现低功耗。

5.1.2 软件复用管脚描述

VDDIO_A

电压场景：1.8V

VDDIO_A 的软件复用管脚如表 5-1 所示。

表5-1 VDDIO_A 的软件复用管脚

Pad 信号	复用信号 0	复用信号 1	复用信号 2
GPIO_00	GPIO[0]	RF_CTRL[0]	-
GPIO_01	GPIO[1]	RF_CTRL[1]	-
GPIO_02	GPIO[2]	RF_CTRL[2]	-
GPIO_03	GPIO[3]	RF_CTRL[3]	-
GPIO_04	GPIO[4]	RF_CTRL[4]	-
GPIO_05	GPIO[5]	RF_CTRL[5]	-
GPIO_06	GPIO[6]	RF_CTRL[6]	RF_PWM
GPIO_07	GPIO[7]	RF_CTRL[7]	RF_PWM
GPIO_08	GPIO[8]	RFFE_CLK	-
GPIO_09	GPIO[9]	RFFE_DATA	-

VDDIO_A 的软件复用管脚描述如表 5-2 所示。

表5-2 VDDIO_A 的软件复用管脚描述

信号名	方向	说明
GPIO[0]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[1]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[2]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[3]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[4]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[5]	I/O	GPIO 信号（输出信号由寄存器配置）。

信号名	方向	说明
GPIO[6]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[7]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[8]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[9]	I/O	GPIO 信号（输出信号由寄存器配置）。
RF_CTRL[0]	O	输出：RF 控制 GPIO 信号。
RF_CTRL[1]	O	输出：RF 控制 GPIO 信号。
RF_CTRL[2]	O	输出：RF 控制 GPIO 信号。
RF_CTRL[3]	O	输出：RF 控制 GPIO 信号。
RF_CTRL[4]	O	输出：RF 控制 GPIO 信号。
RF_CTRL[5]	O	输出：RF 控制 GPIO 信号。
RF_CTRL[6]	O	输出：RF 控制 GPIO 信号。
RF_CTRL[7]	O	输出：RF 控制 GPIO 信号。
RF_PWM	O	输出：RF PWM 信号。
RFFE_CLK	O	MIPI RFFE 接口时钟信号。
RFFE_DATA	I/O	MIPI RFFE 接口数据信号。

VDDIO_B

电压场景：1.8V（默认值）、3.0V

VDDIO_B 的软件复用管脚如表 5-3 所示。

表5-3 VDDIO_B 的软件复用管脚

Pad 信号	复用信号 0	复用信号 1
GPIO_10	GPIO[10]	USIM_CLK
GPIO_11	GPIO[11]	USIM_RST
GPIO_12	GPIO[12]	USIM_DATA

VDDIO_B 的软件复用管脚描述如表 5-4 所示。

表5-4 VDDIO_B 的软件复用管脚描述

信号名	方向	说明
GPIO[10]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[11]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[12]	I/O	GPIO 信号（输出信号由寄存器配置）。
USIM_CLK	O	UICC 输出时钟。
USIM_RST	O	UICC 输出复位。
USIM_DATA	I/O	UICC 数据信息。

VDDIO_C

电压场景：1.8V（默认值）、3.3V

VDDIO_C 的软件复用管脚如表 5-5 所示。

表5-5 VDDIO_C 的软件复用管脚

Pad 信号	复用信号 0	复用信号 1	复用信号 2	复用信号 3	复用信号 4	复用信号 5	复用信号 6
GPIO_13	GPIO[13]	PMU_32K_TEST	DIAG_PLL_LOCK	DIAG[23]	-	-	-
GPIO_14	GPIO[14]	UART_CTS	JTAG_TCK	DIAG[28]	-	-	-
GPIO_15	GPIO[15]	UART_RTS	JTAG_TMS	DIAG[29]	-	-	-
GPIO_16	GPIO[16]	DEBUG_CTS	JTAG_TDO	DIAG[24]	-	-	-
GPIO_17	GPIO[17]	DEBUG_RTS	JTAG_TDI	DIAG[25]	-	-	-
GPIO_18	GPIO[18]	A_PWM_P	JTAG_RSTN	DIAG[26]	LPUART_CTS	-	-

Pad 信号	复用信号 0	复用信号 1	复用信号 2	复用信号 3	复用信号 4	复用信号 5	复用信号 6
GPIO_19	GPIO[19]	A_PWM_N	CLK_32K	DIAG[27]	LPUART_RTS	-	-
GPIO_20	GPIO[20]	LPUART_TXD	UART_TXD	-	-	-	-
GPIO_21	GPIO[21]	LPUART_RXD	UART_RXD	-	-	-	-
GPIO_22	GPIO[22]	DEBUG_TXD	-	-	-	-	-
GPIO_23	GPIO[23]	DEBUG_RXD	-	-	-	-	-
GPIO_24	SSI_CLK	GPIO[24]	I2C1_SCL	-	-	-	-
GPIO_25	SSI_DATA	GPIO[25]	I2C1_SDA	-	-	-	-
GPIO_26	GPIO[26]	SPI0_CLK	-	DIAG[0]	ATE_CLK	-	-
GPIO_27	GPIO[27]	SPI0_RXD	-	DIAG[1]	ATE_GEN_DATA[0]	ATE_SAM_DATA[0]	-
GPIO_28	GPIO[28]	SPI0_CS_N	-	DIAG[2]	ATE_GEN_DATA[1]	ATE_SAM_DATA[1]	-
GPIO_29	GPIO[29]	SPI0_TXD	-	DIAG[3]	ATE_GEN_DATA[2]	ATE_SAM_DATA[2]	-
GPIO_30	GPIO[30]	SPI1_CLK	-	DIAG[4]	ATE_GEN_DATA[3]	ATE_SAM_DATA[3]	-
GPIO_31	GPIO[31]	SPI1_RXD	-	DIAG[5]	ATE_GEN_DATA[4]	ATE_SAM_DATA[4]	-
GPIO_32	GPIO[32]	SPI1_CS_N	UART_CTS	DIAG[6]	ATE_GEN_DATA[5]	ATE_SAM_DATA[5]	-
GPIO_33	GPIO[33]	SPI1_TXD	UART_RTS	DIAG[7]	ATE_GEN_DATA[6]	ATE_SAM_DATA[6]	-
GPIO_34	GPIO[34]	PCM_CLK	-	DIAG[8]	ATE_GEN_DATA[7]	ATE_SAM_DATA[7]	-

Pad 信号	复用信号 0	复用信号 1	复用信号 2	复用信号 3	复用信号 4	复用信号 5	复用信号 6
GPIO_35	GPIO[35]	PCM_SYNC	-	DIAG[9]	ATE_GEN_DATA_VLD	ATE_SAM_DATA_VLD	-
GPIO_36	GPIO[36]	PCM_DIN	-	DIAG[10]	-	-	-
GPIO_37	GPIO[37]	PCM_DOUT	-	DIAG[11]	-	-	-
GPIO_38	GPIO[38]	I2C0_SCL	SPI0_CS_N	DIAG[12]	-	-	-
GPIO_39	GPIO[39]	I2C0_SDA	SPI0_TXD	DIAG[13]	ATE_CLK	-	-
GPIO_40	GPIO[40]	SPI1_CLK	I2C1_SCL	DIAG[14]	ATE_GEN_DATA[0]	ATE_SAM_DATA[0]	-
GPIO_41	GPIO[41]	SPI1_RXD	I2C1_SDA	DIAG[15]	ATE_GEN_DATA[1]	ATE_SAM_DATA[1]	-
GPIO_42	GPIO[42]	SPI1_CS_N	PCM_MCLK	DIAG[16]	ATE_GEN_DATA[2]	ATE_SAM_DATA[2]	A_PWM_N
GPIO_43	GPIO[43]	SPI1_TXD	-	DIAG[17]	ATE_GEN_DATA[3]	ATE_SAM_DATA[3]	A_PWM_P
GPIO_44	GPIO[44]	PWM0	I2C0_SCL	DIAG[18]	ATE_GEN_DATA[4]	ATE_SAM_DATA[4]	-
GPIO_45	GPIO[45]	PWM1	I2C0_SDA	DIAG[19]	ATE_GEN_DATA[5]	ATE_SAM_DATA[5]	-
GPIO_46	GPIO[46]	UART_TXD	SPI0_CLK	DIAG[20]	ATE_GEN_DATA[6]	ATE_SAM_DATA[6]	-
GPIO_47	GPIO[47]	UART_RXD	SPI0_RXD	DIAG[21]	ATE_GEN_DATA[7]	ATE_SAM_DATA[7]	-
GPIO_48	GPIO[48]	-	-	DIAG[22]	ATE_GEN_DATA_VLD	ATE_SAM_DATA_VLD	-
XO_FREQ_SEL	XO_FREQ_SEL	-	-	-	-	-	-

Pad 信号	复用信号 0	复用信号 1	复用信号 2	复用信号 3	复用信号 4	复用信号 5	复用信号 6
SIM_HP	SIM_HP	-	-	-	-	-	-
JTAG_ENABLE	JTAG_ENABLE	-	-	-	-	-	-
RSTN	RSTN	-	-	-	-	-	-
USB_BOOT	USB_BOOT	-	-	-	-	-	-
USB_VBUS	USB_VBUS	-	-	-	-	-	-
XO_SEL	XO_SEL	-	-	-	-	-	-
FUNC_SEL	FUNC_SEL	-	-	-	-	-	-

VDDIO_C 的软件复用管脚描述如表 5-6 所示。

表5-6 VDDIO_C 的软件复用管脚描述

信号名	方向	说明
GPIO[13]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[14]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[15]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[16]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[17]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[18]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[19]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[20]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[21]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[22]	I/O	GPIO 信号（输出信号由寄存器配置）。

信号名	方向	说明
GPIO[23]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[24]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[25]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[26]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[27]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[28]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[29]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[30]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[31]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[32]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[33]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[34]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[35]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[36]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[37]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[38]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[39]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[40]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[41]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[42]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[43]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[44]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[45]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[46]	I/O	GPIO 信号（输出信号由寄存器配置）。

信号名	方向	说明
GPIO[47]	I/O	GPIO 信号（输出信号由寄存器配置）。
GPIO[48]	I/O	GPIO 信号（输出信号由寄存器配置）。
SSI_CLK	I	SSI 时钟信号。
SSI_DATA	I/O	SSI 数据信号
PMU_32K_TEST	O	PMU 32K 时钟观测信号。
UART_CTS	I	UART TX 流控信号。
UART_RTS	O	UART RX 流控信号。
UART_TXD	O	UART TX 数据。
UART_RXD	I	UART RX 数据。
DEBUG_CTS	I	DEBUG_UART TX 流控信号。
DEBUG_RTS	O	DEBUG_UART RX 流控信号。
A_PWM_P	O	AUDIO_PWM 输出信号。
A_PWM_N	O	AUDIO_PWM 输出信号。
LPUART_CTS	I	LPUART TX 流控信号。
LPUART_RTS	O	LPUART RX 流控信号。
LPUART_TXD	O	LPUART TX 数据。
LPUART_RXD	I	LPUART RX 数据。
DEBUG_TXD	O	DEBUG_UART TX 数据。
DEBUG_RXD	I	DEBUG_UART RX 数据。
SPI0_CLK	I/O	SPI0 时钟。
SPI0_RXD	I	SPI0 输入数据。
SPI0_CSN	I/O	SPI0 片选。
SPI0_TXD	I/O	SPI0 输出数据。
SPI1_CLK	I/O	SPI1 时钟。

信号名	方向	说明
SPI1_RXD	I	SPI1 输入数据。
SPI1_CSN	I/O	SPI1 片选。
SPI1_TXD	O	SPI1 输出数据。
PCM_CLK	I/O	I2S SCLK。
PCM_SYNC	I/O	I2S WS。
PCM_DIN	I	I2S RX 数据。
PCM_DOUT	O	I2S TX 数据。
PCM_MCLK	O	I2S Master 时钟。
I2C0_SCL	I/O	I2C0 时钟。
I2C0_SDA	I/O	I2C0 数据。
I2C1_SCL	I/O	I2C1 时钟。
I2C1_SDA	I/O	I2C1 数据。
PWM0	O	PWM0 输出。
PWM1	O	PWM1 输出。
DIAG_PLL_LOCK	O	PLL LOCK 观测。
JTAG_TCK	I	JTAG 时钟输入。
JTAG_TMS	I/O	JTAG 模式选择输入。
JTAG_TDO	O	JTAG 数据输出。
JTAG_TDI	I	JTAG 数据输入。
JTAG_TRSTN	I	JTAG 复位输入。
CLK_32K	O	CLK_32K 时钟输出。
ATE_CLK	I	ATE 维测随路时钟。
ATE_GEN_DATA[0]	O	ATE 维测发送数据。
ATE_GEN_DATA[1]	O	ATE 维测发送数据。

信号名	方向	说明
ATE_GEN_DATA[2]	O	ATE 维测发送数据。
ATE_GEN_DATA[3]	O	ATE 维测发送数据。
ATE_GEN_DATA[4]	O	ATE 维测发送数据。
ATE_GEN_DATA[5]	O	ATE 维测发送数据。
ATE_GEN_DATA[6]	O	ATE 维测发送数据。
ATE_GEN_DATA[7]	O	ATE 维测发送数据。
ATE_GEN_DATA_VLD	O	ATE 维测发送数据有效指示位。
ATE_SAM_DATA[0]	I	ATE 维测数采数据。
ATE_SAM_DATA[1]	I	ATE 维测数采数据。
ATE_SAM_DATA[2]	I	ATE 维测数采数据。
ATE_SAM_DATA[3]	I	ATE 维测数采数据。
ATE_SAM_DATA[4]	I	ATE 维测数采数据。
ATE_SAM_DATA[5]	I	ATE 维测数采数据。
ATE_SAM_DATA[6]	I	ATE 维测数采数据。
ATE_SAM_DATA[7]	I	ATE 维测数采数据。
ATE_SAM_DATA_VLD	I	ATE 维测数采数据有效指示位。
JTAG_ENABLE	I	JTAG 使能。
FUNC_SEL	I	DFT 使能。
RSTN	I	整芯片复位。
USB_BOOT	I	USB BOOT 烧写使能。
USB_VBUS	I	USB 在位指示。
SIM_HPD	I	SIM 卡在位指示。
XO_SEL	I	XO 时钟源选择。
XO_FREQ_SEL	I	XO 时钟频率选择。

VDDIO_F

电压场景：1.8V

VDDIO_F 的软件复用管脚如表 5-7 所示。

表5-7 VDDIO_F 的软件复用管脚

Pad 信号	复用信号 0
SFC_CLK	SFC_CLK
SFC_CSN	SFC_CSN
SFC_IO0	SFC_IO0
SFC_IO1	SFC_IO1
SFC_IO2	SFC_IO2
SFC_IO3	SFC_IO3

VDDIO_F 的软件复用管脚描述如表 5-8 所示。

表5-8 VDDIO_F 的软件复用管脚描述

信号名	方向	说明
SFC_CLK	O	SPI Flash CLK
SFC_CSN	O	SPI Flash CSN
SFC_IO0	I/O	SPI Flash DI
SFC_IO1	I/O	SPI Flash DO
SFC_IO2	I/O	SPI Flash WPN
SFC_IO3	I/O	SPI FLash HOLDN

5.1.3 控制寄存器

管脚驱动强度

可以通过配置 DS0 (Drive Strength)、DS1、DS2 的值改变芯片管脚的驱动电流的强度，驱动强度与 DS0、DS1、DS2 的对应关系如表 5-9、表 5-10 所示，控制寄存器名称与芯片管脚名称一一对应。

表5-9 VDDIO_A、VDDIO_F 管脚驱动强度 (1.8V)

DS2	DS1	DS0	驱动强度 (mA)
0	0	0	3.2
0	0	1	6.3
0	1	0	9.4
0	1	1	12.5
1	0	0	17.1
1	0	1	20.1
1	1	0	23.1
1	1	1	26.1

表5-10 VDDIO_B、VDDIO_C 管脚驱动强度 (3.3V)

DS2	DS1	DS0	驱动强度 (mA)
0	0	0	7.6
0	0	1	11.2
0	1	0	16.4
0	1	1	19.9
1	0	0	21.9
1	0	1	25.3
1	1	0	30.5
1	1	1	33.7

表5-11 VDDIO_B、VDDIO_C 管脚驱动强度 (1.8V)

DS2	DS1	DS0	驱动强度 (mA)
0	0	0	6.4
0	0	1	9.4
0	1	0	13.8
0	1	1	16.6
1	0	0	18.5
1	0	1	21.3
1	1	0	25.5
1	1	1	28.1

5.2 GPIO

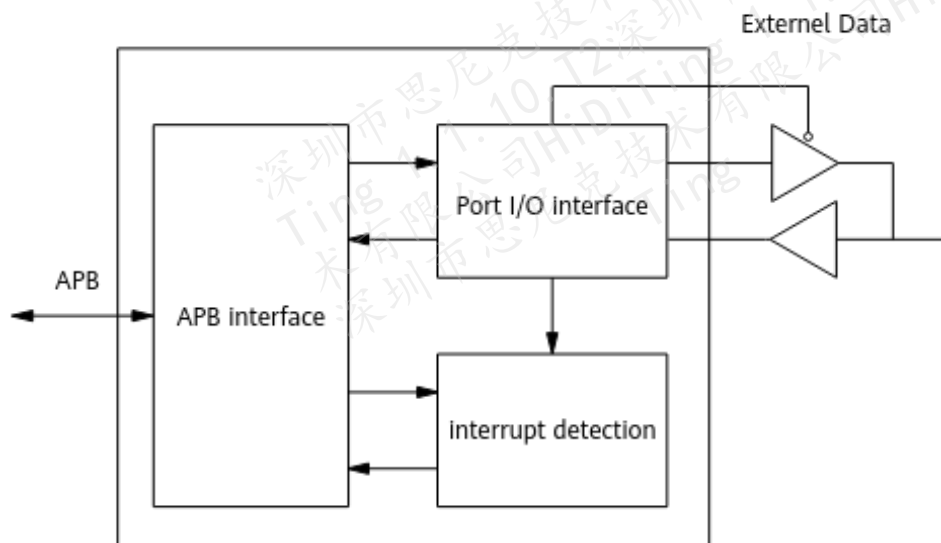
5.2.1 概述

GPIO 是可编程的通用输入/输出接口，用于生成和采集特定应用的输入或输出信号，实现系统和外设之间的通信，方便系统对外设的控制。

GPIO 模块主要接口如下 (如图 5-1 所示)：

- APB 接口
- I/O pad 的外部数据接口
- 中断信号接口

图5-1 GPIO 模块原理图



5.2.2 功能描述

GPIO 接口具有以下功能特点：

- 共 49 个独立的可配置管脚。其中 GPIO10 ~ 15, 17 ~ 48 支持低功耗睡眠时作为唤醒 IO。其他 IO 不支持低功耗下使用。
- 每个 GPIO 管脚都可单独控制传输方向。
- 每个 GPIO 可以单独被配置为外部中断源。
- GPIO 用作中断时有 5 种中断触发方式，中断时触发方式可配：
 - 上升沿触发
 - 下降沿触发
 - 高电平触发
 - 低电平触发
 - 双沿触发

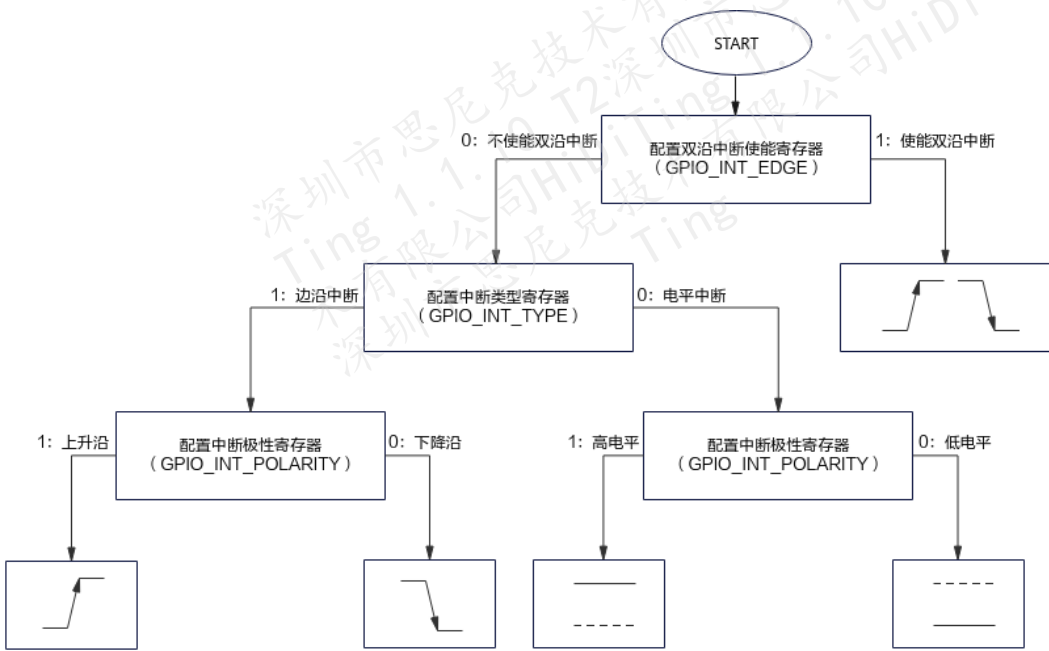
说明

边沿触发中断，需要保证跳转到另一电平，维持 1 个 GPIO 工作时钟后才能产生中断。

5.2.3 工作方式

GPIO 的中断产生配置涉及中断使能、中断屏蔽、中断类型、中断极性等，参考如图 5-2 所示流程进行配置。

图5-2 GPIO 中断配置流程



5.2.3.1 初始化配置

上电复位后，每个 GPIO 可以单独配置为输入或者输出，具体步骤如下：

- 步骤 1 配置 GPIO_SW_OEN(X)数据方向寄存器，数据方向按 bit 单独控制，写入值为 1 表示该 bit 对应的数据方向是输入，写入值为 0 表示该 bit 对应的数据方向是输出。
- 步骤 2 写 GPIO_SW_OUT(X)数据寄存器，如果对应的 bit 数据方向是输出，写入该 bit 的值是对应 I/O 信号的输出，回读值等于最后一次写入的值。如果对应的 bit 数据方向是输入，写入值不起作用。
- 步骤 3 读 GPIO_SW_OUT(X)数据寄存器，如果对应的 bit 数据方向是输入，读该寄存器为外部端口的输入值，软件写入值无效。

----结束

须知

当 GPIO 用作输出时，建议禁止 GPIO 控制器的中断功能，否则当输出信号符合中断产生条件时，会产生 GPIO 中断。

5.2.3.2 边沿中断配置

假设低 8 个 GPIO 输入中断，中断类型均为单边沿上升沿有效，并且输入中断需要去毛刺。

步骤 1 写中断使能寄存器 `GPIO_INT_EN` bit[7:0]=8'h0，配置 8 个 GPIO 中断不使能。

步骤 2 写双沿中断使能寄存器 `GPIO_INT_DEEDGE` bit[7:0]=8'h0，配置不使能双沿中断。

步骤 3 写中断类型寄存器 `GPIO_INT_TYPE` bit[7:0]=8'hFF，配置 8 个输入中断为边沿有效。

步骤 4 写中断极性寄存器 `GPIO_INT_POLARITY` bit[7:0]=8'hFF，配置为 8 个输入中断上升沿有效。

步骤 5 写中断屏蔽寄存器 `GPIO_INT_MASK` bit[7:0]=8'h0，配置不屏蔽中断。

步骤 6 写中断去毛刺控制寄存器 `GPIO_INT_DEBOUNCE` bit[7:0]=8'hFF，配置为去毛刺使能。

步骤 7 写中断使能寄存器 `GPIO_INT_EN` bit[7:0]=8'hFF，使能 8 个输入端口的中断功能。

步骤 8 当有上升沿有效（经过去毛刺）的中断发生后，CPU 收到 GPIO 中断，可以通过查询中断状态寄存器 `GPIO_INTR` 来判断哪个端口发生了中断。

步骤 9 写中断清除寄存器 `GPIO_INT_EOI` [gpio_int_eoi]=1，可以清除掉相应的中断状态。

----结束

说明

中断相关寄存器的配置均独立生效，建议软件先配置中断不使能，再配置相应的中断相关寄存器，最后配置中断使能有效。

当配置的是边沿中断时，可以通过写中断清除寄存器 `GPIO_INT_EOI` 来清中断，如果是电平中断写此寄存器无效。

当配置双沿中断使能寄存器 `GPIO_INT_DEEDGE`，双沿中断有效时，配置中断类型寄存器和中断极性寄存器均无效。

对于边沿中断（包括双沿或单沿），同步功能是默认有效的，不能配置。

双沿中断使能后，建议软件配置中断去毛刺控制寄存器 `GPIO_INT_DEBOUNCE` 为不使能。

5.2.3.3 电平中断配置

假设低 2 个 GPIO 输入中断，中断类型均为高电平有效，并且输入中断需要去毛刺。

步骤 1 写中断使能寄存器 `GPIO_INT_EN` bit[1:0]=2'h0，配置低 2 个 GPIO 中断不使能。

步骤 2 写双沿中断使能寄存器 `GPIO_INT_DEEDGE` bit[1:0]=2'h0，配置不使能双沿中断。

- 步骤 3 写中断类型寄存器 `GPIO_INT_TYPE` bit[1:0]=2'h0, 配置低 2 个输入中断为电平有效。
- 步骤 4 写中断极性寄存器 `GPIO_INT_POLARITY` bit[1:0]=2'hFF, 配置为低 2 个输入中断高电平有效。
- 步骤 5 写中断屏蔽寄存器 `GPIO_INT_MASK` bit[1:0]=2'h0, 配置不屏蔽中断。
- 步骤 6 写中断去毛刺控制寄存器 `GPIO_INT_DEBOUNCE` bit[1:0]=2'h3, 配置低 2 个输入中断为去毛刺使能。
- 步骤 7 写中断使能寄存器 `GPIO_INT_EN` bit[1:0]=2'hFF, 使能低 2 个输入端口的中断功能。
- 步骤 8 当有高电平有效的中断发生后, 经过去毛刺处理, CPU 收到 GPIO 中断, 可以通过查询中断状态寄存器 `GPIO_INTR` 来判断哪个端口发生了中断。

----结束

说明

当设置电平中断时, 中断同步功能默认有效, 不可配置。

电平中断无法通过写 `GPIO_INT_EOI` 中断清除寄存器来清除。

由于需要同步, 因此电平中断必须在 GPIO 有时钟时才能输出。

5.2.4 寄存器概览

GPIO 寄存器概览如表 5-12 如所示。每组 GPIO 包含 8 个 PIN, 不足 8 个 PIN 的 GPIO, 未连接的 PIN 脚对应的寄存器比特配置无效。

表5-12 GPIO 寄存器概览 (GPIO0~9 在 CSYS_SUB, 基址是 0x4040_5000, GPIO10~48 在 AON, 基址是 0x3006_9000, N 表示 GPIO 组序号)

偏移地址	名称	描述
0x0000 + 0x1000×N	<code>GPIO_SW_OUT</code>	GPIO 数据寄存器。
0x0004 + 0x1000×N	<code>GPIO_SW_OEN</code>	GPIO 数据方向寄存器。
0x0008 + 0x1000×N	<code>GPIO_SW_CTL</code>	GPIO 控制模式寄存器。
0x000C + 0x1000×N	<code>GPIO_INT_EN</code>	GPIO 中断使能寄存器。
0x0010 + 0x1000×N	<code>GPIO_INT_MASK</code>	GPIO 中断屏蔽寄存器。
0x0014 + 0x1000×N	<code>GPIO_INT_TYPE</code>	GPIO 中断类型寄存器。

偏移地址	名称	描述
0x0018 + 0x1000×N	GPIO_INT_POLARITY	GPIO 中断极性寄存器。
0x001C + 0x1000×N	GPIO_INT_DEEDGE	GPIO 双沿中断使能寄存器。
0x0020 + 0x1000×N	GPIO_INT_DEBOUNCE	GPIO 中断去毛刺控制寄存器。
0x0024 + 0x1000×N	GPIO_INT_RAW	GPIO 原始中断状态寄存器。
0x0028 + 0x1000×N	GPIO_INTR	GPIO 中断状态寄存器。
0x002C + 0x1000×N	GPIO_INT_EOI	GPIO 中断清除寄存器。
0x0030 + 0x1000×N	GPIO_DATA_SET	GPIO 数据设置寄存器。
0x0034 + 0x1000×N	GPIO_DATA_CLR	GPIO 数据清除寄存器。
0x0038 + 0x1000×N	GPIO_ID	GPIO 版本寄存器。

5.2.5 寄存器描述

GPIO_SW_OUT

GPIO 数据寄存器。

Offset Address: 0x0000 + 0x1000×N Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	gpio_sw_out	第 N 组 GPIO 信号的数据寄存器。 软件模式输出：写入该寄存器的值是该端口 I/O 信号的输出，回读值等于最后一次写入寄存器的值。 软件模式输入：读该寄存器为外部端口的输入值，写该寄存器无效。 硬件模式：写无意义，读回为写入值。 注意：由于端口可以独立控制，因此寄存器每一 bit 均对应一个硬件	0x00000000

Bits	Access	Name	Description	Reset
			I/O 端口。	

GPIO_SW_OEN

GPIO 数据方向寄存器。

Offset Address: $0x0004 + 0x1000 \times N$ Total Reset Value: $0xFFFF_FFFF$

Bits	Access	Name	Description	Reset
31:0	RW	gpio_sw_oen	第 N 组 GPIO 信号的数据方向寄存器。 写入寄存器的值独立控制相应端口的方向。 0: output; 1: input (默认值)。 读回表示各端口的输入输出属性。 0: output; 1: input。	0xFFFFFFF

GPIO_SW_CTL

GPIO 控制模式寄存器。

Offset Address: $0x0008 + 0x1000 \times N$ Total Reset Value: $0x0000_0000$

Bits	Access	Name	Description	Reset
31:0	RW	gpio_sw_ctl	第 N 组 GPIO 信号的控制模式寄存器。 写入寄存器的值可以独立控制相应端口的控制模式。 0: 软件模式 (默认值) ; 1: 硬件模式。	0x00000000

Bits	Access	Name	Description	Reset
			读回表示各端口的控制模式属性。 0：软件模式； 1：硬件模式。 注意：当前平台版本仅支持软件模式，此寄存器无效，不支持配置其它值。	

GPIO_INT_EN

GPIO 中断使能寄存器。

Offset Address: 0x000C + 0x1000×N Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	gpio_int_en	第 N 组 GPIO 信号的中断使能寄存器。 写入寄存器的值可以独立控制相应端口的中断使能。中断功能必须在对应端口为输入属性下才会有效。 0：普通 GPIO 端口（默认值）； 1：中断端口。 读回表示各端口的中断使能状态。 0：普通 GPIO 端口； 1：中断端口。	0x00000000

GPIO_INT_MASK

GPIO 中断屏蔽寄存器。

Offset Address: 0x0010 + 0x1000×N Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
31:0	RW	gpio_int_mask	第 N 组 GPIO 信号的中断屏蔽寄存器。 写入寄存器的值可以独立控制相应端口的中断屏蔽。 0：不屏蔽（默认值）； 1：屏蔽。 读回表示各端口的中断屏蔽状态。 0：不屏蔽； 1：屏蔽。	0x00000000

GPIO_INT_TYPE

GPIO 中断类型寄存器。

Offset Address: 0x0014 + 0x1000×N Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	gpio_int_type	第 N 组 GPIO 信号的中断类型寄存器 写入寄存器的值可以独立控制相应端口的中断类型。 0：电平中断（默认值）； 1：边沿中断。 读回表示各端口的中断类型属性。 0：电平中断； 1：边沿中断。	0x00000000

GPIO_INT_POLARITY

GPIO 中断极性寄存器。

Offset Address: 0x0018 + 0x1000×N Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	gpio_int_polarity	第 N 组 GPIO 信号的中断极性寄存器。 写入寄存器的值可以独立控制相应端口的中断极性。 0: 低电平或者下降沿（默认值）； 1: 高电平或者上升沿。 读回表示各端口的中断极性属性。 0: 低电平或者下降沿； 1: 高电平或者上升沿。	0x00000000

GPIO_INT_DEEDGE

GPIO 双沿中断使能寄存器。

Offset Address: 0x001C + 0x1000×N Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	gpio_int_dedge	第 N 组 GPIO 信号的双沿中断使能寄存器。 写入寄存器的值可以独立控制相应端口的双沿中断使能。 0: 禁止双沿中断； 1: 使能双沿中断。 读回表示各端口的双沿中断使能状态。 0: 禁止双沿中断； 1: 已使能双沿中断。	0x00000000

GPIO_INT_DEBOUNCE

GPIO 中断去毛刺控制寄存器。

Offset Address: $0x0020 + 0x1000 \times N$ Total Reset Value: $0x0000_0000$

Bits	Access	Name	Description	Reset
31:0	RW	gpio_int_debounce	第 N 组 GPIO 信号的中断输入去毛刺控制寄存器。 写入寄存器的值可以独立控制相应端口是否使能中断输入去毛刺功能。 0: 禁止去毛刺 (默认值) ; 1: 使能去毛刺。 读回表示各端口的 Debounce 属性。 0: 禁止去毛刺; 1: 使能去毛刺。 注意: 滤毛刺时钟为 32K 时钟, 可滤除两拍毛刺。	0x00000000

GPIO_INT_RAW

GPIO 原始中断状态寄存器。

Offset Address: $0x0024 + 0x1000 \times N$ Total Reset Value: $0x0000_0000$

Bits	Access	Name	Description	Reset
31:0	RO	gpio_int_raw	第 N 组 GPIO 信号的原始中断状态寄存器。 屏蔽前的中断状态。 0: 无中断; 1: 有中断。	0x00000000

GPIO_INTR

GPIO 中断状态寄存器。

Offset Address: 0x0028 + 0x1000×N Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
[31:0]	RO	gpio_intr	第 N 组 GPIO 信号的中断状态寄存器。 屏蔽之后的中断状态。 0: 无中断; 1: 有中断。	0x00000000

GPIO_INT_EOI

GPIO 中断清除寄存器。

Offset Address: 0x002C + 0x1000×N Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	WO	gpio_int_eoi	第 N 组 GPIO 信号的中断清除寄存器。 写入寄存器的值可以独立控制相应边沿中断端口的中断清除。 0: 不清除中断（默认值）; 1: 清除中断。	0x00000000

GPIO_DATA_SET

GPIO 数据设置寄存器

Offset Address: 0x0030 + 0x1000×N Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	gpio_data_set	GPIO 数据设置寄存器，高有效。	0x00000000

GPIO_DATA_CLR

GPIO 数据清除寄存器

Offset Address: 0x0034 + 0x1000×N Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	gpio_data_clr	GPIO 数据清除寄存器，高有效。	0x00000000

GPIO_ID

GPIO 版本寄存器

Offset Address: 0x0038 + 0x1000×N Total Reset Value: 0x2023_0518

Bits	Access	Name	Description	Reset
31:0	RO	gpio_id	GPIO 版本寄存器。	0x20230518

5.3 UART

5.3.1 概述

通用异步收发器 UART 是一个异步串行的通信接口，UART 主要用于和外部芯片的 UART 进行对接，实现两芯片间的通信。

芯片提供 2 个 UART 接口 (UART0、UART1)，支持 2 线模式。

5.3.2 功能描述

UART 具有以下功能特点：

- 支持 64×8bit 的发送 FIFO 和 64×10bit 的接收 FIFO。
- 支持数据位和停止位的位宽可编程：
 - 数据位可通过编程设定为 5/6/7/8 bit。

- 停止位可通过编程设定为 1/1.5/2 bit。
- 支持奇、偶校验方式或无校验。
- 支持传输速率可编程、支持整数小数分频。
- 支持接收 FIFO 中断、发送 FIFO 中断、接收超时中断、错误中断。
- 支持初始中断状态查询和屏蔽后中断状态查询。
- 支持通过编程禁止 UART 模块或 UART 发送/接收功能以降低功耗。
- 支持硬件流控。
- 支持典型波特率 4800/9600/115200/961200/1M/2M/4M。

5.3.3 接口时序

接口信号

UART 接口信号共包含 2 根基本数据线和 2 根流控线，具体信号描述如表 5-13 所示。

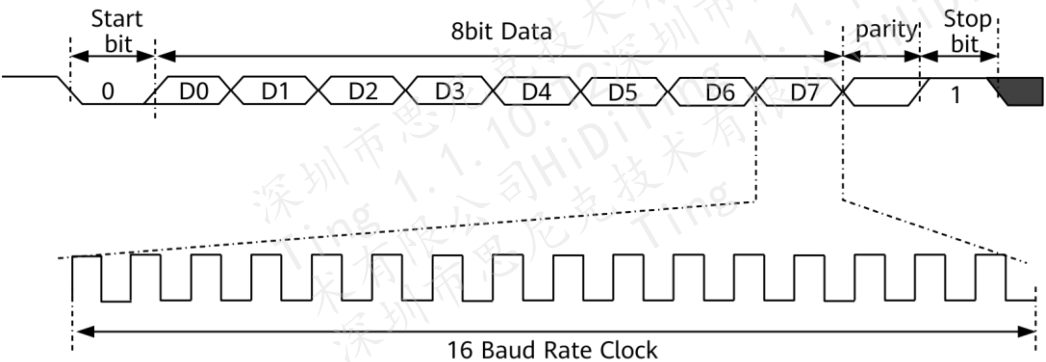
表5-13 UART 接口信号描述

信号名	宽度 (bit)	方向	功能描述
RXD	1	I	输入数据。
TXD	1	O	输出数据。
CTS	1	I	清除发送信号，用于硬件流控，低有效。
RTS	1	O	请求发送信号，用于硬件流控，低有效。

UART 数据格式

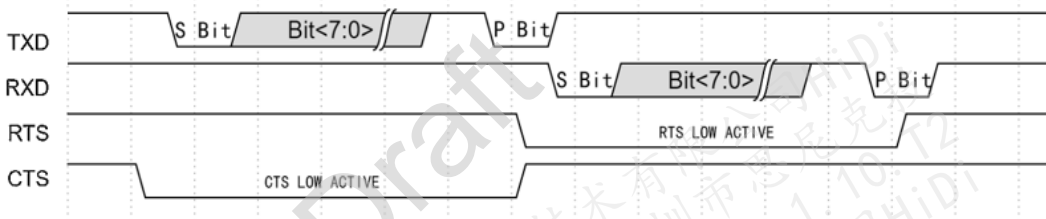
UART 基本数据帧格式如图 5-3 所示。

图5-3 UART 数据帧格式



芯片提供 RTS 和 CTS 两根线作为 UART 硬件流控，二者均为低有效。用户可根据产品设计来选择是否启用该功能，如果启用，则可参考图 5-4 来实现硬件流控功能。

图5-4 UART 流控功能时序



5.3.4 工作方式

5.3.4.1 中断或查询方式下的数据传输

初始化

初始化步骤如下：

- 步骤 1 配置 IO 复用关系，IO 暂不复用为 UART，防止初始化过程中 UART 被对端影响。
- 步骤 2 配置 `HALT_TX[halt_tx]=1`，停止 TX 功能，防止初始化过程中触发 TX。
- 步骤 3 配置 `INTR_EN=0`，屏蔽 UART 所有中断。
- 步骤 4 向 `FIFO_CTL[rx_fifo_rst]`和 `FIFO_CTL[tx_fifo_rst]`写 1，分别复位 RX FIFO 和 TX FIFO。
- 步骤 5 配置 `BAUD_CTL[baud_div]`，根据需要配置波特率过采样倍数。

- 步骤 6 配置 `UART_CTL[div_en]=1`，准备开始配置波特率。
- 步骤 7 配置 `DIV_H`、`DIV_L`、`DIV_FRA`，确定波特率。配置值计算方法为：分频值=工作时钟频率/（波特率×波特率过采样倍数），{`DIV_H`, `DIV_L`}组成分频值的整数部分，`DIV_FRA`=分频率的小数部分×64。
- 步骤 8 配置 `UART_CTL[stp]`、`UART_CTL[pen]`、`UART_CTL[eps]`、`UART_CTL[dlen]`，根据需要配置帧格式，以上寄存器分别控制停止位位宽、奇偶校验使能、奇偶校验选择和数据位长度。
- 步骤 9 配置 `MODEM_CTL[afc_en]`和 `MODEM_CTL[rts]`，根据需要配置自动流控。

表5-14 自动流控配置参考

场景	MODEM_CTL[afc_en]配置值	MODEM_CTL[rts]配置值
打开自动流控	1	1
关闭自动流控，反压对端	0	0
关闭自动流控，正常 RX	0	1

- 步骤 10 配置 `FIFO_CTL[rx_empty_trig]`和 `FIFO_CTL[tx_empty_trig]`，设定发送及接收 FIFO 水线。
- 步骤 11 配置 `FIFO_CTL[fifo_en]=1`，使能 TX FIFO 和 RX FIFO。
- 步骤 12 配置 `INTR_EN=0x1F`，恢复中断使能。
- 步骤 13 配置 `RECEIVE_CTL[receive_enable]=1`，接受数据使能。
- 步骤 14 恢复 IO 复用，将 IO 复用为 UART。
- 步骤 15 配置 `HALT_TX[halt_tx]=0`，使能 TX 功能。

----结束

数据发送

数据发送步骤如下：

- 步骤 1 将发送数据写入 `DATA[data]`，启动数据发送。若为查询模式则跳转至步骤 2，若为中断模式则跳转至步骤 3。

- 步骤 2 查询方式下，如果进行连续数据发送，需要通过读取 `FIFO_STATUS[tx_fifo_full]` 检测 TX FIFO 状态。如果 `FIFO_STATUS[tx_fifo_full]` 为 0，即 TX FIFO 未滿，则可以向 TX FIFO 中发送数据。直到无数据需要发送，跳转至步骤 4。
- 步骤 3 中断方式下，在中断服务程序中查询 `INTR_STATUS[thre_intr_status]` 发送中断状态位，决定是否向 TX FIFO 中发送数据。当 `INTR_STATUS[thre_intr_status]` 置 1，此时 TX FIFO 内数据量小于发送数据水线，可以向 TX FIFO 中发送数据。直到无数据需要发送，跳转至步骤 4。
- 步骤 4 通过检测 `FIFO_STATUS[tx_fifo_empty]` 是否为 1，判断 UART 是否完成全部数据发送。

----结束

数据接收

数据接收的处理方式如下：

- 步骤 1 等待数据接收，若为查询模式则跳转至步骤 2。若为中断模式则跳转至步骤 3。
- 步骤 2 查询方式下，进行数据接收时通过读取 `FIFO_STATUS[rx_fifo_empty]` 检测 RX_FIFO 状态，如果 `FIFO_STATUS[rx_fifo_empty]` 为 0，则 RX_FIFO 非空，可以读取 RX_FIFO 中的数据，跳转至步骤 4。
- 步骤 3 中断方式下，则检测 `INTR_STATUS[data_avail_intr_status]` 接收中断状态位，决定是否读取 RX_FIFO 中的数据。当 `INTR_STATUS[data_avail_intr_status]` 置 1，此时 RX_FIFO 内数据量大于接收 FIFO 水线，可以读取 RX_FIFO 中数据，跳转至步骤 4。
- 步骤 4 回读 `DATA[data]`，读出数据即为 RX 数据。

----结束

5.3.4.2 DMA 方式下的数据传输

初始化

初始化步骤如下：

- 步骤 1 完成 UART 初始化配置。
- 步骤 2 TX 方向，只要 TX FIFO 非滿就会自动上报 TX_SREQ。RX 方向，只要 RX FIFO 非空就会自动上报 RX_SREQ，RX FIFO 内数据量超过水线时则会自动上报 RX_BREQ。
- 步骤 3 DMA 收到 SREQ 或 BREQ 时则会进行数据搬移。

----结束

数据发送

数据发送步骤如下：

- 步骤 1 配置 DMA 数据通道，包括数据传输源地址、目的地址、数据传输个数、传输类型等参数。具体配置请参见“2.6 DMA”章节相关描述。
- 步骤 2 通过 DMA 中断上报或状态查询，判断数据是否发送完成，如果完成则关闭 UART 的 DMA 发送功能，具体请参见“2.6 DMA”章节相关描述。

----结束

数据接收

数据接收的步骤如下：

- 步骤 1 配置 DMA 数据通道，包括数据传输源地址、目的地址、数据传输个数、传输类型等参数。具体配置请参见“2.6 DMA”章节相关描述。
- 步骤 2 通过 DMA 中断上报或状态查询，判断数据是否接收完成，如果完成则关闭 UART 的 DMA 接收功能，具体请参见“2.6 DMA”章节相关描述。

----结束

5.3.5 寄存器概览

UART 寄存器概览如表 5-15 所示。

表5-15 UART 寄存器概览 (UART0 基址是 0x3000_2000, UART1 基址是 0x3000_3000)

偏移地址	名称	描述
0x00	INTR_ID	中断 ID 寄存器。
0x04	DATA	数据寄存器。
0x08	UART_CTL	UART 控制寄存器。
0x0C	DIV_H	分频系数（高位）寄存器。
0x10	DIV_L	分频系数（低位）寄存器。

偏移地址	名称	描述
0x14	DIV_FRA	分频系数（小数部分）寄存器。
0x18	INTR_EN	中断使能寄存器。
0x1C	INTR_STATUS	中断状态寄存器。
0x20	UART_VERSION	UART 版本寄存器。
0x24	FIFO_CTL	FIFO 控制寄存器。
0x28	FAR	FIFO 存取模式使能寄存器。
0x2C	MODEM_CTL	Modem 控制寄存器。
0x30	MODEM_STATUS	Modem 状态寄存器。
0x34	LINE_STATUS	Line 状态寄存器。
0x38	UART_GP_REG	UART 通用寄存器。
0x3C	TX_FIFO_READ	发送 FIFO 读取寄存器。
0x40	RX_FIFO_WRITE	接收 FIFO 写入寄存器。
0x44	FIFO_STATUS	FIFO 状态寄存器。
0x48	TX_FIFO_CNT	发送 FIFO 数据计数器。
0x4C	RX_FIFO_CNT	接收 FIFO 数据计数器。
0x50	HALT_TX	传输挂起寄存器。
0x54	DMA_SW_ACK	DMA 软应答寄存器。
0x58	BAUD_CTL	波特率控制寄存器。
0x5C	STP_CTL	停止位控制寄存器。
0x60	UART_PARAMETER	UART 参数寄存器。
0x64	LP_CTL	低功耗控制寄存器。
0x68	RECEIVE_CTL	接收控制寄存器。
0x6C	WATERLINE_CTL	水线控制寄存器。
0x70	UNIFORM_DIV_CTL	小数分频控制寄存器。

偏移地址	名称	描述
0x74	RTS_RELEASE_CTL	RTS 释放寄存器。

5.3.6 寄存器描述

INTR_ID

中断 ID 寄存器。

Offset Address: 0x00 Total Reset Value: 0x0001

Bits	Access	Name	Description	Reset
15:5	RO	reserved	保留。	0x000
4	RO	fifo_en_s	FIFO 使能控制。 0: FIFO 禁用; 1: FIFO 使能。	0x0
3:0	RO	intr_id	中断 ID。 0x0: Modem 状态; 0x1: 无中断请求; 0x2: THR 空标志; 0x4: 接收数据到达; 0x6: 接收数据线状态; 0x7: 忙状态; 0xC: 字符超时; 其他: 未定义。	0x1

DATA

数据寄存器。

Offset Address: 0x04 Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
15:8	RO	reserved	保留。	0x00
7:0	RW	data	寄存器写入： 若使能 FIFO，写入该寄存器的数据将被发送至发送 FIFO；若 FIFO 未使能，写入该寄存器的数据将被存储于 THR 寄存器（发送 FIFO 的底端存储位置）。 寄存器读取： 该寄存器包含从串行输入端口 (sin) 接收的数据。若 FIFO 未使能，当前数据必须在下一个接收数据到来前读取，否则将被覆盖，并产生 over-run 错误；若使能了 FIFO，通过该寄存器可访问接收 FIFO。若接收 FIFO 已满，且在下一接收数据到来前没有读取该寄存器，虽然 FIFO 内已有的数据不受影响，但后续到来的数据将丢失，并产生 over-run 错误。	0x00

UART_CTL

UART 控制寄存器。

Offset Address: 0x08 Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:8	RO	reserved	保留。	0x00
7	RW	stp	停止位宽。 0: 1bit 停止位;	0x0

Bits	Access	Name	Description	Reset
			1: DLEN 为零时 1.5bit 停止位, 否则 2bit 停止位。 注意: 只有当 STP_CTL[stp_mode]==0 时, 该寄存器才生效。	
6	RW	sps	粘性奇偶校验位选择。 0: 禁用粘性奇偶校验; 1: 使能粘性奇偶校验。	0x0
5	RW	pen	奇偶校验使能。 0: 禁用奇偶校验; 1: 使能奇偶校验。	0x0
4	RW	eps	奇偶校验模式选择。 0: 选择奇校验模式; 1: 选择偶校验模式。	0x0
3:2	RW	dlen	数据长度。 00: 每个字符 5bit 数据; 01: 每个字符 6bit 数据; 10: 每个字符 7bit 数据; 11: 每个字符 8bit 数据。	0x0
1	RW	xbreak	Break 控制位。 0: 串行输出已经释放可用于数据传输; 1: 串行输出强制置于空白状态。	0x0
0	RW	div_en	UART 分频器使能。 0: DIV_L 与 DIV_H 两寄存器仅在 UART 非忙状态可写入; 1: DIV_L 与 DIV_H 两寄存器	0x0

Bits	Access	Name	Description	Reset
			可在任意时刻读写。	

DIV_H

分频系数（高位）寄存器。

Offset Address: 0x0C Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:8	RO	reserved	保留。	0x00
7:0	RW	div_h	分频器分频系数整数部分高8bit。该寄存器仅在UART_CTL[div_en]置位且UART 非忙状态下可以存取。	0x00

DIV_L

分频系数（低位）寄存器。

Offset Address: 0x10 Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:8	RO	reserved	保留。	0x00
7:0	RW	div_l	分频器分频系数整数部分低8bit。该寄存器仅在UART_CTL[div_en]置位且UART 非忙状态下可以存取。	0x00

DIV_FRA

分频系数（小数部分）寄存器。

Offset Address: 0x14 Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
15:6	RO	reserved	保留。	0x000
5:0	RW	div_fra	分频器分频系数小数部分。分频器所用实际分频系数的小数部分为该值除以 2^6 。	0x00

INTR_EN

中断使能寄存器。

Offset Address: 0x18 Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:5	RO	reserved	保留。	0x000
4	RW	ptim_en	可编程 THRE 中断模式使能。 0: 禁用可编程 THRE 中断模式; 1: 使能可编程 THRE 中断模式。	0x0
3	RW	tran_em_intr_en	发送数据为空中断使能。 0: 禁用发送为空中断; 1: 使能发送为空中断。	0x0
2	RW	rece_data_intr_en	接收数据到达中断使能。 0: 禁用接收数据到达中断; 1: 使能接收数据到达中断。	0x0
1	RW	modem_intr_en	Modem 状态中断使能。 0: 禁用 Modem 状态中断; 1: 使能 Modem 状态中断。	0x0
0	RW	rece_line_stat_intr_en	接收数据线状态中断使能。 0: 禁用接收数据线状态中	0x0

Bits	Access	Name	Description	Reset
			断; 1: 使能接收数据线状态中 断。	

INTR_STATUS

中断状态寄存器。

Offset Address: 0x1C Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:6	RO	reserve d	保留。	0x000
5	RO	line_intr _status	接收数据线中断。 0: 无中断; 1: 中断有效。	0x0
4	RO	data_av ail_intr _status	接收数据到达中断。 0: 无中断; 1: 中断有效。	0x0
3	RO	char_to _intr_st atus	字符超时中断。 0: 无中断; 1: 中断有效。	0x0
2	RO	thre_intr _status	THR 空中断。 0: 无中断; 1: 中断有效。	0x0
1	RO	modem _intr_st atus	Modem 状态中断。 0: 无中断; 1: 中断有效。	0x0
0	RO	busy_d et_intr	忙状态监测中断。 0: 无中断; 1: 中断有效。	0x0

UART_VERSION

UART 版本寄存器。

Offset Address: 0x20 Total Reset Value: 0x0002

Bits	Access	Name	Description	Reset
15:0	RO	uart_version	UART 版本寄存器。	0x0002

FIFO_CTL

FIFO 控制寄存器。

Offset Address: 0x24 Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:13	RO	reserved	保留。	0x0
12:10	RW	rx_empty_trig_h	接收空状态触发高水线。 000: 接收 FIFO $\geq 1/8$ 满; 001: 接收 FIFO $\geq 1/4$ 满; 010: 接收 FIFO $\geq 1/2$ 满; 011: 接收 FIFO $\geq 3/4$ 满; 100: 接收 FIFO $\geq 7/8$ 满; 其他: 保留。 注意: 修改寄存器当 UNIFORM_DIV_CTL[uniform_div_enable]配置为 1 时才生效。	0x0
9:7	RW	rx_empty_trig_l	接收空状态触发低水线。 000: 接收 FIFO $\geq 1/8$ 满; 001: 接收 FIFO $\geq 1/4$ 满; 010: 接收 FIFO $\geq 1/2$ 满; 011: 接收 FIFO $\geq 3/4$ 满; 011: 接收 FIFO $\geq 7/8$ 满;	0x0

Bits	Access	Name	Description	Reset
			其他：保留。 注意：改寄存器当 UNIFORM_DIV_CTL[uniform_div_enable]配置为 1 时才生效。	
6	W1_PU LSE	rx_fifo_r st	接收 FIFO 复位请求。 0：不复位； 1：复位。	0x0
5	W1_PU LSE	tx_fifo_r st	发送 FIFO 复位请求。 0：不复位； 1：复位。	0x0
4	RW	fifo_en	FIFO 使能。 0：禁用 FIFO； 1：使能 FIFO。	0x0
3:2	RW	rx_empt y_trig	接收空中断触发条件设置。 00：FIFO 内存在 1 个字符； 01：FIFO 1/4 满； 10：FIFO 1/2 满； 11：距 FIFO 满状态少 2 个字符以 内。	0x0
1:0	RW	tx_empt y_trig	发送空中断触发条件设置。 00：FIFO 为空； 01：FIFO 内存在 2 个字符； 10：FIFO 1/4 满； 11：FIFO 1/2 满。	0x0

FAR

FIFO 存取模式使能寄存器。

Offset Address: 0x28 Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:1	RO	reserved	保留。	0x0000
0	RW	far	使能 FIFO 存取模式。出于测试目的，FIFO 存取模式允许 Master 写入接收 FIFO、读取发送 FIFO。 0：禁用； 1：使能。	0x0

MODEM_CTL

Modem 控制寄存器。

Offset Address: 0x2C Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:6	RO	reserved	保留。	0x000
5	RW	out2	可编程接口 OUT2。 0：out2_n 端口撤离； 1：out2_n 端口有效。 注意：该寄存器无用。	0x0
4	RW	out1	可编程接口 OUT1。 0：out1_n 端口撤离； 1：out1_n 端口有效。 注意：该寄存器无用。	0x0
3	RW	dtr	数据终端 ready。 0：dtr_n 端口撤离； 1：dtr_n 端口有效。 注意：该寄存器无用。	0x0
2	RW	rts	自动流控 RTS 软控信号。 0：反压对端，rts_n=1；	0x0

Bits	Access	Name	Description	Reset
			1: 自动流控 RTS 软控信号。 注意：当 MODEL_CTL[afc_en]==1 且 FIFO_CTL[fifo_en]==1 时，自动流控 RTS 由硬件逻辑接管，此时本寄存器无效。	
1	RW	lb_mode	环回模式控制。 0: 禁用; 1: 使能。	0x0
0	RW	afc_en	自动流控模式使能。 0: 禁用; 1: 使能。	0x0

MODEM_STATUS

Modem 状态寄存器。

Offset Address: 0x30 Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:8	-	reserved	保留。	0x00
7	RO	dsr	数据已准备指示寄存器。用于指示调制解调器控制线 dsr_n 的当前状态。在环回模式 (MODEM_CTL[lb_mode]==1) 下，本寄存器值取自 MODEM_CTL[dsr]。 0: dsr_n 信号线高电平; 1: dsr_n 信号线为低电平。 注意：本芯片 UART 的 dsr_n 恒为 1。	0x0
6	RO	ddsr	dsr_n 跳变指示寄存器，指示 dsr 是否有跳变。	0x0

Bits	Access	Name	Description	Reset
			0: dsr_n 信号线无变化; 1: dsr_n 信号线存在变化。	
5	RO	ri	响铃指示寄存器。用于指示调制解调器控制线 ri_n 的当前状态。在环回模式 (MODEM_CTL[lb_mode==1]) 下, 本寄存器值取自 MODEM_CTL[out1]。 0: ri_n 信号线为高电平; 1: ri_n 信号线为低电平。 注意: 本芯片 UART 的 ri_n 恒为 1。	0x0
4	RO	teri	ri_n 上升沿指示寄存器。 0: 未检测到 ri_n 上升沿; 1: 检测到 ri_n 上升沿。	0x0
3	RO	dcd	数据载波检测寄存器。用于指示 modem 控制线 dcd_n 的当前状态。在环回模式 (MODEM_CTL[lb_mode==1]) 下, 本寄存器值取自 MODEL_CTL[out2]。 0: dcd_n 信号线为高电平; 1: dcd_n 信号线为低电平。 注意: 本芯片 UART 的 dcd_n 恒为 1。	0x0
2	RO	ddcd	dcd_n 跳变指示寄存器, 指示 dcd 是否有跳变。 0: dcd_n 信号线无变化; 1: dcd_n 信号线存在变化。	0x0
1	RO	cts	CTS 信号状态。 0: cts_n 信号线为高电平; 1: cts_n 信号线为低电平。	0x0
0	RO	dcts	该 bit 位用于指示 Modem 的 CTS_N 数据线上从上一次读取 MODEM_STATUS	0x0

Bits	Access	Name	Description	Reset
			寄存器后是否发生过变化。 0: 无变化; 1: 存在变化。	

LINE_STATUS

Line 状态寄存器。

Offset Address: 0x34 Total Reset Value: 0x00C0

Bits	Access	Name	Description	Reset
15:8	-	reserved	保留。	0x00
7	RO	tx_empty_s	发射器空标志。 若 FIFO 使能, 该 bit 在发送器的移位寄存器与发送 FIFO 同时为空时置位; 当 FIFO 禁用时, 该 bit 在 THR 寄存器与移位寄存器同时为空时置位。 0: 发送器非空; 1: 发送器为空。	0x1
6	RO	thre_s	THR 寄存器为空标志。在 INTR_EN[ptim_en]使能的前提下, 如果 FIFO_CTL[fifo_en]也使能, 该 bit 位在发送 FIFO 位满时置位; 当 FIFO_CTL[fifo_en]未使能, 该 bit 位在 THR 寄存器为空时置位。 0: THRE 中断控制禁用; 1: THRE 中断控制使能。	0x1
5	RO	data_available	指示 RBR 或接收 FIFO 内至少存在 1 个字符。非 FIFO 模式下该 bit 位在 RBR 倍读取后清零, 在 FIFO	0x0

Bits	Access	Name	Description	Reset
			模式下该 bit 位在接收 FIFO 为空时清零。 0: 数据未准备好; 1: 数据已准备好。	
4	RO	break_intr	指示串行输入数据中是否检测到 break 序列。 0: 无 break 序列; 1: 检测到 break 序列。	0x0
3	RO	overrun_err	Overrun 错误, 读取 LINE_STATUS 寄存器清零该 bit 位。 0: 无 overrun 错误; 1: 发送 overrun 错误。	0x0
2	RO	parity_err	奇偶校验错误, 读取 LINE_STATUS 寄存器清零该 bit 位。 0: 无奇偶校验错误; 1: 奇偶校验报错。	0x0
1	RO	frame_err	帧错误, 读取 LINE_STATUS 寄存器清零该 bit 位。 0: 无帧错误; 1: 帧错误。	0x0
0	RO	rx_fifo_err	接收 FIFO 错误状态指示。该 bit 位仅在 FIFO 使能时生效。当存在错误的字符位于接收 FIFO 顶部且后续数据无错误, 那么在读取 LSR 后该 bit 位清零。 0: 接收 FIFO 无错误; 1: 接收 FIFO 报错。	0x0

UART_GP_REG

UART 通用寄存器。

Offset Address: 0x38 Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:8	-	reserved	保留。	0x00
7:0	RW	uart_gp_reg	为开发者提供临时存储空间，在 UART IP 内无明确定义。	0x00

TX_FIFO_READ

发送 FIFO 读取寄存器。

Offset Address: 0x3C Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:8	-	reserved	保留。	0x00
7:0	RO	tx_fifo_read	TX FIFO 读数据寄存器。当 FIFO_CTL[fifo_en]被使能，读取该寄存器将返回 TX FIFO 顶部数据，每个连续的读取会对 TX FIFO 进行 POP 操作，并返回当前在 TX FIFO 顶部的数据；当 FIFO_CTL[fifo_en]未被使能，读取该寄存器将返回 DR 中的数据。	0x00

RX_FIFO_WRITE

接收 FIFO 写入寄存器。

Offset Address: 0x40 Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
15:10	-	reserved	保留。	0x00
9	WO	rx_fifo_fe	接收 FIFO 帧错误。 0: 无错误; 1: 帧错误。	0x0
8	WO	rx_fifo_pe	接收 FIFO 奇偶校验错误。 0: 校验位错误; 1: 无错误。	0x0
7:0	WO	rx_fifo_write	RX FIFO 写数据寄存器。该寄存器仅在 FIFO 存取模式使能 (FAR[far]==1) 时生效。当 FIFO_CTL[fifo_en] 使能时, 写入该寄存器的数据将被压入 RX FIFO。每个连续的写操作会连续压入新数据至 RX FIFO 的下一写入位置; 当 FIFO_CTL[fifo_en] 未被使能, 写入该寄存器的数据将被压入 DR。	0x00

FIFO_STATUS

FIFO 状态寄存器。

Offset Address: 0x44 Total Reset Value: 0x000A

Bits	Access	Name	Description	Reset
15:5	-	reserved	保留。	0x000
4	RO	uart_busy	UART 忙状态上报寄存器。 0: UART 处于空闲态或未被激活; 1: UART 处于忙状态 (传输数据)。	0x0

Bits	Access	Name	Description	Reset
3	RO	rx_fifo_empty	RX FIFO 空标志。 0: 接收 FIFO 非空; 1: 接收 FIFO 为空。	0x1
2	RO	rx_fifo_full	RX FIFO 满标志。 0: 接收 FIFO 非满; 1: 接收 FIFO 为满。	0x0
1	RO	tx_fifo_empty	TX FIFO 空标志。 0: 发送 FIFO 非空; 1: 发送 FIFO 为空状态。	0x1
0	RO	tx_fifo_full	TX FIFO 满标志。 0: 发送 FIFO 非满; 1: 发送 FIFO 为满。	0x0

TX_FIFO_CNT

发送 FIFO 数据计数器。

Offset Address: 0x48 Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:7	-	reserved	保留。	0x000
6:0	RO	tx_fifo_level	发送 FIFO 数据计数器。用于指示发送 FIFO 内数据量。	0x00

RX_FIFO_CNT

接收 FIFO 数据计数器。

Offset Address: 0x4C Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
15:7	-	reserved	保留。	0x000
6:0	RO	rx_fifo_level	接收 FIFO 数据计数器。用于指示接收 FIFO 内数据量。	0x00

HALT_TX

传输挂起寄存器。

Offset Address: 0x50 Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:1	-	reserved	保留。	0x0000
0	RW	halt_tx	TX 功能挂起使能。 0: 禁用传输挂起功能; 1: 使能传输挂起功能。	0x0

DMA_SW_ACK

DMA 软应答寄存器。

Offset Address: 0x54 Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:1	-	reserved	保留。	0x0000
0	WC	dma_sw_ack	DMA 软件应答。 0: 未应答; 1: DMA 软件应答。	0x0

BAUD_CTL

波特率控制寄存器。

Offset Address: 0x58 Total Reset Value: 0x007F

Bits	Access	Name	Description	Reset
15:8	-	reserved	保留。	0x00
7:4	RW	sample_phase	接收采样相位。	0x7
3:0	RW	baud_div	波特率过采样倍数。 0x7: 8 倍波特率采样; 0xF: 16 倍波特率采样; 其他: 不支持。	0xF

STP_CTL

停止位控制寄存器。

Offset Address: 0x5C Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:3	-	reserved	保留。	0x0000
2	RW	stp_mode	停止位控制模式。 0: 接收与发送的停止位位宽由 UART_CTL[stp]控制; 1: 发送停止位位宽受 STP_CTL[tx_sp]控制, 接收停止位位宽受 STP_CTL[r_x_sp]控制。	0x0
1	RW	tx_sp	发送停止位位宽。 0: 1bit 停止位; 1: 当 UART_CTL[dlen]为零, 1.5bit 停止位, 否则 2bit 停止位。 注意: 只有当 stp_mode 置位, STP_CTL[tx_sp]才生效。	0x0
0	RW	rx_sp	接收停止位位宽。 0: 1bit 停止位; 1: 当 UART_CTL[dlen]为零,	0x0

Bits	Access	Name	Description	Reset
			1.5bit 停止位，否则 2bit 停止位。 注意：只有当 stp_mode 置位，STP_CTL[rx_sp]才生效。	

UART_PARAMETER

UART 参数寄存器。

Offset Address: 0x60 Total Reset Value: 0x0D04

Bits	Access	Name	Description	Reset
15:13	-	reserved	保留。	0x0
12	RO	shadow	SHADOW 功能使能。 0: 禁用; 1: 使能。	0x0
11	RO	dma_mode	DMA 模式查询。 0: DMA_EXTRA 禁用; 1: DMA_EXTRA 使能。	0x1
10	RO	afce_mode	AFCE 模式查询。 0: AFCE 模式禁用; 1: AFCE 模式使能。	0x1
9:8	RO	apb_data_width	总线接口宽度查询寄存器。 00: APB 数据位宽 8bit; 01: APB 数据位宽 16bit; 10: APB 数据位宽 32bit; 11: 未定义。 注意：本芯片 UART APB 数据位宽固定为 16bit，故该寄存器值固定为 0x1。	0x1

Bits	Access	Name	Description	Reset
7:0	RO	fifo_depth	UART 接收发送 FIFO 深度。 0x0: FIFO 深度为 0; 0x1: FIFO 深度为 16; 0x2: FIFO 深度为 32; 0x4: FIFO 深度为 64; 0x8: FIFO 深度为 128; 0x10: FIFO 深度为 256; 0x20: FIFO 深度为 512; 0x40: FIFO 深度为 1024; 0x80: FIFO 深度为 2048; 其他: 未定义。 注意: 本芯片 UART FIFO 深度固定为 64, 故该寄存器值固定为 0x4。	0x04

LP_CTL

低功耗控制寄存器。

Offset Address: 0x64 Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:1	-	reserved	保留。	0x0000
0	RW	lp_req_en	低功耗请求使能。 0: 低功耗请求禁用, 此时 uart_autocg_bypass 必须设为 1; 1: 低功耗请求使能, 此时 uart_autocg_bypass 必须设为 0。 注意: uart_autocg_bypass 为 SOC 寄存器, 请在《sys_regs_100NL.xlsm》中查	0x0

Bits	Access	Name	Description	Reset
			找。	

RECEIVE_CTL

接收控制寄存器。

Offset Address: 0x68 Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:1	-	reserved	保留。	0x0000
0	RW	receive_enable	FIFO 接收使能。 0: FIFO 接收不使能; 1: FIFO 接收使能。	0x0

WATERLINE_CTL

水线控制寄存器。

Offset Address: 0x6C Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:1	-	reserved	保留。	0x0000
0	RW	double_wl_enable	水线模式控制寄存器。默认情况下: 单水线模式下, 当 rx_empty_trig 条件触发后, auto_rts 触发, 直至 rx_fifo_empty、auto_rst 释放。 双水线模式下, 当 rx_empty_trig_h 条件触发后, auto_rst 触发, 当 rx_empty_trig_l 条件触发后, auto_rst 释放。 0: 单水线控制使能;	0x0

Bits	Access	Name	Description	Reset
			1: 双水线控制使能。 注意：释放 RTS 条件需要与 RTS_RELEASE_CTL[rts_release_sel]配合使用。	

UNIFORM_DIV_CTL

小数分频控制寄存器。

Offset Address: 0x70 Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:1	-	reserved	保留。	0x0000
0	RW	uniform_div_enable	小数分频补偿模式，需要在 DIV_H[div_h]、DIV_L[div_l]、DIV_FRA[div_fra]之前配置。 0: 计数补偿模式； 1: 顺滑补偿模式。	0x0

RTS_RELEASE_CTL

RTS 释放寄存器。

Offset Address: 0x74 Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:1	-	reserved	保留。	0x0000
0	RW	rts_release_sel	RTS 释放条件选择。 当 WATERLINE_CTL[double_wl_enable]配置为 0 时。 0: rx_fifo_empty 时，释放 RTS 流控；	0x0

Bits	Access	Name	Description	Reset
			1: rx_fifo 内数据小于 rx_empty_trig 中指定时, 释放 RTS 流控。 • 当 WATELINE_CTL[double_wl_enable]配置为 1 时。 0: 当 rx_empty_trig_high 小于等于 rx_empty_trig_l 条件 rx_fifo_empty 时释放 RTS 流控, 否则 rx_fifo 内数据小于 rx_empty_trig_l 中指定时, 释放 RTS 流控; 1: 当 rx_fifo 内数据小于 rx_empty_trig_high 和 rx_empty_trig_l 指定的较小数量时, 释放 RTS 流控。	

5.4 LP_UART

5.4.1 概述

LP_UART 是一种低功耗的异步串行通信接口, 不仅可用于 UART 调试, 也可以和其他支持 UART 的芯片进行通信。系统可以通过 LP_UART 从深睡模式唤醒系统而不丢失数据, 同时保持较低功耗。

5.4.2 功能描述

LP_UART 具有以下功能特点:

- 支持 7×8bit 的发送 FIFO 和 31×8bit 的接收 FIFO。
- 支持可编程的数据位宽, 数据位宽可配置为 5bit、6bit、7bit、8bit。
- 支持奇偶校验, 可配置打开/关闭、奇偶校验配置选择 (即选择配置为奇校验还是偶校验)。

- 发送停止位为 1bit。
- 支持源头时钟分频可配置。
- 支持深睡唤醒模式（系统在深睡下仍可正常接收）和正常使用模式（系统在深睡下不可接收）。
- 在正常使用模式支持的最大波特率为 230400bit/s。
- 支持接收数据中断、接收 FIFO 溢出中断、接收帧错误中断、接收校验错误中断、接收 FIFO 非空中断、发送完成中断。
- 中断可配置屏蔽和清除。
- 支持硬件流控。
- 支持系统在深睡模式下仍可正常接收，并可唤醒系统。RX 接收数据后即可唤醒。

5.4.3 工作方式

5.4.3.1 初始化配置

初始化步骤如下：

- 步骤 1 向 `LP_UART_CTRL[regreset]` 写 1，使 LP_UART 的 T/RX 处于复位状态。
- 步骤 2 配置 `LP_UART_CTRL[regclkssel]`，根据需要切换 LP_UART 的工作时钟。
- 步骤 3 如 LP_UART 工作在 CLK_RTC 下，则直接跳转到步骤 4。如 LP_UART 工作在 CLK_LP 下，则配置 `LP_UART_CTRL[regclkdiv]`，内部对 CLK_LP 进行分频。
- 步骤 4 配置 `LP_UART_CTRL[regstopen]` 为 1，开启停止位校验。
- 步骤 5 配置 `LP_UART_CTRL[regparityen]` 为 1，开启奇偶校验。
- 步骤 6 配置 `LP_UART_CTRL[regparityinv]`，根据需要选择奇校验或偶校验。
- 步骤 7 配置 `LP_UART_CTRL[reglength]`，根据需要配置数据位长度。
- 步骤 8 配置 `LP_UART_CTRL[regrdypol]`，根据需要配置流控信号极性。
- 步骤 9 配置 `LP_UART_CTRL[regdenominator]` 和 `LP_UART_CTRL[RegNumerator]` 调整波特率，配置值按照：波特率 = 工作时钟频率 × (Numerator/Denominator) 计算。
- 步骤 10 配置 `LP_UART_CTRL1[lpuartxsync]`，调整采样相位。
- 步骤 11 配置 `LP_UART_CTRL1[rxthreshold]` 和 `LP_UART_CTRL1[idletimeout]`，调整中断上报阈值。
- 步骤 12 配置 `LP_UART_IRQ_CONF[lpuartctrlirqconf]`，根据需要调整中断使能。

步骤 13 向 `LP_UART_CTRL[regreset]` 写 0，初始化完成，解复位。

步骤 14 等待 1μs，等待解复位完成。

----结束

5.4.3.2 数据传输配置

数据发送-查询方式

数据发送步骤如下：

步骤 1 先配置 `LP_UART_IRQ_CONF[lpuartctrlirqconf]`，关闭 TX 中断使能。

步骤 2 将发送数据写入 `LP_UART_TX_DATA[lp_uart_tx_data_in]`，若对端未通过流控信号反压，则逻辑自动开始 TX。

步骤 3 查询方式下，如果进行连续数据发送，需要通过读取 `LP_UART_STATUS[lpuarttxfifo]` 检测 TX_FIFO 内的数据量。如果数据量小于 31，即 TX_FIFO 未滿，则可以向 TX_FIFO 中发送数据。

----结束

数据发送-中断方式

步骤 1 中断方式下，先配置 `LP_UART_IRQ_CONF[lpuartctrlirqconf]`，关闭 TX 中断使能。

步骤 2 向 `LP_UART_TX_DATA[lp_uart_tx_data_in]` 连续写入 n 个数据 (n≤31)。

步骤 3 配置 `LP_UART_IRQ_CONF[lpuartctrlirqconf]`，打开 TX 中断使能。

步骤 4 TX_FIFO 内所有数据发送完成后 LP_UART 将上报 TX 中断，收到中断后查询 `LP_UART_STATUS[TxIrq]`，确认中断有效。

步骤 5 配置 `LP_UART_IRQ_CLR[lpuartirqclr]` 清除 TX 中断。

步骤 6 如仍有数据待发则重复步骤 1 ~ 步骤 5。

----结束

数据接收-查询方式

步骤 1 先配置 `LP_UART_IRQ_CONF[lpuartctrlirqconf]`，关闭 RX FIFO 超时中断和 RX 正常收帧中断。

步骤 2 读取 `LP_UART_STATUS`[`lpuartxfifolvl`]检测 RX FIFO 内的数据个数，如果回读值非 0，则说明 RX FIFO 内有已接收的数据。

步骤 3 读取 `LP_UART_RX_DATA`[`lp_uart_rx_data_in`]，回读 RX 数据，同时可知 RX FIFO 内剩余数据个数，重复步骤 3，直到 RX FIFO 数据个数为 0。

----结束

数据接收-中断方式

步骤 1 配置 `LP_UART_CTRL1`[`rxthreshold`]和 `LP_UART_CTRL1`[`idletimeout`]，调整中断上报阈值。

步骤 2 配置 `LP_UART_IRQ_CONF`[`lpuartctrlirqconf`]，打开 RX FIFO 超时中断和 RX 正常收帧中断。

步骤 3 完成一帧数据收取或 RX FIFO 内有数据超时未取时或 RX FIFO 内数据个数超过阈值，`LP_UART` 将上报中断，收到中断后查询 `LP_UART_STATUS`[`seperaterxirqs`]，确认中断有效。

步骤 4 读取 `LP_UART_RX_DATA`[`lp_uart_rx_data_in`]，回读 RX 数据，同时可知 RX FIFO 内剩余数据个数，重复步骤 4，直到 RX FIFO 数据个数为 0。

步骤 5 配置 `LP_UART_IRQ_CLR`[`lpuartirqclr`]清除 RX 中断。

----结束

5.4.4 寄存器概览

`LP_UART` 寄存器概览如表 5-16 所示。

表5-16 `LP_UART` 寄存器概览 (基址是 0x3006_0600)

偏移地址	名称	描述
0x0000	<code>VERSION_LP_UART</code>	版本号寄存器。
0x0004	<code>LP_UART_CTRL</code>	<code>LP_UART</code> 控制寄存器。
0x0008	<code>LP_UART_CTRL1</code>	<code>LP_UART</code> 控制寄存器 1。
0x000C	<code>LP_UART_STATUS</code>	<code>LP_UART</code> 状态寄存器。
0x0010	<code>LP_UART_IRQ_CLR</code>	<code>LP_UART</code> 中断清除寄存器。

偏移地址	名称	描述
0x0014	LP_UART_TX_DATA	LP_UART TX 数据寄存器。
0x0018	LP_UART_RX_DATA	LP_UART RX 数据寄存器。
0x001C	LP_UART_RX_FREQ_STATUS	LP_UART RX 计数值上报寄存器。
0x0020	LP_UART_IRQ_CONF	LP_UART 中断配置寄存器。
0x0024	FIFO_LEVEL	T/RX FIFO Level 寄存器。
0x0028	TRX_DMA_THRESHOLD	DMA 请求阈值寄存器。

5.4.5 寄存器描述

VERSION_LP_UART

版本号寄存器。

Offset Address: 0x0000 Total Reset Value: 0x2023_1127

Bits	Access	Name	Description	Reset
31:0	RO	version_lp_uart	IP 版本号寄存器，用于查询当前 IP 的版本信息。	0x20231127

LP_UART_CTRL

LP_UART 控制寄存器。

Offset Address: 0x0004 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:28	RW	regclkdiv	时钟分频配置，对 clk_lp 进行分频。 0x0：1 分频； 0x1：2 分频； 0x2：3 分频；	0x0

Bits	Access	Name	Description	Reset
			 0xF: 16 分频。 时钟分频系数= (1+LP_UART_CTRL[RegClkDiv])。	
27	RW	regedgesync	沿同步使能配置寄存器。 0: 只在起始位同步; 1: 在每个 RX 的上升沿或下降沿进行同步。	0x0
26	RW	regclkset	时钟选择寄存器。 0: 选择 32kHz RTC 时钟; 1: 选择 CLK_LP/ (1+LP_UART_CTRL[RegClkDiv])。	0x0
25	RW	regreset	LP_UART 逻辑复位配置。 0: 不复位; 1: 复位。	0x0
24:23	RW	regclkuartctrl	CLK_LP 时钟选择。 00: CLK_LP=0; 01: CLK_LP=CLK_DIG/128; 10: CLK_LP=CLK_DIG/32; 11: CLK_LP=CLK_DIG/64。	0x0
22	RW	regstopen	接收数据帧结束标志 check 使能。 0: 禁止; 1: 使能。	0x0
21	RW	regparityinv	RX/TX 数据奇偶校验选择。 0: 偶校验; 1: 奇校验。	0x0

Bits	Access	Name	Description	Reset
20	RW	regparityen	接收数据帧奇偶检验标志 check 使能，发送数据奇偶位使能。 1：使能； 0：禁止。	0x0
19:18	RW	reglength	帧数据长度配置。 00：帧长=5； 01：帧长=6； 10：帧长=7； 11：帧长=8。	0x0
17:16	RW	regrdypol	bit[17]：TX Ready 极性控制。 0：输入 RDY_IN 为低电平时可发送数据； 1：输入 RDY_IN 为高电平时可发送数据。 bit[16]：RX Ready 极性控制。 0：RX FIFO 有空间输出 RDY_OUT 为低电平； 1：RX FIFO 有空间输出 RDY_OUT 为高电平。	0x0
15:8	RW	regdenominator	波特率分频配置，分母。 注意：0x0 对应最大值 256； buad_rate=clk× (RegNumerator/RegDenominator)。	0x00
7:0	RW	regnumerator	波特率分频配置，分子。 注意：buad_rate=clk× (RegNumerator/RegDenominator)。	0x00

LP_UART_CTRL1

LP_UART 控制寄存器 1。

Offset Address: 0x0008 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:24	RW	rxthreshold	RX FIFO 水线设置寄存器范围为 0 ~ (2 ^{TX_FIFO_DEPTH} -1)。 注意：配置值需要考虑 TX_FIFO_DEPTH，请根据需要适配；假如 FIFO 深度只有 31，RxThreshold 配置超过 31，则会导致 Rxfifo level 不可能超过阈值，irq[5]将无法上报。	0x00
23:16	RW	idletimeout	timeout 阈值配置寄存器，当 RX 进入 idle 状态且 RXFIFO 中有数，计时超过{idleTimeout[7:0]. 8'hff}时上报 timeout 中断。	0x00
15:0	RW	lpuartrxsync	bit[15:8]：无时钟唤醒同步配置。 无时钟唤醒时会按该配置值进行接收数据同步。 bit[7:0]：正常工作同步配置。正常工作时按该配置值进行接收数据同步。该值需要配置为 LP_UART_CTRL[RegDenominator]/2 + LP_UART_CTRL[RegNumerator]。	0x0000

LP_UART_STATUS

LP_UART 状态寄存器。

Offset Address: 0x000C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
31:30	-	reserved	保留。	0x0
29:25	-	reserved	保留。	0x00
24	RO	txirq	TX 中断上报查询。 0: TXFIFO 非空; 1: TXFIFO 为空。	0x0
23:18	RO	seperaterxirqs	bit[23]: RX FIFO 超阈值中断。 0: 正常; 1: 表示 RX FIFO 数据超过水线。 bit[22]: RX FFIFO 超时未取中断。 0: 正常; 1: 表示触发 RX timeout 中断。 bit[21]: RX 奇偶检验错误中断。 0: 正常; 1: 表示 RX 数据存在奇偶校验错误。 bit[20]: RX 帧错误中断。 0: 正常; 1: 表示 RX 数据存在帧错误。 bit[19]: RX FIFO 溢出中断: 0: 正常; 1: 表示 RX FIFO 溢出。 bit[18]: RX 正常收帧中断: 0: 正常。 1: 表示接收到一帧完整且正确的数据。	0x00
17:15	RO	lpuartdebug	bit[17]: TX 状态机。 0: IDLE 默认态;	0x0

Bits	Access	Name	Description	Reset
			1: 正在执行 TX 任务。 bit[16:15]: RX 状态机。 00: IDLE 默认态; 01: 正在执行 RX 任务; 10: 校验状态, 正在检查校验位及停止位; 11: 等待状态, 出现停止位错误时 LPUART 会进入此状态, 等待 RX 信号线恢复高电平。	
14:12	-	reserved	保留。	0x0
11:8	-	reserved	保留。	0x0
7:0	RO	rxstatus	RX 收帧错误状态计数。 bit[7:4]: RX 数据奇偶校验错误计数值上报; bit[3:0]: RX 数据帧错误计数值上报。	0x00

LP_UART_IRQ_CLR

LP_UART 中断清除寄存器。

Offset Address: 0x0010 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:7	-	reserved	保留。	0x00000000
6:0	W1_PULSE	lpuartirqclr	中断清除配置。 bit[6]: TX 中断清除; bit[5]: 保留, 未被使用; bit[4]: RX 超时中断清除; bit[3]: RX 奇偶校验错误中断清除	0x00

Bits	Access	Name	Description	Reset
			除; bit[2]: RX 帧错误中断清除; bit[1]: RX FIFO 溢出中断清除; bit[0]: RX 正常收帧中断清除。 注意: 对应 bit 写 1 清除中断。	

LP_UART_TX_DATA

LP_UART TX 数据寄存器。

Offset Address: 0x0014 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:8	-	reserved	保留。	0x000000
7:0	RW_READ_WRITE	lpuarttxdata	TX 数据配置。	0x00

LP_UART_RX_DATA

LP_UART RX 数据寄存器。

Offset Address: 0x0018 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:16	-	reserved	保留。	0x0000
15:8	RO	lpuartrxfifolvl_backup	RX FIFO 数据 Level。 0x0: RX FIFO 空; 0x1: RX FIFO 内存 1 帧数据; ... 2^RX_FIFO_DEPTH: RX FIFO 满。	0x00

Bits	Access	Name	Description	Reset
			注意：实际位宽与 RX_FIFO_DEPTH 绑定，请根据需要适配。	
7:0	RO	lp_uart_rx_data_in	RX 数据寄存器。	0x00

LP_UART_RX_FREQ_STATUS

LP_UART RX 计数值上报寄存器。

Offset Address: 0x001C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:13	-	reserved	保留。	0x00000
12:0	RO	lp_uart_rx_freq_status	在 RX 信号线首个翻转边沿记录当前 RX 状态。 bit[12:9]: RX 接收数据 bit 计数值; bit[8:0]: baud_rate 计数器计数值。	0x0000

LP_UART_IRQ_CONF

LP_UART 中断配置寄存器。

Offset Address: 0x0020 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:7	-	reserved	保留。	0x0000000
6:0	RW	lp_uart_irq_conf	中断使能配置。 bit[6]: TX 中断使能; bit[5]: RX FIFO 超阈值中断使	0x00

Bits	Access	Name	Description	Reset
			能; bit[4]: RX FIFO 超时中断使能; bit[3]: RX 奇偶校验错误中断使能; bit[2]: RX 帧错误中断使能; bit[1]: RX FIFO 溢出中断使能; bit[0]: RX 正常收帧中断使能。 注意: 配置相应 bit 位为 1 使能中断上报, 配置为 0 屏蔽中断。	

FIFO_LEVEL

T/RX FIFO Level 寄存器。

Offset Address: 0x0024 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:16	-	reserved	保留。	0x0000
15:8	RO	lpuarttxfifolvl	RX FIFO 数据 Level。 0x0: RX FIFO 空; 0x1: RX FIFO 内存 1 帧数据; ... 2^RX_FIFO_DEPTH: RX FIFO 满。 注意: 实际位宽与 RX_FIFO_DEPTH 绑定, 请根据需要适配。	0x00
7:0	RO	lpuarttxfifolvl	TX FIFO 数据 Level: 0x0: TX FIFO 空; 0x1: TX FIFO 内存 1 帧数据;	0x00

Bits	Access	Name	Description	Reset
			... 2^RX_FIFO_DEPTH: TX FIFO 满。 注意：实际位宽与 TX_FIFO_DEPTH 绑定，请根据需要适配。	

TRX_DMA_THRESHOLD

DMA 请求阈值寄存器。

Offset Address: 0x0028 Total Reset Value: 0x0000_1F0F

Bits	Access	Name	Description	Reset
31:16	-	reserved	保留。	0x0000
15:8	RW	tx_bdma_threshhold	TX DMA 的 BREQ 请求阈值。 当 TX FIFO 的 Level 低于（不包含相等）该阈值时 BREQ 将有效。 注意：配置值需要考虑 TX_FIFO_DEPTH，请根据需要适配。	0x1F
7:0	RW	rx_bdma_threshhold	RX DMA 的 BREQ 请求阈值。 当 RX FIFO 的 Level 超过（不包含相等）该阈值时 BREQ 将有效。 注意：配置值需要考虑 RX_FIFO_DEPTH，请根据需要适配。	0x0F

5.5 I2C

5.5.1 概述

I2C 模块的作用是完成 CPU 对 I2C 总线上从设备的数据读写，CPU 可以连续配置多个发送的数据和接收多个数据。I2C 总线上可挂载多个从设备。

5.5.2 功能描述

I2C 具有以下功能特点：

- 应用核集成 2 个 I2C 控制器。
- 支持 2.0 版本的 I2C 总线协议，只支持 Master 模式。
- 支持时钟同步，以及字/字节等待功能。
- 支持中断或轮询操作。
- I2C 模块支持标准地址（7bit）和扩展地址（10bit）。
- 可以工作在两种速度模式下：标准模式（100kbit/s）、快速模式（400kbit/s）。
- I2C 模块支持 General Call 功能。
- 支持 32×8bit 的收发 FIFO，支持 FIFO 使能可配置。
- I2C 模块在 APB 总线上执行 APB Slave 的功能，在 I2C 总线上作为 Master。

5.5.3 接口时序

I2C 总线共包含 2 根通讯线，其中 1 根为信号线 SDA，1 根为时钟线 SCL，具体信号描述如表 5-17 所示。

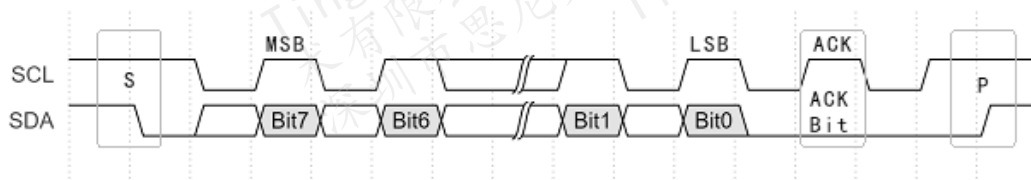
表5-17 I2C 总线描述

信号名	宽度 (bit)	方向	功能描述
SDA	1	B	数据
SCL	1	B	时钟

通讯过程中，协议规定空闲态，数据线 SDA 与时钟线 SCL 均为高电平；当时钟线 SCL 为高电平，将数据线 SDA 由高拉低为数据传输的起始信号；反之，当时钟线 SCL 为高电平，将数据线 SDA 由低拉高为数据传输的终止信号。信号示意图如图 5-5 所示。

总线传输数据为时钟线 SCL 为高电平时对应的数据线 SDA 电平，通常一次传输 8 个 bit 数据后，如果此时与从机通讯完成且正确，从机会对主机发起一个 ACK 信号表示从机的接收成功。不同的从机会有不同的帧格式，详细信息请参照所选用的从机手册说明。

图5-5 I2C 起始和终止信号示意图



5.5.4 工作方式

5.5.4.1 初始化配置

软件需要先完成 I2C 控制器的时钟和复位章节（[5.5.4.1.1 I2C 控制器的时钟和复位](#)），然后初始化配置（[5.5.4.1.2 I2C 初始化配置](#)），最后完成 I2C 控制器 Master 读写操作配置（[5.5.4.2 写操作（中断方式）](#)~[5.5.4.5 读操作（查询方式）](#)）。

5.5.4.1.1 I2C 控制器的时钟和复位

I2C 控制器被 2 个时钟驱动，内部总线时钟 pclk_i2c 和 I2C 工作时钟 clk_i2c。在初始化 I2C 之前，需要复位 I2C 模块，打开 I2C 时钟，配置 I2C 工作时钟分频，解除 I2C 模块的复位。

- 复位：软件需配置 ASUB_CRG 模块的
CRG_ACRG_SOFT_RST_RB_2[`cfg_acrg_soft_rst_i2c_n`]，复位 I2C0，或
CRG_ACRG_SOFT_RST_RB_2[`cfg_acrg_soft_rst_i2c1_n`]，复位 I2C1。
- 工作时钟使能：软件需配置 CRG 模块的
CRG_ACRG_DIV_RB_1[`cfg_acrg_i2c_div_en`]，使能 I2C0 的 clk_i2c，或
CRG_ACRG_DIV_RB_8[`cfg_acrg_i2c1_div_en`]，使能 I2C1 的 clk_i2c。
- 总线时钟使能：配置 CRG 模块的
CRG_ACRG_CLKEN_RB_0[`cfg_acrg_i2c_clken`]，使能 I2C0 的 pclk_i2c，或
CRG_ACRG_CLKEN_RB_0[`cfg_acrg_i2c1_clken`]，使能 I2C1 的 pclk_i2c。
- 解复位：软件需配置 CRG 模块的
CRG_ACRG_SOFT_RST_RB_2[`cfg_acrg_soft_rst_i2c_n`]，解除 I2C0 的复位，

或 `CRG_ACRG_SOFT_RST_RB_2[cfg_acrg_soft_rst_i2c1_n]`，解除 I2C1 的复位。

5.5.4.1.2 I2C 初始化配置

- 步骤 1 配置 `I2C_SCL_H` 和 `I2C_SCL_L`，调整 SCL 的低电平时间和高电平时间。
- 步骤 2 配置 `I2C_ICR`，清除所有中断。
- 步骤 3 配置 `I2C_CTRL[mode_ctrl]=1`，开启 TX FIFO 和 RX FIFO。
- 步骤 4 向 `I2C_CTRL[int_txfifo_over_mask]`，`I2C_CTRL[int_mask]`，`I2C_CTRL[int_ack_err_mask]`和 `I2C_CTRL[int_done_mask]`写 1，解除中断的屏蔽。
- 步骤 5 配置 `I2C_FTRPER`，毛刺过滤设置，默认 15 个时钟周期。在 SCL 为高时，SDA 电平需要保持的时间，小于该时间的 SDA 信号被认为是毛刺，不作为有效数据接收。
- 步骤 6 配置 `I2C_CTRL[i2c_en]=1`，使能 I2C。
- 步骤 7 读 `I2C_TXCOUNT` 和 `I2C_RXCOUNT`，确认 TX FIFO 和 RX FIFO 内的数据个数，如 FIFO 中有残留数据，则向 `I2C_TXCOUNT` 和 `I2C_RXCOUNT` 写入任意值都可清空 FIFO。

----结束

5.5.4.2 写操作（中断方式）

写操作步骤如下：

- 步骤 1 完成 I2C 控制器的初始化（请参见“5.5.4.1.2 I2C 初始化配置”章节）。
- 步骤 2 将从机地址写入 `I2C_TXR[i2c_txr]`。
- 步骤 3 配置 `I2C_COM[op_we]`和 `I2C_COM[op_start]`为 1，将本次操作设定为写操作并发起。
- 步骤 4 中断服务程序里查询 `I2C_SR[int_txfifo_over]`，`I2C_SR[int_txfifo_over]`表示所有数据已完成发送。
- 步骤 5 配置 `I2C_ICR[clr_int_txfifo_over]=1`，清除 `int_txfifo_over` 中断。
- 步骤 6 连续向 `I2C_TXR` 写入数据，TX FIFO 深度为 32，连续写入的数据帧数不得超过 32。
- 步骤 7 配置 `I2C_TXTIDE[txtide]`，根据需要调整 TX FIFO 的溢出阈值，TX FIFO 中的字符个数 $\leq I2C_TXTIDE[txtide]$ 时会触发 TX FIFO 溢出中断。
- 步骤 8 配置 `I2C_COM[op_we]=1`，开启写操作。

- 步骤 9 在中断服务程序中查询 `I2C_SR[int_txfifo_over]`、`I2C_SR[int_txtide]`和 `I2C_SR[int_ack_err]`。
- 步骤 10 若 `I2C_SR[int_ack_err]=1`，说明本次写操作出现错误。配置 `I2C_COM[op_stop]=1`，停止本次操作。
- 步骤 11 若 `I2C_SR[int_txtide]=1`，说明 TX FIFO 内剩余数据量低于阈值。如仍有数据需要发送，则跳转至步骤 12；无多余数据需要发送则跳转至步骤 13。
- 步骤 12 连续向 `I2C_TXR` 写入数据，写入过程中需要保证 TX FIFO 不出现上溢。过程中可查询 `I2C_TXCOUNT[txcount]`，该寄存器表示当前 TX FIFO 中有多少帧数据待发送。
- 步骤 13 查询到 I2C 上报 `I2C_SR[int_txfifo_over]`中断，表示所有数据已全部发送完成。配置 `I2C_ICR[clr_int_txfifo_over]`和 `I2C_ICR[clr_int_txtide]`为 1，清除相关中断。
- 步骤 14 配置 `I2C_COM[op_stop]=1`，停止 I2C 的业务。
- 步骤 15 轮询 `I2C_SR[int_done]`，直到其值为 1，表示总线传输完成。
- 步骤 16 配置 `I2C_ICR[clr_int_done]`，清除 `int_done` 中断。

----结束

5.5.4.3 读操作（中断方式）

读操作步骤如下：

- 步骤 1 完成 I2C 控制器的初始化（请参见“5.5.4.1.2 I2C 初始化配置”章节）。
- 步骤 2 将从机地址写入 `I2C_TXR[i2c_txr]`。
- 步骤 3 配置 `I2C_COM[op_we]`和 `I2C_COM[op_start]`为 1，将本次操作设定为写操作并发起。
- 步骤 4 中断服务程序里查询 `I2C_SR[int_txfifo_over]`，`I2C_SR[int_txfifo_over]`表示所有数据已完成发送。
- 步骤 5 配置 `I2C_SR[clr_int_txfifo_over]=1`，清除 `I2C_SR[clr_int_txfifo_over]`中断。
- 步骤 6 配置 `I2C_RXTIDE[rxtide]`，根据需要调整 RX FIFO 的溢出阈值，RX FIFO 中的字符个数 $\geq$ `I2C_RXTIDE[rxtide]`时会触发 RX FIFO 溢出中断。
- 步骤 7 配置 `I2C_COM[op_rd]=1`，开启读操作。
- 步骤 8 在中断服务程序中查询 `I2C_SR[int_rxtide]`，若该中断为 1 且数据接收尚未结束，则跳转至步骤 9；若该中断为 1 且数据已全部接收完成，则跳转至步骤 10。

- 步骤 9 读取 `I2C_RXR[i2c_rxr]`，只取走一帧数据，再次收完一帧数据后 I2C 仍会上报 `I2C_SR[int_rxtide]`，重复执行步骤 8 ~ 步骤 9，直至数据全部接收完成。
- 步骤 10 配置 `I2C_COM[op_stop]=1`，停止 I2C 的业务。
- 步骤 11 轮询 `I2C_SR[int_done]`，直到其值为 1，表示总线传输完成。
- 步骤 12 配置 `I2C_ICR[clr_int_done]`和 `I2C_ICR[clr_int_rxtide]`，清除相关中断。
- 步骤 13 读取 `I2C_RXR[i2c_rxr]`，取出全部数据。

----结束

5.5.4.4 写操作（查询方式）

请遵循以下步骤：

- 步骤 1 完成 I2C 控制器的初始化（请参见“5.5.4.1.2 I2C 初始化配置”章节）。
- 步骤 2 将从机地址写入 `I2C_TXR[i2c_txr]`。
- 步骤 3 配置 `I2C_COM[op_we]`和 `I2C_COM[op_start]`为 1，将本次操作设定为写操作并发起。
- 步骤 4 轮询 `I2C_SR[int_txfifo_over]`，`I2C_SR[int_txfifo_over]`表示所有数据已完成发送。
- 步骤 5 配置 `I2C_ICR[clr_int_txfifo_over]=1`，清除 `int_txfifo_over` 中断。
- 步骤 6 连续向 `I2C_TXR` 写入数据，TX FIFO 深度为 32，连续写入的数据帧数不得超过 32。
- 步骤 7 配置 `I2C_COM[op_we]=1`，开启写操作。
- 步骤 8 轮询 `I2C_TXCOUNT[txcount]`，根据实际情况，在合适位置连续向 `I2C_TXR` 写入数据，补充 TX 数据。重复步骤 8，直至所有 TX 数据都已写入。
- 步骤 9 轮询 `I2C_SR[int_txfifo_over]`、`I2C_SR[int_ack_err]`。若数据发送过程中出现 `I2C_SR[int_ack_err]=1`，则跳转至步骤 10。若数据发送工作中未出现 `I2C_SR[int_ack_err]`，且 `I2C_SR[int_txfifo_over]=1`，则跳转至步骤 11。
- 步骤 10 若 `I2C_SR[int_ack_err]=1`，说明本次写操作出现错误。配置 `I2C_COM[op_stop]=1`，停止本次操作。
- 步骤 11 配置 `I2C_ICR[clr_int_txfifo_over]`和 `I2C_ICR[clr_int_txtide]`为 1，清除相关中断。
- 步骤 12 配置 `I2C_COM[op_stop]=1`，停止 I2C 的业务。
- 步骤 13 轮询 `I2C_SR[int_done]`，直到其值为 1，表示总线传输完成。

步骤 14 配置 [I2C_ICR](#)[clr_int_done]，清除 int_done 中断。

----结束

5.5.4.5 读操作（查询方式）

请遵循以下步骤：

步骤 1 完成 I2C 控制器的初始化（请参见“[5.5.4.1.2 I2C 初始化配置](#)”章节）。

步骤 2 将从机地址写入 [I2C_TXR](#)[i2c_tsr]。

步骤 3 配置 [I2C_COM](#)[op_we]和 [I2C_COM](#)[op_start]为 1，将本次操作设定为写操作并发起。

步骤 4 轮询 [I2C_SR](#)[int_txfifo_over]，[I2C_SR](#)[int_txfifo_over]表示所有数据已完成发送。

步骤 5 配置 [I2C_ICR](#)[clr_int_txfifo_over]=1，清除 int_txfifo_over 中断。

步骤 6 配置 [I2C_COM](#)[op_rd]=1，开启读操作。

步骤 7 轮询 [I2C_RXCOUNT](#)[rxcount]，根据实际情况，在合适位置读取 [I2C_RXR](#)，防止 RX FIFO 溢出。重复步骤 7，直至所有数据接收完成。

步骤 8 配置 [I2C_COM](#)[op_stop]=1，停止 I2C 的业务。

步骤 9 轮询 [I2C_SR](#)[int_done]，直到其值为 1，表示总线传输完成。

步骤 10 配置 [I2C_ICR](#)[clr_int_done]和 [I2C_ICR](#)[clr_int_rxtide]，清除相关中断。

步骤 11 读取 [I2C_RXR](#)，取出所有数据。

----结束

5.5.5 寄存器概览

I2C 寄存器概览如[表 5-18](#)所示。

表5-18 I2C 寄存器概览（I2C0 的基址是 0x3000_0000，I2C1 的基址是 30000700）

偏移地址	名称	描述
0x00	I2C_CTRL	I2C 控制寄存器。
0x04	I2C_COM	I2C 模块的命令寄存器。
0x08	I2C_ICR	I2C 模块的中断清除寄存器。

偏移地址	名称	描述
0x0C	I2C_SR	I2C 模块状态寄存器。
0x10	I2C_SCL_H	I2C 总线 SCL 信号高电平周期数寄存器。
0x14	I2C_SCL_L	I2C 总线 SCL 信号低电平周期数寄存器。
0x18	I2C_TXR	I2C 发送数据寄存器。
0x1C	I2C_RXR	I2C 接收数据寄存器。
0x20	I2C_FIFOSTATUS	FIFO 状态寄存器。
0x24	I2C_TXCOUNT	发送 FIFO 数据个数寄存器。
0x28	I2C_RXCOUNT	接收 FIFO 数据个数寄存器。
0x2C	I2C_RXTIDE	接收 FIFO 的溢出阈值寄存器。
0x30	I2C_TXTIDE	发送 FIFO 的溢出阈值寄存器。
0x34	I2C_FTRPER	毛刺过滤，SCL 为高时，判断 SDA 电平持续时间，单位为 ic_clk 的时钟个数，持续时间大于该值才认为该电平为接收值。

5.5.6 寄存器描述

I2C_CTRL

I2C 控制寄存器。用于配置 I2C 使能和中断屏蔽。

Offset Address: 0x00 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:13	-	reserved	保留。	0x00000
12	RW	int_txfifo_over_mask	发送 FIFO 数据发送完成中断屏蔽。 0：屏蔽； 1：不屏蔽。	0x0

Bits	Access	Name	Description	Reset
11	RW	mode_ctrl	I2C 工作模式选择。 0: 不使用 FIFO 传输模式; 1: 使用 FIFO 传输模式。	0x0
10	RW	int_txide_mask	发送 FIFO 溢出中断屏蔽。 0: 屏蔽; 1: 不屏蔽。	0x0
9	RW	int_rxtide_mask	接收 FIFO 溢出中断屏蔽。 0: 屏蔽; 1: 不屏蔽。	0x0
8	RW	i2c_en	I2C 使能。 0: 不使能; 1: 使能。	0x0
7	RW	i2c_int_mask	I2C 中断总屏蔽。 0: 屏蔽; 1: 不屏蔽。	0x0
6	RW	int_start_mask	主机开始条件发送结束中断屏蔽。 0: 屏蔽; 1: 不屏蔽。	0x0
5	RW	int_stop_mask	主机停止条件发送结束中断屏蔽。 0: 屏蔽; 1: 不屏蔽。	0x0
4	RW	int_tx_mask	主机发送中断屏蔽。 0: 屏蔽; 1: 不屏蔽。	0x0
3	RW	int_rx_mask	主机接收中断屏蔽。 0: 屏蔽; 1: 不屏蔽。	0x0

Bits	Access	Name	Description	Reset
2	RW	int_ack_err_mask	从机 ACK 错误中断屏蔽。 0：屏蔽； 1：不屏蔽。	0x0
1	RW	int_arb_loss_mask	总线仲裁失败中断屏蔽。 0：屏蔽； 1：不屏蔽。	0x0
0	RW	int_done_mask	总线传输完成中断屏蔽。 0：屏蔽； 1：不屏蔽。	0x0

I2C_COM

I2C 模块的命令寄存器。用于配置 I2C 模块工作时命令。

说明

在系统初始化时配置或配置前，需要清除对应中断标志。I2C_COM bit[3:0]在操作结束后将自动清 0。

Offset Address: 0x04 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:5	-	reserved	保留。	0x00000000
4	RW	op_ack	主机作为接收器是否发送 ACK。 0：发送； 1：不发送。	0x0
3	RW	op_start	产生开始条件操作。 0：操作结束； 1：操作有效。	0x0
2	RW	op_rd	产生读操作。 0：操作结束；	0x0

Bits	Access	Name	Description	Reset
			1: 操作有效。	
1	RW	op_we	产生写操作。 0: 操作结束; 1: 操作有效。	0x0
0	RW	op_stop	产生停止条件操作。 0: 操作结束; 1: 操作有效。	0x0

I2C_ICR

I2C_ICR 为 I2C 模块的中断清除寄存器。

说明

新中断到来时，I2C 模块会自动将 I2C_ICR 相应位清 0。

Offset Address: 0x08 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:10	-	reserved	保留。	0x000000
9	WC	clr_int_txfifo_over	发送 FIFO 数据发送完成中断标志清除。 0: 不清除; 1: 清除。	0x0
8	WC	clr_int_txtide	发送 FIFO 溢出中断标志清除。 0: 不清除; 1: 清除。	0x0
7	WC	clr_int_rxtide	接收 FIFO 溢出中断标志清除。 0: 不清除; 1: 清除。	0x0
6	WC	clr_int_start	主机开始条件发送结束中断标志清除。	0x0

Bits	Access	Name	Description	Reset
			除。 0：不清除； 1：清除。	
5	WC	clr_int_stop	主机停止条件发送结束中断标志清除。 0：不清除； 1：清除。	0x0
4	WC	clr_int_tx	主机发送中断标志清除。 0：不清除； 1：清除。	0x0
3	WC	clr_int_rx	主机接收中断标志清除。 0：不清除； 1：清除。	0x0
2	WC	clr_int_ack_err	从机 ACK 错误中断标志清除。 0：不清除； 1：清除。	0x0
1	WC	clr_int_arb_loss	总线仲裁失败中断标志清除。 0：不清除； 1：清除。	0x0
0	WC	clr_int_done	总线传输完成中断标志清除。 0：不清除； 1：清除。	0x0

I2C_SR

I2C 模块状态寄存器。用于读取 I2C 模块工作状态。

说明

I2C_SR bit[1]表示 I2C 总线仲裁失败。当 I2C_SR bit[1]有效时，当前操作失败。在清 I2C_SR bit[1]之前，需要清除其他中断标志，然后清除 I2C_COM 或向 I2C_COM 写入新的操作命令，最后清除 I2C_SR bit[1]。

Offset Address: 0x0C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:11	-	reserved	保留。	0x000000
10	RO	int_txfifo_over	发送 FIFO 数据发送完成中断标志。 0：无中断标志产生； 1：中断标志产生。	0x0
9	RO	int_txtide	发送 FIFO 溢出中断标志。 0：无中断标志产生； 1：中断标志产生。	0x0
8	RO	int_rxtide	接收 FIFO 溢出中断标志。 0：无中断标志产生； 1：中断标志产生。	0x0
7	RO	bus_busy	总线忙。 0：空闲； 1：忙。	0x0
6	RO	int_start	主机开始条件发送结束中断标志。 0：无中断标志产生； 1：中断标志产生。	0x0
5	RO	int_stop	主机停止条件发送结束中断标志。 0：无中断标志产生； 1：中断标志产生。	0x0
4	RO	int_tx	主机发送中断标志。 0：无中断标志产生； 1：中断标志产生。	0x0

Bits	Access	Name	Description	Reset
3	RO	int_rx	主机接收中断标志。 0：无中断标志产生； 1：中断标志产生。	0x0
2	RO	int_ack_err	从机 ACK 错误中断标志。 0：无中断标志产生； 1：中断标志产生。	0x0
1	RO	int_arb_loss	总线仲裁失败中断标志。 0：无中断标志产生； 1：中断标志产生。	0x0
0	RO	int_done	总线传输完成中断标志。 0：无中断标志产生； 1：中断标志产生。	0x0

I2C_SCL_H

I2C 总线 SCL 信号高电平周期数寄存器。用于配置 I2C 模块工作时 SCL 高电平周期数。

说明

在系统初始化时配置或配置前使 I2C_CTRL bit[7]=0。

Offset Address: 0x10 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:16	-	reserved	保留。	0x0000
15:0	RW	scl_h	配置数值乘 2 等于 SCL 高电平周期数。	0x0000

I2C_SCL_L

I2C 总线 SCL 信号低电平周期数寄存器。用于配置 I2C 模块工作时 SCL 低电平周期数。

说明

在系统初始化时配置或配置前使 I2C_CTRL bit[7]=0。

Offset Address: 0x14 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:16	-	reserved	保留。	0x0000
15:0	RW	scl_l	配置数值乘 2 等于 SCL 低电平周期数。	0x0000

I2C_TXR

I2C 发送数据寄存器。用于配置 I2C 模块工作时发送数据。

说明

不使用 FIFO 模式下，发送结束后，I2C 模块不会修改 I2C_TXR 内容；使用 FIFO 模式下，写入的数据会自动载入到发送 FIFO 中保存直到该数据发送结束。

Offset Address: 0x18 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:8	-	reserved	保留。	0x000000
7:0	RW	i2c_txr	主机发送数据。	0x00

I2C_RXR

I2C 接收数据寄存器。用于主机接收从机数据。

说明

不使用 FIFO 模式下，I2C_RXR 数据在 I2C_SR bit[3]=1 时，数据有效。同时数据将保持到下一个读操作前。使用 FIFO 模式下，读取 I2C_RXR 会直接从接收 FIFO 中取数据。

Offset Address: 0x1C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:8	-	reserved	保留。	0x000000

Bits	Access	Name	Description	Reset
7:0	RO	i2c_rxr	主机接收数据。	0x00

I2C_FIFOSTATUS

FIFO 状态寄存器。

Offset Address: 0x20 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:4	-	reserved	保留。	0x00000000
3	RO	rxfe	接收 FIFO 空状态。 0: 非空; 1: 空。	0x0
2	RO	rxff	接收 FIFO 满状态。 0: 未滿; 1: 满。	0x0
1	RO	txfe	发送 FIFO 空状态。 0: 非空; 1: 空。	0x0
0	RO	txff	发送 FIFO 满状态。 0: 未滿; 1: 满。	0x0

I2C_TXCOUNT

发送 FIFO 数据个数寄存器。

Offset Address: 0x24 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
31:6	-	reserved	保留。	0x00000000
5:0	WC	txcount	读该寄存器返回发送 FIFO 中的字符数，写该寄存器（任意值）将清空发送 FIFO。	0x00

I2C_RXCOUNT

接收 FIFO 数据个数寄存器。

Offset Address: 0x28 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:6	-	reserved	保留。	0x00000000
5:0	WC	rxcount	读该寄存器返回接收 FIFO 中的字符数，写该寄存器（任意值）将清空接收 FIFO。	0x00

I2C_RXTIDE

接收 FIFO 的溢出阈值寄存器。

Offset Address: 0x2C Total Reset Value: 0x0000_0001

Bits	Access	Name	Description	Reset
31:6	-	reserved	保留。	0x00000000
5:0	RW	rxtide	设置 int_rxtide 中断的触发值。 RXFIFO 中的字符个数 ≥ I2C_RXTIDE[rxtide]时会触发接收 FIFO 溢出中断。	0x01

I2C_TXTIDE

发送 FIFO 的溢出阈值寄存器。

说明

TXFIFO 中的字符只有在成功发送后才会被移除。

Offset Address: 0x30 Total Reset Value: 0x0000_0001

Bits	Access	Name	Description	Reset
31:6	-	reserved	保留。	0x0000000
5:0	RW	txtide	设置 int_txtide 中断的触发值。 TXFIFO 中的字符个数≤ I2C_TXTIDE[txtide]时会触发发送 FIFO 溢出中断。	0x01

I2C_FTRPER

I2C_FTRPER 为毛刺过滤，SCL 为高时，判断 SDA 电平持续时间，单位为 ic_clk 的时钟个数，持续时间大于该值才认为该电平为接收值。

Offset Address: 0x34 Total Reset Value: 0x0000_000F

Bits	Access	Name	Description	Reset
31:4	-	reserved	保留。	0x0000000
3:0	RW	ftrper	默认 15 个时钟周期，SDA 需要在 SCL 为高时，电平保持的时间。	0xF

5.6 I2S/PCM

5.6.1 概述

I2S 模块是 APB 总线上的从设备、I2S 总线上的主/从设备。

5.6.2 功能描述

- 支持 Master/Slave 模式。
- 支持 TRX 的左右声道各自独立 FIFO，FIFO 规格为 32X8bit。
- 支持 8kHz/16kHz/32kHz/48kHz/96kHz，在 32 位传输模式时只支持到 48kHz。
- 支持 16/24/32 位的传输工作模式。
- 支持支持 Normal IIS 协议；不支持 TDM IIS 模式。

5.6.3 工作方式

5.6.3.1 接口信号

I2S 接口信号描述如表 5-19 所示。

表5-19 I2S 接口信号

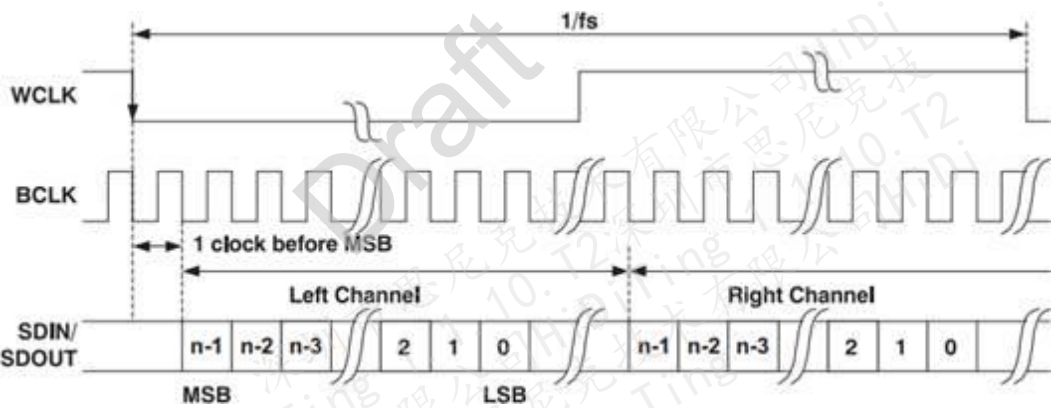
信号名	宽度 (bit)	方向	功能描述
AUDIO_AIAO_BCLK	1	O	串行时钟 SCLK，也称位时钟（BCLK），即对应数字音频的每 1bit 数据，SCLK 都有 1 个脉冲。 SCLK（Serial Clock singal）的频率=2×采样频率×采样位数。 由 Master 产生并控制。
AUDIO_AIAO_WS	1	O	左右声道选择信号 Word Select。 WS 为声道选择信号，即数据发送端所选择的声道： 0：选择左声道； 1：选择右声道。 WS 也称帧时钟，即 LRCLK（Left Right Clock）。WS 频率等于声音的采样率。由 Master 产生并控制。
AUDIO_AIAO_MCLK	1	O	Master Clock。在 I2S 接口的 ADC/DAC 系统中，除 SCK 和 WS 外，CODEC

信号名	宽度 (bit)	方向	功能描述
			(Coder/Decoder) 还需要控制器提供 MCLK (Master Clock) 。由 Master 产生并控制。
AUDIO_AIAO_TX_SD	1	O	输出数据。
AUDIO_AIAO_RX_SD	1	I	输入数据。

5.6.3.2 操作模式

I2S 操作模式如图 5-6 所示。

图5-6 I2S 操作模式



SD 是串行数据，在 I2S 中以二进制补码的形式在数据线上传输。在 WS 变化后的第 1 个 SCK 脉冲，先传输最高位 (MSB)。左声道的数据 MSB 在 WS 下降沿之后第 2 个 SCK/BCLK 上升沿有效，右声道数据的 MSB 在 WS 上升沿之后第 2 个 SCK/BCLK 上升沿有效。

5.6.3.3 应用说明

Master TX 模式

1. 初始化

步骤 1 进行时钟的初始化配置，打开 I2S 需要的源头 264M 时钟。

- 步骤 2 参考 IO_MUX，配置好 I2S 对应的 IO 复用，以及 IO 的驱动能力。
- 步骤 3 设置 `SIO_CT_SET[rst_n]` 为 0x1，解除 I2S 控制器的复位。
- 步骤 4 设置 `SIO_CT_SET[intr_en]` 为 0x1，打开 I2S 的全局中断使能。
- 步骤 5 设置 `SIO_CT_SET[tx_enable]` 为 0x1，打开 I2S TX 发送使能。
- 步骤 6 设置 `SIO_MODE[cfg_i2s_ms_mode_sel]` 为 0x0，设置为 Master 模式。
- 步骤 7 设置 `SIO_MODE[ext_rec_en]`，`SIO_MODE[sio_mode]`，`SIO_DATA_WIDTH_SET[rx_mode]`，设置 I2S 进行通信的协议模式以及位宽。
- 步骤 8 设置 `SIO_FIFO_THRESHOLD[tx_fifo_threshold]`，设置发送 FIFO 的水线阈值。
- 步骤 9 设置 `SIO_INTMASK[tx_intr]` 为 0x0，设置 TX_FIFO 中断不屏蔽。
- 步骤 10 设置地址 `0x4400_1100[12]` 为 0x1，打开 I2S 的工作时钟门控。
- 步骤 11 设置 `CFG_I2S_CRG[cfg_i2s_bclk_div_en]` 为 0x0，关闭分频器使能开关。
- 步骤 12 设置 `CFG_I2S_BCLK_DIV_NUM[cfg_i2s_bclk_div_num]`，配置 MCLK -> BCLK 的分频值。
- 步骤 13 设置 `CFG_I2S_CRG[cfg_i2s_bclk_div_en]` 为 0x1，打开分频器使能开关。
- 步骤 14 设置 `CFG_I2S_FS_DIV_NUM[cfg_i2s_fs_div_num]`，配置 BCLK -> WS 的分频值。
- 步骤 15 设置 `CFG_I2S_FS_DIV_RATIO_NUM[cfg_i2s_fs_div_ratio_num]`，配置 WS 输出占空比。
- 步骤 16 设置 `CFG_I2S_CRG[cfg_i2s_crg_clken]` 为 0x1，打开时钟门控。

----结束

2. 发送数据

- 步骤 1 CPU 接收到中断，或者读取寄存器 `SIO_INTSTATUS[tx_intr]` 为 0x1 时，此时表征 I2S TX_FIFO 中的数据不足水线阈值。
- 步骤 2 写寄存器 `SIO_I2S_LEFT_XD[tx_left_data]` 以及 `SIO_I2S_RIGHT_XD[tx_right_data]`，向 I2S 的左右声道 FIFO 中写入数据。
- 步骤 3 设置 `SIO_INTCLR[tx_intr]` 为 0x1，清除发送 FIFO 中断。
- 步骤 4 设置 `SIO_CT_SET[tx_enable]` 为 0x0，关闭 I2S TX 发送使能。

----结束

Master RX 模式

1. 初始化

- 步骤 1 进行时钟的初始化配置，打开 I2S 需要的源头 PLL 高频时钟。
- 步骤 2 参考 IO_MUX，配置好 I2S 对应的 IO 复用，以及 IO 的驱动能力。
- 步骤 3 设置 `SIO_CT_SET[rst_n]` 为 0x1，解除 I2S 控制器的复位。
- 步骤 4 设置 `SIO_CT_SET[intr_en]` 为 0x1，打开 I2S 的全局中断使能。
- 步骤 5 设置 `SIO_CT_SET[rx_enable]` 为 0x1，打开 I2S RX 接收使能。
- 步骤 6 设置 `SIO_MODE[cfg_i2s_ms_mode_sel]` 为 0x0，设置为 Master 模式。
- 步骤 7 设置 `SIO_MODE[ext_rec_en]`，`SIO_MODE[sio_mode]`，`SIO_DATA_WIDTH_SET[rx_mode]`，设置 I2S 进行通信的协议模式以及位宽。
- 步骤 8 设置 `SIO_FIFO_THRESHOLD[rx_fifo_threshold]`，设置接收 FIFO 的水线阈值。
- 步骤 9 设置 `SIO_INTMASK[rx_intr]` 为 0x0，设置 RX_FIFO 中断不屏蔽。
- 步骤 10 设置地址 `0x4400_1100[12]` 为 0x1，打开 I2S 的工作时钟门控。
- 步骤 11 设置 `CFG_I2S_CRG[cfg_i2s_bclk_div_en]` 为 0x0，关闭分频器使能开关。
- 步骤 12 设置 `CFG_I2S_BCLK_DIV_NUM[cfg_i2s_bclk_div_num]`，配置 MCLK -> BCLK 的分频值。
- 步骤 13 设置 `CFG_I2S_CRG[cfg_i2s_bclk_div_en]` 为 0x1，打开分频器使能开关。
- 步骤 14 设置 `CFG_I2S_FS_DIV_NUM[cfg_i2s_fs_div_num]`，配置 BCLK -> WS 的分频值。
- 步骤 15 设置 `CFG_I2S_FS_DIV_RATIO_NUM[cfg_i2s_fs_div_ratio_num]`，配置 WS 输出占空比。
- 步骤 16 设置 `CFG_I2S_CRG[cfg_i2s_crg_clken]` 为 0x1，打开时钟门控。

----结束

2. 接收数据

- 步骤 1 CPU 接收到中断，或者读取寄存器 `SIO_INTSTATUS[rx_intr]` 为 0x1 时，此时表征 I2S RX_FIFO 中数据量达到了水线阈值。
- 步骤 2 读取寄存器 `SIO_I2S_LEFT_RD[rx_left_data]` 与 `SIO_I2S_RIGHT_RD[rx_right_data]`，将左右声道接收 FIFO 中的数据读取出来。

步骤 3 设置 `SIO_INTCLR[rx_intr]` 为 0x1，清除接收 FIFO 中断。

步骤 4 设置 `SIO_CT_SET[rx_enable]` 为 0x0，关闭 I2S RX 接收使能。

----结束

Slave TX 模式

1. 初始化

步骤 1 进行时钟的初始化配置，打开 I2S 需要的源头 960M 时钟。

步骤 2 参考 IO_MUX，配置好 I2S 对应的 IO 复用，以及 IO 的驱动能力。

步骤 3 设置 `SIO_CT_SET[rst_n]` 为 0x1，解除 I2S 控制器的复位。

步骤 4 设置 `SIO_CT_SET[intr_en]` 为 0x1，打开 I2S 的全局中断使能。

步骤 5 设置 `SIO_CT_SET[tx_enable]` 为 0x1，打开 I2S TX 发送使能。

步骤 6 设置 `SIO_MODE[cfg_i2s_ms_mode_sel]` 为 0x1，设置为 Slave 模式。

步骤 7 设置 `SIO_MODE[ext_rec_en]`，`SIO_MODE[sio_mode]`，
`SIO_DATA_WIDTH_SET[rx_mode]`，设置 I2S 进行通信的协议模式以及位宽。

步骤 8 设置 `SIO_FIFO_THRESHOLD[tx_fifo_threshold]`，设置发送 FIFO 的水线阈值。

步骤 9 设置 `SIO_INTMASK[tx_intr]` 为 0x0，设置 TX_FIFO 中断不屏蔽。

步骤 10 设置地址 `0x4400_1100[12]` 为 0x1，打开 I2S 的工作时钟门控。

步骤 11 设置 `CFG_I2S_CRG[cfg_i2s_bclk_div_en]` 为 0x0，关闭分频器使能开关。

步骤 12 设置 `CFG_I2S_BCLK_DIV_NUM[cfg_i2s_bclk_div_num]`，配置 MCLK -> BCLK 的分频值。

步骤 13 设置 `CFG_I2S_CRG[cfg_i2s_bclk_div_en]` 为 0x1，打开分频器使能开关。

步骤 14 设置 `CFG_I2S_FS_DIV_NUM[cfg_i2s_fs_div_num]`，配置 BCLK -> WS 的分频值。

步骤 15 设置 `CFG_I2S_FS_DIV_RATIO_NUM[cfg_i2s_fs_div_ratio_num]`，配置 WS 输出占空比。

步骤 16 设置 `CFG_I2S_CRG[cfg_i2s_crg_clken]` 为 0x1，打开时钟门控。

----结束

2. 发送数据

- 步骤 1 CPU 接收到中断，或者读取寄存器 `SIO_INTSTATUS`[tx_intr]为 0x1 时，此时表征 I2S TXFIFO 中的数据不足水线阈值。
- 步骤 2 写寄存器 `SIO_I2S_LEFT_XD`[tx_left_data]以及 `SIO_I2S_RIGHT_XD`[tx_right_data]，向 I2S 的左右声道 FIFO 中写入数据。
- 步骤 3 设置 `SIO_INTCLR`[tx_intr]为 0x1，清除发送 FIFO 中断。
- 步骤 4 设置 `SIO_CT_SET`[tx_enable]为 0x0，关闭 I2S TX 发送使能。

----结束

Slave RX 模式

1. 初始化

- 步骤 1 进行时钟的初始化配置，打开 I2S 需要的源头 960M 时钟。
- 步骤 2 参考 IO_MUX，配置好 I2S 对应的 IO 复用，以及 IO 的驱动能力。
- 步骤 3 设置 `SIO_CT_SET`[rst_n]为 0x1，解除 I2S 控制器的复位。
- 步骤 4 设置 `SIO_CT_SET`[intr_en]为 0x1，打开 I2S 的全局中断使能。
- 步骤 5 设置 `SIO_CT_SET`[rx_enable]为 0x1，打开 I2S RX 接收使能。
- 步骤 6 设置 `SIO_MODE`[cfg_i2s_ms_mode_sel]为 0x1，设置为 Slave 模式。
- 步骤 7 设置 `SIO_MODE`[ext_rec_en]，`SIO_MODE`[sio_mode]，`SIO_DATA_WIDTH_SET`[rx_mode]，设置 I2S 进行通信的协议模式以及位宽。
- 步骤 8 设置 `SIO_FIFO_THRESHOLD`[rx_fifo_threshold]，设置接收 FIFO 的水线阈值。
- 步骤 9 设置 `SIO_INTMASK`[rx_intr]为 0x0，设置 RX_FIFO 中断不屏蔽。
- 步骤 10 设置地址 0x4400_1100[12]为 0x1，打开 I2S 的工作时钟门控。
- 步骤 11 设置 `CFG_I2S_CRG`[cfg_i2s_bclk_div_en]为 0x0，关闭分频器使能开关。
- 步骤 12 设置 `CFG_I2S_BCLK_DIV_NUM`[cfg_i2s_bclk_div_num]，配置 MCLK -> BCLK 的分频值。
- 步骤 13 设置 `CFG_I2S_CRG`[cfg_i2s_bclk_div_en]为 0x1，打开分频器使能开关。
- 步骤 14 设置 `CFG_I2S_FS_DIV_NUM`[cfg_i2s_fs_div_num]，配置 BCLK -> WS 的分频值。
- 步骤 15 设置 `CFG_I2S_FS_DIV_RATIO_NUM`[cfg_i2s_fs_div_ratio_num]，配置 WS 输出占空比。

步骤 16 设置 `CFG_I2S_CRG[cfg_i2s_crg_clken]` 为 0x1，打开时钟门控。

----结束

2. 接收数据

步骤 1 CPU 接收到中断，或者读取寄存器 `SIO_INTSTATUS[rx_intr]` 为 0x1 时，此时表征 I2S RX_FIFO 中数据量达到了水线阈值。

步骤 2 读取寄存器 `SIO_I2S_LEFT_RD[rx_left_data]` 与 `SIO_I2S_RIGHT_RD[rx_right_data]`，将左右声道接收 FIFO 中的数据读取出来。

步骤 3 设置 `SIO_INTCLR[rx_intr]` 为 0x1，清除接收 FIFO 中断。

步骤 4 设置 `SIO_CT_SET[rx_enable]` 为 0x0，关闭 I2S RX 接收使能。

----结束

5.6.4 寄存器概览

I2S/PCM 寄存器概览如表 5-20 所示。

表5-20 I2S/PCM 寄存器概览（基址是 0x3000_0300）

偏移地址	名称	描述
0x003C	<code>SIO_VERSION</code>	版本号与测试模式配置。
0x0040	<code>SIO_MODE</code>	模式配置寄存器。
0x0044	<code>SIO_INTSTATUS</code>	中断状态查询寄存器。
0x0048	<code>SIO_INTCLR</code>	中断清除寄存器。
0x004C	<code>SIO_I2S_LEFT_XD</code>	左声道发送数据。
0x0050	<code>SIO_I2S_RIGHT_XD</code>	右声道发送数据。
0x0054	<code>SIO_I2S_LEFT_RD</code>	左声道接收数据。
0x0058	<code>SIO_I2S_RIGHT_RD</code>	右声道接收数据。
0x005C	<code>SIO_CT_SET</code>	I2S_CT 置位寄存器。
0x0060	<code>SIO_CT_CLR</code>	I2S_CT 清除寄存器。

偏移地址	名称	描述
0x0064	SIO_FIFO_THRESHOLD	FIFO 阈值配置寄存器。
0x0068	SIO_RX_STA	RX_FIFO 深度指示。
0x006C	SIO_TX_STA	TX_FIFO 深度指示。
0x0078	SIO_DATA_WIDTH_SET	传输数据长度配置。
0x007C	SIO_I2S_START_POS	声道访问配置寄存器。
0x0080	I2S_POS_FLAG	声道访问状态寄存器。
0x0084	SIO_SIGNED_EXT	高位数据符号扩展使能寄存器。
0x0088	SIO_I2S_POS_MERGE_EN	左右声道数据的操作地址合并使能寄存器。
0x008C	SIO_INTMASK	中断屏蔽寄存器。
0x0090	CFG_I2S_CRG	时钟配置寄存器。
0x0094	CFG_I2S_BCLK_DIV_NUM	BCLK 分频比配置寄存器。
0x0098	CFG_I2S_FS_DIV_NUM	FSCLK 分频器的分频比配置寄存器。
0x009C	CFG_I2S_FS_DIV_RATIO_NUM	FSCLK 分频器的占空比配置寄存器。
0x00A0	SIO_I2S_DUAL_RX_CHN	交替读取接收 FIFO 功能寄存器。
0x00C0	SIO_I2S_DUAL_TX_CHN	交替写入发送 FIFO 功能寄存器。

5.6.5 寄存器描述

SIO_VERSION

版本号与测试模式配置。

Offset Address: 0x003C Total Reset Value: 0x0000_0013

Bits	Access	Name	Description	Reset
31:9	RO	reserved	保留。	0x000000

Bits	Access	Name	Description	Reset
8	RW	sio_loop	SIO 循环和正常模式选择。 0: 正常模式; 1: SIO 发送和接收数据环回模式。用于 SIO 的自测试。 在该模式下, 在 SIO 的对外接口处, SIO 接收串行数据线与 SIO 发送串行数据线直接相连。	0x0
7:0	RO	version	SIO 的版本号。	0x13

SIO_MODE

模式配置寄存器。

Offset Address: 0x0040 Total Reset Value: 0x0000_0080

Bits	Access	Name	Description	Reset
31:8	RO	reserved	保留。	0x000000
7	RW	cfg_i2s_ms_mode_sel	SIO 模块 MASTER/SLAVE 模式选择寄存器。 0: SLAVE 模式; 1: MASTER 模式。	0x1
6	RW	clk_edge	PCM 多路接收模式下, 采样数据时钟边沿选择。 0: 下降沿有效; 1: 上升沿有效。	0x0
5:4	RW	chn_num	多路接收的通道数选择。 00: 2 通道; 01: 4 通道; 10: 8 通道; 11: 16 通道。	0x0

Bits	Access	Name	Description	Reset
3	RW	ext_rec_en	标准模式下, I2S 接收两个声道的数据, 即左声道和右声道。PCM 只接收一个声道的数据。 多路接收模式下, 对于 I2S 或 PCM, 接收通道数可以配置。该模式下, 通道的数据宽度必须为 8bit 或 16bit。 0: 标准的 I2S 或 PCM 接收模式; 1: 扩展的 I2S 或 PCM 多路接收模式。	0x0
2	RO	rxws_select	选择 RX 模块的 ws 来自 txws_dac 还是 rxws_adc。	0x0
1	RW	pcm_mode	PCM 时序模式。 0: 标准模式; 1: 自定义模式。	0x0
0	RW	sio_mode	PCM/I2S 模式选择。 0: I2S 模式; 1: PCM 模式。	0x0

SIO_INTSTATUS

中断状态查询寄存器。

Offset Address: 0x0044 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:6	RO	reserved	保留。	0x00000000
5	RO	tx_left_fifo_under	I2S 模式下, 左声道发送 FIFO 下溢中断状态。PCM 模式下无效。	0x0

Bits	Access	Name	Description	Reset
			0: 未产生中断; 1: 已产生中断。	
4	RO	tx_right_fifo_under	I2S 模式下, 右声道发送 FIFO 下溢中断状态。PCM 模式下, PCM 发送 FIFO 下溢标志。 0: 未产生中断; 1: 已产生中断。	0x0
3	RO	rx_left_fifo_over	I2S 模式下, 左声道接收 FIFO 上溢中断状态。PCM 模式下无效。 0: 未产生中断; 1: 已产生中断。	0x0
2	RO	rx_right_fifo_over	I2S 模式下, 右声道接收 FIFO 上溢中断状态。PCM 模式下, PCM 接收 FIFO 下溢标志。 0: 未产生中断; 1: 已产生中断。	0x0
1	RO	tx_intr	发送 FIFO 低于阈值中断状态。 0: 未产生中断; 1: 已产生中断。	0x0
0	RO	rx_intr	接收 FIFO 高于阈值中断状态。 0: 未产生中断; 1: 已产生中断。	0x0

SIO_INTCLR

中断清除寄存器。

Offset Address: 0x0048 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:6	RO	reserved	保留。	0x00000000
5	WO	tx_left_fifo_under_clr	I2S 模式下, 左声道发送 FIFO 下溢中断清除。PCM 模式下无效。 0: 不清除; 1: 清除。	0x0
4	WO	tx_right_fifo_under_clr	I2S 模式下, 右声道发送 FIFO 下溢中断清除。PCM 模式下, PCM 发送 FIFO 下溢中断清除。 0: 不清除; 1: 清除。	0x0
3	WO	rx_left_fifo_over_clr	I2S 模式下, 左声道接收 FIFO 上溢中断清除。PCM 模式下无效。 0: 不清除; 1: 清除。	0x0
2	WO	rx_right_fifo_over_clr	I2S 模式下, 右声道接收 FIFO 上溢中断清除。PCM 模式下, PCM 接收 FIFO 下溢中断清除。 0: 不清除; 1: 清除。	0x0
1	WO	tx_intr_clr	发送 FIFO 低于阈值中断清除。 0: 不清除;	0x0

Bits	Access	Name	Description	Reset
			1: 清除。	
0	WO	rx_intr_clr	接收 FIFO 高于阈值中断清除。 0: 不清除; 1: 清除。	0x0

SIO_I2S_LEFT_XD

左声道发送数据。

Offset Address: 0x004C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	WO	tx_left_data	I2S 左声道发送数据。	0x00000000

SIO_I2S_RIGHT_XD

右声道发送数据。

Offset Address: 0x0050 Total Reset Value: 0x00000000

Bits	Access	Name	Description	Reset
31:0	WO	tx_right_data	I2S 模式[31:0]为右声道发送数据; PCM 模式[15:0]为 PCM 发送数据。	0x00000000

SIO_I2S_LEFT_RD

左声道接收数据。

Offset Address: 0x0054 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RO	rx_left_data	I2S 左声道接收数据。	0x00000000

SIO_I2S_RIGHT_RD

右声道接收数据。

Offset Address: 0x0058 Total Reset Value: 0x00000000

Bits	Access	Name	Description	Reset
31:0	RO	rx_right_data	I2S 模式[31:0]为右声道接收数据; PCM 模式[15:0]为 PCM 接收数据。	0x00000000

SIO_CT_SET

I2S_CT 置位寄存器。

Offset Address: 0x005C Total Reset Value: 0x0000_8000

Bits	Access	Name	Description	Reset
31:16	RO	reserved	保留。	0x0000
15	RW	rst_n	I2S/PCM 通道复位，低电平有效。 它对 I2S/PCM 接收和发送模块（包括 FIFO）进行复位，因此发送和接收 FIFO 状态寄存器会变为 0。但不会复位 CPU 接口寄存器模块。 复位置位寄存器： 0：复位寄存器不置位，复位保持原始状态； 1：复位寄存器置位，即复位不生效。 和 SIO_CT_CLR 寄存器搭配使用。	0x1
14	RW	intr_en	中断全局使能置位寄存器。	0x0

Bits	Access	Name	Description	Reset
			0: 不置位 (保持原状态) ; 1: 置位 (使能) 。 和 SIO_CT_CLR 寄存器搭配使用。	
13	RW	rx_enable	接收通道使能置位寄存器。 0: 不置位 (保持原状态) ; 1: 置位 (使能) 。 和 SIO_CT_CLR 寄存器搭配使用。	0x0
12	RW	tx_enable	发送通道使能置位寄存器。 0: 不置位 (保持原状态) ; 1: 置位 (使能) 。 和 SIO_CT_CLR 寄存器搭配使用。	0x0
11	RW	rx_fifo_disable	接收 FIFO 禁止置位寄存器。 0: 不置位 (保持原状态) ; 1: 置位 (禁止) 。 和 SIO_CT_CLR 寄存器搭配使用。	0x0
10	RW	tx_fifo_disable	发送 FIFO 禁止置位寄存器。 0: 不置位 (保持原状态) ; 1: 置位 (禁止) 。 和 SIO_CT_CLR 寄存器搭配使用。	0x0
9	RW	rx_data_merge_en	数据接收合并使能置位寄存器, 只在 I2S 模式下且数据位宽为 16bit 时有效。 0: 不置位 (保持原状态) ; 1: 置位 (使能) 。	0x0

Bits	Access	Name	Description	Reset
			和 SIO_CT_CLR 寄存器搭配使用。 数据接收合并使能寄存器使能时，左右声道数据拼成一个 32bit 数据在 FIFO 中进行存储，左声道 16bit 数据占据高 16 位，右声道 16bit 数据占据低 16 位。这样可以提高 FIFO 的利用率，以提供更长时间的缓冲能力。 CPU 从接收 FIFO 中读数据时，按照如下顺序：从左声道 FIFO 读出一个 32bit 数据（左右声道各 16bit 合并成的一个 32bit 数据），再从右声道 FIFO 读出一个 32bit 数据，如此反复。	
8	RW	tx_data_merge_en	数据发送合并使能置位寄存器，只在 I2S 模式下且数据位宽为 16bit 时有效。 0：不置位（保持原状态）； 1：置位（使能）。 和 SIO_CT_CLR 寄存器搭配使用。 数据发送使能寄存器使能时，左右声道数据拼成一个 32bit 数据在 FIFO 中进行存储，左声道 16bit 数据占据高 16 位，右声道 16bit 数据占据低 16 位。这样可以提高 FIFO 的利用率，以提供更长时间的缓冲能力。 在这种情况下，CPU 向发送 FIFO 中写入数据时，按照如	0x0

Bits	Access	Name	Description	Reset
			下顺序：向左声道 FIFO 写入一个 32bit 数据（左右声道各 16bit 合并成的一个 32bit 数据），再向右声道 FIFO 写入一个 32bit 数据，如此反复。	
7:4	RO	reserved	接收 FIFO 阈值。 当 rx_right_depth>(rx_fifo_threshold + 1)时，报接收中断和 DMA 请求。	0x0
3:0	RO	reserved	发送 FIFO 阈值。 当 tx_right_depth<(tx_fifo_threshold+1)时，报发送中断和 DMA 请求。	0x0

SIO_CT_CLR

I2S_CT 清除寄存器。

Offset Address: 0x0060 Total Reset Value: 0x0000_8000

Bits	Access	Name	Description	Reset
31:16	RO	reserved	保留。	0x0000
15	RW	rst_n_clr	I2S/PCM 通道复位，低电平有效。 它对 I2S/PCM 接收和发送模块（包括 FIFO）进行复位，因此发送和接收 FIFO 状态寄存器会变为 0。但不会复位 CPU 接口寄存器模块。 复位清除寄存器： 0：复位寄存器不清除，复位保持原始状态；	0x1

Bits	Access	Name	Description	Reset
			1: 复位寄存器清除, 即复位生效。	
14	RW	intr_en_clr	中断全局使能清除寄存器。 0: 不清除 (保持原始状态) ; 1: 清除 (禁止) 。	0x0
13	RW	rx_enable_clr	接收通道使能清除寄存器。 0: 不清除 (保持原始状态) ; 1: 清除 (禁止) 。	0x0
12	RW	tx_enable_clr	发送通道使能清除寄存器。 0: 不清除 (保持原始状态) ; 1: 清除 (禁止) 。	0x0
11	RW	rx_fifo_disable_clr	接收 FIFO 禁止清除寄存器。 0: 不清除 (保持原始状态) ; 1: 清除 (使能) 。	0x0
10	RW	tx_fifo_disable_clr	发送 FIFO 禁止清除寄存器。 0: 不清除 (保持原始状态) ; 1: 清除 (使能) 。	0x0
9	RW	rx_data_merge_en_clr	数据接收合并使能清除寄存器, 只在 I2S 模式下且数据位宽为 16bit 时有效。 0: 不清除 (保持原始状态) ; 1: 清除 (禁止) 。	0x0
			数据接收合并使能寄存器使能时, 左右声道数据拼成一个	

Bits	Access	Name	Description	Reset
			32bit 数据在 FIFO 中进行存储，左声道 16bit 数据占据高 16 位，右声道 16bit 数据占据低 16 位。这样可以提高 FIFO 的利用率，以提供更长时间的缓冲能力。 CPU 从接收 FIFO 中读数据时，按照如下顺序：从左声道 FIFO 读出一个 32bit 数据（左右声道各 16bit 合并成的一个 32bit 数据），再从右声道 FIFO 读出一个 32bit 数据，如此反复。	
8	RW	tx_data_merge_en_clr	数据发送合并使能清除寄存器，只在 I2S 模式下且数据位宽为 16bit 时有效。 0：不清除（保持原始状态）； 1：清除（禁止）。 数据发送合并使能寄存器使能时，左右声道数据拼成一个 32bit 数据在 FIFO 中进行存储，左声道 16bit 数据占据高 16 位，右声道 16bit 数据占据低 16 位。这样可以提高 FIFO 的利用率，以提供更长时间的缓冲能力。 在这种情况下，CPU 向发送 FIFO 中写入数据时，按照如下顺序：向左声道 FIFO 写入一个 32bit 数据（左右声道各 16bit 合并成的一个 32bit 数据），再向右声道 FIFO 写入一个 32bit 数据，如此反复。	0x0

Bits	Access	Name	Description	Reset
7:4	RO	reserved	接收 FIFO 阈值。 当 rx_right_depth?(rx_fifo_threshold+1)时, 报接收中断和 DMA 请求。	0x0
3:0	RO	reserved	发送 FIFO 阈值。 当 tx_right_depth<(tx_fifo_threshold+1)时, 报发送中断和 DMA 请求。	0x0

SIO_FIFO_THRESHOLD

FIFO 阈值配置寄存器。

Offset Address: 0x0064 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:16	RO	reserved	保留。	0x0000
15:8	RW	rx_fifo_threshold	接收 FIFO 阈值。 当 rx_right_depth?(rx_fifo_threshold+1)时, 报接收中断和 DMA 请求。	0x00
7:0	RW	tx_fifo_threshold	发送 FIFO 阈值。 当 tx_right_depth<(tx_fifo_threshold+1)时, 报发送中断和 DMA 请求。	0x00

SIO_RX_STA

RX_FIFO 深度指示。

Offset Address: 0x0068 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:16	RO	reserved	保留。	0x0000
15:8	RO	rx_left_depth	左声道接收 FIFO 深度指示。 只在 I2S 模式下有效。	0x00
7:0	RO	rx_right_depth	I2S 模式下, 为右声道接收 FIFO 深度指示。 PCM 模式下, 为 PCM 接收 FIFO 深度指示。	0x00

SIO_TX_STA

TX_FIFO 深度指示。

Offset Address: 0x006C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:16	RO	reserved	保留。	0x0000
15:8	RO	tx_left_depth	左声道发送 FIFO 深度指示。 只在 I2S 模式下有效。	0x00
7:0	RO	tx_right_depth	I2S 模式下, 为右声道发送 FIFO 深度指示。 PCM 模式下, 为 PCM 发送 FIFO 深度指示。	0x00

SIO_DATA_WIDTH_SET

传输数据长度配置。

Offset Address: 0x0078 Total Reset Value: 0x0000_0009

Bits	Access	Name	Description	Reset
31:6	RO	reserved	保留。	0x00000000
5:3	RW	rx_mode	接收数据长度配置位。	0x1

Bits	Access	Name	Description	Reset
			000: 8bit; 001: 16bit; 010: 18bit; 011: 20bit; 100: 24bit; 101: 32bit; 110~111: 保留。 对于 I2S 模式, 支持 16/18/20/24/32bit; 对于 PCM 模式, 支持 8/16bit; 对于多路接收, 两种模式只支持 8/16bit。	
2:0	RW	tx_mode	发送数据长度配置位。 对于 I2S 模式 000: 保留; 001: 16bit; 010: 18bit; 011: 20bit; 100: 24bit; 101: 32bit; 110~111: 保留。 000: 8bit 001: 16bit; 010: 32bit; 011: 64bit; 100: 128bit; 101~111: 保留。	0x1

SIO_I2S_START_POS

声道访问配置寄存器。

Offset Address: 0x007C Total Reset Value: 0x00000000

Bits	Access	Name	Description	Reset
31:2	RO	reserved	保留。	0x00000000
1	RW	start_pos_write	写发送 FIFO 时。 0: 从左声道开始访问; 1: 从右声道开始访问。	0x0
0	RW	start_pos_read	读接收 FIFO 时。 0: 从左声道开始访问; 1: 从右声道开始访问。	0x0

I2S_POS_FLAG

声道访问状态寄存器。

Offset Address: 0x0080 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:2	RO	reserved	保留。	0x00000000
1	RO	next_pos_write	写发送 FIFO 时。 0: 下一次从左声道开始访问; 1: 下一次从右声道开始访问。	0x0
0	RO	next_pos_read	读接收 FIFO 时。 0: 下一次从左声道开始访问; 1: 下一次从右声道开始访问。	0x0

SIO_SIGNED_EXT

高位数据符号扩展使能寄存器。

Offset Address: 0x0084 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RW	signed_ext_en	高位数据符号扩展使能。 0: 禁止; 1: 使能。	0x0

SIO_I2S_POS_MERGE_EN

左右声道数据的操作地址合并使能寄存器。

Offset Address: 0x0088 Total Reset Value: 0x00000000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RW	merge_en	在 I2S 模式下, 对左右声道数据的操作地址合并使能。 0: 禁止; 1: 使能。	0x0

SIO_INTMASK

中断屏蔽寄存器。

Offset Address: 0x008C Total Reset Value: 0x00000000

Bits	Access	Name	Description	Reset
31:6	RO	reserved	保留。	0x00000000
5	RW	tx_left_fifo_under_mask	I2S 模式下, 左声道发送 FIFO 下溢中断屏蔽。PCM 模式下无效。	0x0

Bits	Access	Name	Description	Reset
			0: 不屏蔽; 1: 屏蔽。	
4	RW	tx_right_fifo_under_mask	I2S 模式下, 右声道发送 FIFO 下溢中断屏蔽。PCM 模式下, PCM 发送 FIFO 下溢中断屏蔽。 0: 不屏蔽; 1: 屏蔽。	0x0
3	RW	rx_left_fifo_over_mask	I2S 模式下, 左声道接收 FIFO 上溢中断屏蔽。PCM 模式下无效。 0: 不屏蔽; 1: 屏蔽。	0x0
2	RW	rx_right_fifo_over_mask	I2S 模式下, 右声道接收 FIFO 上溢中断屏蔽。PCM 模式下, PCM 接收 FIFO 下溢中断屏蔽。 0: 不屏蔽; 1: 屏蔽。	0x0
1	RW	tx_intr_mask	发送 FIFO 低于阈值中断屏蔽。 0: 不屏蔽; 1: 屏蔽。	0x0
0	RW	rx_intr_mask	接收 FIFO 高于阈值中断屏蔽。 0: 不屏蔽; 1: 屏蔽。	0x0

CFG_I2S_CRG

时钟配置寄存器。

Offset Address: 0x0090 Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
15:5	RO	reserved	保留。	0x000
4	RW	cfg_i2s_fs_sel	SIO_BCLK 时钟相位需静态配置。 0: FSCLK 原相位; 1: FS_CLK 反相, 波形取反。	0x0
3	RO	reserved	无效	0x0
2	RW	cfg_i2s_bclk_sel	SIO_BCLK 时钟相位,需静态配置。 0: BLK 原相位; 1: BCLK 反相, 波形取反。	0x0
1	RW	cfg_i2s_crg_clocken	SIO_BCLK 时钟门控寄存器。 0: 关闭 BCLK; 1: 开启 BCLK。	0x0
0	RW	cfg_i2s_bclk_div_en	SIO 模块 bclk_div_en, BCLK 分频器使能信号。	0x0

CFG_I2S_BCLK_DIV_NUM

BCLK 分频比配置寄存器。

Offset Address: 0x0094 Total Reset Value: 0x0001

Bits	Access	Name	Description	Reset
15:7	RO	reserved	保留。	0x000
6:0	RW	cfg_i2s	SIO 模块 BCLK 分频器的分频	0x01

Bits	Access	Name	Description	Reset
		_bclk_div_num	比。 注意：在配置分频比之前需要先配置 CFG_I2S_CRG<cfg_i2s_bclk_div_en>为 0，配置完分频比后，再配置 CFG_I2S_CRG<cfg_i2s_bclk_div_en>为 1。	

CFG_I2S_FS_DIV_NUM

FSCLK 分频器的分频比配置寄存器。

Offset Address: 0x0098 Total Reset Value: 0x0080

Bits	Access	Name	Description	Reset
15:10	RO	reserved	保留。	0x00
9:0	RW	cfg_i2s_fs_div_num	SIO 模块 FSCLK 分频器的分频比。	0x080

CFG_I2S_FS_DIV_RATIO_NUM

FSCLK 分频器的占空比配置寄存器。

Offset Address: 0x009C Total Reset Value: 0x0001

Bits	Access	Name	Description	Reset
15:10	RO	reserved	保留。	0x00
9:0	RW	cfg_i2s_fs_div_ratio_num	SIO 模块 FSCLK 分频器的占空比。	0x001

SIO_I2S_DUAL_RX_CHN

交替读取接收 FIFO 功能寄存器。

Offset Address: 0x00A0 Total Reset Value: 0x00000000

Bits	Access	Name	Description	Reset
31:0	RO	rx_data	接收到的数据。 结合 SIO_I2S_START_POS<start_pos_read>配置使用，写入此寄存器内容，相当于交替读取左右接收 FIFO。	0x00000000

SIO_I2S_DUAL_TX_CHN

交替写入发送 FIFO 功能寄存器。

Offset Address: 0x00C0 Total Reset Value: 0x00000000

Bits	Access	Name	Description	Reset
31:0	WO	tx_data	发送的数据。 结合 SIO_I2S_START_POS<start_pos_write>配置使用，写入此寄存器内容，相当于交替写入左右发送 FIFO。	0x00000000

5.6.6 注意事项

WS 从 MCLK 经过两个分频比 (aiao_fsclk_div 和 aiao_bclk_div) 分频而来，
 $WS = MCLK / (FSCLK_DIV \times BCLK_DIV)$

BCLK 从 MCLK 经过 aiao_bclk_div 分频而来， $BCLK = MCLK / BCLK_DIV$ 。

两个分频比的配置要注意：

- 一定要保证 WS 与采样频率一致。
- 尽量满足 “ $BCLK = 2 \times \text{采样频率} \times \text{采样位数}$ ” 的公式，如不满足，则需要保证以下两点要求：
 - $BCLK / (2 \times \text{采样频率})$ 要大于采样位数。
 - CODEC 的 BCLK 也要与 IIS 输出的 BCLK 保持一致。

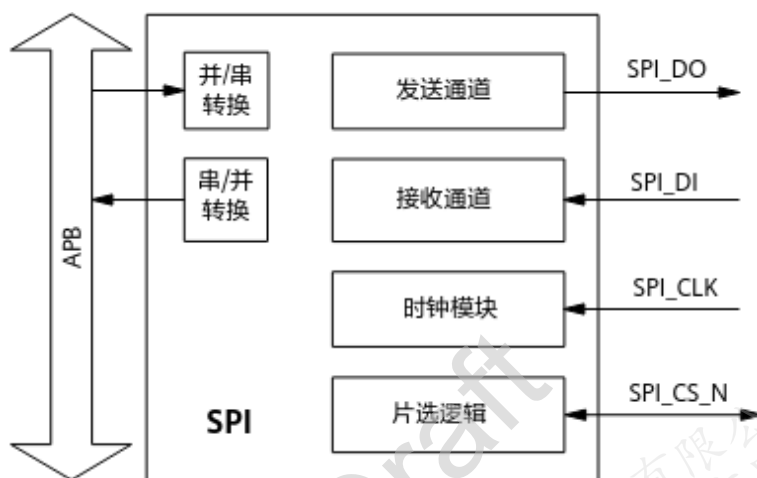
5.7 SPI

5.7.1 概述

SPI 实现数据的串并、并串转换，可以作为 Master 或 Slave 与外部设备进行同步串行通信（外围设备必须支持 SPI 帧格式）。

芯片的 SPI 工作参考时钟为 198MHz。SPI 功能框图如图 5-7 所示。

图5-7 SPI 功能框图



5.7.2 功能描述

SPI 具有以下功能特点：

- 支持 2 个 SPI 控制器。
- 支持接口时钟频率可编程。
 - 作为主设备：最大支持接口时钟频率的 8 分频，即 24.75MHz。
 - 作为从设备：最大支持接口时钟频率为 16MHz。
- 支持的 FIFO 规格：收/发分开的字宽为 128×16bit 的 FIFO（发送 FIFO 和接收 FIFO 各 1 个）。
- 支持 SPI 帧格式，分为以下三种：
 - Motorola 帧格式
 - TI 帧格式
 - National Microwire 帧格式
- 串行数据帧长度可编程：4bit ~ 16bit。

- 支持发送 FIFO 中断、接收 FIFO 中断、接收超时中断和接收 FIFO 溢出中断独立屏蔽。
- 内部提供环回测试模式。
- 支持 DMA 操作，但不支持作为 DMA 的流控设备。

5.7.3 接口时序

SPI 总线共包含 4 根线，其中 2 根为数据线，1 根为时钟线，1 根为片选线。具体信号描述如表 5-21 所示。

表5-21 SPI 接口信号描述

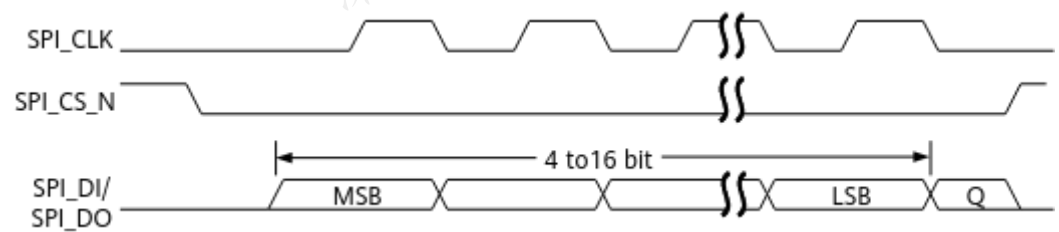
信号名	宽度 (bit)	方向	功能描述
RXD	1	I	输入数据。
TXD	1	O	输出数据。
CK	1	B	时钟信号。
CSN	1	B	片选信号，低有效。

按照 Motorola 帧格式，不同工作状态的通讯时序详细说明如下。

SPO=0、SPH=0

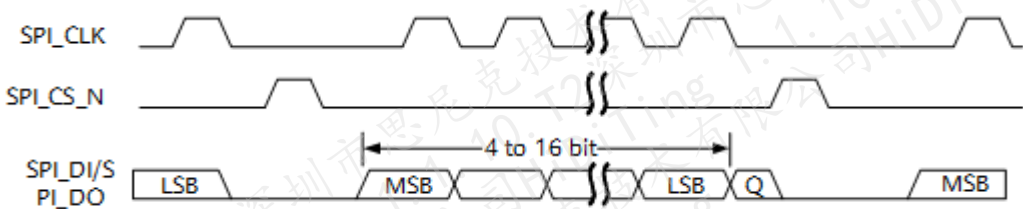
SPI 单帧帧格式如图 5-8 所示。

图5-8 SPI 单帧帧格式 (SPO=0、SPH=0)



SPI 连续帧帧格式如图 5-9 所示。

图5-9 SPI 连续帧格式 (SPO=0、SPH=0)



在该模式下，当 SPI 处于空闲状态时：

- SPI_CLK 信号设置为低电平。
- SPI_CS_N 信号设置为高电平。
- 发送数据线 SPI_DO 强制为低电平。

当 SPI 处于使能状态，而且发送 FIFO 内有有效数据时，设置 SPI_CS_N 信号为低电平表示开始传输数据。此时使能 Slave 的数据放入 Master 的输入 SPI_DI。半个 SPI_CLK 时钟周期之后，有效的 Master 数据传输到 SPI_DO。此时 Master 和 Slave 数据都已经有效，SPI_CLK 管脚在接下来的半个 SPI_CLK 时钟周期之后变为高电平。数据在 SPI_CLK 时钟的上升沿被捕获，在时钟的下降沿被传送。

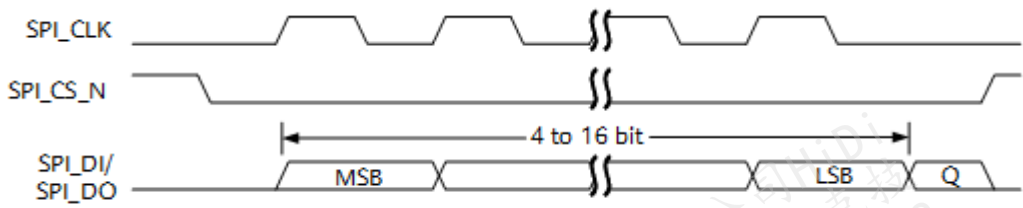
如果传输单个 word (4bit~16bit)，当捕捉到最后 1bit 数据时，SPI_CS_N 在接下来的 1 个 SPI_CLK 时钟周期之后恢复为高电平。

如果是连续传输，SPI_CS_N 信号在每个 word (4bit~16bit) 传输之间会拉高 1 个时钟周期。这是因为 SPH 为 0 时，Slave 选择管脚会固定其内部串行设备寄存器的数据，使它不会变化。因此在连续传输时，主设备必须在每个 word (4bit~16bit) 传输之间将 SPI_CS_N 信号拉高。连续传输结束时，SPI_CS_N 在捕捉到最后 1bit 后的 1 个 SPI_CLK 时钟周期之后恢复为高电平。

SPO=0、SPH=1

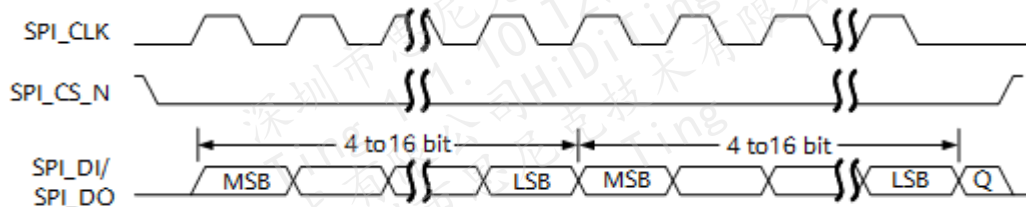
SPI 单帧帧格式如图 5-10 所示。

图5-10 SPI 单帧帧格式 (SPO=0、SPH=1)



SPI 连续帧格式如图 5-11 所示。

图5-11 SPI 连续帧格式 (SPO=0、SPH=1)



在该模式下，当 SPI 处于空闲状态时：

- SPI_CLK 信号设置为低电平。
- SPI_CS_N 信号设置为高电平。
- 发送数据线 SPI_DO 强制为低电平。

当 SPI 为使能状态，且发送 FIFO 内有有效数据时，设置 SPI_CS_N 信号为低电平表示开始传输数据。此时 Slave 的数据立刻发送到 Master 的接收数据线 SPI_DI。半个 SPI_CLK 时钟周期之后，Master 的有效数据在自身的传输线上有效，同时 SPI_CLK 产生第一个上升沿。数据在 SPI_CLK 时钟的下降沿被捕获，在时钟的上升沿被传送。

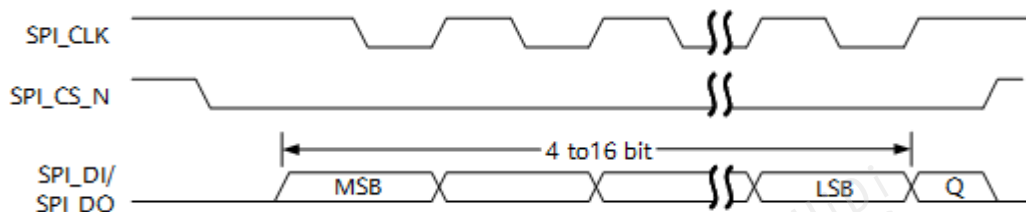
如果传输单个 word (4bit ~ 16bit)，当捕捉到最后 1bit 数据时，SPI_CS_N 在接下来的 1 个 SPI_CLK 时钟之后恢复为高电平。

如果是连续传输，在传输数据 word (4bit ~ 16bit) 之间 SPI_CS_N 保持为低电平。连续传输结束时，SPI_CS_N 在最后 1bit 捕获后的 1 个 SPI_CLK 时钟之后恢复为高电平。

SPO=1、SPH=0

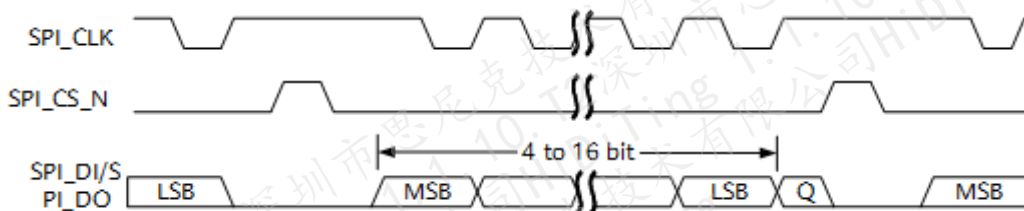
SPI 单帧格式如图 5-12 所示。

图5-12 SPI 单帧格式 (SPO=1、SPH=0)



SPI 连续帧格式如图 5-13 所示。

图5-13 SPI 连续帧格式 (SPO=1、SPH=0)



在该模式下，当 SPI 处于空闲状态时：

- SPI_CLK 信号设置为高电平。
- SPI_CS_N 信号设置为高电平。
- 发送数据线 SPI_DO 强制为低电平。

当 SPI 为使能状态，且发送 FIFO 内有有效数据时，设置 SPI_CS_N 信号为低电平表示开始传输数据。此时 Slave 的数据立刻发送到 Master 的接收数据线 SPI_DI。半个 SPI_CLK 周期之后，Master 的有效数据传送到 SPI_DO。再过半个 SPI_CLK 时钟周期之后，SPI_CLK Master 管脚设置为低电平，这表示数据在 SPI_CLK 时钟的下降沿被捕获，在 SPI_CLK 时钟的上升沿被传送。

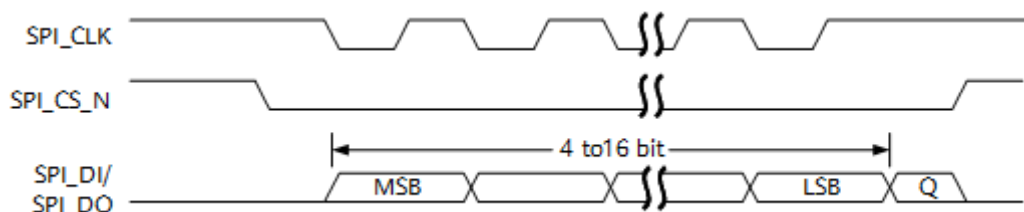
如果传输单个 word (4bit ~ 16bit)，当捕捉到最后 1bit 数据时，SPI_CS_N 在接下来的 1 个 SPI_CLK 时钟之后恢复为高电平。

如果是连续传输，SPI_CS_N 信号在每个 word (4bit ~ 16bit) 传输之间必须拉高。这是因为当 SPH 为 0 时，Slave 选择管脚固定其内部串行设备寄存器的数据，使它不会变化。SPI_CS_N 在捕捉到最后 1bit 数据后的 1 个 SPI_CLK 时钟周期之后恢复为高电平。

SPO=1、SPH=1

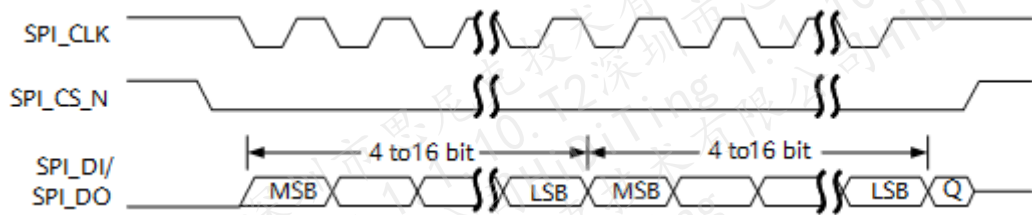
SPI 单帧帧格式如图 5-14 所示。

图5-14 SPI 单帧帧格式 (SPO=1、SPH=1)



SPI 连续帧帧格式如图 5-15 所示。

图5-15 SPI 连续帧格式 (SPO=1、SPH=1)



在该模式下，当 SPI 处于空闲状态时：

- SPI_CLK 信号设置为高电平。
- SPI_CS_N 信号设置为高电平。
- 发送数据线 SPI_DO 强制为低电平。

当 SPI 为使能状态，且发送 FIFO 内有有效数据时，设置 SPI_CS_N Master 信号为低电平表示开始传输数据。此时 Slave 的数据立刻发送到 Master 的接收数据线 SPI_DI。半个 SPI_CLK 时钟周期之后，Master 数据在自身的传输线上有效，同时时钟 SPI_CLK 产生第 1 个下降沿。数据在 SPI_CLK 时钟的上升沿被捕获，在时钟的下降沿被传送。

如果传输单个 word (4bit ~ 16bit)，SPI_CS_N 在传输的最后 1bit 捕获后的 1 个 SPI_CLK 时钟周期之后恢复为高电平。

如果是连续传输，SPI_CS_N 信号始终保持为低电平。SPI_CS_N 在捕获到最后 1bit 后的 1 个 SPI_CLK 时钟周期之后恢复到高状态。对于连续传输，SPI_CS_N 在传输过程中一直保持为低电平，结束方式与单个传输方式相同。

5.7.4 工作方式

工作模式

SPI 既可作为 MASTER 也可作为 SLAVE，配置 SPI_MS_SELECT[spi_ms_select]可选择工作模式。

SPI 支持 SPI/SSP/MICROWIRE 协议，配置 SPI_CTRA[prs]可选择协议。

中断处理

SPI 有 6 个中断，所有中断都是独立中断源、可屏蔽、高电平有效。

- SPI_MST_INTR

多主机中断，当 SPI（指代本芯片的 SPI，后文均按此处理）做 MASTER，串行总线上存在另一个 MASTER 选择 SPI 做 SLAVE 时上报此中断。读 SPI_MSTICR 或读 SPI_ICR 可清除该中断。

- SPI_TX_OVERFLOW_INTR
TX FIFO 已经为满，仍向 TX FIFO 写入数据时上报此中断。读 SPI_TFOFIER 或读 SPI_ICR 可清除该中断。
- SPI_TX_EMPTY_INTR
TX FIFO 内数据帧数不超过阈值时上报该中断，可配置 SPI_TWLR[twl]调整阈值。该中断说明 TX FIFO 内的数据个数较少，如有数据需要发送则应及时填入 TX FIFO。
- SPI_RX_UNDERFLOW_INTR
当 RX FIFO 已经为空，仍读取 RX DATA 时上报此中断。读 SPI_RFFICR 或读 SPI_ICR 可清除该中断。
- SPI_RX_OVERFLOW_INTR
当 RX FIFO 已经为满，串行总线仍在传入数据时上报此中断。读 SPI_RXOICR 或读 SPI_ICR 可清除该中断。
- SPI_RX_FULL_INTR
当 RX FIFO 内的数据个数超过阈值时上报该中断。配置 SPI_RWLR[rwl]可调整该阈值。该中断说明 RX FIFO 内数据量较多，要保证 RX FIFO 不溢出则需要及时从 RX FIFO 取走数据。

6 个原始中断可以通过 SPI_INSR 寄存器查询；也可以通过配置 SPI_INMAR 相应位域进行中断屏蔽，屏蔽后的中断可以通过 SPI_RAINSR 寄存器查询。

初始化

初始化步骤如下：

步骤 1 向寄存器 SPI_ER[start_en]写“0”，禁止 SPI。

步骤 2 配置 SPI_CTRA：

- [prs]，选择协议，可选 SPI/SSP/MICROWIRE。
- [trsm]，选择传输模式，该寄存器不影响传输本身，只用于指示接收或发送数据是否有效。
- [dfs32]，配置数据帧长度。
- [cfs16]，配置控制帧长度，仅在 MICROWIRE 协议下有效。
- [scpol]，调整时钟极性，仅在 SPI 协议下有效。

- [scph], 调整时钟相位, 仅在 SPI 协议下有效。
- [ssn_te], 配置片选信号翻转使能。
- [soe], 配置 SLAVE 输出使能, 仅在 SPI 做 SLAVE 时有效。

步骤 3 配置 SPI_CTRB[nrdf], 配置数据帧数量。

步骤 4 配置 SPI_BRS[FRDV], 配置时钟分频, $F_{\text{clk_out}} = F_{\text{spi_clk}} / \text{FRDV}$, 仅在 SPI 做 MASTER 时有效。

步骤 5 配置 SPI_TWLR[twl], 调整 TX FIFO 的阈值。

步骤 6 配置 SPI_RWLR[rwl], 调整 RX FIFO 的阈值。

步骤 7 配置 SPI_INMAR 为 0, 解除所有中断的屏蔽。

步骤 8 配置 SPI_TDER[txdde], 调整 TXD 驱动沿。

步骤 9 配置 SPI_RSDR[rxdsd], 调整 RXD 采样延迟。

----结束

查询方式下的数据传输

具体步骤如下:

步骤 1 向寄存器 SPICR1[sse]写 “1”, 使能 SPI。

步骤 2 将需发送的数据连续写到寄存器 SPIDR。

步骤 3 轮询寄存器 SPISR, 直到 SPISR[bsy]为 0 (表示总线不忙)、SPISR[tfe]为 1 (表示发送 FIFO 已空)、SPISR[rne]为 1 (表示接收 FIFO 非空)。

步骤 4 读出数据, 需保证读空接收 FIFO (可通过查询 SPISR[rne]得到)。

说明

Motorola SPI/PI 同步串行接口的全双工特性, 每发一个数据就接收一个数据, 即使只需发送数据, 也需要清空接收 FIFO。

若期望发送单帧, 在步骤 2 中, 软件须保证仅写入 1 个有效 Byte/Half-word。

若期望发送连续帧, 在步骤 2 中, 软件须保证有效的 Byte/Half-word 被连续写入发送 FIFO, 一旦发送 FIFO 不为空, SPI 开始发送, 直到发送 FIFO 为空, 结束连续帧。若非连续写入发送 FIFO, 在中途, 发送 FIFO 可能空, SPI 提前结束连续帧, 导致连续帧被截断。

步骤 5 向寄存器 SPICR1[sse]写 “0”, 禁止 SPI。

----结束

中断方式下的数据传输

具体步骤如下：

步骤 1 向寄存器 SPICR1[sse]写“1”，使能 SPI。

步骤 2 将需发送的数据连续写到寄存器 SPIDR。

步骤 3 等待中断 SPIRXINTR，读出数据。循环直到读出所有数据。

说明

Motorola SPI/TI 同步串行接口的全双工特性，每发一个数据就接收一个数据，即使只需发送数据，也需要清空接收 FIFO。

若期望发送单帧，在步骤 2 中，软件须保证仅写入 1 个有效 Byte/Half-word。

若期望发送连续帧，在步骤 2 中，软件须保证有效的 Byte/Half-word 被连续写入发送 FIFO，一旦发送 FIFO 不为空，SPI 开始发送，直到发送 FIFO 为空，结束连续帧。若非连续写入发送 FIFO，在中途，发送 FIFO 可能空，SPI 提前结束连续帧，导致连续帧被截断。

步骤 4 向寄存器 SPICR1[sse]写“0”，禁止 SPI。

----结束

5.7.4.1 SPI/SSP 协议+MASTER 模式数据收发配置

请按照如下流程配置：

步骤 1 完成初始化配置。

步骤 2 向寄存器 SPI_ER[start_en]写“1”，使能 SPI，SPI 使能之后才可将 TX 数据写入 FIFO。

步骤 3 连续写入 SPI_DRNM，将数据提前写入 TX FIFO，防止发送过程中断流。TX FIFO 深度为 128，注意写入数据量不超过 TX FIFO 深度，防止溢出。

步骤 4 配置 SPI_SLENR[ssef]=1，使能 SLAVE，开始数据收发。若为中断模式，则跳转至步骤 5。若为查询模式，则跳转至步骤 6。

步骤 5 中断服务程序中：

- 查询 SPI_INSR[tfeis]，中断上报则说明 TX FIFO 内数据已发送一部分，查询 SPI_TLR[ttf] (TX FIFO 内目前的数据数量)，连续写入 SPI_DRNM 直至 TX FIFO 为满，不可溢出。

- 查询 `SPI_INSR[rfuis]`，中断上报则说明 RX FIFO 内数据量较多。读取 `SPI_RLR[rfl]` (RX FIFO 内目前的数据量)，基于 RX FIFO 内的数据量连续读取 `SPI_DRNM`，从 RX FIFO 中取出数据，直到 RX FIFO 为空，不可溢出。
- 如无数据待发送，也无数据待收取，则查询 `SPI_WSR[tfe]`，`SPI_WSR[sbf]`和 `SPI_WSR[rfne]`。直到 `SPI_WSR[sbf]=0`，表示 SPI 无数据正在传输；
`SPI_WSR[rfne]=0`，表示 RX FIFO 为空；`SPI_WSR[tfe]=1`，表示 TX FIFO 为空。
跳转至步骤 7。

步骤 6 轮询以下寄存器：

- 轮询 `SPI_TLR[tf]`，根据实际情况，TX FIFO 内数据量低于预期时连续写入 `SPI_DRNM` 直至 TX FIFO 为满。
- 查询 `SPI_RLR[rfl]`，根据实际情况，RX FIFO 内数据量高于预期时连续读取 `SPI_DRNM` 直至 RX FIFO 为空。
- 如无数据待发送，也无数据待收取，则查询 `SPI_WSR[tfe]`，`SPI_WSR[sbf]`和 `SPI_WSR[rfne]`。直到 `SPI_WSR[sbf]=0`，表示 SPI 无数据正在传输；
`SPI_WSR[rfne]=0`，表示 RX FIFO 为空；`SPI_WSR[tfe]=1`，表示 TX FIFO 为空。
跳转至步骤 7。

步骤 7 配置 `SPI_SLENR[ssef]=0`，结束本次数据收发。

步骤 8 向寄存器 `SPI_ER[start_en]`写“0”，禁止 SPI。

----结束

5.7.4.2 MICROWIRE 协议+MASTER 模式数据收发配置

请按照如下流程配置：

步骤 1 完成初始化配置。

步骤 2 配置 `SPI_MCR`：

- `[mtrc]`：定义 Microwire 串行协议的数据传输方向。
- `[mss]`：定义 Microwire 传输是否是连续的。
- `[mh]`：定义是否去需要握手。

步骤 3 向寄存器 `SPI_ER[start_en]`写“1”，使能 SPI，SPI 使能之后才可将 TX 数据写入 FIFO。

步骤 4 连续写入 `SPI_DRNM`，将数据提前写入 TX FIFO，防止发送过程中断流。TX FIFO 深度为 128，注意写入数据量不超过 TX FIFO 深度，防止溢出。

步骤 5 配置 `SPI_SLENR`[ssef]=1，使能 SLAVE，开始数据收发。若为中断模式，则跳转至步骤 5。若为查询模式，则跳转至步骤 6。

步骤 6 中断服务程序中：

- 查询 `SPI_INSR`[tfeis]，中断上报则说明 TX FIFO 内数据已发送一部分，查询 `SPI_TLR`[tfl] (TX FIFO 内目前的数据数量)，连续写入 `SPI_DRNM` 直至 TX FIFO 为满，不可溢出。
- 查询 `SPI_INSR`[rfuis]，中断上报则说明 RX FIFO 内数据量较多。读取 `SPI_RLR`[rfl] (RX FIFO 内目前的数据量)，基于 RX FIFO 内的数据量连续读取 `SPI_DRNM`，从 RX FIFO 中取出数据，直到 RX FIFO 为空，不可溢出。
- 如无数据待发送，也无数据待收取，则查询 `SPI_WSR`[tfe]，`SPI_WSR`[sbf]和 `SPI_WSR`[rfne]。直到 `SPI_WSR`[sbf]=0，表示 SPI 无数据正在传输；
`SPI_WSR`[rfne]=0，表示 RX FIFO 为空；`SPI_WSR`[tfe]=1，表示 TX FIFO 为空。
跳转至步骤 8。

步骤 7 轮询以下寄存器：

- 轮询 `SPI_TLR`[tfl]，根据实际情况，TX FIFO 内数据量低于预期时连续写入 `SPI_DRNM` 直至 TX FIFO 为满。
- 查询 `SPI_RLR`[rfl]，根据实际情况，RX FIFO 内数据量高于预期时连续读取 `SPI_DRNM` 直至 RX FIFO 为空。
- 如无数据待发送，也无数据待收取，则查询 `SPI_WSR`[tfe]，`SPI_WSR`[sbf]和 `SPI_WSR`[rfne]。直到 `SPI_WSR`[sbf]=0，表示 SPI 无数据正在传输；
`SPI_WSR`[rfne]=0，表示 RX FIFO 为空；`SPI_WSR`[tfe]=1，表示 TX FIFO 为空。
跳转至步骤 8。

步骤 8 配置 `SPI_SLENR`[ssef]=0，结束本次数据收发。

步骤 9 向寄存器 `SPI_ER`[start_en]写“0”，禁止 SPI。

----结束

5.7.4.3 SPI/SSP 协议+SLAVE 模式数据收发配置

请按照如下流程配置：

步骤 1 完成初始化配置。

- 步骤 2 向寄存器 `SPI_ER[start_en]` 写 “1”，使能 SPI，SPI 使能之后才可将 TX 数据写入 FIFO。
- 步骤 3 连续写入 `SPI_DRNM`，将数据提前写入 TX FIFO，防止发送过程中断流。TX FIFO 深度为 128，注意写入数据量不超过 TX FIFO 深度，防止溢出。
- 步骤 4 等待 MASTER 发起访问，若为中断模式，则跳转至步骤 5。若为查询模式，则跳转至步骤 6。
- 步骤 5 中断服务程序中：
- 查询 `SPI_INSR[tfeis]`，中断上报则说明 TX FIFO 内数据已发送一部分，查询 `SPI_TLR[tf]` (TX FIFO 内目前的数据数量)，连续写入 `SPI_DRNM` 直至 TX FIFO 为满，不可溢出。
 - 查询 `SPI_INSR[rfois]`，中断上报则说明 RX FIFO 内数据量较多。读取 `SPI_RLR[rfl]` (RX FIFO 内目前的数据量)，基于 RX FIFO 内的数据量连续读取 `SPI_DRNM`，从 RX FIFO 中取出数据，直到 RX FIFO 为空，不可溢出。
 - 如无数据待发送，也无数据待收取，则查询 `SPI_WSR[tfe]`，`SPI_WSR[sbf]` 和 `SPI_WSR[rfe]`。直到 `SPI_WSR[sbf]=0`，表示 SPI 无数据正在传输；
`SPI_WSR[rfe]=0`，表示 RX FIFO 为空；`SPI_WSR[tfe]=1`，表示 TX FIFO 为空。
跳转至步骤 7。
- 步骤 6 轮询以下寄存器：
- 轮询 `SPI_TLR[tf]`，根据实际情况，TX FIFO 内数据量低于预期时连续写入 `SPI_DRNM` 直至 TX FIFO 为满。
 - 查询 `SPI_RLR[rfl]`，根据实际情况，RX FIFO 内数据量高于预期时连续读取 `SPI_DRNM` 直至 RX FIFO 为空。
 - 如无数据待发送，也无数据待收取，则查询 `SPI_WSR[tfe]`，`SPI_WSR[sbf]` 和 `SPI_WSR[rfe]`。直到 `SPI_WSR[sbf]=0`，表示 SPI 无数据正在传输；
`SPI_WSR[rfe]=0`，表示 RX FIFO 为空；`SPI_WSR[tfe]=1`，表示 TX FIFO 为空。
跳转至步骤 7。
- 步骤 7 向寄存器 `SPI_ER[start_en]` 写 “0”，禁止 SPI。

----结束

5.7.4.4 MICROWIRE 协议+SLAVE 模式数据收发配置

请按照如下流程配置：

步骤 1 完成初始化配置。

步骤 2 配置 `SPI_MCR`：

- `[mtrc]`：定义 Microwire 串行协议的数据传输方向。
- `[mss]`：定义 Microwire 传输是否是连续的。
- `[mh]`：定义是否去需要握手。

步骤 3 向寄存器 `SPI_ER[start_en]`写“1”，使能 SPI，SPI 使能之后才可将 TX 数据写入 FIFO。

步骤 4 连续写入 `SPI_DRNM`，将数据提前写入 TX FIFO，防止发送过程中断流。TX FIFO 深度为 128，注意写入数据量不超过 TX FIFO 深度，防止溢出。

步骤 5 等待 MASTER 发起访问，若为中断模式，则跳转至步骤 5。若为查询模式，则跳转至步骤 6。

步骤 6 中断服务程序中：

- 查询 `SPI_INSR[tfeis]`，中断上报则说明 TX FIFO 内数据已发送一部分，查询 `SPI_TLR[tfll]` (TX FIFO 内目前的数据数量)，连续写入 `SPI_DRNM` 直至 TX FIFO 为满，不可溢出。
- 查询 `SPI_INSR[rfeis]`，中断上报则说明 RX FIFO 内数据量较多。读取 `SPI_RLR[rfrl]` (RX FIFO 内目前的数据量)，基于 RX FIFO 内的数据量连续读取 `SPI_DRNM`，从 RX FIFO 中取出数据，直到 RX FIFO 为空，不可溢出。
- 如无数据待发送，也无数据待收取，则查询 `SPI_WSR[tfe]`，`SPI_WSR[sbf]`和 `SPI_WSR[rfe]`。直到 `SPI_WSR[sbf]=0`，表示 SPI 无数据正在传输；`SPI_WSR[rfe]=0`，表示 RX FIFO 为空；`SPI_WSR[tfe]=1`，表示 TX FIFO 为空。跳转至步骤 8。

步骤 7 轮询以下寄存器：

- 轮询 `SPI_TLR[tfll]`，根据实际情况，TX FIFO 内数据量低于预期时连续写入 `SPI_DRNM` 直至 TX FIFO 为满。
- 查询 `SPI_RLR[rfrl]`，根据实际情况，RX FIFO 内数据量高于预期时连续读取 `SPI_DRNM` 直至 RX FIFO 为空。
- 如无数据待发送，也无数据待收取，则查询 `SPI_WSR[tfe]`，`SPI_WSR[sbf]`和 `SPI_WSR[rfe]`。直到 `SPI_WSR[sbf]=0`，表示 SPI 无数据正在传输；`SPI_WSR[rfe]=0`，表示 RX FIFO 为空；`SPI_WSR[tfe]=1`，表示 TX FIFO 为空。跳转至步骤 8。

步骤 8 向寄存器 `SPI_ER[start_en]` 写 “0”，禁止 SPI。

----结束

5.7.4.5 DMA 配置流程

数据发送

数据发送步骤如下：

步骤 1 配置 `SPI_DCR[tden]`=0，关闭 TX DMA 使能。

步骤 2 配置 `SPI_DTDL[tdl]`，调整 TX_BREQ 上报阈值，当 TX FIFO 内的数据量不超过该阈值时 SPI 将向 DMA 上报 TX_BREQ。

步骤 3 配置 `SPI_DCR[tden]`=1，打开 TX DMA 使能。

步骤 4 配置 DMA 数据通道，包括数据传输源地址、目的地址、数据传输个数、传输类型等参数。具体配置请参见“2.6 DMA”章节相关描述。

步骤 5 通过 DMA 中断上报或状态查询，判断数据是否发送完成，如果完成则关闭 SPI 的 DMA 发送功能，具体请参见“2.6 DMA”章节相关描述。

----结束

数据接收

数据接收的步骤如下：

步骤 1 配置 `SPI_DCR[rden]`=0，关闭 RX DMA 使能。

步骤 2 配置 `SPI_DRDL[rld]`，调整 RX_BREQ 上报阈值，当 RX FIFO 内的数据量超过该阈值时将向 DMA 上报 RX_BREQ。

步骤 3 配置 `SPI_DCR[rden]`=1，打开 RX DMA 使能。

步骤 4 配置 DMA 数据通道，包括数据传输源地址、目的地址、数据传输个数、传输类型等参数。具体配置请参见“2.6 DMA”章节相关描述。

步骤 5 通过 DMA 中断上报或状态查询，判断数据是否接收完成，如果完成则关闭 SPI 的 DMA 接收功能，具体请参见“2.6 DMA”章节相关描述。

----结束

5.7.5 寄存器概览

SPI 寄存器概览如表 5-22 所示。

表5-22 SPI 寄存器概览 (SPI0 基址是 0x3000_0100, SPI1 基址是 0x0x3000_0200)

偏移地址	名称	描述
0x0000	SPI_ER	SPI 使能寄存器。
0x0004	SPI_CTRA	SPI 控制寄存器 A。
0x0008	SPI_CTRB	SPI 控制寄存器 B。
0x0010	SPI_MCR	Microwire 控制寄存器。
0x0014	SPI_BRS	波特率控制寄存器。
0x0018	SPI_DCR	DMA 控制寄存器。
0x001C	SPI_DRDL	DMA 接收流水线寄存器。
0x0020	SPI_DTDL	DMA 发送流水线寄存器。
0x0024	SPI_RSDR	RXD 采样延迟寄存器。
0x0028	SPI_TDER	TXD 驱动边沿寄存器。
0x002C	SPI_DRNM	SPI 数据寄存器。
0x00BC	SPI_RAINSR	原始中断状态寄存器。
0x00C0	SPI_INSR	中断状态寄存器。
0x00C4	SPI_INMAR	中断屏蔽寄存器。
0x00C8	SPI_SLENR	Slave 使能寄存器。
0x00CC	SPI_TWLR	发送流水线寄存器。
0x00D0	SPI_TLR	发送数据量查询寄存器。
0x00D4	SPI_TFOFIER	TX FIFO 中断清除寄存器。
0x00D8	SPI_RWLR	接收流水线寄存器。
0x00DC	SPI_RLR	接收数据量查询寄存器。

偏移地址	名称	描述
0x00E0	SPI_RFFICR	RX FIFO Underflow 中断清除寄存器。
0x00E4	SPI_WSR	状态寄存器。
0x00E8	SPI_MSTICR	多 master 中断清除寄存器。
0x00EC	SPI_ID	版本信息寄存器。
0x00F0	SPI_RSVD	保留寄存器。
0x00F4	SPI_RXOICR	RX FIFO Overflow 中断清除寄存器。
0x00F8	SPI_ICR	中断清除寄存器。
0x00FC	SPI_APB.CG	APB 强制时钟门控控制寄存器。

5.7.6 寄存器描述

SPI_ER

SPI_ER 为 SPI 使能寄存器。

Offset Address: 0x0000 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	-	reserved	保留。	0x00000000
0	RW	start_en	SPI 使能。 使能或禁用全部 SPI 操作。禁用时，所有串行传输立即挂起，TX FIFO 和 RX FIFO 缓存被清空，ssi_sleep 输出被置位（延迟后），通知系统此时可安全地停止 SPI 时钟，以节省功耗；使能时，某些 SPI 控制寄存器内容不可更改。	0x0

Bits	Access	Name	Description	Reset
			0: 禁止串行传输; 1: 使能串行传输。	

SPI_CTRA

SPI_CTRA 为 SPI 控制寄存器 A。

Offset Address: 0x0004 Total Reset Value: 0x0000_E004

Bits	Access	Name	Description	Reset
31:24	-	reserved	保留。	0x00
23:22	RW	prs	帧格式。选择传输数据所使用的串行协议。 00 (MOTOROLA_SPI) : 摩托罗拉协议; 01 (TEXAS_SSP) : TI 协议; 10 (NS_MICROWIRE) : National Microwire 协议; 11 (RESERVED) : 保留。	0x0
21:20	RO	enhff	SPI 帧格式。 00 (STD_ENHFF) : 标准 SPI 帧格式; 01 (DUAL_ENHFF) : 双线 SPI 帧格式; 10 (QUAD_ENHFF) : 4 线 SPI 帧格式; 11 (OCTAL_ENHFF) : 8 线 SPI 帧格式。 注意: 本芯片 SPI 为单线, 故本寄存器恒为 00。	0x0
19:18	RW	trsm	传输模式。	0x0

Bits	Access	Name	Description	Reset
			选择串行通信传输模式。该寄存器不影响传输本身，只用于指示接收或发送数据是否有效。 transmit-only 模式：SPI 接收到的外部器件发送的数据无效，数据不会存储在 RX FIFO 内，且会被下个传输所覆盖。 receive-only 模式：发送的数据无效。第一次写入 TX FIFO 后，相同的数据会在传输持续时间内重复发送。 transmit&receive 模式：发送与接收数据均有效。传输会持续到 TX FIFO 为空。接收到了外部器件数据被存储于可被 host 处理器读取的 RX FIFO 内。 eeprom-read 模式：当发送控制数据时，接收数据无效。当全部控制数据发送至 EEPROM 后，接收数据开始有效而发送数据变为无效。该模式下，TX FIFO 内全部数据均被视为控制数据。该模式仅在 SPI 被配置为主器件时才有效。 00：收发模式 (Transmit & Receive) ； 01：只发模式 (Transmit Only) ； 10：只收模式 (Receive Only) ； 11：EEPROM 读取模式 (EEPROM Read) 。	
17:13	RW	dfs32	32bit 传输模式的数据帧长度。该寄存器仅在 SSI_MAX_XFER_SIZE 配置为 32 时生效。 当数据帧长度配置为小于 32bit，接收	0x07

Bits	Access	Name	Description	Reset
			逻辑自动将数据向右调整，RX FIFO 的高位 bit 被零填充。设计者需保证发送数据写入 TX FIFO 前同样经过了高位零填充处理。发送逻辑在发送数据时会忽略未使用的高位 bit。 0x03 (FRAME_04BITS) : 4bit 串行数据传输； 0x04 (FRAME_05BITS) : 5bit 串行数据传输； 0x05 (FRAME_06BITS) : 6bit 串行数据传输； 0x0D (FRAME_14BITS) : 14bit 串行数据传输； 0x0E (FRAME_15BITS) : 15bit 串行数据传输； 0x0F (FRAME_16BITS) : 16bit 串行数据传输； 0x10 (FRAME_17BITS) : 17bit 串行数据传输； 0x11 (FRAME_18BITS) : 18bit 串行数据传输； 0x12 (FRAME_19BITS) : 19bit 串行数据传输； 0x1D (FRAME_30BITS) : 30bit 串行数据传输； 0x1E (FRAME_31BITS) : 31bit 串行数据传输； 0x1F (FRAME_32BITS) : 32bit 串行数据传输。	

Bits	Access	Name	Description	Reset
12:9	RO	dfs16	数据帧长度。该寄存器仅在 SSI_MAX_XFER_SIZE 配置为 16 时生效。若 SSI_MAX_XFER_SIZE 配置为 32，写入该寄存器无任何影响。 选择数据帧长度，当数据帧长度配置为小于 16bit，接收逻辑自动将数据向右调整，RX FIFO 的高位 bit 被零填充。设计者需保证发送数据写入 TX FIFO 前同样经过了高位零填充处理。发送逻辑在发送数据时会忽略未使用的高位。 0x3 (FRAME_04BITS) : 4bit 串行数据传输； 0x4 (FRAME_05BITS) : 5bit 串行数据传输； 0x5 (FRAME_06BITS) : 6bit 串行数据传输； 0xD (FRAME_14BITS) : 14bit 串行数据传输； 0xE (FRAME_15BITS) : 15bit 串行数据传输； 0xF (FRAME_16BITS) : 16bit 串行数据传输。 注意：本芯片 SSI_MAX_XFER_SIZE 配置为 32，故此寄存器无效。	0x0
8:5	RW	cfs16	控制帧长度。选择 Microwire 协议格式的控制字长度。 0x0 (SIZE_01_BIT) : 1bit 控制帧长度； 0x1 (SIZE_02_BIT) : 2bit 控制帧	0x0

Bits	Access	Name	Description	Reset
			长度; 0x2 (SIZE_03_BIT) : 3bit 控制帧长度; 0xD (SIZE_14_BIT) : 14bit 控制帧长度; 0xE (SIZE_15_BIT) : 15bit 控制帧长度; 0xF (SIZE_16_BIT) : 16bit 控制帧长度。	
4	RW	scpol	选择时钟极性。 仅当帧格式 (SPI_CTRA[prs]) 设为摩托罗拉协议时有效。用于选择非激活状态下串行时钟的极性 (当 SPI master 未传输数据时)。 0 (SCLK_LOW) : 非激活状态时钟为低; 1 (SCLK_HIGH) : 非激活状态时钟为高。	0x0
3	RW	scph	串行时钟相位。 仅当帧格式 (SPI_CTRA[prs]) 设为摩托罗拉协议时有效。该寄存器控制串行时钟与 slave 选择信号之间的关系。 当 SCPH=0, 数据在串行时钟第一个边沿被采样, 当 SCPH=1, 串行时钟在 slave 选择信号激活后开始翻转, 数据在第二个时钟边沿被采样。 0 (SCPH_MIDDLE) : 串行时钟在第一个数据位中间翻转; 1 (SCPH_START) : 串行时钟在第	0x0

Bits	Access	Name	Description	Reset
			一个数据位的起始位置翻转。	
2	RW	ssn_te	Slave 选择信号翻转使能。 当时钟相位 (SPI_CTRA[scph]) 设为 0, 该寄存器控制 slave 选择信号 (ss*_n)数据传输间隙的表现。 0: ss*_n 信号将在数据传输期间保持为低, sclk 信号也将持续翻转; 1: ss*_n 信号线将在连续的数据帧间翻转且串行时钟 (sclk) 会在 ss*_n 为高时保持默认值。 注意: 该寄存器仅在 SSI_SCPH0_SSTOGGLE 设为 1 时生效。	0x1
1	RW	srlt	移位寄存器环。 仅用于测试。若使能, 发送移位寄存器的输出将被连接至接收移位寄存器的输入。 当 SPI 配置为从器件, 回环模式所使用的 ss_in_n 与 ssi_clk 必须由外部器件提供。 0 (NORMAL_MODE): 常规模式; 1 (TESTING_MODE): 测试模式, Tx&Rx 移位寄存器互联。	0x0
0	RW	soe	Slave 输出使能。 仅在 SPI 配置为从器件有效。当配置为主器件, 该寄存器无任何功能。 该寄存器使能或禁止 Slave 器件输出的 ssi_oe_n 信号的设置。当 SLV_OE=1, ssi_oe_n 不会被激活。 当 ssi_oe_n 控制 Slave 器件 TXD 信号的三态 buffer 时, 若 SLV_OE=1,	0x0

Bits	Access	Name	Description	Reset
			TXD 输出将呈现高阻态。 该功能在主器件以广播模式发送数据时非常有效，仅有一个从器件会在主器件的 rxd 数据线上回送数据。 0 (ENABLED) : Slave 输出使能; 1 (DISABLED) : 禁止 Slave 输出。	

SPI_CTRB

SPI_CTRB 为 SPI 控制寄存器 B。

Offset Address: 0x0008 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:16	-	reserved	保留。	0x0000
15:0	RW	nrdf	数据帧数量。 当 TRSM=10 或 TRSM=11, 该寄存器设置 SPI 连续接收的数据帧数量。SPI 持续接收数据直到数据帧个数等于该寄存器加 1, 最大允许 64KB 的连续数据接收。 当 SPI 配置为从器件, 传输将在 Slave 被选中时一直进行, 因此, 该寄存器在 SSI_IS_MASTER=0 时无任何作用。	0x0000

SPI_MCR

SPI_MCR 为 Microwire 控制寄存器。

Offset Address: 0x0010 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
31:3	-	reserved	保留。	0x00000000
2	RW	mtrc	Microwire 控制, 定义 Microwire 串行协议的数据传输方向。 0: 接收数据, SPI MacroCell 从外部串行器件接收数据; 1: 发送数据, SPI MacroCell 向外部串行器件发送数据。	0x0
1	RW	mss	Microwire 传输模式。定义 Microwire 传输是否是连续的。 0: 非连续 Microwire 传输, 发送或接收每个数均需要 1 个控制字; 1: 连续 Microwire 传输, 一个数据块发送或接收仅需要 1 个控制字。	0x0
0	RW	mh	Microwire 握手使能。仅在 SPI 被配置为 Master 器件时有效, 当被配置为 Slave 器件时该寄存器无意义。用于使能或禁止 Microwire 协议的 busy/ready 握手接口。当使能时, SPI 在最后 1bit 数据传输完成之后, SR 寄存器的 Busy 状态清除之前, 检查目标 Slave 器件的 ready 状态。 0: 握手接口禁用; 1: 握手接口使能。	0x0

SPI_BRS

SPI_BRS 为波特率控制寄存器。

Offset Address: 0x0014 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:16	-	reserved	保留。	0x0000
15:0	RW	frdv	时钟分频器。 该寄存器最低 bit 位被强制设为 0，不受寄存器写入的影响。保证该寄存器的值永远是偶数。若该寄存器设为 0，SPI 时钟输出将被禁止。SPI 时钟频率按如下公式计算： $F_{sclk_out} = F_{ssi_clk} / FRDV$ FRDV 是 2 到 65534 之间的任意偶数。例如： $F_{ssi_clk} = 3.6864\text{MHz}, FRDV = 2$ $F_{sclk_out} = 3.6864 / 2 = 1.8432\text{MHz}$	0x0000

SPI_DCR

SPI_DCR 为 DMA 控制寄存器。

Offset Address: 0x0018 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:2	-	reserved	保留。	0x00000000
1	RW	rden	接收 DMA 使能。 0：接收 DMA 禁用； 1：接收 DMA 使能。	0x0
0	RW	tden	发送 DMA 使能。 0：发送 DMA 禁用； 1：接收 DMA 使能。	0x0

SPI_DRDL

SPI_DRDL 为 DMA 接收流水线寄存器。

Offset Address: 0x001C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:7	-	reserved	保留。	0x0000000
6:0	RW	rdl	接收流水线。 该寄存器控制接收逻辑在何时发起 DMA 请求。 watermark level = DMARDL+1。 dma_rx_req 将在 RX FIFO 内数据量大于等于该寄存器加 1 时产生 (且要求 RDEN=1) 。	0x00

SPI_DTDL

SPI_DTDL 为 DMA 发送流水线寄存器。

Offset Address: 0x0020 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:7	-	reserved	保留。	0x0000000
6:0	RW	tdl	发送流水线。 该寄存器控制发送逻辑在何时发起 DMA 请求。 watermark level = DMATDL。 dma_tx_req 将在 TX FIFO 内有效数据量等于或小于该值时产生 (且要求 TDEN=1) 。	0x00

SPI_RSDR

SPI_RSDR 为 RXD 采样延迟寄存器。

Offset Address: 0x0024 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:8	-	reserved	保留。	0x000000
7:0	RW	rxdsd	RXD 采样延迟。 该寄存器用于延迟 RXD 输入端口的采样时间，延迟单位为 ssi_clk 周期。 注意：若该寄存器所配置的值超出内部移位寄存器深度 (SSI_RX_DLY_SR_DEPTH) ，零延迟采样将作用在 RXD 端口。	0x00

SPI_TDER

SPI_TDER 为 TXD 驱动边沿寄存器。

Offset Address: 0x0028 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:8	-	reserved	保留。	0x000000
7:0	RO	txdde	TXD 驱动沿。 该值决定了发送数据的驱动沿。 该寄存器最大值= (SPI_BRS/2) - 1。	0x00

SPI_DRNM

SPI_DRNM 为 SPI 数据寄存器。

Offset Address: 0x002C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:0	RW	datar	写入/读出数据。写入该寄存器必须将数据右对齐，读取的数据已被自动右对齐。 读操作：读出 RX FIFO 缓存的数据； 写操作：数据存入 TX FIFO。	0x00000000

SPI_RAINSR

SPI_RAINSR 为原始中断状态寄存器。

Offset Address: 0x00BC Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:16	-	reserved	保留。	0x0000
15	RO	mmcris	多 Master 连接原始中断状态。该寄存器在 SPI 被配置为 Slave 器件时不可见。 0：中断未产生（SPI_INMAR[mmcim]作用前）； 1：中断产生（SPI_INMAR[mmcim]作用前）。	0x0
14:9	-	reserved	保留。	0x00
8	RO	tfofris	TX FIFO 溢出原始中断状态。 0：中断未产生（SPI_INMAR[tfofim]作用前）； 1：中断产生（SPI_INMAR[tfofim]作用前）。	0x0
7	RO	tferis	TX FIFO 为空原始中断。 0：中断未产生（SPI_INMAR[tfeim]作用前）；	0x0

Bits	Access	Name	Description	Reset
			1: 中断产生 (SPI_INMAR[tfeim]作用前)。	
6:3	-	reserved	保留。	0x0
2	RO	rfufri	RX FIFO 下溢原始中断状态。 0: 中断未产生 (SPI_INMAR[rfufim]作用前) ; 1: 中断产生 (SPI_INMAR[rfufim]作用前)。	0x0
1	RO	rfofri	RX FIFO 溢出原始中断状态: 0: 中断未产生 (SPI_INMAR[rfofim]作用前) ; 1: 中断产生 (SPI_INMAR[rfofim]作用前)。	0x0
0	RO	rffri	RX FIFO 为满原始中断状态: 0: 中断未产生 (SPI_INMAR[rffim]作用前) ; 1: 中断产生 (SPI_INMAR[rffim]作用前)。	0x0

SPI_INSR

SPI_INSR 为中断状态寄存器。

Offset Address: 0x00C0 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:16	-	reserved	保留。	0x0000
15	RO	mmcis	多 Master 连接中断状态。该寄存器在 SPI 被配置为 Slave 器件时不可见。 0: 中断未产生	0x0

Bits	Access	Name	Description	Reset
			(SPI_INMAR[mmcim]作用后) ; 1: 中断产生 (SPI_INMAR[mmcim]作用后) 。	
14:9	-	reserved	保留。	0x00
8	RO	tfofis	TX FIFO 溢出中断状态。 0: 中断未产生 (SPI_INMAR[tfofim]作用后) ; 1: 中断产生 (SPI_INMAR[tfofim] 作用后) 。	0x0
7	RO	tfeis	TX FIFO 为空中断。 0: 中断未产生 (SPI_INMAR[tfeim]作用后) ; 1: 中断产生 (SPI_INMAR[tfeim] 作用后) 。	0x0
6:3	-	reserved	保留。	0x0
2	RO	rfufis	RX FIFO 下溢中断状态。 0: 中断未产生 (SPI_INMAR[rfufim]作用后) ; 1: 中断产生 (SPI_INMAR[rfufim] 作用后) 。	0x0
1	RO	rfofis	RX FIFO 溢出中断状态。 0: 中断未产生 (SPI_INMAR[rfofim]作用后) ; 1: 中断产生 (SPI_INMAR[rfofim] 作用后) 。	0x0
0	RO	rffis	RX FIFO 为满中断状态。 0: 中断未产生 (SPI_INMAR[rffim]作用后) ; 1: 中断产生 (SPI_INMAR[rffim] 作用后) 。	0x0

Bits	Access	Name	Description	Reset
			作用后)。	

SPI_INMAR

SPI_INMAR 为中断屏蔽寄存器

Offset Address: 0x00C4 Total Reset Value: 0x0000_8187

Bits	Access	Name	Description	Reset
31:16	-	reserved	保留。	0x0000
15	RW	mmcim	多 Master 连接中断屏蔽。该寄存器在 SPI 被配置为 Slave 器件时不可见。 0：中断屏蔽； 1：中断未屏蔽。	0x1
14:9	-	reserved	保留。	0x00
8	RW	tfofim	TX FIFO 溢出中断屏蔽。 0：中断屏蔽； 1：中断未屏蔽。	0x1
7	RW	tfeim	TX FIFO 为空中断屏蔽。 0：中断屏蔽； 1：中断未屏蔽。	0x1
6:3	-	reserved	保留。	0x0
2	RW	rfufim	RX FIFO 下溢原始中断屏蔽。 0：中断屏蔽； 1：中断未屏蔽。	0x1
1	RW	rfofim	RX FIFO 溢出中断屏蔽。 0：中断屏蔽； 1：中断未屏蔽。	0x1

Bits	Access	Name	Description	Reset
0	RW	rffim	RX FIFO 为满中断屏蔽。 0：中断屏蔽； 1：中断未屏蔽。	0x1

SPI_SLENR

SPI_SLENR 为 Slave 使能寄存器。

Offset Address: 0x00C8 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	-	reserved	保留。	0x00000000
0	RW	ssef	Slave 选择使能标志。 该寄存器的每一 bit 代表 1 个来自 SPI Master 的 Slave 选择信号 (ss_x_n)。若该寄存器某 bit 位设为 1，相应的 Slave 选择信号被激活。应注意，置位或清零该寄存器并不影响相应的 Slave 选择输出信号，直到一个传输开始。 启动一个传输前，用户需要使能相应 Slave 器件所对应的 bit 位。在非广播模式，该寄存器中仅应有 1bit 被置位。 0：无从器件选择； 1：从器件被选择。	0x0

SPI_TWLR

SPI_TWLR 为发送水线寄存器。

Offset Address: 0x00CC Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:7	-	reserved	保留。	0x00000000
6:0	RW	twl	TX FIFO 水线设置。 控制 TX FIFO 产生中断的水线，FIFO 深度被配置为 2~256，若用户向该寄存器写入的值大于或等于 FIFO 深度，则该寄存器将不可写入并保持当前值。当 TX FIFO 内数据量小于等于该寄存器值，TX FIFO 空中断将被触发。	0x00

SPI_TLR

SPI_TLR 为发送数据量查询寄存器。

Offset Address: 0x00D0 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:8	-	reserved	保留。	0x00000000
7:0	RO	tfl	TX FIFO 内有效数据个数。	0x00

SPI_TFOFIER

SPI_TFOFIER 为 tx fifo 中断清除寄存器。

Offset Address: 0x00D4 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	-	reserved	保留。	0x00000000
0	RC	ctfofi	清除 TX FIFO 溢出中断。 该寄存器反应中断状态，读该寄存器将清除 ssi_txo_intr 中断，写该寄存器无效。	0x0

SPI_RWLR

SPI_RWLR 为接收水线寄存器

Offset Address: 0x00D8 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:7	-	reserved	保留。	0x00000000
6:0	RW	rwl	RX FIFO 水线设置。 控制 RX FIFO 产生中断的水线位置。若用户向该寄存器写入的值大于或等于 FIFO 深度，则该寄存器将不可写入并保持当前值。当 RX FIFO 内数据量大于或等于该寄存器值加 1，RX FIFO 的满中断将被触发。	0x00

SPI_RLR

SPI_RLR 为接收数据量查询寄存器。

Offset Address: 0x00DC Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:8	-	reserved	保留。	0x00000000
7:0	RO	rfl	RX FIFO 内有效数据个数。	0x00

SPI_RFFICR

SPI_RFFICR 为 RX FIFO Underflow 中断清除寄存器。

Offset Address: 0x00E0 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
31:1	-	reserved	保留。	0x00000000
0	RC	crfufi	清除 RX FIFO 下溢中断。 该寄存器反应中断状态，读该寄存器将清除 ssi_rxu_intr 中断，写该寄存器无效。	0x0

SPI_WSR

SPI_WSR 为状态寄存器。

Offset Address: 0x00E4 Total Reset Value: 0x0000_1800

Bits	Access	Name	Description	Reset
31:16	-	reserved	保留。	0x0000
15	RO	sbf	SSI 忙标志。 0: SPI 空闲或禁用; 1: SPI 正在传输数据。	0x0
14:13	-	reserved	保留。	0x0
12	RO	tfe	TX FIFO 空标志。当 TX FIFO 彻底为空时，该 bit 位置位。当 TX FIFO 内存在一个或更多的有效数据，该 bit 位清零。该 bit 位并不产生中断。 0: TX FIFO 非空; 1: TX FIFO 为空。	0x1
11	RO	tfnf	TX FIFO 非满标志。当 TX FIFO 内存在一个或多个空存储空间时，该 bit 位置位；当 FIFO 为满时该 bit 位清零。 0: TX FIFO 为满;	0x1

Bits	Access	Name	Description	Reset
			1: TX FIFO 非满。	
10:6	-	reserved	保留。	0x00
5	RO	rffe	RX FIFO 满标志。当 RX FIFO 彻底满时, 该 bit 位置位。当 RX FIFO 仍有 1 个或更多的存储空间该 bit 位清零。 0: RX FIFO 非满; 1: RX FIFO 满。	0x0
4	RO	rfne	RX FIFO 非空标志。当 RX FIFO 内存在 1 个或更多有效数据, 该 bit 位置位; 当 RX FIFO 为空时, 该 bit 位清零。该 bit 位可被软件轮询来确定 RX FIFO 彻底变空。 0: RX FIFO 空; 1: RX FIFO 非空。	0x0
3:1	-	reserved	保留。	0x0
0	RO	dcfe_tfee	MASTER 模式下: 数据冲突错误。 当 SPI MASTER 在执行数据传输, ss_in_n 信号被其他 MASTER 驱动时, 该 bit 位置位, 通知处理器最后一次数据传输在完成前被挂起。该 bit 位在读取后自动清零。 0: 无数据冲突; 1: 数据冲突错误产生。 SLAVE 模式下: 发送错误。 当 TX FIFO 为空时, 若一个传输开始, 该 bit 位置位。该 bit 位仅在 SPI 配置为 Slave 器件时才可置位。上次传输的数据将在 TXD 数	0x0

Bits	Access	Name	Description	Reset
			据线上重复发送。该 bit 位在读取时自动清零。 0: 无错误; 1: 发送错误产生。	

SPI_MSTICR

SPI_MSTICR 为多 Master 中断清除寄存器。

Offset Address: 0x00E8 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	-	reserved	保留。	0x00000000
0	RC	cmmci	清除多 Master 连接中断。 该寄存器反应中断状态，读该寄存器将清除 ssi_mst_intr 中断，写该寄存器无任何影响。 0: 无中断; 1: 多 Master 连接中断有效。	0x0

SPI_ID

SPI_ID 为版本信息寄存器。

Offset Address: 0x00EC Total Reset Value: 0x0000_0002

Bits	Access	Name	Description	Reset
31:0	RO	id	存储了十六进制格式的 IP 版本信息。版本信息的每个数字以 ASCII 码表示，例如： 32_30_31_2A 代表版本 2.01*。	0x00000002

SPI_RSVD

SPI_RSVD 为保留寄存器。

Offset Address: 0x00F0 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:13	-	reserved	保留。	0x00000
12	RW	cfg_ssp_sclk_mask	在旁路 SSP 模式, SCLK 屏蔽修改。 0: 不屏蔽随路时钟; 1: 屏蔽旁路随路时钟。 注意: 仅在 SPI_CTRA[[prs]配置为 SSP 协议下有效。	0x0
11	-	reserved	保留。	0x0
10	RW	wait_bypass_en	等待状态旁路信号。 0: 不等待; 1: 等待。 注意: 仅在 Master 模式下有效。	0x0
9:0	RW	wait_number	指令发送后等待接收数据的时间。 单位为 SPI 工作时钟 cycles。 注意: 仅在 Master 模式下有效。	0x000

SPI_RXOICR

SPI_RXOICR 为 RX FIFO Overflow 中断清除寄存器。

Offset Address: 0x00F4 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	-	reserved	保留。	0x00000000
0	RC	crfofi	清除 RX FIFO 溢出中断。 该寄存器反应中断状态, 读该寄存	0x0

Bits	Access	Name	Description	Reset
			器清除 ssi_rxo_intr 中断，写该寄存器无影响。	

SPI_ICR

SPI_ICR 为中断清除寄存器。

Offset Address: 0x00F8 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	-	reserved	保留。	0x00000000
0	RC	cis	清除中断。 该寄存器在 ssi_txo_intr、ssi_rxu_intr、ssi_rxo_intr 与 the ssi_mst_intr 四个中断中任意一个有效时置位。读该寄存器将清除这四个中断。写该寄存器无影响。	0x0

SPI_APB.CG

SPI_APB.CG 为 APB 强制时钟门控控制寄存器。

Offset Address: 0x00FC Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:16	-	reserved	保留。	0x0000
15:0	WO	spi_apb_cg	控制 APB 总线强制门控信号。该寄存器用作关闭 APB 总线时钟自动门控。默认自动 CG 开启。 0xdead: 关闭自动 CG, APB 总线时钟常开; 0xbeef: 恢复自动 CG;	0x0000

Bits	Access	Name	Description	Reset
			others: 不影响自动 CG 状态。	

5.8 PWM

5.8.1 概述

PWM 模块用于生成 PWM 信号，例如可用于调节灯的亮度，控制蜂鸣器等工作。

芯片提供 2 个 PWM：PWM0 和 PWM1，两者功能完全一致，且支持同时输出 PWM 信号。

5.8.2 功能描述

PWM 模块具有以下功能特点：

- 可以对源头时钟进行分频，支持软件配置；将分频后的时钟作为周期刻度，分配高低电平长度。
- 高低电平分配的周期长度范围均为 1 ~ 65535，支持软件配置。
- 支持单次模式场景与重复模式场景。
- 支持 CPU 动态调整占空比。
- 支持配置开始/停止工作。
- PWM 支持输出的最大频率是 XO 时钟二分频。

5.8.3 工作方式

PWM0 和 PWM1 配置流程相同，只有寄存器地址的区别。

工作时需要先打开时钟门控：

PWM0：配置 0x30005020 寄存器 bit4 为 1，打开时钟门控（默认打开）。

PWM1：配置 0x30005020 寄存器 bit5 为 1，打开时钟门控（默认打开）。

5.8.3.1 单脉冲输出

步骤 1 配置 PWM 高低电平计数值，写寄存器 [PHASE\[15:0\]](#)和 [PHASE\[31:16\]](#)分别配置输出脉冲的低电平和高电平计数值。

步骤 2 配置 PWM 时钟分频，配置 [SCALER](#)[scaler]。

步骤 3 配置 PWM 启动，配置 [START](#)[start]为 1，启动 PWM。

----结束

5.8.3.2 repeat 模式

步骤 1 配置 PWM 高低电平计数值，写寄存器 [PHASE](#)[15:0]和 [PHASE](#)[31:16]分别配置输出脉冲的低电平和高电平计数值。

步骤 2 配置 PWM 时钟分频，配置 [SCALER](#)[scaler]。

步骤 3 配置 PWM repeat 周期数，配置 [REPEAT_NUMB](#)[repeat_num]。

步骤 4 配置 PWM 启动，配置 [START](#)[start]为 1，启动 PWM。

----结束

5.8.3.3 continuous 模式

步骤 1 配置 PWM 高低电平计数值，写寄存器 [PHASE](#)[15:0]和 [PHASE](#)[31:16]分别配置输出脉冲的低电平和高电平计数值。

步骤 2 配置 PWM 时钟分频，配置 [SCALER](#)[scaler]。

步骤 3 配置 PWM 为 continuous 模式，配置 [CTRL_SET](#)[continuous_set]为 1。

步骤 4 配置 PWM 启动，配置 [START](#)[start]为 1，启动 PWM。

----结束

5.8.4 寄存器概览

PWM0 寄存器概览如[表 5-23](#) 所示。

PWM0 基地址是 0x3000_0500；PWM1 基地址是 0x3000_0600。

表5-23 PWM0 寄存器概览（基址是 0x3000_0500）

偏移地址	名称	描述
0x0000	VERSION	IP 版本寄存器。

偏移地址	名称	描述
0x0004	START	START 寄存器。
0x0010	CTRL_SET	CTRL_SET 寄存器。
0x0014	CTRL_CLR	CTRL_CLR 寄存器。
0x0018	CTRL_STATUS	CTRL_STATUS 寄存器。
0x0020	REPEAT_NUMB	REPEAT_NUMB 寄存器。
0x0024	SCALER	SCALER 寄存器。
0x0028	PHASE	PWM PHASE 配置寄存器。
0x002C	UPWM_STATUS	PWM_STATUS 寄存器。
0x0030	IRQ	IRQ 寄存器。
0x0034	IRQ_CLR	IRQ_CLR 寄存器。

5.8.5 寄存器描述

VERSION

IP 版本寄存器。

Offset Address: 0x0000 Total Reset Value: 0x2024_0115

Bits	Access	Name	Description	Reset
31:0	RO	version	IP 版本寄存器。	0x20240115

START

START 寄存器。

Offset Address: 0x0004 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	W1_PU	start	PWM 启动寄存器，写 1 产生	0x0

Bits	Access	Name	Description	Reset
	LSE		单个时钟周期的高电平脉冲。 1: PWM 启动; 0: 无效。	

CTRL_SET

CTRL_SET 寄存器。

Offset Address: 0x0010 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:5	RO	reserve d	保留。	0x00000000
4	WO	repeat_i rq_en_s et	CTRL 寄存器, 置位操作, 支持单 bit 操作。 1: 对应 bit 置 1; 0: 无效。	0x0
3	WO	data_irq _en_set	CTRL 寄存器, 置位操作, 支持单 bit 操作。 1: 对应 bit 置 1; 0: 无效。	0x0
2	WO	exp_data _in_set	CTRL 寄存器, 置位操作, 支持单 bit 操作。 1: 对应 bit 置 1; 0: 无效。	0x0
1	WO	continuous_set	CTRL 寄存器, 置位操作, 支持单 bit 操作。 1: 对应 bit 置 1; 0: 无效。	0x0
0	WO	stop_set	CTRL 寄存器, 置位操作, 支持单 bit 操作。 1: 对应 bit 置 1;	0x0

Bits	Access	Name	Description	Reset
			0: 无效。	

CTRL_CLR

CTRL_CLR 寄存器。

Offset Address: 0x0014 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:5	RO	reserved	保留	0x00000000
4	WO	repeat_irq_en_clr	CTRL 寄存器, 清零操作, 支持单 bit 操作。 1: 对应 bit 置 0; 0: 无效。	0x0
3	WO	data_irq_en_clr	CTRL 寄存器, 清零操作, 支持单 bit 操作。 1: 对应 bit 置 0; 0: 无效。	0x0
2	WO	exp_data_in_clr	CTRL 寄存器, 清零操作, 支持单 bit 操作。 1: 对应 bit 置 0; 0: 无效。	0x0
1	WO	continuous_clr	CTRL 寄存器, 清零操作, 支持单 bit 操作。 1: 对应 bit 置 0; 0: 无效。	0x0
0	WO	stop_clr	CTRL 寄存器, 清零操作, 支持单 bit 操作。 1: 对应 bit 置 0; 0: 无效。	0x0

CTRL_STATUS

CTRL_STATUS 寄存器，即 CTRL 寄存器的回读值。

Offset Address: 0x0018 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:5	RO	reserved	保留。	0x00000000
4	RO	repeat_irq_en	重复发送 PWM 中断使能： 1：使能； 0：不使能。	0x0
3	RO	data_irq_en	数据异常中断使能： 1：使能； 0：不使能。	0x0
2	RO	exp_data_in	1：数据为预期数据； 0：无效。	0x0
1	RO	continuous	连续发送 PWM 信号使能： 1：使能； 0：不使能。	0x0
0	RO	stop	1：立即停止 PWM 发送； 0：无效。	0x0

REPEAT_NUMB

REPEAT_NUMB 寄存器。

Offset Address: 0x0020 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:8	RO	reserved	保留。	0x00000000
7:0	RW	repeat_num	PWM repeat 模式下输出高低电平的 cycle 数-1（实际输出 cycle 数	0x00

Bits	Access	Name	Description	Reset
			=REPEAT_NUMB+1) 。	

SCALER

SCALER 寄存器。

Offset Address: 0x0024 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:4	RO	reserved	保留。	0x00000000
3:0	RW	scaler	PWM PHASE 计数的颗粒度, $\text{clk_pwm} \times (2^{\text{SCALER}})$; 如 scaler=2; high_phase=3; clk_pwm=5MHz, 则输出高电平持续时间为 $3 \times (1/5 \times 2^2) = 2.4\mu\text{s}$ 。	0x0

PHASE

PWM PHASE 配置寄存器。

Offset Address: 0x0028 Total Reset Value: 0x00000000

Bits	Access	Name	Description	Reset
31:0	RW_REG_AND_VLD	phase	[15:0] LOW_PHASE_IN - PWM 信号低电平持续的 CLK 数; [31:16] HIGH_PHASE_IN - PWM 信号高电平持续的 CLK 数。	0x00000000

UPWM_STATUS

PWM_STATUS 寄存器。

Offset Address: 0x002C Total Reset Value: 0x00000000

Bits	Access	Name	Description	Reset
31:5	RO	reserved	reserved	0x00000000
4:0	RO	upwm_status	• [4]: PHASE 值更新指示信号, 1 代表更新成功 (前提是 data_irq_en 为 1) ; • [3]: Error, 高有效; • [2]: NewData 占空比配置生效, 高有效。 • [1:0]: state 状态机当前状态: 00: IDLE; 01: LOW (输出低电平) ; 10: HIGH (输出高电平) 。	0x00

IRQ

IRQ 寄存器。

Offset Address: 0x0030 Total Reset Value: 0x00000000

Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	RO	irq	中断上报寄存器, 高有效。	0x0

IRQ_CLR

IRQ_CLR 寄存器。

Offset Address: 0x0034 Total Reset Value: 0x00000000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

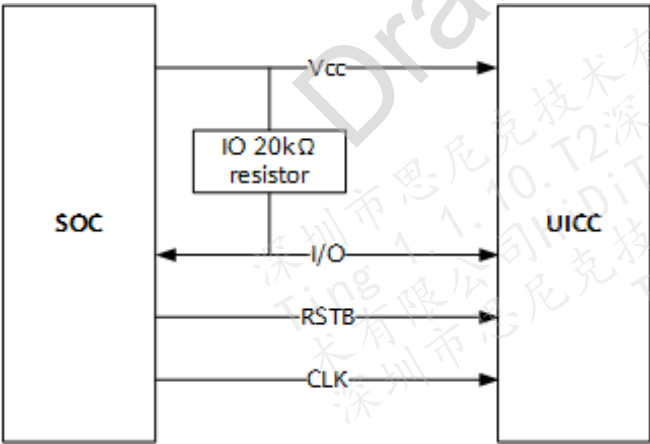
Bits	Access	Name	Description	Reset
31:1	RO	reserved	保留。	0x00000000
0	W1_PU LSE	irq_clr	如果定义中断类型为电平中断，则配置该寄存器写清。可读、可写，自清。 写 1 产生 1 个 cycle 的脉冲。读，基本上为 0。	0x0

5.9 UICC

5.9.1 概述

UICC 接口是芯片与 SIM（Subscriber Identity Module）卡通信的接口。

图5-16 UICC Physical interface



注：RSTB（Reset 信号）

5.9.2 功能描述

UICC 接口具有以下功能特点：

- 芯片为主设备接口，SIM 卡为从设备。
- 支持 SIM 卡的激活、去激活、复位、参数、协议配置。

- 支持对 SIM 卡读写访问操作。

5.9.3 工作方式

激活冷复位流程

- 步骤 1 配置 `UICC_CLKRST[uiicc_clk_en]` 为 1 打开 UICC 工作时钟。
- 步骤 2 配置 `UICC_CLKRST[uiicc_module_reset]` 为 0，解复位 UICC 逻辑。
- 步骤 3 配置 `UICC_CLKRST[uiicc_reset_pol]` 选择输出复位极性。（可选）
- 步骤 4 配置 `UICC_CLKRST[uiicc_reset]` 为 1 输出复位信号给 SIM 卡。
- 步骤 5 配置 `UICC_CLKRST[uiicc_clk_div2]`，输出 1 ~ 5M 时钟信号给 SIM 卡。
- 步骤 6 等待 400 个 UICC IO 时钟周期后，配置 `UICC_CLKRST[uiicc_reset]` 为 0，SIM 卡解复位。
- 步骤 7 等待 ATR 响应，如果大于 4000 个 UICC IO 时钟周期未返回数据，启动流程失败。
- 步骤 8 等待中断，读 `UICC_FIFO_LVL[rx_overflow]`，判断无 overflow，根据 `rxfifo_lvl` 的值从 `UICC_RX_BYTE[uiicc_rx_byt]` 读出接收数据并解码。

----结束

正常传输

TX:

- 步骤 1 根据 ATR 协议的值配置 `UICC_CONF[conf_d]` 和 `UICC_CONF[conf_f]`。
- 步骤 2 配置 `UICC_CONF[conf_gt]`，根据 ATR 解帧协议配置。
- 步骤 3 配置 `UICC_CONF[conf_tx_parity_en]` 奇偶校验使能，配置 `UICC_CONF[conf_opendrain]` 输出开漏模式，配置 `UICC_CONF[conf_usegtforretx]` 使能 TX 重传间隔配置。（可选）
- 步骤 4 配置 `UICC_CLKRST[uiicc_clk_div2]`，输出时钟信号给 SIM 卡，配置值使得输出时钟不超过协议时钟且不低于 1M。
- 步骤 5 配置 `UICC_CLKRST[uiicc_flip_tx_parity]` 奇偶校验选择。（可选）
- 步骤 6 向 `UICC_TX_BYTE[uiicc_tx_byt]` 写入发送数据，UICC 开始传输。

步骤 7 等待中断，读 `UICC_FIFO_LVL`，`UICC_FIFO_LVL[txfifo_lvl]` 为 0 表示当前 tx 发送完成，等待接收或发送剩余数据。

说明

如果希望对端收到错误数据时重传数据，配置 `UICC_CONF[conf_tx_parity_en]` 为 1。

----结束

RX:

步骤 1 配置 `UICC_CLKRST[uicc_flip_rx_parity]` 奇偶校验选择。（可选）

步骤 2 配置 `UICC_CONF[conf_rx_parity_en]` 奇偶校验使能。（可选）

步骤 3 等待中断，查询 `UICC_FIFO_LVL[rxfifo_lvl]` 寄存器，读 `UICC_RX_BYTE[uicc_rx_byt]` 直到把 RXFIFO 读空。

说明

如果需要丢弃有奇偶校验错误的接收数据，配置 `UICC_CONF[conf_rx_parity_en]` 为 1。

----结束

热复位（可在工作过程配置）

步骤 1 配置 `UICC_CLKRST[uicc_reset_pol]` 选择输出复位极性。（可选）

步骤 2 配置 `UICC_CLKRST[uicc_reset]` 为 1 输出复位信号给 SIM 卡。

步骤 3 等待 400 个 UICC IO 时钟周期后，配置 `UICC_CLKRST[uicc_reset]` 为 0，SIM 卡解复位。

步骤 4 等待 ATR，如果大于 4000 个 UICC IO 时钟周期未返回数据，则启动流程失败。

步骤 5 等待中断，读 `UICC_FIFO_LVL`，判断无 overflow，根据 `rxfifo_lvl` 的值读出接收数据并解码。

----结束

时钟停止

在与对端设备没有数据传输时可以配置停止输出时钟。

步骤 1 配置 `UICC_CLKRST[uicc_clk_stop_high]` 停止时钟极性。

步骤 2 等待 1860 个 UICC IO 时钟周期，配置 `UICC_CLKRST[uicc_clk_div2]` 为 0，UICC 停止输出时钟。

步骤 3 需要传输数据，配置 `UICC_CLKRST[uicc_clk_div2]` 为非零值，等待 700 个 UICC IO 时钟周期后开始数据传输。

----结束

去激活

步骤 1 配置 `UICC_CLKRST[uicc_reset_pol]` 选择输出复位极性。（可选）

步骤 2 配置 `UICC_CLKRST[uicc_reset]` 为 1 输出复位信号。

步骤 3 配置 `UICC_CLKRST[uicc_clk_div2]` 为 0。

步骤 4 配置 `UICC_CLKRST[uicc_module_reset]` 为 1，复位 UICC 逻辑。

说明

txoverflow 和 rxoverflow 时走去激活流程。

----结束

5.9.4 寄存器概览

UICC 寄存器概览如表 5-24 所示。

表5-24 UICC 寄存器概览（基地址: 0x3000_0400）

偏移地址	名称	描述
0x0004	<code>UICC_CLKRST</code>	UICC 时钟复位控制寄存器。
0x0008	<code>UICC_CONF</code>	UICC 控制寄存器。
0x000C	<code>UICC_STATUS</code>	UICC 状态寄存器。
0x0010	<code>UICC_ETU_COUNT</code>	UICC ETU 计数值上报寄存器。
0x0014	<code>UICC_ETU_THRESHOLD</code>	UICC ETU 阈值配置寄存器。
0x0018	<code>UICC_FIFO_LVL</code>	FIFO 状态寄存器。
0x001C	<code>UICC_TX_BYTE</code>	UICC TX 数据寄存器。

偏移地址	名称	描述
0x0020	UICC_RX_BYTE	UICC RX 数据寄存器。
0x0024	UICC_INTR_CLR	UICC 中断清除寄存器。
0x0028	UICC_INTR	UICC 中断寄存器。
0x002C	UICC_DEBUG	UICC 维测状态寄存器。
0x0030	UICC_ID	UICC 版本寄存器

5.9.5 寄存器描述

UICC_CLKRST

UICC 时钟复位控制寄存器。

Offset Address: 0x0004 Total Reset Value: 0x0000_0100

Bits	Access	Name	Description	Reset
31:12	RO	reserved	保留。	0x00000
11	RW	uicc_module_reset	UICC 控制器软复位寄存器。 0: 复位无效; 1: 复位有效。	0x0
10	RW	uicc_clk_en	UICC 控制器时钟使能寄存器。 0: 时钟关闭; 1: 时钟打开。	0x0
9	RW	uicc_reset_pol	UICC 输出复位极性控制寄存器。 0: 复位低有效; 1: 复位高有效。	0x0
8	RW	uicc_reset	UICC 输出复位配置寄存器。	0x1
7	RW	uicc_flip_rx_parity	Rx 奇偶校验选择寄存器。	0x0

Bits	Access	Name	Description	Reset
		ty	0: not flip(even); 1: flip(odd)。	
6	RW	uicc_flip_tx_parity	Tx 奇偶校验选择寄存器。 0: not flip(even); 1: flip(odd)。	0x0
5	RW	uicc_clk_stop_high	UICC 不工作时输出时钟极性控制。 0: 时钟线为低电平; 1: 时钟线为高电平。	0x0
4:0	RW	uicc_clk_div2	UICC 工作时钟分频配置寄存器, 分频系数为配置值的两倍, 配置 0 无效。	0x00

UICC_CONF

UICC 控制寄存器。

Offset Address: 0x0008 Total Reset Value: 0x00C1_0174

Bits	Access	Name	Description	Reset
31:24	RW	conf_gt	Tx 发送间隔时间配置, 发送间隔为 12+conf_gt 个 etucount。	0x00
23	RW	conf_tx_parity_en	Tx 奇偶校验使能。 0: 禁止; 1: 使能。	0x1
22	RW	conf_rx_parity_en	Rx 奇偶校验使能。 0: 禁止; 1: 使能。	0x1
21:16	RW	conf_d	etu rate 配置分子, 配置为 0 表示 64。	0x01

Bits	Access	Name	Description	Reset
15:14	RO	reserve	保留。	0x0
13	RW	conf_us egtforre tx	Tx 重传间隔时间配置使能。 0: 禁止; 1: 使能。 未使能默认为 13 个 etucount, 使能后保持和发送 间隔一样的时间配置。	0x0
12	RW	conf_op endrain	开漏输出使能: 0: 禁止; 1: 使能。	0x0
11:0	RW	conf_f	etu rate 配置分母。	0x174

UICC_STATUS

UICC 状态寄存器。

Offset Address: 0x000C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:16	RO	reserve d	保留。	0x0000
15:8	RO	uicc_tx_ parity_e rrors	Tx 方向奇偶校验错误计数值 上报寄存器。	0x00
7:0	RO	uicc_rx_ parity_ errors	Rx 数据奇偶校验错误计数值 上报寄存器。	0x00

UICC_ETU_COUNT

UICC ETU 计数值上报寄存器。

Offset Address: 0x0010 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
31:0	RO	uicc_et u_count	ETU 计数值上报寄存器。	0x00000000

UICC_ETU_THRESHOLD

UICC ETU 阈值配置寄存器。

Offset Address: 0x0014 Total Reset Value: 0xFFFF_FFFF

Bits	Access	Name	Description	Reset
31:0	RW	uicc_et u_thres hold	UICC ETU 阈值配置寄存器。	0xFFFFFFFF

UICC_FIFO_LVL

FIFO 状态寄存器。

Offset Address: 0x0018 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:25	RO	reserve d	保留。	0x00
24	RO	tx_overf low	TX FIFO 溢出。 0: 无溢出; 1: 溢出。	0x0
23:16	RO	txfifo_lvl	Tx FIFO 中写入的据量。	0x00
15:9	RO	reserve d	保留。	0x00
8	RO	rx_overf low	Rx FIFO 溢出。 0: 无溢出; 1: 溢出。	0x0
7:0	RO	rxfifo_lvl	Rx FIFO 中接收数据量。	0x00

UICC_TX_BYTE

UICC TX 数据寄存器。

Offset Address: 0x001C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:8	RO	reserved	保留。	0x000000
7:0	WO	uicc_tx_byt	UICC Tx 数据配置寄存器。	0x00

UICC_RX_BYTE

UICC RX 数据寄存器。

Offset Address: 0x0020 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:8	RO	reserved	保留。	0x000000
7:0	RO	uicc_rx_byt	UICC Rx 数据寄存器。	0x00

UICC_INTR_CLR

UICC 中断清除寄存器。

Offset Address: 0x0024 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:2	RO	reserved	保留。	0x00000000
1	W1_PU LSE	uicc_etu_intr_clr	UICC ETU 超时中断清除寄存器。配置 1 清除中断，回读为 0。配置为脉冲中断时该寄存器无效。 0: 无效; 1: 清中断。	0x0
0	W1_PU LSE	uicc_int_r_clr	UICC 正常功能中断清除寄存	0x0

Bits	Access	Name	Description	Reset
			器。配置 1 清除中断，回读为 0。配置为脉冲中断时该寄存器无效。 0：无效； 1：清中断。	

UICC_INTR

UICC 中断寄存器。

Offset Address: 0x0028 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:2	RO	reserved	保留。	0x00000000
1	RO	uicc_etu_intr	UICC ETU 超时中断上报。 0：中断无效； 1：中断无效。 当中断为脉冲中断时不能触发。	0x0
0	RO	uicc_int_r	UICC 接收/发空/fifo overflow 中断。 0：中断无效； 1：中断无效。 当中断为脉冲中断时不能触发。	0x0

UICC_DEBUG

UICC 维测状态寄存器。

Offset Address: 0x002C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
31:7	RO	reserved	保留。	0x00000000
6	RO	uicc_etu_intr_alarm	UICC 二次电平中断报警，当配置为电平类型中断时，uicc_etu_intr 一次中断上报后软件没有及时清除，再次触发中断时会给出报警指示。 0：报警指示无效； 1：报警指示生效，中断未及时处理。	0x0
5	RO	uicc_intr_alarm	UICC 二次电平中断报警，当配置为电平类型中断时，uicc_intr 一次中断上报后软件没有及时清除，再次触发中断时会给出报警指示。 0：报警指示无效； 1：报警指示生效，中断未及时处理。	0x0
4	RO	uicc_atr_ts	UICC ATR_TS 状态上报，用于指示 UICC 当前是否在接收 ATR TS 字符。 0：UICC 不处于 ATR_TS 接收状态； 1：UICC 处于 ATR_TS 接收状态。	0x0
3:0	RO	uicc_state	UICC 状态机上报，用于定位问题读取状态。 0：idle； 1：rx_data； 2：rx_check； 3：rx_err；	0x0

Bits	Access	Name	Description	Reset
			4: rx_good; 5: rx_end; 6: tx_data; 7: tx_check; 8: tx_retransmit; 9: tx_end。	

UICC_ID

UICC 版本寄存器。

Offset Address: 0x0030 Total Reset Value: 0x20240207

Bits	Access	Name	Description	Reset
31:0	RO	uicc_id	UICC 版本寄存器。	0x20240207

5.10 USB2.0

5.10.1 概述

该系统支持 1 个 USB 2.0 Device 接口。

5.10.2 功能描述

功能特点

该系统支持 1 个 USB 2.0 Device 模块，此模块具有以下特点：

- 支持 USB2.0 协议的 FS (Full-Speed)、HS (High-Speed) 两种模式，USB 接口速率分别到达 12Mbit/s 和 480Mbit/s。
- 支持高速传输的 UTMI+接口协议。

USB 2.0 Device 控制器具有以下功能：

- 对传输的控制和处理。

- 对数据包的解析和打包。
- 对 USB 传输信号的编码和解码。
- 支持内部 DMA 控制器。
- 支持 7 个 in 端点，支持 4 个 out 端点。

工作原理

USB 2.0 Device 支持以下几种标准的传输方式：

- Control Transfer（控制传输）

主要用于 USB Host 与 USB Device 端点 0 之间的传输，某些特定型号的 USB 设备的控制传输可能用到其他的端点。控制传输是双向传输，数据量通常较小，可以传输 8Byte、16Byte、32Byte 或 64Byte 的数据，依赖于设备和传输速度。

- Bulk Transfer（批量传输）

主要用于没有带宽和间隔时间要求的情况下发送和接收大量的数据，这种类型的设备适合于传输非常慢和大量被延迟的传输，可以等到所有其他类型的数据的传送完成之后再传送和接收数据。它的特点是以错误检测和重传的方式保证 USB Host 与 USB Device 的数据被无差错地发送。

- Interrupt Transfer（中断传输）

主要用于少量、分散、不可预测的数据的传输。中断传输方式下，定时查询设备是否有中断数据要发送。

5.11 Vsensor

5.11.1 概述

Vsensor 作为电压传感器，实现将模拟信号转化为数字信号的功能，支持测量 AIO PAD(AIO0、AIO1、AIO2、AIO3)管脚上的模拟信号。

5.11.2 功能描述

Vsensor 具有以下功能特点：

- 支持 4 路模拟输入通道，通过配置进行通道选择，分时复用。
- 支持 12bit ADC，最高采样率为 0.2Msps。
- 支持电压检测范围。0 通道 0.5~4.8V，1,2,3 通道支持到 0.1V~1.5V。

- 支持平均操作模式，支持 16 次累加取平均值。
- 支持单次采样模式，支持单次采样后直接上报。
- 支持中断使能，中断上报，中断清除。

5.11.3 工作方式

状态查询方式下的数据采集

- 步骤 1 配置 `VSENSOR_0[vs_chan_sel]`，选择需要测量的通道；配置 `vs_chan_en` 为 0x1，打开选择通道。
- 步骤 2 配置 `VSENSOR_0[vs_inputbuffer_chop_en]`为 0x1、配置 `VSENSOR_0[vs_comp_chop_en]`为 0x1，使能 chopper 模式。
- 步骤 3 配置 `VSENSOR_0[bgr_top_en]`为 0x1，使能内部基准信号。
- 步骤 4 配置 `VSENSOR_3[bgr_trim_temp]`为 0xE，优化采样精度。
- 步骤 5 配置 `VSENSOR_4[vs_ctrl_mode]`为 0x0，选择自动产生时序。
- 步骤 6 配置 `VSENSOR_6[vs_start_pre_cycle]`为 0x1，`VSENSOR_6[vs_start_keep_cycle]`为 0x3，`VSENSOR_6[vs_start_separate_cycle]`为 0x13，构建自动生成的时序。
- 步骤 7 配置 `VSENSOR_6[equ_mode]`，选择平均值模式。
- 步骤 8 配置 `VSENSOR_9[over_int_en]`为 0x0，`VSENSOR[done_int_en]`为 0x1，`VSENSOR[done_int_mask]`为 0x1，使能完成中断，并屏蔽上报。
- 步骤 9 读取 `VSENSOR_0[vs_en]`的值，若为 0，则配置 `VSENSOR_0[vs_en]`为 0x1，开启自动采样，否则配置 `VSENSOR_5[vsensor_start]`为 0x1，开启自动采样。
- 步骤 10 轮询读 `VSENSOR_STATUS_1[done_int_sts]`，直至读到 1，读取 `VSENSOR_STATUS_2[vs_data_rpt]`的值。
- 步骤 11 配置 `VSENSOR_10[vsensor_int_clr]`为 0x1，清除中断状态。
- 步骤 12 继续采样则重复步骤 8 ~ 步骤 10。
- 步骤 13 结束采样则配置 `VSENSOR_0[vs_en]`为 0x0。

----结束

中断响应方式下的数据采集

- 步骤 1 配置 **VSSENSOR_0**[vs_chan_sel], 选择需要测量的通道; 配置 **VSSENSOR_0**[vs_chan_en]为 0x1, 打开选择通道。
- 步骤 2 配置 **VSSENSOR_0**[vs_inputbuffer_chop_en]为 0x1、配置 **VSSENSOR_0**[vs_comp_chop_en]为 0x1, 使能 chopper 模式。
- 步骤 3 配置 **VSSENSOR_0**[bgr_top_en]为 0x1, 使能内部基准信号。
- 步骤 4 配置 **VSSENSOR_3**[bgr_trim_temp]为 0xE, 优化采样精度。
- 步骤 5 配置 **VSSENSOR_4**[vs_ctrl_mode]为 0x0, 选择自动产生时序。
- 步骤 6 配置 **VSSENSOR_6**[vs_start_pre_cycle]为 0x1, **VSSENSOR_6**[vs_start_keep_cycle]为 0x3, **VSSENSOR_6**[vs_start_separate_cycle]为 0x13, 构建自动生成的时序。
- 步骤 7 配置 **VSSENSOR_6**[equ_mode], 选择平均值模式。
- 步骤 8 配置 **VSSENSOR_9**[over_int_en]为 0x0, **VSSENSOR**[done_int_en]为 0x1, **VSSENSOR**[done_int_mask]为 0x0, 使能完成中断, 不屏蔽上报。
- 步骤 9 读取 **VSSENSOR_0**[vs_en]的值, 若为 0, 则配置 **VSSENSOR_0**[vs_en]为 0x1, 开启自动采样, 否则配置 **VSSENSOR_5**[vsensor_start]为 0x1, 开启自动采样。
- 步骤 10 等待中断响应, 读取 **VSSENSOR_STATUS_2**[vs_data_rpt]的值。
- 步骤 11 配置 **VSSENSOR_10**[vsensor_int_clr]为 0x1, 清除中断状态。
- 步骤 12 继续采样则重复步骤 8 ~ 步骤 10。
- 步骤 13 结束采样则配置 **VSSENSOR_0**[vs_en]为 0x0。

----结束

说明

- vs_en 配置为 0x0 结束采样后, 到下一次采样 vs_en 配置为 0x1, 之间的间隔应该大于 40μs。
- 芯片处于 PSM 模式下, AIO PAD 电平翻转频率不能超过 32kHz, 该约束包含所有的 AIO PAD(AIO0,AIO1,AIO2,AIO3)。

5.11.4 寄存器概览

VSSENSOR_RB 寄存器概览如表 5-25 所示。

表5-25 VSENSOR_RB 寄存器概览 (基址是 0x3002_8000)

偏移地址	名称	描述
0x020	VSENSOR_0	VSENSOR 配置寄存器 0。
0x024	VSENSOR_1	VSENSOR 配置寄存器 1。
0x028	VSENSOR_2	VSENSOR 配置寄存器 2。
0x02C	VSENSOR_3	VSENSOR 配置寄存器 3。
0x030	VSENSOR_4	VSENSOR 配置寄存器 4。
0x034	VSENSOR_5	VSENSOR 配置寄存器 5。
0x038	VSENSOR_6	VSENSOR 配置寄存器 6。
0x03C	VSENSOR_7	VSENSOR 配置寄存器 7。
0x040	VSENSOR_8	VSENSOR 配置寄存器 8。
0x044	VSENSOR_9	VSENSOR 配置寄存器 9。
0x048	VSENSOR_10	VSENSOR 配置寄存器 10。
0x060	VSENSOR_STATUS_0	VSENSOR 状态寄存器 0。
0x064	VSENSOR_STATUS_1	VSENSOR 状态寄存器 1。
0x068	VSENSOR_STATUS_2	VSENSOR 状态寄存器 2。

5.11.5 寄存器描述

VSENSOR_0

VSENSOR 配置寄存器 0。

Offset Address: 0x020 Total Reset Value: 0x0800_0498

Bits	Access	Name	Description	Reset
31:28	-	reserved	保留。	0x0
27	RW	vs_comp_chop_en	比较器 chopper 功能使能信号。	0x1

Bits	Access	Name	Description	Reset
			0: 不使能; 1: 使能。	
26	RW	vs_vrefex_sel	参考电压选择信号。 0: 选择内部 VREF; 1: 选择外灌 VREF。	0x0
25:23	RW	vs_test_sel	IP 模拟电压测试选择信号, 在 vs_test_en=1 时有效, 信号从 VTEST18 端口输出。 000: 基准测试输出电压 BGR_TEST_PRE; 001: 基准输出电压 BGR_VREF0P8_PRE; 010: DAC 输出电压; 011: 共模 buffer 输出电压 VCM; 100: REF buffer 输出电压 VREF; 101: 输入 buffer 输出电压 AIN, 可 bypass; 110: 基准产生的共模 buffer 参考电压; 111: 基准产生的 REF buffer 参考电压。	0x0
22:21	RW	vs_vref_sel	内部参考电平选择信号 (AVDD=1.8±10%) 。 00: 1.5V; 01: 1.54V; 11: 1.52V; 10: 1.48V。	0x0
20	RW	bgr_top_en	内部基准使能信号, 需在 vs_en=1	0x0

Bits	Access	Name	Description	Reset
			前配置为 1。 0：关闭； 1：使能。	
19:17	RW	vs_clk_div	IP 工作时钟的分频控制信号，在 vs_clk 的基础上分频后送给 Vsensor 使用。 000~111：1~128 分频； 000：1 分频； 001：2 分频； 010：4 分频； ... 110：64 分频； 111：128 分频。 注意：该寄存器用于调试。	0x0
16	RW	vs_start_apb	IP 转换功能控制信号，上升沿有效，高电平需≥4 个 vs_clk 周期。两个上升沿间隔需> 单次转换时间。 当 VSENSOR_4[vs_ctrl_mode]配置为 1 时生效。	0x0
15	RW	vs_en	IP 使能信号。 0：禁止； 1：使能。 默认配置 0，静态配置完成后，配置为 1。	0x0
14:11	RW	vs_chan_sel	通道选择信号（在 vs_chan_en=1 时有效）。 0000~1111：VS_IN0~VS_IN15	0x0
10	RW	vs_buff_sel	输入 buffer 选择控制开关。	0x1

Bits	Access	Name	Description	Reset
			0: 输入经过 mux 后直接送到 ADC 采样; 1: 输入经过 buffer 后送到 ADC 采样。	
9:8	RW	vs_itrim	内部比较器偏置电流配置信号。 00: 10 μ A; 01: 11 μ A; 10: 12 μ A; 11: 9 μ A。	0x0
7	RW	vs_inputbuffer_chop_en	input buffer 的 chopper 功能使能信号。 0: 禁止; 1: 使能。	0x1
6:4	RW	vs_set_samp	采样时间控制信号 (控制采样时间占 ADC 工作时钟的周期数)。 000~111: 3~10 个 ADC 工作时钟周期; 001 (配置大于 001 时, ADC 采样率降低)。	0x1
3:2	RW	vs_bgssel_ref	REF buffer 输入参考电压选择信号。 00: 0.8V; 01: 0.85V; 10: 0.9V; 11: 0.95V。	0x2
1	RW	vs_buff_inc_en	用于电流检测模式, 抬升输入电流对应的电压, 抬升使能信号。 0: 正常模式, 无电压抬升; 1: 抬升输入电压。	0x0

Bits	Access	Name	Description	Reset
0	RW	vs_chan_en	通道开关控制信号。 0：所有通道均关断； 1：选通通道由 vs_chan_sel<3:0> 配置。	0x0

VSENSOR_1

VSENSOR 配置寄存器 1。

Offset Address: 0x024 Total Reset Value: 0x0000_8000

Bits	Access	Name	Description	Reset
31:0	RW	bgr_cfg	内部基准控制信号。 bit[31:16]：预留； bit[15]：内部 buffer 使能； bit[14:12]：测试信号选择控制位； bit[11:10]：buffer2 偏置电流控制位； bit[9:8]：buffer1 偏置电流控制位； bit[7:6]：环路稳定性补偿控制位 bit[5:4]：预留； bit[3]：外灌电流使能控制位； bit[2]：ECO 使能控制位； bit[1]：启动电路修调控制位； bit[0]：测试使能控制位。	0x00008000

VSENSOR_2

VSENSOR 配置寄存器 2。

Offset Address: 0x028 Total Reset Value: 0x0001_0000

Bits	Access	Name	Description	Reset
31:26	-	reserved	保留。	0x00
25	RW	vs_vcmex_sel	VCM 选择信号： 0：选择内部 VCM； 1：选择外灌 VCM。	0x0
24:21	RW	vs_iset	内部电流档位配置。 vs_iset[3:2]配置输入 buffer 偏置电流： 00：4μA； 01：5μA； 10：6μA； 11：3μA。 vs_iset[1:0]配置 REF buffer 偏置电流： 00：4μA； 01：5μA； 10：6μA； 11：3μA。	0x0
20:13	RW	vs_rsv	预留输入寄存器。 bit[7:6]：选择内部逻辑信号输出到 DTEST18 端口测试， vs_test_en=1 时有效。 00：samp 信号； 01：比较器输出； 10：比较器 chop 信号； 11：时钟。 bit[5:4]：VCMbuffer 编制电流控制位 VCM_IBIAS_SEL。 bit[3:2]：基准电压选择控制位 BG_VCM_SEL。	0x08

Bits	Access	Name	Description	Reset
			bit[1:0]: buffer 偏置电流控制位 ISET_BGBUF。	
12	RW	vs_test_en	IP 测试通道开关控制信号。 0: 关闭到 VTEST18 和 DTEST18 的测试通道; 1: 打开到 VTEST18 和 DTEST18 的测试通道。	0x0
11:6	RW	bgr_trim_dc	内部基准电压 trim 寄存器。 需要在 ATE 阶段完成校准, 然后将 trim 结果烧写进 eFuse。	0x00
5:0	RW	bgr_trim_temp	内部基准电压温漂 trim 寄存器, 需要在 ATE 阶段完成校准, 然后将 trim 结果烧写进 eFuse (该寄存器需在高温条件下 trim, 若无高温测试, 则不使用)。	0x00

VSENSOR_3

VSENSOR 配置寄存器 3。

Offset Address: 0x02C Total Reset Value: 0x0000_0001

Bits	Access	Name	Description	Reset
31:1	-	reserved	保留。	0x00000000
0	RW	vs_reset_apb	IP 复位信号, 需在 IP 使能 100μs 后, 转换开始前完成解复位。 0: IP 处于解复位态; 1: IP 处于复位态。 当 VSENSOR_4[vs_ctrl_mode]配置为 1 时生效。	0x1

VSENSOR_4

VSENSOR 配置寄存器 4。

Offset Address: 0x030 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	-	reserved	保留。	0x00000000
0	RW	vs_ctrl_mode	控制时序生成模式。 0: 通过配置 VSENSOR_5[vsensor_start]后自动产生; 1: 通过配置 VSENSOR_3[vs_reset_apb]、VSENSOR_0[vs_start_apb]构造。	0x0

VSENSOR_5

VSENSOR 配置寄存器 5。

Offset Address: 0x034 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	-	reserved	保留。	0x00000000
0	W1_PULSE	vsensor_start	写 1 后将自动生成 vs_start 时序。 当 VSENSOR_4[vs_ctrl_mode]配置为 0 时生效。	0x0

VSENSOR_6

VSENSOR 配置寄存器 6。

Offset Address: 0x038 Total Reset Value: 0x0001_3311

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
31:17	-	reserved	保留。	0x0000
16:12	RW	vs_start_separate_cycle	vs_start 自动生成模式下，每两个 vs_start 脉冲之间间隔的时钟周期数。 0x00 ~ 0x11：保留； 0x12：19 个周期； 0x13：20 个周期； ... 0x1F：32 个周期。	0x13
11	-	reserved	保留。	0x0
10:8	RW	vs_start_keep_cycle	vs_start 自动生成模式下，每个 vs_start 脉冲维持的时钟周期数（vsensor 工作时钟）。 000：保留； 001：保留； 010：保留； 011：4 个周期； 100：5 个周期； ... 111：8 个周期。	0x3
7	-	reserved	保留。	0x0
6:4	RW	vs_start_pre_cycle	vs_start 自动生成模式下，配置 vsensor_start 同步后到第一个 vs_start 信号上升沿的间隔时钟周期数（vsensor 工作时钟）。 000：保留； 001：2 个周期； 010：3 个周期；	0x1

Bits	Access	Name	Description	Reset
			... 111: 8 个周期。	
3:1	-	reserved	保留。	0x0
0	RW	equ_mode	vsensor 数据平均操作模式。 0: 单次完成后上报, 不做平均处理; 1: 16 次累加取平均值后上报。	0x1

VSENSOR_7

VSENSOR 配置寄存器 7。

Offset Address: 0x03C Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:28	-	reserved	保留。	0x0
27:16	RW	vs_data_th_min	vsensor 采样数据阈值下限, 当采样得到的数据低于此数时, 将会上报超阈值下限中断。	0x000
15:12	-	reserved	保留。	0x0
11:0	RW	vs_data_th_max	vsensor 采样数据阈值上限, 当采样得到的数据超过此数时, 将会上报超阈值上限中断。	0x000

VSENSOR_8

VSENSOR 配置寄存器 8。

Offset Address: 0x040 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
31:12	-	reserved	保留。	0x00000
11:0	RW	vs_data_offset	vsensor 采样数据偏移校准值。 0: 平均值 + vs_data_offset[10:0]; 1: 平均值 - vs_data_offset[10:0]。 采样计算后的平均值经过偏移校准后, 加操作如果溢出时返回 0x3FF, 减操作如果下溢时返回 0x0。	0x000

VSENSOR_9

VSENSOR 配置寄存器 9。

Offset Address: 0x044 Total Reset Value: 0x0000_0030

Bits	Access	Name	Description	Reset
31:6	-	reserved	保留。	0x00000000
5	RW	over_int_en	数据超过阈值上限或者低于阈值下限中断使能。 0: 禁止; 1: 使能。	0x1
4	RW	done_int_en	平均采样完成中断使能。 0: 禁止; 1: 使能。	0x1
3:2	-	reserved	保留。	0x0
1	RW	over_int_mask	数据超过阈值上限或者低于阈值下限中断屏蔽位。 0: 不屏蔽;	0x0

Bits	Access	Name	Description	Reset
			1: 屏蔽。	
0	RW	done_int_mask	平均采样完成中断屏蔽位。 0: 不屏蔽; 1: 屏蔽。	0x0

VSENSOR_10

VSENSOR 配置寄存器 10。

Offset Address: 0x048 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	-	reserved	保留。	0x00000000
0	W1_PULSE	vsensor_int_clr	中断清除信号。 0: 不清除中断; 1: 清除中断。	0x0

VSENSOR_STATUS_0

VSENSOR 状态寄存器 0。

Offset Address: 0x060 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:1	-	reserved	保留。	0x00000000
0	RO	working_rpt	自动生成时序情况下, 工作状态指示。 0: 工作完成; 1: 正在工作。	0x0

VSENSOR_STATUS_1

VSENSOR 状态寄存器 1。

Offset Address: 0x064 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:6	-	reserved	保留。	0x00000000
5	RO	over_int_mask_sts	数据超过阈值上限或者低于阈值下限中断屏蔽后状态。	0x0
4	RO	done_int_mask_sts	数据采样完成中断屏蔽后状态。	0x0
3:2	-	reserved	保留。	0x0
1	RO	over_int_sts	数据超过阈值上限或者低于阈值下限中断屏蔽前状态。	0x0
0	RO	done_int_sts	数据采样完成中断屏蔽前状态。	0x0

VSENSOR_STATUS_2

VSENSOR_STATUS_2 为 VSENSOR 状态寄存器 2。

Offset Address: 0x068 Total Reset Value: 0x0000_0000

Bits	Access	Name	Description	Reset
31:12	-	reserved	保留。	0x000000
11:0	RO	vs_data_rpt	采样完成后，经过偏移校准后的最终采样值。	0x000

5.12 Tsensor

5.12.1 概述

Tsensor 用于检测 die 的结温，并以二进制形式输出温度信息。

5.12.2 功能描述

Tsensor 模块具有以下功能特点：

- 上报获取的 Tsensor IP 温度值。
 - 辅助算法校准，即计算 16 次温度采样平均值。
 - 支持软件可配是否计算单点 16 次平均值。
 - 利用中断的方式实时上报当前校准后的温度值。
- 支持高温低温门限的使用。
 - 通过中断的方式实时上报当前温度是否超过高温门限或低于低温门限。
 - 支持软件分别可配高温门限和低温门限。
- 支持超高温保护的使用。
 - 通过中断的方式实时上报当前温度是否超过超高温的温度门限。
 - 支持软件可配超高温的温度门限。
- 支持周期采样温度上报。
 - 支持软件可配周期采样间隔。
- 支持 ATE 校准温度值。
 - 支持通过配置 TRIM 值的方式实现对 Tsensor IP 的温度校准。
 - Tsensor 的精度： $\pm 3^{\circ}\text{C}$ ，温度范围： $-40^{\circ}\text{C}\sim +125^{\circ}\text{C}$ 。

5.12.3 工作方式

Tsensor 模块的工作模式分为以下 3 种：

- 正常检测温度模式。
- 高低温门限中断模式。
- 过温保护中断模式。

说明

以上 3 种模式均为在检测温度值基础上进行，检测温度的模式有多种，此处检测温度的模式均为周期采样配合 16 次单点平均计算。

5.12.3.1 正常检测温度模式

正常检测温度模式配置步骤如下：

步骤 1 写 **TSENSOR_STS**[tsensor_clr]为 0x1，清除所有模式的状态信息。

- 步骤 2 写 `TSENSOR_TEMP_INT_CLR`[`tsensor_int_clr`]为 0x1，清除 Tsensor 中断信号。
- 步骤 3 写 `TSENSOR_TEMP_INT_EN`[`tsensor_done_int_en`]为 0x1，打开 Tsensor 的采集温度完成中断使能。
- 步骤 4 写 `TSENSOR_CTRL`[`tsensor_mode`]为 0x0，选择 16 次平均值的单次上报方式。
- 步骤 5 写 `TSENSOR_CTRL`[`tsensor_enable`]为 0x1，开启 Tsensor 的使能信号。
- 步骤 6 写 `TSENSOR_AUTO_REFRESH_PERIOD`[`tsensor_auto_refresh_period`]，设置合适的周期采样的时间间隔。
- 步骤 7 写 `TSENSOR_AUTO_REFRESH_CFG`[`tsensor_auto_refresh_enable`]为 0x1，开启周期采样的使能信号。
- 步骤 8 读 `TSENSOR_TEMP_INT_STS`[`tsensor_done_int_sts`]为 0x1，等待 16 点平均计算温度模式下的中断信号的产生。
- 步骤 9 读 `TSENSOR_STS`[`tsensor_data`]，获取 16 点平均单次上报模式下的温度值。

----结束

5.12.3.2 高低温门限中断模式

高低温门限中断模式配置步骤如下：

- 步骤 1 写 `TSENSOR_STS`[`tsensor_clr`]为 0x1，清除自动模式下产生的 rdy 信号。
- 步骤 2 写 `TSENSOR_TEMP_INT_CLR`[`tsensor_int_clr`]为 0x1，清除 Tsensor 中断信号。
- 步骤 3 写 `TSENSOR_TEMP_INT_EN`[`tsensor_out_thresh_int_en`]为 0x1，打开 Tsensor 的超门限范围中断使能。
- 步骤 4 写 `TSENSOR_TEMP_HIGH_LIMIT`[`tsensor_temp_high_limit`]、
`TSENSOR_TEMP_LOW_LIMIT`[`tsensor_temp_low_limit`]，设置合适的高低温门限值。
- 步骤 5 写 `TSENSOR_CTRL`[`tsensor_mode`]为 0x0，选择 16 次平均值的单次上报方式。
- 步骤 6 写 `TSENSOR_CTRL`[`tsensor_enable`]为 0x1，开启 Tsensor 的使能信号。
- 步骤 7 写 `TSENSOR_AUTO_REFRESH_PERIOD`[`tsensor_auto_refresh_period`]，设置合适的周期采样的时间间隔。
- 步骤 8 写 `TSENSOR_AUTO_REFRESH_CFG`[`tsensor_auto_refresh_enable`]为 0x1，开启周期采样的使能信号。

步骤 9 读 `TSENSOR_TEMP_INT_STS`[`tsensor_out_thresh_int_sts`]是否为 1，如果为 1，则当前的温度值高于高温门限或低于低温门限。

----结束

5.12.3.3 过温保护中断模式

过温保护中断模式配置步骤如下：

- 步骤 1 写 `TSENSOR_STS`[`tsensor_clr`]为 0x1，清除自动模式下产生的 rdy 信号。
- 步骤 2 写 `TSENSOR_TEMP_INT_CLR`[`tsensor_int_clr`]为 0x1，清除 Tsensor 中断信号。
- 步骤 3 写 `TSENSOR_TEMP_INT_EN`[`tsensor_overtemp_int_en`]为 0x1，打开 Tsensor 的过温中断使能。
- 步骤 4 写 `TSENSOR_OVER_TEMP`[`tsensor_overtemp_thresh`]，设置合适的过温门限值。
- 步骤 5 写 `TSENSOR_OVER_TEMP`[`tsensor_overtemp_thresh_en`]为 0x1，打开过温保护使能信号。
- 步骤 6 写 `TSENSOR_CTRL`[`tsensor_mode`]为 0x0，选择 16 次平均值的单次上报方式。
- 步骤 7 写 `TSENSOR_CTRL`[`tsensor_enable`]为 0x1，开启 Tsensor 的使能信号。
- 步骤 8 写 `TSENSOR_AUTO_REFRESH_PERIOD`[`tsensor_auto_refresh_period`]，设置合适的周期采样的时间间隔。
- 步骤 9 写 `TSENSOR_AUTO_REFRESH_CFG`[`tsensor_auto_refresh_enable`]为 0x1，开启周期采样的使能信号。

读 `TSENSOR_TEMP_INT_STS`[`tsensor_overtemp_int_sts`]是否为 1，如果为 1，则当前的温度值超过了设置的过温门限值。

----结束

5.12.4 寄存器概览

Tsensor 寄存器概览如表 5-26 所示。

表5-26 Tsensor 寄存器概览（基址是 0x3002_A000）

偏移地址	名称	描述
0x0000 + 0x1000×	TSENSOR_CTL_ID	TSENSOR CTL ID 寄存

偏移地址	名称	描述
CH_NUM		器。
0x0010 + 0x1000 × CH_NUM	TSENSOR_REG0	通用寄存器。
0x0014 + 0x1000 × CH_NUM	TSENSOR_REG1	通用寄存器。
0x0018 + 0x1000 × CH_NUM	TSENSOR_REG2	通用寄存器。
0x001C + 0x1000 × CH_NUM	TSENSOR_REG3	通用寄存器。
0x0300 + 0x1000 × CH_NUM	TSENSOR_START	TSENSOR 启动寄存器。
0x0304 + 0x1000 × CH_NUM	TSENSOR_CTRL	TSENSOR 控制寄存器。
0x0308 + 0x1000 × CH_NUM	TSENSOR_STS	TSENSOR 状态寄存器。
0x0310 + 0x1000 × CH_NUM	TSENSOR_CTRL1	TSENSOR 控制寄存器 1。
0x0314 + 0x1000 × CH_NUM	TSENSOR_TEMP_HIGH_LIMIT	TSENSOR 温度门限上限。
0x0318 + 0x1000 × CH_NUM	TSENSOR_TEMP_LOW_LIMIT	TSENSOR 温度门限下限。
0x031C + 0x1000 × CH_NUM	TSENSOR_OVER_TEMP	TSENSOR 过温控制寄存器。
0x0320 + 0x1000 × CH_NUM	TSENSOR_TEMP_INT_EN	TSENSOR 中断使能寄存器。
0x0324 + 0x1000 × CH_NUM	TSENSOR_TEMP_INT_CLR	TSENSOR 中断清除寄存器。
0x0328 + 0x1000 × CH_NUM	TSENSOR_TEMP_INT_STS	TSENSOR 中断状态寄存器。

偏移地址	名称	描述
0x0330 + 0x1000×CH_NUM	TSENSOR_AUTO_REFRESH_PERIOD	TSENSOR 自动检测周期寄存器。
0x0334 + 0x1000×CH_NUM	TSENSOR_AUTO_REFRESH_CFG	TSENSOR 自动检测控制寄存器。

Tsensor 寄存器偏移地址中变量的取值范围和含义如表 5-27 所示。

表5-27 Tsensor 寄存器偏移地址变量表

变量名称	取值范围	描述
CH_NUM	0	Tsensor 组数

5.12.5 寄存器描述

TSENSOR_CTL_ID

TSENSOR_CTL_ID 为 TSENSOR CTL ID 寄存器

Offset Address: 0x0000 + 0x1000×CH_NUM Total Reset Value: 0x0106

Bits	Access	Name	Description	Reset
[15:0]	RO	tsensor_ctl_id	TSENSOR CTL ID 寄存器。	0x0106

TSENSOR_REG0

TSENSOR_REG0 为通用寄存器。

Offset Address: 0x0010 + 0x1000×CH_NUM Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
[15:0]	RW	tsensor_reg0	Tsensor 通用寄存器 0。	0x0000

TSensor_REG1

TSensor_REG1 为通用寄存器。

Offset Address: $0x0014 + 0x1000 \times CH_NUM$ Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
[15:0]	RW	tsensor_reg1	Tsensor 通用寄存器 1。	0x0000

TSensor_REG2

TSensor_REG2 为通用寄存器。

Offset Address: $0x0018 + 0x1000 \times CH_NUM$ Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
[15:0]	RW	tsensor_reg2	Tsensor 通用寄存器 2。	0x0000

TSensor_REG3

TSensor_REG3 为通用寄存器。

Offset Address: $0x001C + 0x1000 \times CH_NUM$ Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
[15:0]	RW	tsensor_reg3	Tsensor 通用寄存器 3。	0x0000

TSensor_START

TSensor_START 为 TSensor 启动寄存器。

Offset Address: $0x0300 + 0x1000 \times CH_NUM$ Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
[15:1]	-	reserved	保留。	0x0000
[0]	W1_P	tsensor_start	自动模式下，写 1 刷新一次温度	0x0

Bits	Access	Name	Description	Reset
	ULSE		码, 回读 tsensor_data_auto 获取当前温度值, 当 tsensor_rdy_auto 为 1 时, 表明温度值有效。 写 0 无效。	

TSENSOR_CTRL

TSENSOR_CTRL 为 TSENSOR 控制寄存器。

Offset Address: 0x0304 + 0x1000×CH_NUM Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
[15:3]	-	reserved	保留。	0x0000
[2:1]	RW	tsensor_mode	模式选择 00: 16 点平均单次上报模式 (支持配置 period 周期定时使用); 01: 16 点平均循环上报模式 (无间隔持续检测并刷新码字); 10、11: 单点循环上报模式 (该模式不比较阈值, 仅上报温度码)。	0x0
[0]	RW	tsensor_enable	TSENSOR_CTRL 开关 0: 关闭 TENSOR_CTRL; 1: 打开 TENSOR_CTRL。	0x0

TSENSOR_STS

TSENSOR_STS 为 TSENSOR 状态寄存器。

Offset Address: 0x0308 + 0x1000×CH_NUM Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
------	--------	------	-------------	-------

Bits	Access	Name	Description	Reset
[15:12]	-	reserved	保留。	0x0
[11:2]	RO	tsensor_data	所有模式下获取到的温度值。 10bit 温度区间码值输出，线性分布； -40C~dec'146 125C~dec'943 即是 tsensor 直接输出码值对温度°C单位换算公式 $T_{\text{°C}} = [\text{BIN2DEC}(\text{temp_out} < 9:0>) - 146] / (943 - 146) * [125 - (-40)] + (-40)$ 。	0x000
[1]	RO	tsensor_rdy	所有模式下。 0：检测未启动或手动检测中； 1：tsensor_data 值为有效的温度值。	0x0
[0]	W1_PULSE	tsensor_clr	清除所有模式的状态。 0：无效； 1：清除。	0x0

TSENSOR_CTRL1

TSENSOR_CTRL1 为 TSENSOR 控制寄存器 1。

Offset Address: 0x0310 + 0x1000×CH_NUM Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
[15:5]	-	reserved	保留。	0x000
[4]	RW	temp_scan_dft	DFT 使能 0：功能输出； 1：DFT 输出。	0x0

Bits	Access	Name	Description	Reset
[3]	RW	temp_set	0: 功能模式下 temp_out 正常输出 (默认) ; 1: 功能模式下 temp_out 可以通过数模接口输入来配置输出值。	0x0
[2:1]	RW	temp_ct_sel	对于输入 1M 精准时钟情况下: 00: 0.512ms; 01: 0.256ms; 10: 1.024ms; 11: 2.048ms。	0x0
[0]	RW	temp_calib	0: 选择开启校准算法 (默认) ; 1: 不开启校准算法。	0x0

TSENSOR_TEMP_HIGH_LIMIT

TSENSOR_TEMP_HIGH_LIMIT 为 TSENSOR 温度门限上限。

Offset Address: 0x0314 + 0x1000×CH_NUM Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
[15:10]	-	reserved	保留。	0x00
[9:0]	RW	tsensor_temp_high_limit	过高温度阈值门限。	0x000

TSENSOR_TEMP_LOW_LIMIT

TSENSOR_TEMP_LOW_LIMIT 为 TSENSOR 温度门限下限。

Offset Address: 0x0318 + 0x1000×CH_NUM Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
[15:10]	-	reserved	保留。	0x00

Bits	Access	Name	Description	Reset
[9:0]	RW	tsensor_temp_low_limit	过低温度阈值门限。	0x000

TSENSOR_OVER_TEMP

TSENSOR_OVER_TEMP 为 TSENSOR 过温控制寄存器。

Offset Address: 0x031C + 0x1000×CH_NUM Total Reset Value: 0x03FF

Bits	Access	Name	Description	Reset
[15:11]	-	reserved	保留。	0x00
[10]	RW	tsensor_overnmp_thresh_en	16 点平均单次上报模式或 16 点平均循环上报模式下过温保护阈值判断使能。 0: 过温保护阈值判断使能关闭; 1: 过温保护阈值判断使能打开。	0x0
[9:0]	RW	tsensor_overnmp_thresh	过温保护阈值门限。	0x3FF

TSENSOR_TEMP_INT_EN

TSENSOR_TEMP_INT_EN 为 TSENSOR 中断使能寄存器

Offset Address: 0x0320 + 0x1000×CH_NUM Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
[15:3]	-	reserved	保留。	0x0000
[2]	RW	tsensor_overnmp_int_en	TSENSOR 过温保护中断使能。	0x0
[1]	RW	tsensor_out_thresh_int_en	TSENSOR 温度超门限范围中断使能。 0: 禁止;	0x0

Bits	Access	Name	Description	Reset
			1：使能。	
[0]	RW	tsensor_done_int_en	TSENSOR 温度采集完毕中断使能。 0：禁止； 1：使能。	0x0

TSENSOR_TEMP_INT_CLR

TSENSOR_TEMP_INT_CLR 为 TSENSOR 中断清除寄存器。

Offset Address: 0x0324 + 0x1000×CH_NUM Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
[15:1]	-	reserved	保留。	0x0000
[0]	W1_PULSE	tsensor_int_clr	0：无效； 1：清除中断。	0x0

TSENSOR_TEMP_INT_STS

TSENSOR_TEMP_INT_STS 为 TSENSOR 中断状态寄存器。

Offset Address: 0x0328 + 0x1000×CH_NUM Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
[15:3]	-	reserved	保留。	0x0000
[2]	RO	tsensor_overtemp_int_sts	TSENSOR 过温保护中断状态。	0x0
[1]	RO	tsensor_out_thresh_int_sts	TSENSOR 温度超门限范围中断状态。	0x0
[0]	RO	tsensor_done_int_sts	TSENSOR 温度采集完毕中断状态。	0x0

TSensor_AUTO_REFRESH_PERIOD

TSensor_AUTO_REFRESH_PERIOD 为 TSENSOR 自动检测周期寄存器。

Offset Address: 0x0330 + 0x1000×CH_NUM Total Reset Value: 0xFFFF

Bits	Access	Name	Description	Reset
[15:0]	RW	tsensor_auto_refresh_period	TSensor 自动检测周期, 32k 时钟周期数。	0xFFFF

TSensor_AUTO_REFRESH_CFG

TSensor_AUTO_REFRESH_CFG 为 TSENSOR 自动检测控制寄存器。

Offset Address: 0x0334 + 0x1000×CH_NUM Total Reset Value: 0x0000

Bits	Access	Name	Description	Reset
[15:1]	-	reserved	保留。	0x0000
[0]	RW	tsensor_auto_refresh_enable	16 点平均单次上报模式下周期检测使能。 0: 定时周期检测关闭; 1: 定时周期检测打开。	0x0

5.13 Audio PWM

5.13.1 概述

Audio PWM 将音频数据转化为对应的占空比方波输出。Audio PWM 与硬件 RC 电路和 PA 连接实现音频播放。

5.13.2 功能描述

Audio PWM 具有以下功能特点:

- 支持将音频数据点转换为方波输出。
- 支持数据传输完成后中断上报，支持中断使能，中断屏蔽。

Draft

6 安全

6.1 概述

6.2 功能描述

6.1 概述

Hi2131 系列芯片支持安全相关功能。

6.2 功能描述

安全具有以下功能特点：

- 支持芯片安全启动。
- 支持 EFUSE，用于存储关键信息。
- 支持自研安全算法能力，包括：PBKDF2、SHA256、ECC 算法。

7 JTAG

7.1 概述

7.2 功能描述

7.1 概述

Hi2131 系列芯片内部集成 A、C 双核，基于内部集成的 Coresight 调试架构，可使用 JTAG 和 SWD 接口进行调试。Hi2131 系列芯片支持 JLINK 仿真器。

7.2 功能描述

芯片调试接口默认复用为其他功能。使用调试功能时，需要将 JTAG_ENABLE 管脚置为高电平，此时对应 GPIO 作为调试接口。支持 EFUSE 熔断 JTAG 调试功能。

须知

默认 JTAG 管脚复用为其他功能，有两种方法可改为 JTAG 调试功能：

- 管脚 JTAG_ENABLE 在板级接上拉电阻。
- 软件配置 IO 复用关系。

JTAG 接口信号如表 7-1 所示。

表7-1 JTAG 接口信号

信号名称	含义
------	----

信号名称	含义
JTAG_TCK	JTAG/SWD 时钟输入。
JTAG_TDI	JTAG 数据输入。
JTAG_TMS	JTAG/SWD 模式选择。
JTAG_TRST	JTAG 复位输入。
JTAG_TDO	JTAG 数据输出。

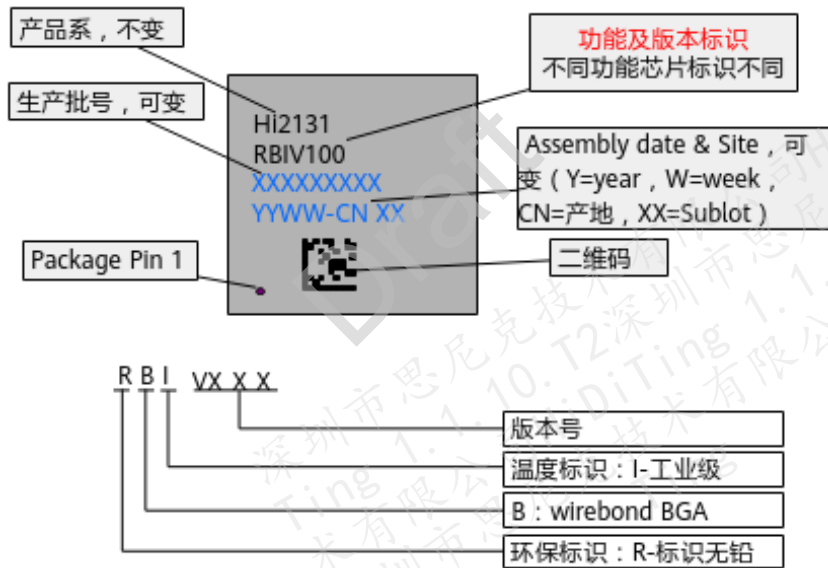
说明

除支持 JTAG 接口外，同时兼容 SWD 接口，涉及到接口信号为表 7-1 中的 JTAG_TCK 和 JTAG_TMS，信号含义与 JTAG 相同。

A 订购须知

芯片 Mark 命名规则如图 A-1、图 A-2 所示。

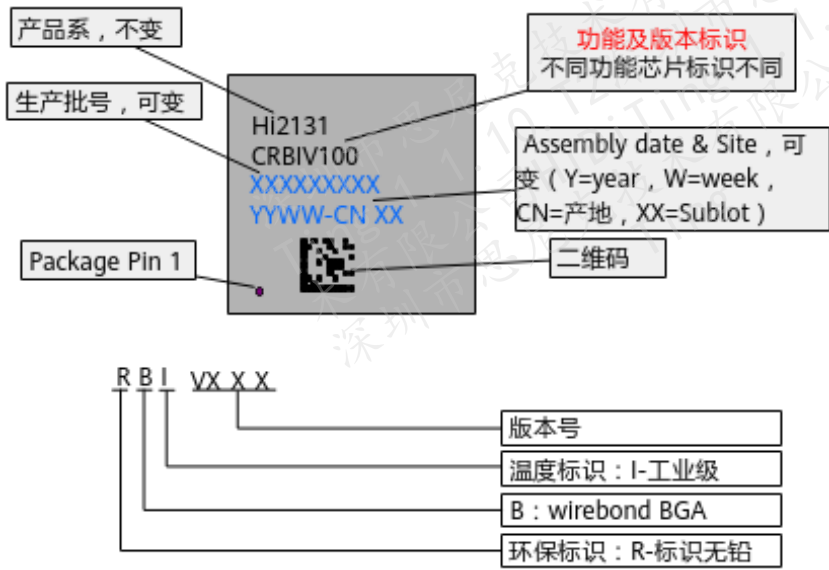
图A-1 芯片 Mark 命名规则 1



表A-1 芯片应用的配置 1

Part Number	功能	封装类型	封装尺寸	Pitch
Hi2131RBI V100	支持 CAT1 协议，支持 OpenCPU 增强，4MB FLASH KGD。	TFBG A	5.8mm×5.8mm	0.4mm

图A-2 芯片 Mark 命名规则 2



表A-2 芯片应用的配置 2

Part Number	功能	封装类型	封装尺寸	Pitch
Hi2131CR BIV100	支持 CAT1 协议，支持轻量化 OpenCPU，主要面向通用物联场景，2MB FLASH KGD。	TFBG A	5.8mm× 5.8mm	0.4mm

B 缩略语

Numerics

3GPP 3rd Generation Partnership Project 第三代合作伙伴计划

A

ADC Analog-to-Digital Converter 模数转换器

AES Advanced Encryption Standard 高级加密标准

AHB Advanced High Performance Bus 先进高性能总线

AIO Analog Input/Output 模拟输入/输出

ANA Analog 模拟

AO Always-On 常电区

AP Application Processor 应用处理器

APB Advanced Peripheral Bus 高级外设总线

ATE Auto Test Equipment 自动测试设备

B

BGA Ball Grid Arra 球栅阵列

C

CH China 中国

CPU Central Processing Unit 中央处理单元

D

DAC	Digital-to-Analog Converter	数模转换器
DCXO	Digitally Compensated Crystal Oscillator	数字补偿晶体振荡器
DFE	Digital Front-End	数字前端
DMA	Direct Memory Access	直接内存访问模块
DRX	Discontinuous Reception	非连续接收
DS	Drive Strength	驱动强度
DT	Deutsche Telekom AG	德国电信股份有限公司

E

EAL	Evaluation Assurance Level	评估保证级
ECC	Elliptic Curve Cryptography	椭圆加密算法
ECO		
EDGE	Edge Detector	边沿检测器
EFC	External Flash Controller	外置 Flash 控制器
EFUSE	Electronic Fuse	一次性可编程器件

F

FDD	Frequency Division Duplex	频分双工
FIFO	First In First Out	先进先出
FOTA	Firmware Over-The-Air	空中固件升级

G

GPIO	General Purpose Input/Output	通用输入输出
-------------	------------------------------	--------

I

I2C	Inter-Integrated Circuit	内部整合电路
------------	--------------------------	--------

I2S	Inter-Integrated Circuit Sound	集成电路内置音频
IO	Input Output	输入输出
IPC	Inter-Processor Communication	处理器间通信
IPv4	Internet Protocol version 4	第四版互联网协议
IPv6	Internet Protocol version 6	第六版因特网协议
IRQ	Interrupt Quest	中断请求
J		
JTAG	Joint Test Action Group	联合测试行为组织，是一种国际标准的测试协议
K		
KGD	Known Good Die	已知合格裸片
KM	Key Managerment	密钥管理
L		
LDO	Low Drop Out	低压差输出
LP_UART	Low Power Universal Asynchronous Receiver & Transmitter	低功耗通用异步收发器
M		
MCU		
MPU	Memory Protection Unit	存储器保护单元
N		
NMI	Non-Maskable Interrupt	不可屏蔽中断
O		
OSC	Oscillator	振荡器

P

PA	Power Amplifier	功率放大器
PBKDF2	Password-Based Key Derivation Function 2	基于密码的密钥衍生函数
PIO	Programmable Input/Output	可程序输入输出
PKE	Public-Keyencryption	公开密钥加密
PLL	Phase-Locked Loops	锁相环
PMP	Physical Memory Protectio	
PMU	Power Management Unit	电源管理模块
POR	Power On Reset	上电复位
PSM	Power Saving Mode	节电模式
PWM	Pulse-Width Modulation	脉冲宽度调制
REF	Reference	基准
RF	Radio Frequency	射频
RFFE	RF Front-End	射频前端
ROM	Read-Only Memory	只读存储器
RTC	Real-Time Clock	实时时钟
RX	Receiver	接收器

S

SCL	Serial Clock Line	串行时钟线
SDA	Serial Data and Address	串行数据地址线
SE	Secure Element	安全单元
SFC	SPI Flash Controller	串行接口闪存控制器
SHA	Secure Hash Algorithm	安全散列算法
SIM	Subscriber Identity Module	用户识别模块

SOC	System-On-a-Chip	片上系统
SP	Security Processor	安全处理器
SPI	Serial Peripheral Interface	串行外设接口
SR	Service Request	业务请求
SRAM	Static Random Access Memory	静态随机存取存储器
SSI	Synchronous Serial Interface	同步串行接口
SWD	Serial Wire Debug	串行线调试
T		
TCP	Transmission Control Protocol	传输控制协议
TCXO	Temperature Compensated Crystal Oscillator	温度补偿晶体振荡器
TDD	Time Division Duplex	时分双工
TFBGA	Thin & Fine Ball Grid Array	薄型精细球栅阵列封装
TI	Texas Instruments	德州仪器
TX	Transmitter	发送器
U		
UART	Universal Asynchronous Receiver & Transmitter	通用异步收发器
UDP	User Datagram Protocol	用户数据报协议
UE	User Equipment	用户终端
UICC	Universal Integrated Circuit Card	通用集成电路卡
USB	Universal Serial Bus	通用串行总线
V		
VBAT	Voltage of Battery	电池电压
W		

WDT	Watchdog Timer	看门狗计时器
X		
XO	Crystal Oscillators	晶振
Z		
ZUC	ZUC Stream Cipher Algorithm	祖冲之序列密码算法